

# HYDRA

A Digital Management System for  
PSG Electrical and Cables / Trite Solar

*Project Report — Task 1 Documentation*

## INSY7315 — Information Systems 3E (WIL)

Client: PSG Electrical and Cables / Trite Solar ([tritesolar.co.za](http://tritesolar.co.za))

WIL Coordinators: Denzyl Soobramoney Govender, Gareth Grant

### Team HYDRA

Kaedon Naidoo (Group Leader) — ST10293141

Yashodha Govender — ST10318651

Darsh Solmayi — ST10288567

Katelyn Narain — ST10311395

Yajur Mohan — ST10440284

Lonwabo Gumede — ST10270409

Date: 17 August 2026

## Table of Contents

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 1.1 About the Team .....                                                 | 9  |
| 1.2 About the Client.....                                                | 9  |
| 1.3 Purpose of the Project .....                                         | 10 |
| 1.4 Project Background .....                                             | 10 |
| 1.5 Scope Statement.....                                                 | 13 |
| 2. Requirements .....                                                    | 15 |
| 2.1 User Roles & User Stories .....                                      | 15 |
| 2.1.1 User Roles .....                                                   | 15 |
| 2.1.2 User Stories.....                                                  | 17 |
| 2.2 UX Journey Map .....                                                 | 23 |
| 2.2.1 Customer Journey.....                                              | 25 |
| 2.2.2 Employee (Electrician) Journey .....                               | 26 |
| 2.2.3 Admin Journey.....                                                 | 27 |
| 2.2.4 Design Rationale for the Three-Lane, Cross-Linked Structure .....  | 27 |
| 2.2.5 Cross-Checking Against the Design Class and Sequence Diagrams..... | 28 |
| 2.3 Work Breakdown Structure (WBS) .....                                 | 28 |
| 2.3.1 Phase Rationale and Dependency Logic .....                         | 30 |
| 2.3.2 Resource Distribution Across the Team .....                        | 32 |
| 2.3.3 Critical Path and Schedule Risk.....                               | 33 |
| 2.4 Non-Functional Requirements .....                                    | 33 |
| 3. Analysis Artifacts .....                                              | 38 |
| 3.1 Domain Class Diagram .....                                           | 38 |
| Purpose of the diagram .....                                             | 38 |
| Methodology: how the domain model was produced and validated .....       | 39 |
| A worked example: tracing one job through the domain model .....         | 39 |
| Class descriptions .....                                                 | 40 |
| Relationships and multiplicities.....                                    | 46 |
| Deliberate design decisions and alternatives considered .....            | 47 |
| 3.2 Design Class Diagram.....                                            | 49 |
| Purpose of the diagram and what changes from the domain model .....      | 49 |
| Attribute typing, keys and structural detail.....                        | 50 |
| Key classes and their behaviour .....                                    | 54 |
| Design principles applied.....                                           | 56 |
| Mapping the design class diagram to the codebase .....                   | 56 |
| Relationships, keys and visibility .....                                 | 56 |
| 4. Implementation Artifacts .....                                        | 58 |
| 4.1 Sequence and State Diagrams.....                                     | 58 |
| Purpose of this section and the behavioural modelling approach.....      | 58 |

|                                                                                               |     |
|-----------------------------------------------------------------------------------------------|-----|
| State diagrams.....                                                                           | 59  |
| Sequence diagrams.....                                                                        | 78  |
| Traceability matrix.....                                                                      | 91  |
| A note on traceability .....                                                                  | 92  |
| 4.2 Deployment Plan .....                                                                     | 92  |
| Infrastructure and environment overview .....                                                 | 92  |
| Physical and logical nodes .....                                                              | 93  |
| Network segregation and security .....                                                        | 94  |
| Monitoring, observability and operational visibility .....                                    | 94  |
| Rollout strategy and rationale.....                                                           | 94  |
| Disaster recovery and business continuity within the deployment .....                         | 95  |
| 4.3 Data Schema: Choice of Technology .....                                                   | 95  |
| Chosen technology stack.....                                                                  | 95  |
| Justification: project requirements and data shape.....                                       | 96  |
| Justification: team skill set.....                                                            | 96  |
| Justification: cost .....                                                                     | 97  |
| Justification: scalability and future growth .....                                            | 97  |
| Alternatives considered.....                                                                  | 97  |
| Data normalisation and schema design discipline .....                                         | 98  |
| Indexing and query performance strategy.....                                                  | 99  |
| Summary.....                                                                                  | 100 |
| 4.4 Data Schema: Entity Relationship Diagram .....                                            | 100 |
| Purpose and structure of the ERD set .....                                                    | 100 |
| Customer / Website ERD .....                                                                  | 101 |
| Employee ERD.....                                                                             | 103 |
| Admin ERD .....                                                                               | 105 |
| Access Control ERD.....                                                                       | 106 |
| Notation legend and constraint summary.....                                                   | 108 |
| 5. Architecture Artifacts .....                                                               | 109 |
| 5.1 Design Patterns.....                                                                      | 109 |
| 5.1.1 The problem the Repository pattern solves.....                                          | 109 |
| 5.1.2 Anatomy of a single repository triad — worked example.....                              | 110 |
| 5.1.3 Applying the pattern across the domain — the remaining five triads .....                | 110 |
| 5.1.4 UML notation used in Figure 31, and the dependency-inversion principle it encodes ..... | 111 |
| 5.1.5 Why the Repository pattern, and not an alternative .....                                | 113 |
| 5.1.6 Integration with the wider architecture .....                                           | 113 |
| 5.2 Architecture Patterns .....                                                               | 114 |
| 5.2.1 Choice of pattern and alternatives considered .....                                     | 114 |
| 5.2.2 The four layers, described component by component .....                                 | 115 |

|                                                                           |     |
|---------------------------------------------------------------------------|-----|
| 5.2.3 Request flow — a worked example traced through all four layers..... | 116 |
| 5.2.4 Why strict layering matters, and what it protects.....              | 117 |
| 5.2.5 Trade-offs acknowledged .....                                       | 118 |
| 5.3 Cloud Architecture.....                                               | 118 |
| 5.3.1 Chosen cloud provider and justification.....                        | 119 |
| 5.3.2 Architecture overview — tier by tier.....                           | 120 |
| 5.3.3 The missed-call and notification pipeline in detail.....            | 121 |
| 5.3.4 Autoscaling, availability and resilience considerations .....       | 123 |
| 5.3.5 Network segregation and communication protocols.....                | 123 |
| 5.3.6 Alignment with non-functional requirements.....                     | 124 |
| 5.3.6 Alignment with non-functional requirements.....                     | 124 |
| 6. Security .....                                                         | 125 |
| 6.1 Authentication and Authorisation.....                                 | 125 |
| 6.1.1 Credential submission and transport .....                           | 125 |
| 6.1.2 Password storage — bcrypt hashing .....                             | 126 |
| 6.1.3 Token issuance and structure (JSON Web Tokens) .....                | 126 |
| 6.1.4 Token lifetime, refresh and secure client-side storage .....        | 127 |
| 6.1.5 Role-based access control (RBAC) .....                              | 127 |
| 6.2 Data Protection.....                                                  | 128 |
| 6.2.1 Data classification .....                                           | 129 |
| 6.2.2 Protection in transit.....                                          | 129 |
| 6.2.3 Protection at rest and key management.....                          | 129 |
| 6.2.4 Data protection on mobile and offline scenarios .....               | 130 |
| 6.2.5 Data minimisation and retention .....                               | 130 |
| 6.3 Input Validation and Injection Prevention .....                       | 131 |
| 6.3.1 SQL injection prevention via the Repository pattern .....           | 131 |
| 6.3.2 Schema and boundary validation .....                                | 131 |
| 6.3.3 File upload validation .....                                        | 131 |
| 6.3.4 Cross-site scripting (XSS) and output encoding.....                 | 132 |
| 6.3.5 Cross-site request forgery (CSRF) resistance.....                   | 132 |
| 6.3.6 Rate limiting and security headers .....                            | 132 |
| 6.4 POPIA Compliance .....                                                | 132 |
| 6.4.1 Accountability.....                                                 | 133 |
| 6.4.2 Processing limitation and minimality .....                          | 133 |
| 6.4.3 Purpose specification .....                                         | 133 |
| 6.4.4 Further processing limitation .....                                 | 133 |
| 6.4.5 Information quality.....                                            | 133 |
| 6.4.6 Openness .....                                                      | 134 |
| 6.4.7 Security safeguards.....                                            | 134 |



|                                                                       |     |
|-----------------------------------------------------------------------|-----|
| 6.4.8 Data subject participation .....                                | 134 |
| 6.4.9 Cross-border transfer and hosting location .....                | 134 |
| 6.4.10 Consent capture and the missed-call feature specifically ..... | 135 |
| 6.4.11 Breach notification .....                                      | 135 |
| 6.5 Payment Security .....                                            | 135 |
| 6.5.1 No card data in HYDRA's own database .....                      | 135 |
| 6.5.2 Tokenisation and reduced PCI-DSS scope .....                    | 135 |
| 6.5.3 Gateway selection considerations.....                           | 136 |
| 6.5.4 Webhook integrity and idempotency.....                          | 136 |
| 6.5.5 Reconciliation, refunds and disputes .....                      | 136 |
| 6.6 Backup and Recovery Strategy .....                                | 137 |
| 6.6.1 Recovery objectives .....                                       | 137 |
| 6.6.2 Backup configuration.....                                       | 137 |
| 6.6.3 Geo-redundancy and restore testing .....                        | 137 |
| 6.6.4 Disaster recovery runbook and business continuity .....         | 137 |
| 6.7 Audit Logging .....                                               | 138 |
| 6.7.1 A distributed, entity-attached approach .....                   | 138 |
| 6.7.2 Log content, retention and integrity .....                      | 138 |
| 6.7.3 Monitoring, alerting and review cadence .....                   | 138 |
| 6.8 Risk Analysis.....                                                | 139 |
| 6.8.1 Risk methodology and risk appetite .....                        | 139 |
| 6.8.2 Interpreting the register and ongoing review .....              | 142 |
| 7. DevOps.....                                                        | 144 |
| 7.1 Version Control Workflow .....                                    | 144 |
| 7.1.1 Repository structure and branching strategy.....                | 144 |
| 7.1.2 Pull requests and code review.....                              | 145 |
| 7.1.3 Commit message conventions.....                                 | 145 |
| 7.1.4 Merge strategy, tagging and release branches .....              | 146 |
| 7.2 Continuous Integration and Automated Build .....                  | 146 |
| 7.2.1 Pipeline triggers and stages.....                               | 146 |
| 7.2.2 Static analysis and linting .....                               | 146 |
| 7.2.3 Automated test execution .....                                  | 146 |
| 7.2.4 Automated build and artefact production .....                   | 147 |
| 7.2.5 Pipeline visibility and notifications .....                     | 147 |
| 7.3 Testing Strategy .....                                            | 147 |
| 7.3.1 The testing pyramid.....                                        | 147 |
| 7.3.2 Unit testing .....                                              | 148 |
| 7.3.3 Integration testing .....                                       | 148 |
| 7.3.4 Manual UAT, smoke testing and end-to-end verification .....     | 148 |

|                                                                                    |     |
|------------------------------------------------------------------------------------|-----|
| 7.3.5 Test data management and non-functional testing .....                        | 149 |
| 7.4 Deployment Strategy and Environments .....                                     | 149 |
| 7.4.1 Environment structure .....                                                  | 149 |
| 7.4.2 Staging as a production-equivalent environment .....                         | 149 |
| 7.4.3 Deployment mechanism — slot swapping .....                                   | 150 |
| 7.4.4 Rollback strategy .....                                                      | 150 |
| 7.4.5 Database schema migrations .....                                             | 150 |
| 7.5 Release Process and Release Management .....                                   | 151 |
| 7.5.1 Release cadence and milestones .....                                         | 151 |
| 7.5.2 Parallel release paths for API and client applications .....                 | 151 |
| 7.5.3 Staged mobile rollout .....                                                  | 151 |
| 7.5.4 Release notes and versioning transparency .....                              | 151 |
| 7.5.5 Hotfix process .....                                                         | 152 |
| 7.6 Monitoring, Logging and Observability .....                                    | 152 |
| 7.6.1 Application performance monitoring .....                                     | 152 |
| 7.6.2 Alerting .....                                                               | 152 |
| 7.6.3 Relationship to audit logging and security monitoring .....                  | 152 |
| 7.7 DevOps Culture and Team Collaboration Practices .....                          | 153 |
| 7.7.1 Agile ceremonies and iteration structure .....                               | 153 |
| 7.7.2 Task tracking and project visibility .....                                   | 153 |
| 7.7.3 Documentation as a first-class deliverable .....                             | 153 |
| 7.7.4 Retrospectives and continuous improvement .....                              | 153 |
| 8. Running Costs .....                                                             | 155 |
| 8.1 Cost Categories .....                                                          | 155 |
| 8.1.1 Application hosting (compute) .....                                          | 156 |
| 8.1.2 Database hosting .....                                                       | 156 |
| 8.1.3 Background and scheduled compute .....                                       | 157 |
| 8.1.4 Object storage .....                                                         | 157 |
| 8.1.5 Secrets management .....                                                     | 157 |
| 8.1.6 CDN and edge networking .....                                                | 157 |
| 8.1.7 Domain name .....                                                            | 158 |
| 8.1.8 SSL/TLS certificate .....                                                    | 158 |
| 8.1.9 Notification gateway (SMS, WhatsApp and email) .....                         | 158 |
| 8.1.10 Payment gateway fees .....                                                  | 159 |
| 8.1.11 Cost categories deliberately excluded from the infrastructure figures ..... | 159 |
| 8.2 Cost at Multiple Scales .....                                                  | 159 |
| 8.2.1 Low-volume scenario (100 jobs/month) .....                                   | 160 |
| 8.2.2 Medium-volume scenario (1,000 jobs/month) .....                              | 160 |
| 8.2.3 High-volume scenario (10,000 jobs/month) .....                               | 161 |

|                                                                          |     |
|--------------------------------------------------------------------------|-----|
| 8.2.4 Cost per job as a normalised comparison metric .....               | 161 |
| 8.2.5 Sensitivity to exchange rate and third-party pricing changes ..... | 161 |
| 8.3 Free-Tier vs Paid-Tier Trade-offs .....                              | 162 |
| 8.3.1 What is realistically available at no cost.....                    | 162 |
| 8.3.2 Limitations and risks of free and lowest-cost tiers.....           | 162 |
| 8.3.3 Recommended migration path.....                                    | 162 |
| 8.3.4 Governance of tier changes after handover.....                     | 163 |
| 8.4 Two-Year Cost Projection .....                                       | 163 |
| 8.4.1 Growth assumptions and their justification.....                    | 163 |
| 8.4.2 Interpreting the projection.....                                   | 163 |
| 8.4.3 Risk factors and assumptions that could alter the projection.....  | 164 |
| 8.4.4 Recommended review cadence .....                                   | 164 |
| 9. Change Management .....                                               | 165 |
| 9.1 Training Plan .....                                                  | 165 |
| 9.1.1 Rationale for a role-differentiated training design.....           | 165 |
| 9.1.2 Office/Admin staff training.....                                   | 166 |
| 9.1.3 Electrician (field/employee) training.....                         | 166 |
| 9.1.4 Shared documentation asset and long-term availability .....        | 166 |
| 9.1.5 Measuring training effectiveness .....                             | 167 |
| 9.2 Handover Plan .....                                                  | 167 |
| 9.2.1 Formal handover deliverables .....                                 | 167 |
| 9.2.2 The handover session itself .....                                  | 167 |
| 9.2.3 Explicit boundaries of post-handover responsibility.....           | 168 |
| 9.2.4 Documentation as the primary post-handover risk mitigation.....    | 168 |
| 9.3 Transition Support Period .....                                      | 168 |
| 9.3.1 Rationale for a defined, time-boxed window.....                    | 168 |
| 9.3.2 Parallel running as a low-risk operational safety net .....        | 168 |
| 9.3.3 Support channel and escalation .....                               | 169 |
| 9.3.4 Formal conclusion of the transition support period .....           | 169 |
| 9.4 Adoption Roadmap.....                                                | 169 |
| 9.4.1 Sequencing rationale .....                                         | 169 |
| 9.4.2 Mapping the roadmap to Kotter's model.....                         | 170 |
| 9.4.3 Mapping the roadmap to ADKAR at the individual level.....          | 170 |
| 9.4.4 Contingency if a phase does not proceed smoothly .....             | 170 |
| 9.5 Managing Resistance to Change.....                                   | 170 |
| 9.5.1 Where resistance is most likely to arise, and why .....            | 170 |
| 9.5.2 Mitigation 1 — deliberately minimal scope .....                    | 171 |
| 9.5.3 Mitigation 2 — phased, staggered exposure .....                    | 171 |
| 9.5.4 Mitigation 3 — genuine co-design rather than imposed change.....   | 171 |

---

|                                                                               |     |
|-------------------------------------------------------------------------------|-----|
| 9.5.5 Reactive support if resistance surfaces despite these mitigations ..... | 172 |
| 9.5.6 Reinforcement — sustaining adoption beyond the transition period .....  | 172 |
| 10. Appendices .....                                                          | 173 |
| Appendix A — Diagram-to-Section Mapping and Coverage Summary .....            | 173 |
| Appendix B — Meeting Minutes .....                                            | 173 |
| B.1 Minutes of Meeting — 18 April 2026 .....                                  | 173 |
| B.2 Minutes of Meeting — 16 May 2026 .....                                    | 175 |
| B.3 Client Meeting Attendance Confirmation .....                              | 176 |
| Appendix C — Partnership Agreement .....                                      | 178 |
| Appendix D — Glossary of Terms .....                                          | 181 |
| References .....                                                              | 182 |
| Appendix E — Prototype and Interactive Demonstrations .....                   | 186 |
| E.1 System Overview .....                                                     | 186 |
| E.2 Website Prototype .....                                                   | 187 |
| E.2.1 As-Built Website Prototype Screens .....                                | 190 |
| E.3 Customer App — Live Interactive Prototype .....                           | 194 |
| E.4 Employee App — Flow and High-Fidelity Mockup .....                        | 200 |
| E.5 Admin App — Flow and High-Fidelity Mockup .....                           | 200 |



## 1. Introduction

### 1.1 About the Team

HYDRA is the title chosen by a team of six students created to develop and construct a comprehensive digital management system for PSG Electrical and Cables and its renewable-energy branch, Trite Solar. The name embodies the team's multi-faceted, interdisciplinary character: each member manages a specific aspect of the system — data, design, development, infrastructure, and delivery — yet all six collaborate towards a unified final product. Table 1 outlines the team, the number of students, and the main areas of responsibility.

**Table 1: HYDRA team members and responsibility areas**

| Team Member       | Student Number | Primary Role                         | Core Responsibility Areas                                                                                                                                    |
|-------------------|----------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kaedon Naidoo     | ST10293141     | Project Lead / Business Analyst      | Client liaison, requirements gathering, Work Breakdown Structure and project plan, User Acceptance Testing coordination, training and handover documentation |
| Darsh Somayi      | ST10288567     | Lead Developer / Solutions Architect | Client liaison, Design Class Diagram and design pattern selection, backend API development, authentication and role-based access control                     |
| Katelyn Narain    | ST10311395     | Business Analyst / Data Architect    | Entity-Relationship Diagram and domain modelling, Design Class Diagram, database implementation, handover documentation                                      |
| Yashodha Govender | ST10318651     | UX/UI Designer                       | UX journey mapping, wireframing, frontend user-interface build                                                                                               |
| Yajur Mohan       | ST10440284     | Cloud/DevOps Engineer                | System architecture and cloud design, backend API support, Azure deployment and CI/CD pipeline configuration                                                 |
| Lonwabo Gumede    | ST10270409     | Frontend Developer / QA Tester       | Frontend implementation, unit and integration testing                                                                                                        |

The contribution area for each member was allocated based on their proven strengths and academic expertise, which was then formalised through the Work Breakdown Structure (refer to Section 2.3) that links every task in the 133-day project schedule to a designated resource. This guarantees that accountability can be traced from the initial planning document to the completed HYDRA submission.

### 1.2 About the Client

PSG Electrical and Cables, in collaboration with Trite Solar ([tritesolar.co.za](http://tritesolar.co.za)), is a South African company providing residential and commercial electrical installation, repair, and compliance inspection services, as well as solar photovoltaic installation through its Trite Solar division. The client can be contacted at 072 752 0848 or [Keorang@gmail.com](mailto:Keorang@gmail.com).

Currently, the business primarily relies on manual, paper, or spreadsheet processes to handle essential operations: recording customer job requests, creating quotes, allocating electricians to tasks, monitoring

materials utilized on-site, producing invoices, noting payments, and providing compliance certificates (Certificates of Compliance) mandated by South African electrical regulation (Republic of South Africa, 2009). With the expansion of the business into electrical and solar services, this manual method has turned into a major operational hindrance: tracking job status in real-time is challenging, quotes and invoices often contain transcription mistakes, and there is an absence of a centralized system connecting a customer's request to the electrician's field tasks, materials used, and the final compliance paperwork.

This establishes a distinct and urgent demand for a digital solution that merges these separate manual processes into a unified, role-sensitive platform — one that a customer, a field electrician, and office/management personnel can each utilize effectively, without repeating data entry or losing sight of a job's progress.

### **1.3 Purpose of the Project**

The aim of the HYDRA project is to create and develop a comprehensive web platform and associated mobile application that digitalizes and integrates the essential business processes of PSG Electrical and Cables and Trite Solar. In particular, the system is designed to:

Substitute handwritten, paper job logging with a systematic digital process encompassing job requests, estimates, electrician allocation, task execution, materials monitoring, invoicing, payment, and compliance inspection reporting.

Provide customers with immediate access to their job's status and a compliance record that they can access whenever needed.

Provide field electricians with a mobile-centric solution for checking their schedules, confirming site arrivals, recording materials, and submitting inspection reports without having to return to the office for paperwork.

Provide management and administrative personnel with a central control point for allocating tasks, overseeing payments and inventory, handling electrician payroll based on recorded hours, and prioritizing customer inquiries — featuring a basic automated response system for inquiries received as missed calls.

Implement customer-retention features — a rewards and discount initiative — that cannot be achieved with the existing manual system, establishing the mobile app (instead of the website) as the main continuous interface between PSG Electrical and Cables/Trite Solar and its clientele.

### **1.4 Project Background**

The HYDRA initiative emerged from a Work-Integrated Learning (WIL) collaboration between the student team and PSG Electrical and Cables/Trite Solar, arranged through a partnership agreement with Emeris. WIL placements of this nature are well acknowledged in South African higher education as a means to bridge the divide between academic learning and practical workplace skills: a systematic, project-oriented collaboration involving a higher education institution, an overseeing academic entity, and an industry partner is perceived in the literature to yield a "win-win" scenario where students acquire real client-facing experience while the industry partner benefits from a customized solution to a real operational challenge (Lubbe and Svensson, 2024).

The engagement commenced with a preliminary client meeting on 18 April 2026 at Umdloti Mall, overseen by Kaedon Naidoo as the group leader, with Darsh Somayi recording the minutes and all six HYDRA members in attendance. The initial meeting included introductions of stakeholders, discussions on the client's business objectives, the intended functionalities of the website and app, target audience specifications, and platform needs, as well as preliminary conversations regarding the project timeline and budget; it ended with the

agreement to start development on the website and Android app, to provide a prototype for client feedback prior to full-scale development, and to establish a single point of contact for ongoing communication between both teams.

This initial meeting established the framework for the entire engagement: the HYDRA team conducted a systematic series of client-facing meetings and internal team discussions throughout the Planning and Design phases, with each session recorded and centered on a key, decision-influencing topic. Table 2 below encapsulates all 20 meetings conducted from the initial client engagement to the conclusion of the Design phase, all finished prior to 5 August 2026. A complete, unaltered version of the Minutes of Meeting documents for the two meetings highlighted in the checklist — 18 April 2026 and 16 May 2026 — is included in Appendix C, allowing this section to summarize their results while the full original records are accessible for confirmation.

**Table 2: HYDRA project meetings, 18 April 2026 – 4 August 2026**

| # | Date        | Type     | Significant Topic                                                    | Key Outcome                                                                                                                                                                                                                    |
|---|-------------|----------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | 18 Apr 2026 | Client   | Stakeholder introductions, business goals, and platform purpose      | Development to begin with website and Android app; prototype-first approach agreed; single point of contact established                                                                                                        |
| 2 | 23 Apr 2026 | Internal | Team role allocation and communication protocol                      | Responsibility areas assigned per member (see Table 1); weekly internal check-in cadence agreed                                                                                                                                |
| 3 | 29 Apr 2026 | Internal | Initial Work Breakdown Structure drafting                            | Six-phase project structure (Planning, Design, Development, Testing, Deployment, Handover) drafted for client review                                                                                                           |
| 4 | 05 May 2026 | Client   | Service catalogue and electrical compliance certificate requirements | Confirmed that both electrical and solar service categories must be represented as configurable Service_Type records; compliance-certificate data (CoC number, compliance status) confirmed as a mandatory output of every job |
| 5 | 11 May 2026 | Internal | Draft ERD review — core entities                                     | Customer, Job, Electrician, Quote and Invoice entities and relationships reviewed and approved as the foundation for further modelling                                                                                         |
| 6 | 16 May 2026 | Client   | Confirming the three-role login structure and full feature scope     | Customer, Employee and Admin apps confirmed as separate login-gated experiences;                                                                                                                                               |

| #  | Date        | Type     | Significant Topic                                          | Key Outcome                                                                                                                    |
|----|-------------|----------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
|    |             |          |                                                            | rewards/discounts, QR check-in, and missed-call AI messaging confirmed as required features                                    |
| 7  | 21 May 2026 | Internal | UX journey mapping kickoff and persona definition          | Customer, Electrician and Admin personas defined as the basis for the UX journey map                                           |
| 8  | 27 May 2026 | Client   | Branding guidelines and website content handover           | Client provided logo, colour palette and initial service/portfolio content for use in wireframes                               |
| 9  | 02 Jun 2026 | Internal | Wireframe and navigation-flow review                       | Screen-by-screen navigation flow for the customer journey signed off internally ahead of client presentation                   |
| 10 | 08 Jun 2026 | Internal | ERD extension — rewards, discounts and employee scheduling | Rewards_Account, Discount, Employee_Schedule and Timesheet entities added to the data model                                    |
| 11 | 13 Jun 2026 | Internal | Design Class Diagram drafting and design pattern selection | Repository pattern with dependency inversion selected for data-access layer, to be applied consistently across all entities    |
| 12 | 19 Jun 2026 | Client   | QR check-in feature walkthrough                            | Client approved the QR scan-on-arrival concept as the mechanism for confirming job start and triggering the milestone timeline |
| 13 | 25 Jun 2026 | Internal | Cloud architecture and Azure service selection             | Azure App Service, Azure Database for PostgreSQL and Azure Blob Storage confirmed as the hosting stack                         |
| 14 | 01 Jul 2026 | Client   | Rewards and discounts programme scoping                    | Points-earning rule (per completed and paid job) and discount-code redemption model confirmed                                  |
| 15 | 07 Jul 2026 | Internal | Database schema finalisation                               | Full schema reviewed for normalisation and foreign-key integrity ahead of implementation                                       |

| #  | Date        | Type     | Significant Topic                                          | Key Outcome                                                                                                                                                                                        |
|----|-------------|----------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 16 | 13 Jul 2026 | Client   | Missed-call AI messaging and data-privacy considerations   | Client confirmed the requirement for automated SMS response to missed calls; team raised POPIA consent and data-minimisation requirements for contact/call-log access, to be addressed in the NFRs |
| 17 | 18 Jul 2026 | Internal | Authentication and role-based access control design review | JWT-based authentication with RBAC confirmed as the security model across all three apps                                                                                                           |
| 18 | 24 Jul 2026 | Internal | API contract and endpoint planning                         | REST endpoint list drafted ahead of the Development phase's Backend API task                                                                                                                       |
| 19 | 30 Jul 2026 | Client   | Mid-design progress demo                                   | Client reviewed wireframes, ERD and system architecture together and approved progression into Development                                                                                         |
| 20 | 04 Aug 2026 | Internal | Design-phase sign-off and Development readiness review     | Team confirmed all Design-phase deliverables complete and ready to hand into the Development phase per the WBS                                                                                     |

## 1.5 Scope Statement

In scope for this phase of the HYDRA project:

- A public-facing marketing website (Home, About, Services, FAQ, Portfolio, Contact Us) requiring no login;
- A customer-facing mobile application, including all website content plus job request and tracking, quote review and acceptance, invoicing and payment, a rewards and discounts programme, a live job-milestone timeline, and QR-code-based job check-in confirmation;
- An employee (electrician) mobile application, including calendar/schedule viewing, QR-code scanning to confirm job arrival, materials logging, inspection report submission, and time-clock functionality for hours worked;
- An administrative web/mobile application, including job assignment, quote and invoice generation, payment and stock monitoring, payroll processing from submitted timesheets, triage of website contact-form queries, and a basic automated (AI-assisted) SMS/message response to missed calls;
- A relational database (Azure Database for PostgreSQL) implementing the full data model described in Section 4.4 of this document, covering all three applications;
- Role-based authentication and access control differentiating customer, employee and admin permissions across a single shared backend;

-Deployment to Microsoft Azure with an automated CI/CD pipeline.

Out of scope for this phase:

-Native iOS application development (the mobile application is scoped, per client meeting minutes, primarily around Android, with the possibility of a future iOS release considered but not committed to in this phase);

-Integration with third-party accounting or ERP software (e.g. Sage, Xero);

-E-commerce functionality (online sale of electrical materials or solar components to the public);

-Multi-language localisation;

-IoT/hardware integration (e.g. smart electrical monitoring devices), beyond the QR-code check-in mechanism described above.

This scope statement will be revisited at subsequent project milestones as client feedback is incorporated, consistent with the iterative review points built into the Work Breakdown Structure (Section 2.3).



## 2. Requirements

### 2.1 User Roles & User Stories

#### 2.1.1 User Roles

The HYDRA data model (refer to the Client, Employee, and Admin Entity-Relationship Diagrams) outlines three main authenticated roles, along with one visitor context that is not authenticated. Every role corresponds directly to a login-secured application, with each supported by specific attributes (username, password\_hash, role) on its related entity, upheld via role-based access control (RBAC) at the API level. RBAC is a commonly used security framework where permissions are linked to roles instead of specific users, ensuring that every account receives only the essential access needed to carry out its duties — adhering to the security principle of least privilege (Pathlock, 2026). In practical terms, this signifies that each API request directed at the HYDRA backend includes a signed JSON Web Token with a role claim, which is verified by a common authentication middleware prior to allowing access to any controller; the controller subsequently executes a more detailed authorization check (for instance, ensuring that the customer\_id included in a job record corresponds with the customer\_id on the requesting token) before delivering or altering any data. This dual-layer method — an initial broad role verification succeeded by a detailed ownership verification — is intentional, as RBAC alone merely addresses "is this user allowed to access this kind of endpoint", while the ownership verification resolves the distinct and equally crucial query "is this particular user allowed to view this specific record", which is vital in a system like HYDRA where three clients, two electricians, and an office administrator could query the same Job table at once and need to view only the relevant subset of rows for each.

A total of four access contexts are identified—one unauthenticated and three authenticated. Each context is outlined below along with the specific business justifications, from PSG Electrical and Cables/Trite Solar's viewpoint as a small, owner-operated electrical and solar services company, regarding the purpose of the role, the actions it is allowed to perform, and importantly, the actions it is expressly prohibited from undertaking.

Guest (not logged in). Any individual from the public visiting the marketing site. A visitor can access all public information (Home, About, Services, FAQ, Portfolio) and send a Contact Us inquiry, but cannot view, create, or edit any job, billing, or personal-account details. This serves as the initial step aimed at transforming visitors into registered clients. From a business viewpoint, this position exists because most potential customers of a small electrical and solar contractor will first discover the company through a search engine result, a referral, or local advertising — rather than having login credentials — and any need to create an account before accessing the company's past work portfolio or FAQs would unnecessarily elevate the obstacle to that initial interaction. The Visitor context is intentionally maintained with minimal friction: no personal data is collected or retained until a Contact Us request is submitted, at which time only the essential fields required to process that request (name, contact number, email, and the message itself) are recorded in the Contact\_Query entity, and still no Customer account record is generated unless the visitor clearly decides to register. This design decision is inherently a direct, structural implementation of data-minimisation, a fundamental principle of the Protection of Personal Information Act, which mandates that personal information be collected only to the level that is sufficient, pertinent, and not excessive for its intended purpose (Republic of South Africa, 2013) — the Visitor role, by its very nature, does not have a purpose that would warrant collecting more than what is necessary for a Contact Us form. From a security-engineering viewpoint, the Visitor context represents the system's most vulnerable public interface, as it can be accessed by anyone with internet access and no authentication; therefore, the Contact Us submission

endpoint is subject to rate limiting and server-side validation (refer to Section 6.3, Input Validation and Injection Prevention) precisely because it allows write access for a completely anonymous user.

**Client.** A customer of PSG Electrical and Cables/Trite Solar who has created an account. A client can ask for a new task, examine and approve estimates, monitor the current status and milestone timeline of an ongoing job, see and settle invoices, access past payments, earn and use loyalty rewards and discounts, and check compliance/inspection reports for finished work. Upon the first login, the user is led through a brief onboarding tutorial and an initial profile configuration step, as outlined in the project brief. Each of these capabilities aligns directly with a particular entity or group of entities in the foundational data model, and it is beneficial to trace that alignment explicitly, as it transforms the Customer role's permission boundary into something that can be validated against the schema instead of remaining an informal description: requesting a job creates a new entry in Job; reviewing and accepting a quote accesses and alters a record in Quote; tracking a live timeline retrieves entries from Job\_Milestone filtered by job\_id; viewing and paying invoices accesses Invoice and creates Payment; viewing previous payments retrieves Payment filtered by invoice\_id linked back through Invoice to the specific customer's customer\_id; earning and redeeming rewards accesses and modifies Rewards\_Account, Rewards\_Transaction, Discount, and Customer\_Discount; and viewing compliance reports retrieves Inspection\_Report filtered by the job\_id of jobs the customer personally requested. In all of these instances, the API layer's ownership verification (outlined previously) limits the query to rows where the pertinent foreign key connects to the authenticated customer's own customer\_id — a customer account can never create a request, no matter how the client-side application is altered, that fetches another customer's job, invoice, or inspection report, since that restriction is enforced on the server side instead of merely concealed by the application's user interface.

The guided onboarding walkthrough mentioned earlier is designed for a specific, client-oriented purpose instead of being a standard "nice to have" feature: the current customer base of PSG Electrical and Cables/Trite Solar has historically interacted exclusively via phone prior to HYDRA, making it improbable for new HYDRA users to be acquainted with concepts like reviewing digital quotes, scanning QR codes to verify an electrician's arrival, or monitoring a live milestone timeline — all of which represent entirely new interaction methods for this unique customer segment rather than a minor evolution of an existing digital practice. The onboarding walkthrough and the first profile setup stage serve as the means to close this gap, and their incorporation is directly shown in the usability non-functional requirement elaborated on in Section 2.4.

**Electrician (Worker).** A field technician working for PSG Electrical and Cables/Trite Solar. An electrician has the ability to check their designated tasks and schedule (filled by themselves or an administrator), scan a customer's QR code upon arrival to validate the job booking and initiate the job timeline, record materials utilized for a job, update the status of job milestones, submit compliance inspection reports, clock in and out to track hours worked, and send leave requests. Electricians lack access to the schedules, payroll, or customers of other electricians who are not assigned to them. Similar to the Customer role mentioned earlier, each capability corresponds to a particular entity: assigned jobs and calendar relate to Job (filtered by electrician\_id) and Employee\_Schedule; QR scanning corresponds to Job\_Checkin, which captures a timestamp and GPS location as proof of arrival; logging materials corresponds to a new entry in Job\_Material (or a similar join structure) that subsequently decreases Material.stock\_level; updating milestone status corresponds to Job\_Milestone; submitting a compliance report corresponds to Inspection\_Report; clocking in and out corresponds to Timesheet; and submitting leave corresponds to Leave\_Request.

The clear limitation that an electrician cannot view other electricians' schedules or payroll is not a random oversight but a purposeful design choice supported by two specific rationales. The initial reason is a clear privacy rationale: an electrician's personal data, including hours worked, leave taken, and salary, pertains to

that particular employee, and there is no functional necessity for a coworker to access this information, making the restriction of visibility a straightforward, minimal application of the least-privilege principle previously applied to the Visitor and Customer roles (Pathlock, 2026). The second reason pertains to the intimate, tight-knit work culture at businesses like PSG Electrical and Cables/Trite Solar: in a small group where electricians often work physically close and know each other well, the lack of a technical barrier to accessing a colleague's schedule or salary could realistically lead to interpersonal tension or feelings of favoritism if one electrician observed another regularly getting more or higher-paying job assignments — limiting this visibility at the system level eliminates that potential for workplace discord before it has a chance to develop, instead of depending on an informal assumption that employees will refrain from checking.

**Admin / Office Staff.** Internal employees tasked with handling routine operational duties. An admin possesses the ability to access and control all customer and electrician records, allocate electricians to tasks, create price estimates and bills, track payments and material inventory, accept or reject leave applications, handle payroll using submitted work logs, review website contact-form submissions, and handle the automated missed-call notification system. Given that a small electrical/solar company of this scale usually does not require a dedicated reporting interface, the Manager/Owner role outlined in the guidelines is integrated as an authorization level within the single Admin role (marked by the role attribute on the Admin profile) instead of as a separate, fifth application—where the owner logs into the same Admin system but receives complete access, including data export and payroll authorization, while office staff may be limited to certain functions (such as task assignment and quoting only) through the same role-based access RBAC control system.

This design choice—a single Admin application with an internal permission tier instead of a separate fifth Manager/Owner application—requires more thorough justification than a brief explanation can offer, as it reflects a deliberate trade-off rather than an automatic decision. A completely distinct Manager/Owner application would demand its own front-end code, its own UX workflows, and ongoing maintenance throughout the system's lifespan, for a role that, in a small business like PSG Electrical and Cables/Trite Solar (a compact, owner-run contracting company with a limited number of office and field employees), is typically filled by one or at most two individuals. Moreover, the owner's daily tasks closely mirror those of the office administrator—both need access to quotes, invoices, job assignments, and payments; the owner merely requires extra functions like payroll approval and data export for accounting or tax needs. Developing and sustaining an entire fifth application for such a narrow permission gap would not be a practical allocation of the project's limited development hours under the Work Breakdown Structure (see Section 2.3) and would introduce a fourth codebase for the client to manage post-handover, contradicting the maintainability non-functional requirement outlined in Section 2.4. By implementing the distinction as a role attribute on the Admin entity, the owner and office administrator can use the same application, login process, and codebase, differing only in the UI elements and API endpoints made available to them after authentication. This approach is not only more cost-effective to develop but also far more affordable to maintain for a client without an internal development team once HYDRA's direct involvement concludes.

### 2.1.2 User Stories

User stories follow the standard "As a [role], I want to [action], so that [benefit]" structure. This method of documenting needs, as opposed to drafting a traditional specification, ensures each requirement stays tied to both a distinct role and a clear business advantage, representing a widely accepted agile approach to requirements engineering (AltexSoft, 2023). Acceptance criteria are supplied for the nine main stories to enable independent verification of each, adhering to the "Testable" element of the INVEST framework used for evaluating user-story quality (AltexSoft, 2023). The complete INVEST framework checks whether a story meets six criteria: Independent (can be built and delivered without strict dependence on another story), Negotiable (serves as a discussion starting point rather than a rigid agreement), Valuable (provides a

meaningful advantage to the requesting role, not just a technical task), Estimable (clear enough for the team to reasonably gauge effort), Small (narrowly defined to fit within a single development iteration, matching the two-to-five-day task detail in Section 2.3's WBS) and Testable (stated in a way that allows confirmation of fulfillment or failure) (AltexSoft, 2023). All stories listed below were assessed against these six standards during requirements collection (WBS task 1.1), not just testability; when two stories showed interdependence that broke the Independent rule (for instance, a client couldn't "accept a quote" without a prior story on how the quote is produced), the stories were intentionally combined under the same module heading so readers can still see their connection, even though each remains its own standalone value unit.

To maintain the same level of rigor across all twenty-four stories listed below, rather than focusing acceptance criteria and prioritization solely on a few prominent stories, each story now includes both a Given/When/Then acceptance criterion and a MoSCoW priority rating—Must have, Should have, Could have, or Won't have this time—a commonly used requirements-prioritization method that originated in the Dynamic Systems Development Method and is currently maintained by the Agile Business Consortium (2014). This approach is particularly well-suited to a fixed-deadline academic project like this one, where the 133-day schedule outlined in the WBS (Section 2.3) cannot be extended if the scope exceeds expectations.

A Must-have story is one that, if missing, would prevent HYDRA from delivering its core value proposition to PSG Electrical and Cables/Trite Solar entirely; a Should-have story is important but the system would remain minimally functional without it; a Could-have story provides additional value but can be delayed past the current submission if time constraints arise without significantly impacting the client relationship. Importantly, none of the stories below are rated Won't have, because every story included here passed an earlier informal triage during requirements gathering that had already eliminated anything deemed clearly outside the scope of this engagement.

## Module: Job Management

**1.** *As a Customer, I want to request a job/service, so that an electrician can be assigned to resolve my electrical or solar problem.*

**Priority:** Must have — this is the single entry point for every paying transaction the business conducts through HYDRA; without it, no other story in the Job Management, Quoting, or Invoicing & Payments modules has anything to act on.

**Acceptance criteria:** Given a logged-in customer, when they select a service type and submit a job request with a site address, then a new Job record is created with status "Requested" and becomes visible to Admin.

**Additional detail:** The service type selected is drawn from the Service\_Type lookup entity described in Section 4.4 rather than free text, both so that Admin's dashboard can filter and report on demand by category (a capability the client specifically values given Trite Solar's ongoing expansion from electrical work into solar installation) and so that a future, not-yet-in-scope automatic-quoting feature would have a clean, structured field to key off. The site address is captured as a discrete field on Job rather than inferred from the customer's profile address, because a returning customer may reasonably be requesting work at a second property (a rental unit, a family member's home, or a small-business premises) distinct from their own registered residential address.

**2.** *As an Admin, I want to assign an electrician to a job, so that work is scheduled efficiently and the customer knows who to expect.*

**Priority:** Must have.

**Acceptance criteria:** Given a job with an accepted quote, when an admin selects an available electrician and confirms, then the job's electrician\_id is set, the job status updates to "Scheduled", and an entry is written to the job-assignment log.

**Additional detail:** The requirement that a quote be *accepted* before assignment is possible (rather than allowing assignment on a merely requested, unquoted job) is a deliberate business-process constraint: it prevents an electrician's time being provisionally scheduled against a job the customer has not yet actually committed to paying for, which would otherwise risk wasted travel and blocked calendar slots if the customer subsequently declined the quote. The job-assignment log exists as its own append-only record, separate from the single current electrician\_id value stored on Job, specifically so that a later reassignment (for example, if the originally assigned electrician becomes unavailable) does not erase the history of who was originally offered the work — a distinction elaborated further in Section 3.1 (Domain Class Diagram) and Section 4.4 (ERD).

**3. As an Electrician, I want to view my assigned jobs, so that I know my schedule for the day.**

**Priority:** Must have.

**Acceptance criteria:** Given a logged-in electrician, when they open the calendar screen, then all jobs and schedule events assigned to them are listed in chronological order.

**Additional detail:** Because South African field technicians frequently work across a wide geographic area with variable mobile data coverage, this screen is designed to load a reasonably short lookahead window (the current day and the immediately following days) by default rather than the electrician's entire job history, directly supporting the performance non-functional requirement discussed in Section 2.4.

**4. As an Electrician, I want to scan the customer's QR code on arrival, so that the job booking and timeline are automatically confirmed.**

**Priority:** Must have.

**Acceptance criteria:** Given an electrician on site with the customer's app open, when the electrician scans the customer-generated QR code, then a check-in record is created with a timestamp and GPS location, and the job status updates to "In Progress" on both apps.

**Additional detail:** Capturing a GPS location alongside the timestamp gives the business a verifiable, evidentiary record of both *when* and *where* work began, which protects PSG Electrical and Cables/Trite Solar against any future dispute over whether, or when, an electrician actually attended a given site — a safeguard with direct financial significance given that payroll (Story 18, below) is ultimately calculated from the timesheet data this check-in event is the trigger for.

**5. As a Customer, I want to view a live timeline of job milestones, so that I understand the progress of my repair or installation and when it will be complete.**

**Priority:** Should have — materially improves the customer experience and directly reduces the "black box" problem described in Section 2.2, but the core transaction (job requested, quoted, completed, invoiced, paid) remains functional even in a build where milestone-level detail were temporarily unavailable to the customer.

**Acceptance criteria:** Given an in-progress job, when the electrician marks a milestone complete, then the customer's app immediately reflects the updated milestone status and estimated completion.

## **Module: Quoting**

**6. As a Customer, I want to view and accept a quote, so that I can approve the cost before work begins.**

**Priority:** Must have.

**Acceptance criteria:** Given a quote generated by Admin, when the customer opens it, then labour cost, materials cost and total amount are displayed, and an "Accept" action updates the quote status and triggers job scheduling.

**Additional detail:** Breaking the displayed quote into labour cost and materials cost separately, rather than presenting only a single combined total, gives the customer transparency into how the price was arrived at — a deliberate trust-building design choice for a business whose existing, pre-HYDRA quoting process was verbal and informal, and where a first-time HYDRA customer has no prior digital interaction with the business to anchor their trust in a number presented without justification.

**7. As an Admin, I want to create and send a quote for a requested job, so that the customer has a clear cost estimate before work is scheduled.**

**Priority:** Must have.

**Acceptance criteria:** Given a newly requested job, when an admin enters labour and estimated materials cost, then a quote record is generated and a notification is sent to the customer.

## Module: Materials & Compliance

**8. As an Electrician, I want to log materials used on a job, so that costs are tracked accurately.**

**Priority:** Must have.

**Acceptance criteria:** Given an in-progress job, when the electrician adds a material and quantity, then the material's stock level is decremented and the job's materials cost is recalculated.

**Additional detail:** Recalculating the job's materials cost at the moment of logging, rather than only at invoicing time, means Admin has an accurate, real-time view of a job's accumulating cost even while it is still in progress — relevant where a job's actual materials usage diverges meaningfully from what was originally estimated in the quote (Story 7), giving Admin advance warning before an invoice is generated rather than only discovering the discrepancy afterwards.

**9. As an Electrician, I want to submit an inspection report, so that compliance is documented and a certificate can be issued.**

**Priority:** Must have — this story exists to satisfy a binding legal obligation, not merely a business preference. Under the Electrical Installation Regulations made in terms of the Occupational Health and Safety Act, a Certificate of Compliance referencing the relevant SANS 10142-1 wiring standard must be issued for electrical work performed in South Africa (Republic of South Africa, 2009), and HYDRA cannot be considered fit for purpose for an electrical contracting business without a mechanism to generate and retain that record.

**Acceptance criteria:** Given a completed job, when the electrician submits findings and a compliance status, then an inspection report record is created with a certificate number and becomes visible to the customer.

**10. As an Admin, I want to track material stock levels, so that supplies can be reordered before they run out.**

**Priority:** Should have — improves operational efficiency and reduces the risk of a job being delayed by an unavailable part, but the core job-execution and billing workflow does not structurally depend on automated stock alerts; in their absence, stock could still be tracked, only less proactively, by an admin manually reviewing the material catalogue.

**Acceptance criteria:** Given the material catalogue, when stock for an item falls below its reorder level, then the item is flagged on the admin dashboard.

## Module: Invoicing & Payments

**11. As an Admin, I want to generate an invoice from a completed job, so that the customer can be billed.**

**Priority:** Must have.

**Acceptance criteria:** Given a job marked "Completed", when an admin generates an invoice, then the invoice reflects the accepted quote amount and any applied discount, with a due date.

**Additional detail:** Requiring that the job first reach the "Completed" status before an invoice can be generated at all (rather than allowing an invoice to be raised speculatively against a merely scheduled or in-progress job) directly enforces, at the level of a user story and its acceptance criterion, the same business rule that is separately encoded as a legal state transition in the Job Lifecycle state diagram referenced in Section 4.1 — a customer is never billed for work that has not yet actually been finished.

**12. As a Customer, I want to make a payment against an invoice, so that my account is settled.**

**Priority:** Must have.



**Acceptance criteria:** Given an outstanding invoice, when the customer submits a payment, then a payment record is created, the invoice status updates accordingly, and reward points are credited to the customer's account.

**Additional detail:** The phrase "the invoice status updates accordingly" is deliberately not written as a single fixed outcome, because HYDRA's billing model explicitly supports partial payment: a payment that does not cover the full amount outstanding moves the invoice to a "Partially Paid" status rather than "Paid", and only a payment (or the cumulative sum of several payments) that brings the outstanding balance to zero triggers the "Paid" status and the associated reward-point credit described in the second half of this acceptance criterion — this partial-payment cardinality is discussed further, and justified in relational-modelling terms, in Section 4.4.

## Module: Rewards & Discounts

**13.** *As a Customer, I want to earn reward points for completed and paid jobs, so that I am incentivised to keep using the app rather than a competitor.*

**Priority:** Should have — meaningfully supports the client's stated goal of building repeat business and differentiating HYDRA from a purely transactional, phone-based booking process, but the system remains fully functional as a job-management and billing tool in its absence.

**Acceptance criteria:** Given an invoice that has just reached "Paid" status, when the payment settlement event is processed, then a Rewards\_Transaction record is created crediting the customer's Rewards\_Account with points calculated as a function of the invoice total, and the customer's updated points balance is reflected immediately on their dashboard.

**14.** *As a Customer, I want to redeem discounts against future invoices, so that my loyalty is recognised in a tangible way.*

**Priority:** Should have.

**Acceptance criteria:** Given a customer with a sufficient points balance and an available discount offer, when the customer selects "Redeem" against a specific eligible invoice, then a Customer\_Discount record is created linking the discount to that invoice, the invoice's amount due is reduced accordingly, and the redeemed points are deducted from the customer's Rewards\_Account balance.

## Module: Employee Scheduling & Hours

**15.** *As an Electrician, I want to clock in and out of each shift, so that my hours are accurately recorded and passed to Admin for payment.*

**Priority:** Must have — payroll (Story 18) is structurally dependent on this data existing; without it, electricians could not be paid accurately for hours actually worked.

**Acceptance criteria:** Given a logged-in electrician at the start or end of a shift, when they select "Clock In" or "Clock Out", then a Timesheet record is created or updated with the corresponding timestamp, and the total hours worked for that shift is calculated automatically once both timestamps are present.

**16.** *As an Electrician, I want to submit a leave request, so that I can plan time away from work.*

**Priority:** Should have.

**Acceptance criteria:** Given a logged-in electrician, when they submit a leave request specifying a start date, end date and reason, then a Leave\_Request record is created with status "Pending" and becomes visible to Admin for approval.

**17.** *As an Admin, I want to approve or decline leave requests, so that adequate staffing is maintained.*

**Priority:** Should have.

**Acceptance criteria:** Given a pending leave request, when an admin approves or declines it, then the request's status updates accordingly, the requesting electrician is notified, and, if approved, the electrician's availability is reflected on the scheduling and job-assignment screens for the relevant date range.



**18.** *As an Admin, I want to process payroll from submitted timesheets, so that electricians are paid accurately and on time.*

**Priority:** Must have.

**Acceptance criteria:** Given a pay period with confirmed timesheet records for an electrician, when an admin runs payroll for that period, then a Payroll record is generated summing the electrician's total hours across all associated Timesheet rows, applying the relevant pay rate, and producing a gross pay, deductions and net pay figure ready for admin sign-off.

## **Module: Admin Operations & Website Queries**

**19.** *As an Admin, I want to receive and triage contact-form queries submitted through the website, so that genuine leads are converted into scheduled jobs quickly.*

**Priority:** Must have — this is the primary conversion mechanism between an anonymous website visitor and a paying Customer, and without it, the marketing website described in the Visitor role above would have no functional purpose beyond static information display.

**Acceptance criteria:** Given a newly submitted Contact\_Query, when an admin opens the admin dashboard's enquiries screen, then the query appears with the visitor's contact details and message, and the admin can mark it as "In Progress", "Converted to Job" or "Closed" to track its resolution.

**20.** *As an Admin, I want the system to send an automatic message to a caller after a missed call, so that a potential customer enquiry is not lost simply because no one was available to answer the phone.*

**Priority:** Should have — this is a genuine value-add specifically requested in the client brief and reflects the real operational reality of a small business where the office phone is not always staffed, but the system's core job-management and billing functions do not depend on it, and a build without it would still deliver the majority of HYDRA's value to the client.

**Acceptance criteria:** Given a missed call logged on the admin device, when the call goes unanswered beyond a configured threshold, then a Missed\_Call\_Log record is created, an automated message is generated and sent via the AI\_Message\_Log mechanism, and the outgoing message content and delivery status are recorded for admin review.

**Additional detail:** Because this feature requires the admin application to access phone contacts and call-log information — a materially more sensitive category of device data than any other feature in HYDRA requires — its scope is deliberately and explicitly limited to the admin's own work device, with explicit consent obtained during admin onboarding, and only the minimum information necessary to identify the caller and generate a relevant reply is ever persisted; this constraint is discussed further under Compliance in Section 2.4 and under POPIA Compliance in Section 6.4.

**21.** *As an Admin, I want to export customer and employee data, so that I can generate reports to support business decisions.*

**Priority:** Could have — valuable for longer-term business planning and accounting reconciliation, but not required for HYDRA's core day-to-day job-management, quoting, billing and scheduling workflows to function correctly, and can reasonably be deferred to a later iteration of the system if project time runs short.

**Acceptance criteria:** Given an admin with export permissions (that is, an Admin account with the Owner/Manager permission tier described in Section 2.1.1), when they select a data category and date range and confirm the export, then a structured export file is generated containing only the requested category of data, and the export event itself is logged for audit purposes.

## **Module: Website & Onboarding**

**22.** *As a Visitor, I want to browse services, FAQs and the portfolio without registering, so that I can evaluate the business's credibility before committing to sign up.*

**Priority:** Must have.

**Acceptance criteria:** Given an unauthenticated visitor on the marketing website, when they navigate to the Services, FAQ or Portfolio pages, then the relevant content loads without any login prompt or account-creation requirement.

**23.** *As a new user, I want a guided walkthrough of the app immediately after sign-up, so that I understand its features before I start using it independently.*

**Priority:** Should have.

**Acceptance criteria:** Given a customer completing registration for the first time, when they reach their dashboard for the first time, then a short, dismissible guided walkthrough highlights the core screens (requesting a job, viewing quotes, tracking status, and viewing rewards) before the walkthrough is marked complete and does not reappear on subsequent logins.

**24.** *As a Visitor, I want to submit a contact enquiry through the website, so that I can get in touch with PSG Electrical and Cables/Trite Solar even before creating an account.*

**Priority:** Must have.

**Acceptance criteria:** Given an unauthenticated visitor on the Contact Us page, when they submit a valid name, contact detail and message, then a Contact\_Query record is created with status "New" and becomes immediately visible on the admin's enquiries screen described in Story 19.

Every story above maps to a specific entity or relationship in the HYDRA data model (Customer, Electrician, Admin, Job, Quote, Invoice, Payment, Material, Inspection\_Report, Rewards\_Account, Discount, Job\_Milestone, Job\_Checkin, Contact\_Query, Employee\_Schedule, Timesheet, Leave\_Request, Payroll, Missed\_Call\_Log and AI\_Message\_Log), ensuring full traceability between what the system must let a user do and the underlying database structure designed to support it. Taken as a set, the twenty-four stories above also give a complete, checkable trace against the six user capabilities listed for each role in Section 2.1.1: every capability named for the Visitor, Customer, Electrician and Admin roles has at least one corresponding user story above with an explicit acceptance criterion, and, conversely, no story above introduces a capability that was not already anticipated by the role descriptions — a consistency check that was performed explicitly while finalising this section, in the same spirit as the cross-checking exercise described for the UX Journey Map in Section 2.2.5 below.

## 2.2 UX Journey Map

To illustrate how each of the three main roles progresses within the HYDRA system, a UX journey map was created to display the Customer, Employee, and Admin journeys simultaneously, with dashed cross-links marking points where an action in one role's journey initiates a subsequent step in another role's journey. This type of journey map is a widely recognized UX research and design method, as described by the Nielsen Norman Group as "a representation of the steps a person follows to achieve a goal" (Gibbons, 2018). Unlike a basic wireframe, a journey map highlights the series of screens a user encounters and the emotional or contextual factors that link one stage to the next, which aligns perfectly with HYDRA's structure since it is not a single-user application but an integrated, multi-role system where one role's activity often enables or depends on another's. Such a journey map is also specifically advised as an early deliverable to be developed before detailed interface design starts, primarily because it compels the design team to consider the user's holistic experience of a process, rather than focusing on isolated screens or components that a wireframe-first approach might otherwise produce (Garrett, 2011).

It makes sense to clarify why a three-lane, interconnected format was selected instead of the usual single-persona journey map created for a customer-facing product: HYDRA is not three separate applications that just happen to share a backend, but one unified business process—a task progressing from request, through quoting, assignment, execution and compliance, to billing and payment—seen from three distinct perspectives at the same time. A traditional single-lane customer journey map, like those typically made for

a purely consumer-facing e-commerce or content product, would only document the customer's side of the process and completely leave out the electrician's and administrator's equivalent steps, even though those steps are, in every case, the real drivers that move the customer's own journey forward from one stage to the next. Developing three separate, isolated journey maps instead of one combined version was also examined and dismissed for the same basic reason: three disconnected diagrams would still miss the critical transition points—the exact instant a customer's quote acceptance turns into an admin's job-assignment task, or the exact instant an electrician's QR-code scan updates a customer's "in progress" status—and it is these very transitions that most clearly decide whether the final system will truly feel seamless and responsive to its three different user groups, or end up feeling like three distinct tools awkwardly linked together.

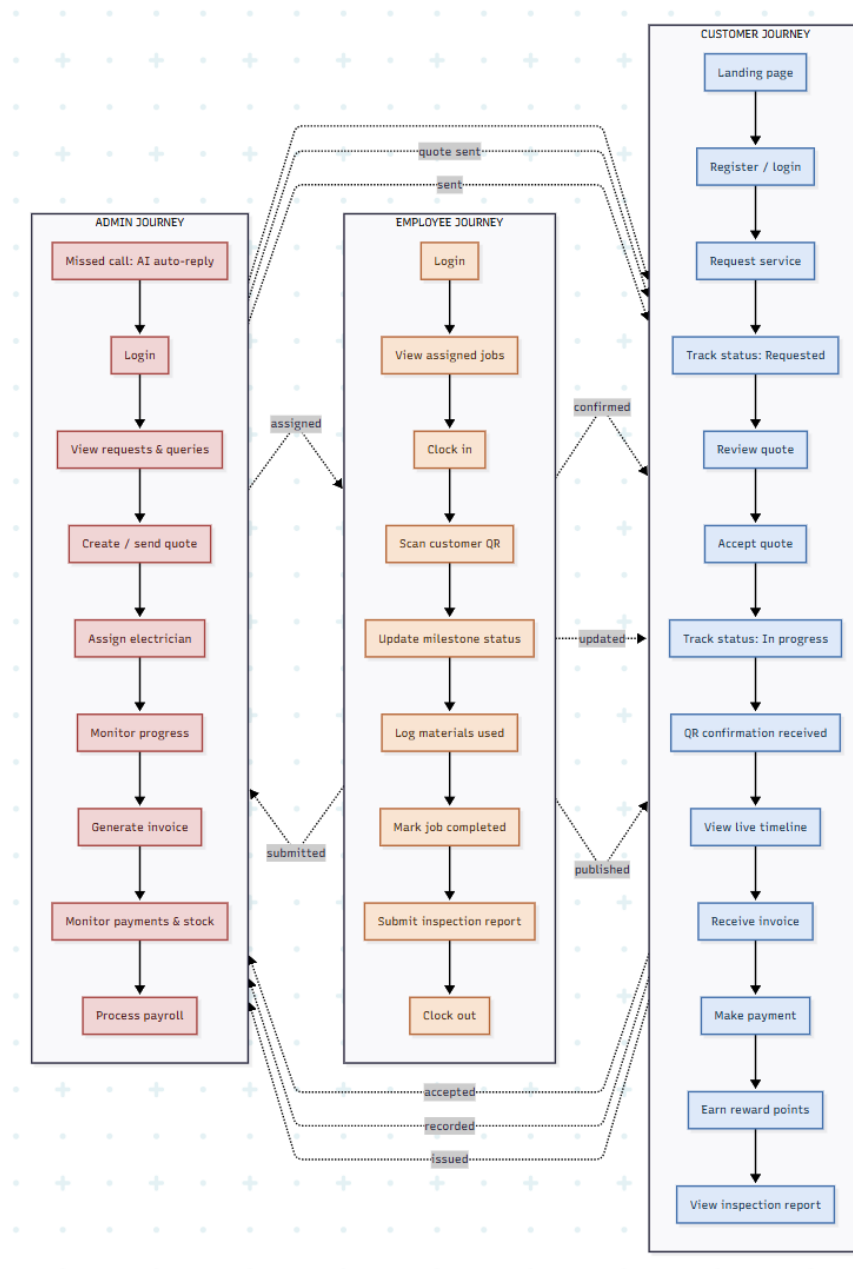


Figure 1: HYDRA UX Journey Map — Customer, Employee and Admin journeys with cross-role triggers

### 2.2.1 Customer Journey

The customer's path starts at the Landing Page, which is the initial display a visitor sees, whether they arrive through the company website or open the app for the very first time. From there, the journey moves to Register/Login, where a new user creates an account (or a returning user logs back in) — this step also includes, as outlined in the project brief, guiding first-time users through a profile setup before a brief introduction to the app's functions. The customer then arrives at Request Service, where they pick the appropriate service type (such as electrical repair, installation, or solar work) and enter their location details, which generates a new job marked as "Requested." This opening sequence of three screens is intentionally kept brief — just three steps from arrival to the first key action of submitting a job — specifically because, as noted in Section 2.1.1, PSG Electrical and Cables/Trite Solar's current customer base is more comfortable calling directly than working through a multi-step digital registration process, and adding even one extra screen between arrival and the first service request increases the risk of the customer leaving the process and dialing the business instead — something that, from the company's point of view, undermines much of HYDRA's intended value, even if the call is later handled manually.

The following screen, Track Job Status: Requested, ensures customers can monitor their request while waiting for Admin to reply. This approach was intentionally chosen to prevent the "black box" issue often found in traditional phone-based bookings, where customers have no way to track progress without contacting the company directly.

Despite its basic appearance—showing only the current status and submission date—this screen resolves a persistent problem highlighted by the client during discussions about their old process. Before the HYDRA system, customers who requested a service call had no way of knowing if their request was received, being processed, or ignored, except by calling the office again. This meant two calls for a single job—a first call to request service and a second just to check the status—wasting staff time on both.

The Track Job Status screen eliminates the need for the second call for any customer who checks the app instead, cutting down on the administrative workload that the Admin role in Section 2.1.1 handles.

Once Admin generates and submits a quote, the customer proceeds to the Review Quote stage and then to Accept Quote; accepting serves as the pivotal event (illustrated by a dashed cross-link to the Admin workflow) that prompts Admin to arrange the job and allocate an electrician. This represents the most critical cross-role connection in the entire journey map, as it marks the point where a mere exchange of information (a proposed price being reviewed) transforms into a definitive agreement that utilizes actual electrician hours and available material inventory—which is exactly why, as outlined in User Story 6 (Section 2.1.2), the Review Quote screen explicitly separates the price into labor and materials components instead of displaying only a total sum, enabling the customer to make that decision with certainty rather than hesitation.

With the job assigned, the customer's display switches to Track Job Status: In Progress, and soon after to QR Confirmation Received — the point when the assigned electrician scans the customer's in-app QR code upon reaching the job location. This one interaction, linked from the Employee workflow, instantly tells the customer the electrician has arrived, adjusts the job status, and launches the live progress timeline on the View Live Timeline screen, where the customer can observe each job stage (for example diagnosis, part installation, testing) shift from scheduled to finished almost immediately as the electrician updates it on-site. The QR-scan confirmation method stands out as a deliberate design choice, since it was not the only method evaluated: a faster-to-build option — an electrician simply tapping "I have arrived" in their own app with no customer involvement at all — would have cut one step from the electrician's process. That simpler method was intentionally set aside in favor of the QR-scan approach because it forces the customer's own device, physically located at the customer's address, to produce the code being scanned, delivering to both parties

a verifiable, time-stamped, location-based record that the right electrician arrived at the right place at a specific moment — a far stronger proof than a single-side confirmation from the electrician alone, and one directly tied to the data-integrity non-functional requirement outlined in Section 2.4.

When the electrician indicates that the job is finished and Admin creates the invoice, the client moves on to Receive Invoice and Make Payment. The moment the payment succeeds, Earn Reward Points is activated, embodying the loyalty/discount framework designed to maintain user interaction with the app between assignments. The process ends at View Inspection Report / Compliance Certificate, providing the client with a lasting, accessible compliance record—a requirement driven by the legal importance of the Certificate of Compliance in South African electrical work (Republic of South Africa, 2009). Assigning the reward-point credit right after payment, instead of as a separate action the client must manually claim, is an intentional design decision meant to strengthen the loyalty system at the precise instant the client is most involved with the app (having just finished a payment), rather than obliging an additional, later app visit solely to claim points—each extra step needed to obtain a reward noticeably lowers the share of clients who will actually take part in a loyalty programme, so merging "pay" and "earn points" into what appears, from the client's viewpoint, as a single seamless action was deemed crucial for the feature's real-world effectiveness rather than just its theoretical presence.

### 2.2.2 Employee (Electrician) Journey

The employee starts their process at Login, then proceeds to View Assigned Jobs, where the electrician checks their daily or weekly schedule. This schedule is filled either by Admin through job assignments or by the electrician themselves for personal events like leave. Before beginning any work, the electrician completes Clock In, which creates a timesheet entry later used by Admin for payroll calculations. Starting the day with a clock-in step, before reviewing job specifics, is a carefully planned decision based on standard labour-tracking methods. Work hours are logged from the moment the electrician begins their shift rather than only when they reach a specific job site. This approach is important because a technician's day also includes travel between jobs, vehicle preparation, and other work-related tasks that should be recorded on the timesheet, even if they don't link to a single Job record.

On reaching the job location, the electrician completes the Scan Customer QR step, which mirrors the customer's earlier "QR Confirmation Received" action. This scan serves two purposes: it verifies the electrician's presence to the customer and activates the job's active-progress status for the technician. Next, the electrician proceeds through the tasks of Update Milestone Status (visible in real time on the customer's timeline display), Log Materials Used (which reduces inventory counts in the Material database and updates project expenses), and Mark Job Completed. This core phase of the process—arrival, status update, material logging, and completion—is structured for efficiency, minimizing typing and screen transitions. This design reflects the harsh, hands-on reality of the role: an electrician often performs these app actions with one hand while standing on a ladder, wearing heavy-duty gloves, or in dim lighting inside a ceiling space or electrical panel, a far cry from the controlled office environment where most digital tools are tested. The unique demands of this role directly shape the usability requirements outlined in Section 2.4.

The last two actions employees must take are Submit Inspection Report and Clock Out. Once published, Submit Inspection Report links to the customer's "View Inspection Report" screen, and Clock Out closes the timesheet entry. Clock Out also links to the Admin process and updates payroll directly. Submit Inspection Report is positioned as the final job-specific task in the electrician's workflow, scheduled after all milestones rather than mixed in with them. This placement reflects that compliance sign-off should capture the electrician's final, thorough assessment of the completed installation—covering any final checks made only

after every other part of the work is done—rather than an ongoing commentary recorded mid-job when the full outcome remains uncertain.

### 2.2.3 Admin Journey

The admin journey differs from the other two by featuring an entry point that doesn't come from a login screen: Missed Call: AI Auto-Reply. This indicates the need for the admin application to track missed business calls and send a simple automated message to the caller, which — linked back to the Customer journey — can serve as the initial touchpoint that directs a new customer to the Landing Page. This intentionally unconventional starting point on the diagram — an automated system event happening without any admin being logged in at that time — is included specifically to highlight a true business reality that a login-first diagram would completely hide: for a considerable portion of PSG Electrical and Cables/Trite Solar's potential new business, the initial interaction the company has with a prospective customer is not a webpage visit or a form submission, but rather a phone call that goes unanswered due to staff being busy elsewhere, and the automated reply function exists specifically to make sure that moment is not just lost.

From Login, the admin navigates through View Requests & Queries (which displays website contact-form submissions and results of missed-call follow-ups), Create/Send Quote, and Assign Electrician — the latter being cross-linked to the Employee journey's "View Assigned Jobs" phase. The administrator subsequently Oversees Progress for all ongoing tasks, Issues Invoice after work is finished (linked to the customer's "Receive Invoice" stage), and Oversees Payments & Inventory, which gets linked signals from both the customer's payment verification and the issuance of reward points. The process ends at Payroll Processing, initiated when electricians log off during the pay period. In contrast to the Customer and Employee journeys, which each adhere to a generally linear flow that relates to a single job's lifecycle, the Admin journey is intentionally illustrated (and should be perceived) as a collection of recurring, parallel monitoring loops instead of a singular start-to-finish route — an administrator does not navigate through View Requests & Queries, Create/Send Quote, Assign Electrician, and Monitor Progress just once daily, but rather cycles through various subsets of these screens repeatedly and in any order dictated by incoming inquiries, ongoing jobs, and payment events, which explains why the Admin lane on the journey map features significantly more cross-links converging into and out of it compared to the Customer or Employee lanes.

### 2.2.4 Design Rationale for the Three-Lane, Cross-Linked Structure

Integrating all three roles onto one canvas instead of creating three separate diagrams was an intentional design decision. Since HYDRA's main value lies in the coordination among three isolated user groups, a journey map displaying just one role in isolation would overlook the crucial interactions that enhance the system's worth — the QR check-in connecting customer and electrician, the quote acceptance event connecting customer and admin, and the clock-out event connecting electrician and admin. The dashed cross-links thus fulfill the same analytical role that Nielsen Norman Group research highlights as vital for effective journey mapping: revealing the handoff points and dependencies that a limited, single-persona perspective would overlook (Gibbons, 2018). Explicitly counting these cross-links throughout the complete diagram provides concrete evidence of this interdependence instead of just an assertion: the map features six unique cross-role trigger points — quote acceptance (Customer → Admin), electrician assignment (Admin → Employee), QR-code arrival confirmation (Employee → Customer, simultaneously updating the Admin's monitoring view), milestone updates (Employee → Customer), inspection-report publication (Employee → Customer), and clock-out (Employee → Admin, informing payroll) — indicating that about twenty-four total steps depicted across all three lanes, a quarter directly rely on, or initiate, an action in another lane, a ratio that reinforces the assertion that HYDRA's value resides in this cross-role coordination rather than in the journey of any single role in isolation.



### 2.2.5 Cross-Checking Against the Design Class and Sequence Diagrams

The UX journey diagram was created subsequent to the domain class diagram and the state diagrams, serving primarily as a validation phase: every screen indicated in the diagram was verified against the design class diagram (Section 3.2) to ensure that a matching method existed to back it up (for instance, the Customer's "Accept quote" step correlates directly with `Customer.acceptQuote(quoteId)`, and "Make payment" connects to `Customer.makePayment(invoiceId, amount)`), while every cross-role link was validated against the sequence diagrams (Section 4.1) to ensure that a corresponding message exchange had indeed been designed to facilitate it. If a screen or step could not be linked to a current class method or sequence message, this suggested either a missing function to be incorporated into the design class diagram or a step that was solely UX-related (a visual confirmation without any state change) that did not need backend support — this verification method serves as a practical, streamlined instance of the larger notion that a system's user experience design and its fundamental technical design should be checked against each other instead of being created separately (Garrett, 2011; Sommerville, 2016). This exercise highlighted a noteworthy example that merits explicit documentation, demonstrating the exercise's inherent value: the customer-facing "View Live Timeline" screen initially lacked a corresponding operation within the design class diagram because it serves as a read-only perspective of existing `Job_Milestone` data instead of an action altering system state — this established that no additional method was necessary for its support (a simple query on pre-existing data sufficed) and did not indicate a fundamental design gap needing resolution, showcasing that this cross-check's objective is as much to verify that a screen is sufficiently supported as it is to identify one that is not.

## 2.3 Work Breakdown Structure (WBS)

The HYDRA Work Breakdown Structure breaks down the entire 133-day project — starting with initial client engagement on 20 May 2026 and concluding with final submission on 20 November 2026 — into six consecutive phases: Planning, Design, Development, Testing, Deployment, and Handover. Aligned with the Project Management Institute's recommendations, the WBS focuses on deliverables instead of just an activity list: each task here signifies a distinct, assignable piece of work that has a defined output, a designated resource, an estimated time frame, and — when relevant — a predecessor task that needs to be finished before it can start (Project Management Institute, 2019). This framework enables monitoring of project advancement in relation to a schedule baseline and highlights any delays promptly compared to the remaining critical path. A deliverable-focused WBS of this nature is clearly differentiated, according to the guidance from the Project Management Institute, from a solely activity-focused breakdown (merely a list of tasks without a defined outcome), based on the idea that a deliverable-focused structure provides each task in the hierarchy with a clear, measurable "done" condition — the WBS is considered complete for a specific task not when a certain duration has elapsed, but when the actual artefact that task guarantees (a completed ERD, a functioning authentication module, an approved UAT session) is present (Project Management Institute, 2019). In the context of HYDRA, this indicates that each of the sixteen distinct tasks outlined in the table below was established from the beginning based on a specific artefact to which this documentation set or the functioning system can be compared — the output for the ERD task is the Entity Relationship Diagram included in Section 4.4, the result of the Design Class Diagram task is the diagram shown in Section 3.2, and so forth — instead of being centered around an undefined period whose conclusion would depend on the team's subjective assessment.

The HYDRA Work Breakdown Structure decomposes the full 133-day project — from initial client engagement on 20 May 2026 through to final submission on 20 November 2026 — into six sequential phases: Planning, Design, Development, Testing, Deployment and Handover. Consistent with Project Management Institute guidance, the WBS is deliverable-oriented rather than simply a list of activities: each task below represents a discrete, assignable unit of work with a clear output, a named resource, an estimated duration, and — where

applicable — a predecessor task that must be complete before it can begin (Project Management Institute, 2019). This structure allows project progress to be tracked against a schedule baseline and makes any slippage immediately visible against the remaining critical path. A deliverable-oriented WBS of this kind is also explicitly distinguished, within the Project Management Institute's own guidance, from a purely activity-oriented breakdown (a simple to-do list of tasks with no defined output), on the basis that a deliverable-oriented structure gives every task in the hierarchy an unambiguous, verifiable "done" condition — the WBS is complete for a given task not when a certain amount of time has been spent on it, but when the specific artefact that task promises (a finalised ERD, a working authentication module, a signed-off UAT session) actually exists (Project Management Institute, 2019). Applied to HYDRA specifically, this means every one of the sixteen individual tasks listed in the table below was defined, from the outset, around a concrete artefact this documentation set or the working system itself can be checked against — the ERD task's output is the Entity Relationship Diagram appearing in Section 4.4, the Design Class Diagram task's output is the diagram appearing in Section 3.2, and so on — rather than around an open-ended block of time whose completion would otherwise be a matter of the team's own subjective judgement.

**Table 3: HYDRA Work Breakdown Structure**

| ID  | Task                                                             | Duration  | Start       | Finish      | Predecessor(s) | Resources Needed            | Team Member(s)               |
|-----|------------------------------------------------------------------|-----------|-------------|-------------|----------------|-----------------------------|------------------------------|
| 0   | HYDRA – PSG Electrical and Trite Solar Digital Management System | 133 days  | 20 May 2026 | 20 Nov 2026 | —              | —                           | All                          |
| 1   | PLANNING                                                         | 14 days   | 20 May 2026 | 08 Jun 2026 | —              | —                           | —                            |
| 1.1 | Client Meetings & Requirements Gathering                         | 2 weeks   | 20 May 2026 | 02 Jun 2026 | —              | Client availability         | Darsh Somayi, Kaedon Naidoo  |
| 1.2 | Work Breakdown Structure & Project Plan                          | 4 days    | 03 Jun 2026 | 08 Jun 2026 | 1.1            | Meeting minutes, MS Project | Kaedon Naidoo                |
| 2   | DESIGN                                                           | 51 days   | 04 Jun 2026 | 13 Aug 2026 | —              | —                           | —                            |
| 2.1 | ERD & Domain Modelling                                           | 5 days    | 04 Jun 2026 | 10 Jun 2026 | 1.1            | Draw.io / dbdiagram.io      | Katelyn Narain               |
| 2.2 | UX Journey Maps & Wireframes                                     | 6.4 weeks | 05 Jun 2026 | 20 Jul 2026 | 1.1            | Figma / Canva               | Yashodha Govender            |
| 2.3 | Design Class Diagram & Design Patterns                           | 5 days    | 10 Jul 2026 | 16 Jul 2026 | 2.1            | UML modelling tool          | Darsh Somayi, Katelyn Narain |
| 2.4 | System Architecture & Cloud Design                               | 10 days   | 31 Jul 2026 | 13 Aug 2026 | 2.1            | Azure documentation         | Yajur Mohan                  |
| 3   | DEVELOPMENT                                                      | 20 days   | 03 Aug 2026 | 28 Aug 2026 | —              | —                           | —                            |
| 3.1 | Database Setup – PostgreSQL                                      | 3 days    | 03 Aug 2026 | 05 Aug 2026 | 2.1            | Azure PostgreSQL access     | Katelyn Narain               |

| ID  | Task                                       | Duration | Start       | Finish      | Predecessor(s) | Resources Needed                      | Team Member(s)                      |
|-----|--------------------------------------------|----------|-------------|-------------|----------------|---------------------------------------|-------------------------------------|
| 3.2 | Backend API – Node.js                      | 2 weeks  | 06 Aug 2026 | 19 Aug 2026 | 2.3, 3.1       | Node.js, GitHub repository            | Darsh Somayi, Yajur Mohan           |
| 3.3 | Frontend – React                           | 2 weeks  | 03 Aug 2026 | 14 Aug 2026 | 2.2            | React, branding assets                | Lonwabo Gumedede, Yashodha Govender |
| 3.4 | Authentication & Role-Based Access Control | 5 days   | 20 Aug 2026 | 26 Aug 2026 | 3.2            | bcrypt, JWT libraries                 | Darsh Somayi                        |
| 4   | TESTING                                    | 10 days  | 27 Aug 2026 | 09 Sep 2026 | —              | —                                     | —                                   |
| 4.1 | Unit and Integration Testing               | 6 days   | 27 Aug 2026 | 03 Sep 2026 | 3.2, 3.3, 3.4  | Jest / testing framework              | Lonwabo Gumedede                    |
| 4.2 | User Acceptance Testing with Client        | 4 days   | 04 Sep 2026 | 09 Sep 2026 | 4.1            | Client availability, test environment | Kaeson Naidoo                       |
| 5   | DEPLOYMENT                                 | 4 days   | 10 Sep 2026 | 15 Sep 2026 | —              | —                                     | —                                   |
| 5.1 | Azure Deployment and CI/CD Pipeline        | 4 days   | 10 Sep 2026 | 15 Sep 2026 | 4.2            | Azure account, GitHub Actions         | Yajur Mohan                         |
| 6   | HANDOVER                                   | 5 days   | 16 Sep 2026 | 22 Sep 2026 | —              | —                                     | —                                   |
| 6.1 | Training, Documentation and Handover       | 5 days   | 16 Sep 2026 | 22 Sep 2026 | 5.1            | User guide, training material         | Kaeson Naidoo, Katelyn Narain       |
| 7   | HYDRA Final Project Submission             | 0 days   | 20 Nov 2026 | 20 Nov 2026 | 6.1            | —                                     | —                                   |

\_(Full Gantt chart with dependency arrows and quarter-by-quarter timeline is provided as a separate artefact, HYDRA\_WBS\_GANTT\_CHART.)\_

### 2.3.1 Phase Rationale and Dependency Logic

Planning (14 days) lays the groundwork for all following stages. Client Meetings & Requirements Gathering (1.1) lacks a predecessor, as it serves as the authentic beginning of the project, and is intentionally assigned to Darsh Somayi and Kaeson Naidoo together — Kaeson, as project lead, guarantees that the meeting is accurately recorded and that action items are monitored, while Darsh's initial participation allows the team's lead developer to understand requirements directly rather than through documentation, minimizing the possibility of misinterpreted technical requirements in the later Development phase. The two-week timeframe designated for this particular task is a conscious decision rather than a standard assumption: since PSG Electrical and Cables/Trite Solar operates as a business with field personnel and an active clientele, planning based on the client's real availability — instead of the team's preferred speed — was seen as the

critical constraint on this task's timeframe, and two weeks was deemed adequate to allow for at least one follow-up meeting if the initial discussion uncovered issues that couldn't be addressed immediately, without excessively postponing the commencement of the Design phase reliant on it. The Work Breakdown Structure (1.2) cannot commence until the requirements gathering is largely finished, thus its reliance on 1.1; its four-day duration signifies that creating the WBS mainly involves synthesis and scheduling once the foundational requirements are established, rather than necessitating additional external contributions.

Design (51 days) is purposely the longest phase, highlighting that HYDRA's complexity mainly stems from aligning three separate applications with a common data model rather than from any individual technically challenging feature. ERD & Domain Modelling (2.1) and UX Journey Maps & Wireframes (2.2) rely solely on the requirements-gathering meeting (1.1) and can consequently proceed simultaneously — this is evident in their shared start dates. The parallelisation was an intentional scheduling choice instead of a random outcome of task sequencing: since the ERD (Katelyn Narain) and the UX journey/wireframe work (Yashodha Govender) rely on the same foundational set of client requirements while generating truly independent artefacts — one outlining the data the system must retain, and the other detailing the screens and flows a user will navigate — there was no technical rationale to make one wait for the other, and executing them simultaneously is what enables the entire fifty-one-day Design phase to be accommodated within a timeline that still provides sufficient time for Development, Testing, Deployment, and Handover prior to the client's operational demands. The Design Class Diagram & Design Patterns (2.3), on the other hand, relies on the ERD (2.1) being completed first, as details at the implementation level (types, methods, PK/FK relationships) cannot be effectively developed until the conceptual entities and their relationships are established — trying to design methods like `Invoice.applyPayment()` (refer to Section 3.2) prior to finalizing the core Invoice and Payment entities and their cardinality in the ERD would have posed the risk of necessitating significant revisions to the design class diagram once the ERD was ultimately finalized, which is exactly the type of rework that a dependency-ordered WBS aims to avoid. System Architecture & Cloud Design (2.4) has the same dependency, as the cloud architecture needs to be structured around the validated data entities and their storage/security needs, and its ten-day duration (the most extended single task in the Design phase following the UX work) mirrors the real unfamiliarity at the project's beginning that some team members experienced with Azure's specific service offerings—time intentionally included in the timeline for the accountable team member, Yajur Mohan, to refer to official Azure documentation and verify architectural decisions against it rather than presuming previous expertise that hadn't yet been shown.

Development (20 days) commences once its design prerequisites are fulfilled. Configuration of the database (3.1) relies on the completed ERD (2.1); Development of the Backend API (3.2) has a dual reliance on the Design Class Diagram (2.3) and the finalized database (3.1), as the API layer utilizes the classes and repositories outlined in 2.3 with respect to the schema established in 3.1. Frontend development (3.3) relies solely on the UX wireframes (2.2) and can thus move forward concurrently with backend tasks — a strategic scheduling choice that shortens the overall timeline by enabling Lonwabo Gumede and Yashodha Govender to create the interface while Darsh Somayi and Yajur Mohan develop the API supporting it. The parallel-track framework in Development reflects the same reasoning previously utilized in the Design phase, and its tangible impact is evident in the dates listed in Table 3: Frontend (3.3) starts on 3 August, coinciding with the start of Database Setup (3.1), and operates simultaneously with the initial portion of the Backend API task (3.2) — without this parallelisation, a strictly sequential development phase (database first, then backend, then frontend, and finally authentication) would have taken approximately thirty-two days instead of the twenty days planned, resulting in a twelve-day difference that the twelve-day interval between UAT completion and Deployment commencement could not have accommodated. Authentication & RBAC (3.4) is planned to be implemented last in Development, as it relies on the already established API (3.2), given that access control is enforced on endpoints that need to be created first; its deliberate placement as the last

Development task, coming just before Testing, serves another purpose — since authentication and role-based access control inherently span every part of the already constructed system, scheduling this task after most of the API's endpoints are in place reduces the need for rework whenever a new endpoint is introduced during 3.2, as access-control rules are applied over a near-complete API structure instead of being added gradually alongside it.

Testing (10-day) commencement period cannot start until all three development streams have converged. This is why Unit and Integration Testing lists 3.2, 3.3 and 3.4 as joint predecessors in its 4.1 version. This triple dependency is not merely a scheduling issue but a direct reflection of what integration testing is actually verifying: that the three independently-built streams function correctly together as a single system. This cannot be meaningfully assessed until each stream exists individually. The streams in question are a backend API, a frontend interface, and an authentication layer wrapping both. User Acceptance Testing is scheduled to take place after internal testing is complete. This reflects good practice in not exposing the client to a build that has not first passed the team's own quality checks. Presenting an unstable or defect-riddled build directly to the client during UAT would risk damaging the client's confidence in the system at a critical point in the project. This is just before the Deployment and Handover phases that follow.

Deployment (4 days) and Handover (5 days) are separate, single-task stages arranged one after the other—deployment can't start until the client formally approves UAT, and training/handover can't begin until the live system matches what the client will actually operate. The four-day Deployment timeframe is intentionally short, recognizing that at this point in the timeline, the Azure setup—covering App Service, PostgreSQL, Blob Storage, Key Vault, and the CI/CD pipeline outlined in Section 7—has already been designed during the Design phase (task 2.4) and largely set up and tested in Development and Testing rather than being built fresh now. Deployment, as defined here, resembles moving a proven configuration into production rather than constructing infrastructure from scratch, and that's why a four-day window is achievable.

The seven-week period separating Handover completion on 22 September 2026 and Final Project Submission on 20 November 2026 serves as a carefully planned buffer. It creates space for finalizing documentation, completing academic reports, and accommodating any last-minute modifications requested by the client after UAT, without endangering the core development and deployment schedule. This buffer was calculated intentionally rather than chosen randomly, designed to handle three specific types of remaining risks that the team deemed realistic post-Handover — small adjustments the client may request only after PSG Electrical and Cables/Trite Solar personnel have actively used the live system for several weeks (beyond the limited UAT session in task 4.2); the extensive effort required to compile documentation and write this report, which is intentionally excluded from WBS tasks competing with development work and instead folded into this buffer; and typical unexpected delays that any project with a fixed deadline should reasonably expect, in line with the project management principle that a schedule lacking any contingency is itself a risk to the project's success (Kerzner, 2017).

### **2.3.2 Resource Distribution Across the Team**

Workload was intentionally divided among all six team members instead of being focused on just one person. When looking at the number of unique tasks assigned, Kaedon Naidoo, Darsh Somayi, and Katelyn Narain each handle four tasks, Yajur Mohan takes on three, and Yashodha Govender and Lonwabo Gumede each manage two—but counting tasks alone does not fully reflect their contributions, since Yashodha's UX Journey Maps & Wireframes task (2.2) alone lasts 6.4 weeks, making it one of the longest tasks in the entire WBS, and Lonwabo's Frontend build (3.3, shared with Yashodha) adds another two weeks to his workload. When considering effort in terms of person-days rather than task count, the workload is evenly spread across the team, meeting the goal that no single person ends up with an unfairly large share of the project.

Beyond this total balance, it is important to observe how each member's task allocations closely align with a clear individual expertise, as this was a planned approach rather than an accidental result: Katelyn Narain's four assignments (ERD & Domain Modelling, Design Class Diagram & Design Patterns, Database Setup, and—together with Kaedon—Training/Handover) create a unified data-and-design progression from initial concept to actual database setup; Darsh Somayi's four tasks (Requirements Gathering, Design Class Diagram, Backend API, and Authentication & RBAC) establish a cohesive backend-and-security workflow; Yajur Mohan's three responsibilities (System Architecture & Cloud Design, Backend API, and Azure Deployment) form a solid infrastructure-and-cloud sequence; and Kaedon Naidoo's four duties (Requirements Gathering, WBS & Project Plan, UAT with Client, and Training/Handover) create a clear project-management-and-client-interaction thread. This connection between task assignment and specialization was implemented as a deliberate way to mitigate risk: giving a task to the member with the strongest relevant prior experience with its tools and methods (Azure documentation to Yajur, UML modelling to Katelyn and Darsh, bcrypt/JWT libraries to Darsh) reduces the estimating uncertainty tied to each task's stated duration, since duration forecasts are inherently more dependable when made by—or for—someone already broadly experienced with the required work compared to assigning it to whoever happens to be free at that point in the timeline.

### 2.3.3 Critical Path and Schedule Risk

Tracing the Predecessors column from the Planning phase to Final Project Submission in the Gantt chart shows one unbroken critical path: Requirements Gathering → WBS & Project Plan → ERD & Domain Modelling → Design Class Diagram → Backend API → Authentication → Unit/Integration Testing → UAT → Azure Deployment → Handover → Final Submission. This means any delay in any of these tasks would push back the entire project's end date, while a delay in something like UX Journey Maps & Wireframes—which has extra schedule flexibility alongside the ERD work—would not necessarily impact the deadline as long as it still finishes before the Design Class Diagram task that relies on it. Pinpointing the critical path is a standard part of project management because it directs the team's focus to where schedule checks are most needed: tasks not on this path have some built-in cushion before they risk the overall 20 November 2026 submission date, but critical-path tasks do not (Kerzner, 2017; Project Management Institute, 2021).

Two particular tasks along this crucial path require extra risk analysis because of their placement and length. Backend API (3.2), lasting two weeks, holds the longest duration of any single task on the critical path and faces two dependencies: Design Class Diagram (2.3) and Database Setup (3.1). Any delay in either of those earlier tasks directly impacts this one, and any delay here leaves no buffer before the start of Authentication (3.4), Testing (4.1), and all following phases, effectively making it the sole point where the entire project timeline is most vulnerable to a single task running over. UX Journey Maps & Wireframes (2.2), though not on the critical path, deserves mention for another reason: at 6.4 weeks, it is the longest task in the entire WBS. While its schedule slack means small delays won't postpone the project overall, that slack is limited—it must finish before Frontend development (3.3) can truly start—and its unusually long duration compared with other Design-phase tasks was seen, during planning, as a true reflection of the scope of the three-lane, cross-linked journey-mapping exercise outlined in Section 2.2, rather than inflated scheduling, and was kept at full length instead of being shortened to match the five-to-ten-day length of similar Design tasks.

## 2.4 Non-Functional Requirements

Non-functional requirements (NFRs) outline the required performance standards for the system rather than defining its actual functions. Since NFRs in student projects are often poorly defined—reduced to broad general checklists without considering the specific needs of the client—the following NFRs include explanations of their importance for PSG Electrical and Cables/Trite Solar, along with details on how each will be implemented and evaluated. The eight key aspects covered correspond closely to the widely accepted



ISO/IEC 25010 software product quality framework, which organizes quality into categories such as performance efficiency, security, usability, reliability, maintainability, and compatibility (International Organization for Standardization, 2011). This ensures the NFRs are grounded in an established industry standard rather than an arbitrary set of criteria. It is important to clarify that ISO/IEC 25010 further breaks down each of these main categories into more detailed sub-categories—for instance, reliability includes maturity, availability, fault tolerance, and recoverability, while security covers confidentiality, integrity, non-repudiation, accountability, and authenticity. Where applicable, the requirements below reference this detailed breakdown instead of treating each category as a single, uniform demand. This approach is most evident in the reliability and availability requirement, which is structured around the standard's own availability and recoverability sub-categories rather than a vague statement like "the system must function properly."

**Performance.** The system needs to deliver quote calculations, job-status updates, and dashboard data without noticeable delays—aiming for response times under two seconds for standard API calls during normal usage. This requirement exists because electricians often access or modify job details while working on-site, frequently using 3G/4G mobile data instead of Wi-Fi; a slow application at a job site would directly weaken the QR check-in and milestone-update functions that are essential to the client's needs. The two-second target does not apply to every interaction in the system: background tasks that don't require immediate responses, like bulk data exports (User Story 21), are excluded from this standard since a report taking several minutes is an acceptable compromise the requesting admin consciously agrees to, whereas a delay of two seconds or more on the QR-scan confirmation screen—an action happening in real time, in front of a customer, at the exact moment an electrician arrives on site—would visibly and immediately damage the professionalism of that moment in ways a slow background export never could. Performance will be tracked through response-time logging in Azure Application Insights and stress-tested before deployment (refer to the CI/CD pipeline outlined in Section 2.3), with extra focus on the payload size of the endpoints supporting the Employee App's primary field-based workflows (View Assigned Jobs, Scan Customer QR, Log Materials Used), which are intentionally built to return only the essential data needed for the current screen rather than large, complex object structures that would take longer to send over a limited mobile connection.

**Security.** Authentication must rely on the industry-standard bcrypt password hashing and JSON Web Tokens (JWT) for handling sessions, while role-based access control is applied at the API level to guarantee that a customer's account cannot view another customer's job information and an electrician's account cannot see payroll or admin-only records. This is especially critical for HYDRA because, according to the client's specifications, the admin application needs to read phone contacts and call logs to enable the missed-call auto-messaging function—a considerably more sensitive data-access scenario than a normal CRUD system and one that amplifies the impact of any authorization error. RBAC, founded on the least-privilege principle, makes certain that users of the three applications receive only the permissions required for their particular role (Pathlock, 2026). The specification intentionally defines both an authentication method (bcrypt hashing, ensuring that even a total database breach would not reveal usable clear-text passwords) and an authorization method (JWT-embedded role claims validated by the two-step role-then-ownership check outlined in Section 2.1.1), because these address two separate failure vectors: a weak or reversible password store could let an attacker pose as a valid user, whereas an inadequate or absent authorization check could permit a properly authenticated individual (a legitimate customer logged into their own account) to retrieve another person's data—both issues must be resolved for the security requirement to be satisfied, and neither alone suffices. The security controls will be verified by the authentication/RBAC unit tests planned for WBS task 3.4 and 4.1, with additional details on input validation and injection prevention presented in Section 6 of this report.

**Usability.** The system must be user-friendly for people with varying technical skills—ranging from office workers to field electricians to occasional app users. This is important because the client’s client base mostly includes homeowners and small business owners who may not be comfortable with digital tools, so a confusing interface could lead them to call the office instead of using the app, defeating the project’s purpose. This will be handled with the guided onboarding process outlined in the project brief and through mobile-friendly design confirmed during UAT (WBS task 4.2). Usability, unlike some other non-functional requirements discussed here, involves distinct design needs for HYDRA’s three roles rather than one uniform approach. For the Customer App, usability focuses on ease of use for infrequent, non-technical users, supported by the onboarding walkthrough in Section 2.2.1. For the Employee App, it emphasizes quick access and minimal effort under field conditions, achieved through the simplified job-execution flow in Section 2.2.2. For the Admin App, it centers on managing complex tasks like handling multiple jobs, queries, and payments efficiently for daily users who will become skilled over time, unlike the more occasional or field-based usage of the other roles. Recognizing these three distinct usability needs, instead of treating usability as a single standard across all apps, is a direct result of the three-lane UX journey mapping exercise in Section 2.2, which clearly showed how different each role’s usage context really is.

**Reliability and Availability.** The system needs to stay accessible during business hours, supported by a documented backup and recovery plan, because any interruption in job-tracking during active work (such as an electrician unable to log in via QR code or an admin unable to assign a new job request) disrupts ongoing work instead of just causing minor inconvenience. This will be accomplished through Azure’s managed PostgreSQL service featuring automated, scheduled backups and the ability to restore data to any point in time, and it will be evaluated against an internal uptime target suitable for a small-business system like this. Compared to the ISO/IEC 25010 sub-characteristics mentioned earlier, this requirement actually covers two separate qualities that are often confused but should be treated distinctly: availability (the percentage of business hours the system remains reachable and functional) and recoverability (the system’s capacity to restore normal functioning without data loss after an outage occurs) (International Organization for Standardization, 2011). HYDRA’s strategy for availability relies on Azure App Service’s built-in managed infrastructure and automatic scaling rather than any custom redundancy the team would have to develop and maintain, which fits well for a project of this size and complexity; its focus on recoverability centers on the point-in-time restore function of the PostgreSQL database, which is more critical for a system like HYDRA than a basic nightly full backup, as it enables the business to revert to a state just before a specific error (for instance, a payroll run processed with incorrect timesheet data) instead of losing all changes since the last backup.

**Scalability.** The system needs to grow alongside increases in customers, jobs, and—since Trite Solar has added solar installation services—new service types, without needing a complete rebuild. This is important because the client is moving beyond just electrical work into solar installation, so the data model and app setup must already allow for extra service types and possible new staff positions instead of being fixed to today’s operations. This is handled in two ways: the `Service_Type` entity is set up as a lookup table rather than a fixed list, and the app layer runs on Azure App Service with autoscale turned on. It’s worth mentioning that this solution works at two levels, not just one: in terms of data, the `Service_Type` lookup table means adding a new work category—like if Trite Solar later included battery-storage installation—only needs a new entry in that table, not a database change or code update anywhere else; and for infrastructure, Azure App Service’s autoscale settings let the app handle more users at once—such as if the business’s customer base jumps after the app gains traction or if marketing for the new solar service drives a sudden spike in visitors to the public site in Section 2.1.1—without needing manual setup changes when the growth happens.

**Maintainability.** The codebase needs to be neat, modular, and thoroughly documented so that PSG Electrical and Cables/Trite Solar—or a later development team—can later expand or adjust it once the HYDRA team’s work concludes. This is essential because, as outlined in the WIL structure of this project, the client relationship includes a fixed handover point (WBS task 6.1); unlike an internal team, HYDRA will not remain indefinitely to troubleshoot or enhance the system. This is achieved through the repository pattern and dependency-inversion design (refer to the Design Patterns diagram) applied uniformly across all entities, along with the training and documentation deliverable included in the Handover phase. This requirement is connected to, yet separate from, the single-admin-application design choice detailed in Section 2.1.1: both stem from the same core limitation—that PSG Electrical and Cables/Trite Solar will not keep an in-house development team post-handover—but while the single-application decision tackles maintainability by reducing the number of codebases the client must eventually manage, the repository-pattern decision focuses on maintainability by ensuring any single codebase is simple to adjust safely when a future developer—potentially with no prior knowledge of HYDRA’s original design—needs to implement changes. The uniform, repeated structure of the repository pattern across every entity, explained further in Section 5.1, means such a developer only has to learn this one pattern to confidently navigate the data-access layer for any entity, instead of having to independently figure out a different convention for each one.

**Compliance.** The system needs to accurately gather and retain the data necessary for South African electrical compliance certification. According to the Electrical Installation Regulations issued under the Occupational Health and Safety Act, every electrical installation must undergo inspection and receive a valid Certificate of Compliance, which must reference the applicable technical standard (SANS 10142-1) used during testing (Republic of South Africa, 2009). This requirement is not optional but a legal obligation tied to the client’s field, and it is explicitly reflected in the `Inspection_Report` entity’s `certificate_no` and `compliance_status` fields. Additionally, since the admin app handles customer and caller contact details for the missed-call messaging function, the system must adhere to the Protection of Personal Information Act, which regulates how South African businesses process personal data (Republic of South Africa, 2013). This will be managed by limiting phone-contact and call-log access strictly to the admin’s work device, securing explicit consent during onboarding, and ensuring the `Contact` and `Missed_Call_Log` entities store only the essential data needed for the messaging feature. This Compliance NFR differs from the other eight requirements in this section because it stems from mandatory external laws rather than an internal quality goal set by the project team. This difference significantly affects how the requirement is handled in the rest of the report: while falling short of a target like Performance, for example, could be a reasonable compromise under time constraints, missing the Compliance requirement—such as an inspection report lacking a valid certificate number or a missed-call feature storing more caller data than POPIA’s data-minimisation principle allows—would expose PSG Electrical and Cables/Trite Solar to real legal and regulatory consequences. Therefore, across the WBS, ERD, and Security section of this report (Section 6), it is treated as an absolute requirement rather than a flexible objective.

**Data Integrity.** Records for quotes, invoices, payments, and materials used must always stay consistent within the system—any invoice’s total must link back to its original quote and any discounts applied, and each payment must connect to a specific invoice. This is important because these records hold direct financial significance and, for inspection reports, legal importance; an inconsistency among a quote, invoice, and payment isn’t just a minor error but a potential billing conflict. The system enforces this through structured foreign-key connections within the database schema (as shown in the Domain and Design Class Diagrams) and through calculation methods assigned to the responsible entity (for example, `calculateTotal()` on `Quote` or `calculateNetPay()` on `Payroll`), instead of allowing totals to be calculated inconsistently across different parts of the application. The design principle behind this rule—where a calculation should occur in a single, defined location in the code, on the entity that controls the relevant data, rather than being recalculated

independently wherever it's needed—mirrors the principle explained in more detail in Section 3.2 and Section 5.2 regarding method placement in the design class diagram and the separation of concerns in the layered architecture; the Data Integrity Non-Functional Requirement described here essentially serves as the business justification for a technical design choice outlined elsewhere in this document, and both should be viewed as directly related rather than as separate decisions that happen to align.

### 3. Analysis Artifacts

#### 3.1 Domain Class Diagram

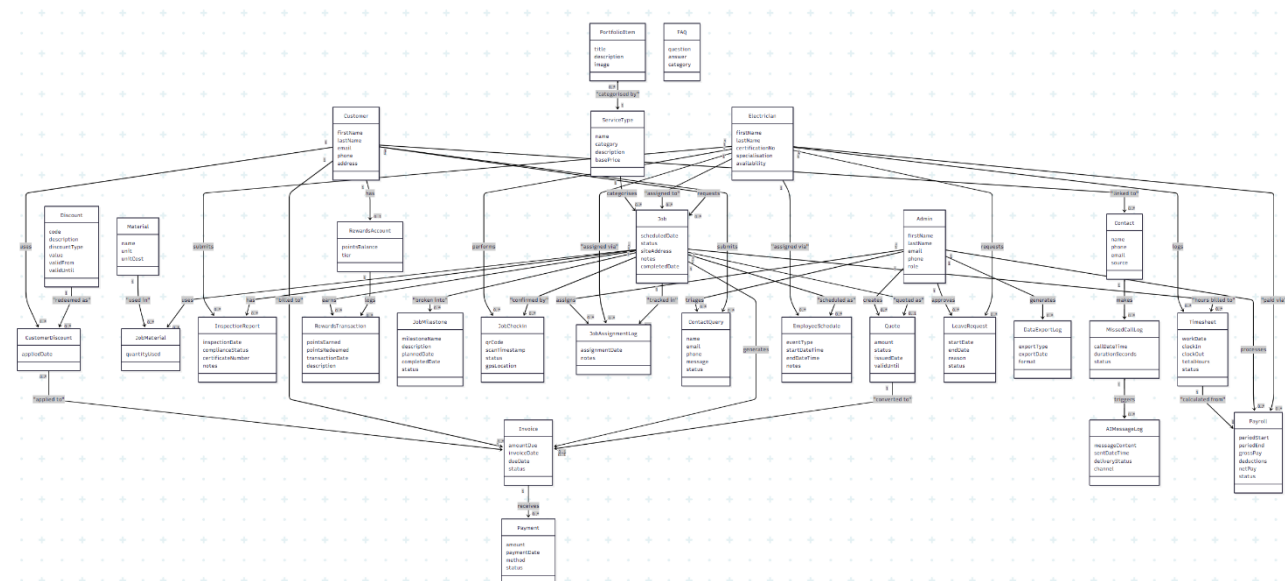


Figure 2: HYDRA Domain Class Diagram

#### Purpose of the diagram

The domain class diagram represents the real-world business principles that the HYDRA system is based on, regardless of implementation details such as database types, method signatures, or access modifiers (Sommerville 2016). Before that image is converted into a technical design, it serves to provide the client, PSG Electrical and Cables / Trite Solar, and the development team with a shared, non-technical picture of the data the company already works with, including customers, jobs, quotes, invoices, electricians, and so forth (Pressman and Maxim, 2020; Larman, 2012). This arrangement is important from both a pedagogical and practical standpoint: object-oriented analysis and design methodology views the domain model as a link between a client's purely verbal requirements discussion and a formal, implementable structure, and moving directly to implementation-level classes runs the risk of introducing unintentional technical limitations that do not accurately represent the client's own business (Larman, 2012). Every class in the diagram represents an idea that the company understands on its own; each association, which uses UML multiplicity notation to indicate the number of instances of one class that can relate to another, indicates a true relationship between those concepts (Object Management Group, 2017). The HYDRA domain model is purposefully designed around four access surfaces — the public website, the Customer App, the Employee App, and the Admin App — but a single, shared domain, because a customer's job, an electrician's timesheet, and an administrator's invoice are all facets of the same underlying business process rather than separate systems. The Access Control ERD in Section 4.4 is made possible by the "one domain, many views" approach; if every app had its own private data model, there would be nothing for that diagram to reconcile.

Each class depicted in the diagram is described in the subsections that follow, along with an explanation of how the classes relate to one another, a fully worked example that traces a single, realistic job thru every stage of the model, and a discussion of the deliberate modelling decisions made along the way, including normalisation, association classes, cardinality choices, and the alternative structures that were considered and rejected.

## Methodology: how the domain model was produced and validated

In accordance with accepted practice for obtaining a domain model directly from client conversation rather than from documentation alone, the domain class diagram was not created in a single sitting by a single team member working alone; rather, it was constructed iteratively and validated against the client at two different points in the project timeline (Larman, 2012).

The first draft, which covered only the five entities Customer, Job, Electrician, Quote, and Invoice, was created internally by Katelyn Narain and reviewed by the larger team at the internal meeting of May 11, 2026 (meeting 5, Section 1.4). This was done thru an informal CRC-card-style exercise in which each candidate class's name, core responsibilities (what it needs to know or do), and collaborators (which other classes it needs to speak with) on a separate card and physically verified that each responsibility identified during the initial client meeting on April 18, 2026, could be linked to a single card.

This CRC approach was specifically chosen because it compels the modeller to ask, "whose job is it to know this?" for each piece of information the client mentions. This question reveals hidden classes (like the eventual need for a Contact entity, distinct from Customer) far more reliably than working solely from a narrative requirements description (Larman, 2012).

The three-role login structure (Customer, Employee, and Admin) and the entire downstream feature set—rewards and discounts, QR-based check-in, and missed-call AI messaging—were then formally confirmed with the client on May 16, 2026 (meeting 6, Section 1.4). This led to the addition of RewardsAccount, RewardsTransaction, Discount, CustomerDiscount, JobCheckin, Contact, MissedCallLog, and AIMessageLog to the model. On June 8, 2026 (meeting 10), the model was expanded once again to include EmployeeSchedule and Timesheet. On July 7, 2026 (meeting 15), the entire schema was examined for normalisation and foreign-key integrity prior to deployment.

Therefore, rather than being over-engineered with speculative entities the client never actually asked for, the domain model presented in this section is the fourth and final iteration of a model that was purposefully allowed to grow only as new, client-confirmed requirements demanded it. This discipline is consistent with the general principle that a domain model should represent what the business genuinely needs captured, not every concept a modeller can imagine might one day be useful (Larman, 2012; Sommerville, 2016).

## A worked example: tracing one job through the domain model

Because a class diagram alone is a static picture, it is useful to walk through a single, concrete, realistic scenario end to end, showing which class instances are created and which relationships are exercised at each step, grounded in the kind of job PSG Electrical and Cables / Trite Solar actually performs.

Consider a residential customer — call her Thandeka Mkhize, of Ballito, KwaZulu-Natal — who has noticed her home's distribution board tripping intermittently and, separately, is interested in a solar panel installation quote. She first appears in the system as a Customer record (firstName: "Thandeka", lastName: "Mkhize", email, phone, address: "12 Dolphin Crescent, Ballito"), created when she registers through the Customer App. Because registration automatically provisions a loyalty account (per the has relationship described below), a RewardsAccount record is created for her at the same moment, with a starting pointsBalance of zero and tier of the entry-level tier.

Thandeka requests a job through the app, selecting the ServiceType "Fault Finding" from the categories the business has configured (alongside "Solar Panel Installation", "Compliance Certificate", and others). This creates a Job record linked to her customerId, the chosen serviceTypeId, and her supplied siteAddress, with status initially set to "Requested" (matching the Job Lifecycle state diagram's initial state, Section 4.1) and no



electricianId yet assigned. An administrator reviews the request and, using the `Admin.createQuote()` operation described in Section 3.2, produces a Quote record referencing this Job, with an amount based on the ServiceType's basePrice adjusted for the specifics of the callout, and a status of "sent". Thandeka reviews and accepts the quote through the Customer App (`Quote.accept()`), which triggers `Quote.convertToInvoice()` is not yet called at this point — that only happens once the work is actually done — but the act of acceptance does move the Job forward, and the admin now assigns an electrician.

Suppose Sipho Zulu, an Electrician with a specialisation of "General Electrical" and a valid certificationNo, is assigned via `Admin.assignJob()`, populating the Job's electricianId and creating a JobAssignmentLog record capturing the assignment date and the assigning admin. This assignment is also reflected as an EmployeeSchedule entry against Sipho's calendar, so that the conflict-detection logic described in the Employee Calendar and Scheduling sequence diagram (Section 4.1) can prevent him being double-booked. On the day of the job, Sipho arrives at 12 Dolphin Crescent and scans Thandeka's QR code from within the Employee App; this creates a JobCheckin record (qrCode, scanTimestamp, status: "confirmed", gpsLocation), which is what moves the Job's status to "InProgress" in the state diagram. While working, Sipho draws two units of circuit breaker and one reel of twin-and-earth cable from stock; each of these is recorded as a JobMaterial row (one row per distinct Material used, each carrying its own quantityUsed and costAtTime, so that the cost the business actually incurred on this specific job — as opposed to the current, possibly different, list price of that Material — is preserved permanently).

On completion, Sipho submits an InspectionReport (inspectionDate, complianceStatus: "Compliant", certificateNumber, and an attached certificate file reference), which is what allows the Job's status to move to its terminal "Closed" state per the compliance gate discussed in Section 4.1. An admin then triggers `Job.generateInvoice()`, converting the accepted Quote into an Invoice record referencing the Job, the Quote, and Thandeka's customerId, with an amountDue calculated from the quote amount plus the actual JobMaterial costs. Thandeka pays the invoice in full through the Customer App (`Customer.makePayment()`), creating a Payment record against the Invoice and moving the Invoice's status to "Paid" via `Invoice.applyPayment()` and `Invoice.markPaid()`. Because the job is now paid and closed, a RewardsTransaction record is created against her RewardsAccount, adding points via `RewardsAccount.addPoints()`, and her running pointsBalance is updated accordingly. Finally, Sipho's hours for the day are captured on a Timesheet record (clockIn, clockOut, totalHours), which is later read, alongside every other Timesheet entry for the pay period, when an admin runs `Admin.processPayroll()` to generate his Payroll record for that period.

This single scenario exercises eighteen of the domain model's classes and more than a dozen of its relationships, and demonstrates concretely why the model needs to be as richly interconnected as it is: a single, ordinary job — nothing exceptional about it, no cancellations, no disputes, no partial payments — still touches identity, scheduling, compliance, stock, billing and loyalty simultaneously, and every one of those concerns needed its own class precisely because none of them can be safely collapsed into another without losing information the business genuinely needs to keep.

## Class descriptions

Each of the classes listed below is explained in four sections: what it stands for in the business; its complete list of attributes, each of which has a purpose; why the class is required as a separate concept rather than being combined with another; and, when applicable, a note explaining how it was specifically scoped to PSG Electrical and Cables/Trite Solar's actual operation rather than using a generic template.

**Customer** — Represents a person or household that has registered to request electrical or solar services from PSG Electrical and Cables / Trite Solar. Its attributes are `firstName` and `lastName` (kept as two separate fields rather than one combined name field, so that the system can address a customer correctly in automated messages — "Hi Thandeka" rather than "Hi Thandeka Mkhize" — and so that sorting and searching by surname is possible for administrative reporting), `email` and `phone` (the two channels through which the system communicates status updates, invoices and reward notifications), and `address` (the customer's primary residential or billing address, distinct from any individual job's `siteAddress`, since a customer may request work at a property other than their own registered address — for example, booking a job for an elderly relative). It is the anchor for everything a customer does in the system: submitting queries, requesting jobs, earning rewards and receiving invoices. It is needed because the business must be able to identify who a job, quote or payment belongs to, and because a persistent customer identity is what allows the rewards programme (see `RewardsAccount`, below) to accumulate value across multiple, separate visits rather than resetting with every transaction — a pattern well established in customer relationship management literature as central to converting one-off clients into repeat business (Elmasri and Navathe, 2016). For a business such as PSG Electrical and Cables/Trite Solar, where a single solar installation customer is highly likely to also need future maintenance, compliance re-certification, or referral-driven repeat electrical work, this persistent identity is not a "nice to have" but a direct enabler of the client's own stated goal of using the mobile app as an ongoing retention tool rather than a one-off booking form.

**Electrician** — Represents a field technician employed by the business who carries out jobs. Its attributes are `firstName` and `lastName`, `certificationNo` (the electrician's registration/certification number, required because South African electrical work legally requires a qualified, certified person to sign off compliance work — see `InspectionReport` below), `specialisation` (distinguishing, for example, a technician qualified for general electrical fault-finding from one qualified specifically for solar photovoltaic installation, which matters directly for PSG Electrical and Cables/Trite Solar's dual electrical-and-solar service offering), and `availability` (a simple indicator used alongside `EmployeeSchedule` to support assignment decisions). It is needed to support job assignment, scheduling, compliance record-keeping and payroll. Storing the certification number directly on the `Electrician` entity, rather than in a separate compliance table, was a deliberate simplification appropriate to the scale of PSG Electrical and Cables / Trite Solar's current workforce; a larger organisation with electricians holding multiple, expiring certifications (for example, separate registrations for electrical and solar work) might normalise this further into its own `Certification` entity with its own expiry-tracking logic, but doing so here would have added complexity without a corresponding business benefit given the size of the client's current team (Silberschatz, Korth and Sudarshan, 2020).

**Admin** — Represents office staff who manage the day-to-day operation of the business: triaging queries, creating quotes, assigning electricians, approving leave and processing payroll. Its attributes are `firstName`, `lastName`, `email`, `phone` and `role` (differentiating, per the discussion in Section 2.1, a business-owner-level Admin from a more limited office-staff Admin, without requiring an entirely separate class or application). It is needed because several workflows in HYDRA (quoting, assignment, payroll, data export) are only ever initiated by an administrator, not by a customer or electrician, and because the system's role-based access model (discussed further in Section 4.4's Access Control ERD and Section 6.1's authentication/authorisation discussion) depends on being able to distinguish an administrator's identity from a customer's or electrician's at the data level, not only at the application level.

**ServiceType** — Represents a category of work the business offers (for example, "Fault Finding", "Solar Panel Installation", "Compliance Certificate", "Distribution Board Upgrade"), together with a `name`, `category` (broadly, "Electrical" or "Solar", reflecting the two trading identities of PSG Electrical and Cables and Trite Solar respectively), `description` and `basePrice`. It is needed so that jobs, portfolio items and quotes can all be

classified consistently and so that pricing has a starting reference point that an admin can adjust from when preparing a job-specific quote, rather than pricing every job from scratch. Modelling `ServiceType` as its own configurable entity, rather than hard-coding a fixed list of service categories into the application, was a deliberate choice so that the client can add new service categories themselves (for example, if Trite Solar later introduces battery storage installation as a new offering) without requiring a code change or a new release from the development team — directly supporting the change-management and long-term maintainability goals discussed in Sections 2.4 and 9.

`PortfolioItem` — Represents a completed piece of work the business wants to showcase publicly (on the website's portfolio page), with a title, description, image, a `serviceTypeId` foreign key (so portfolio items can be filtered or browsed by category — a prospective solar customer can view only solar-related portfolio items), a `completedDate` and a `featured` boolean flag allowing the business to highlight its best or most recent work on the website's home page. It is needed to support marketing and to give prospective customers evidence of the quality and type of work PSG Electrical and Cables / Trite Solar performs — directly serving the "public, no login" access tier established in the Access Control ERD (Section 4.4), since portfolio content must be visible to an anonymous website visitor who has not registered and may never register.

`FAQ` — Represents a single frequently-asked question and its answer, shown on the public website, with a question, answer, category (for example, grouping questions under "Pricing", "Compliance", "Scheduling") and a `displayOrder` allowing the business to control which questions appear first. It is needed to reduce the volume of routine queries reaching the Admin App, freeing administrator time for the higher-value triage and quoting work described under the Admin class above — a direct, practical response to the operational bottleneck identified in Section 1.2, where the client's current manual process has no equivalent self-service mechanism at all.

`Job` — The central class of the whole domain. It represents a single unit of electrical or solar work requested by a customer, from first request through to completion. Its attributes are `scheduledDate` (when the work is planned to occur), `status` (tracking the job through the lifecycle described in the Job Lifecycle state diagram, Section 4.1), `siteAddress` (the physical location the work will be performed, which — as noted under Customer above — may differ from the customer's own registered address), `notes` (free-text detail supplied at the time of request, capturing the nature of the problem or request in the customer's own words) and `completedDate`. Almost every other class in the system — quotes, invoices, milestones, check-ins, materials, assignment logs — exists to support or record something that happens to a Job. It is needed because it is the object around which the whole business process (and the whole HYDRA system) is organised; in entity-relationship terms, Job functions as the system's principal "fact" entity, with almost every other table in the schema either describing it, categorising it, or recording an event that happened to it (Connolly and Begg, 2015).

`Quote` — Represents a priced proposal sent to a customer for a specific job, with an amount, a status (for example, sent, accepted, declined — matching the `QuoteReceived/JobBooked/QuoteDeclined` states shown in the Customer Job Booking state diagram, Section 4.1), an `issuedDate` and a `validUntil` expiry date. It is needed because the business does not simply perform work on request; it must first cost the work and obtain the customer's agreement, and that agreement needs to be recorded before an invoice can be raised. Modelling Quote as a distinct, persistent class — rather than as a transient calculation performed and discarded — also gives the business a defensible audit trail of what was actually agreed with a customer, which matters both for dispute resolution (a customer querying why an invoice amount differs from what they believe they were quoted) and for the kind of consumer-protection accountability increasingly expected of service businesses operating in South Africa (Republic of South Africa, 2013). The `validUntil` expiry date specifically reflects a real commercial concern raised during the requirements-gathering process: material

costs for solar components in particular can shift materially over a period of weeks, and an unexpiring quote would expose the business to being held to a price it can no longer honour.

**Invoice** — Represents the bill raised against a completed (or quoted) job, with an `amountDue`, `invoiceDate`, `dueDate` and `status`. It is needed to formalise what the customer owes and to drive the payment and collections process, and it deliberately carries its own `dueDate` independent of the Job's own dates, since the payment terms clock and the work-completion clock are conceptually different things that can, in practice, diverge (for example, a job may be completed on one date but invoiced several days later once materials costs from `JobMaterial` are finalised).

**Payment** — Represents a single payment made by a customer against an invoice, with an `amount`, `paymentDate`, `method` (for example, EFT, card, or another mechanism supported by the payment gateway discussed in Section 6.5) and `status`. It is a separate class from `Invoice`, rather than a single "paid" flag on the invoice, because the business allows partial payments — an invoice can receive more than one payment over time before it is fully settled, directly reflecting a common real-world scenario for higher-value solar installation jobs, where a customer may pay a deposit up front and the balance on completion. This decision is not cosmetic: it is a direct consequence of applying "first normal form" thinking to the billing domain, since a single invoice-level flag cannot represent a repeating, multi-valued fact (multiple payments, each with its own date, amount and method) without either losing information or resorting to an unmanageable set of extra "payment 2", "payment 3" columns (Silberschatz, Korth and Sudarshan, 2020).

**Material** — Represents a stock item (cabling, circuit breakers, distribution boards, solar panels, inverters, mounting hardware, and so on) that the business purchases and uses on jobs, with a `name`, `unit of measure` (for example, "metre" for cable, "unit" for a breaker or panel) and `unitCost`. It is needed to support job costing and stock control, giving the business visibility, at a per-job level, into what it actually costs to deliver each unit of work, rather than only tracking labour and quoted price — a distinction that matters considerably more for Trite Solar's higher-material-cost solar installations than it might for a business dealing purely in low-cost electrical consumables.

**JobMaterial** — A junction/association class that resolves the many-to-many relationship between `Job` and `Material`: any job can use many materials, and any material can be used on many jobs. It records the `quantityUsed` of a specific material on a specific job, and, at the design level (Section 3.2), the `costAtTime` that material was charged at when it was actually used — deliberately distinct from `Material`'s own current `unitCost`, since the price of solar components in particular is prone to change over time, and a job completed six months ago should always show what it actually cost the business at the time, not be silently recalculated against today's price list. It is needed because the relationship itself (which materials, and how many, were used on which job) carries information that cannot be stored on either `Job` or `Material` alone. This is a textbook application of the standard relational technique for resolving many-to-many relationships through an intersection entity (Elmasri and Navathe, 2016; Connolly and Begg, 2015).

**JobMilestone** — Represents one step in a larger job's progress (for example, "Site survey completed", "Wiring installed", "Panels mounted", "System commissioned" for a solar job), with a `milestoneName`, `description`, `plannedDate`, `completedDate` and `status`. It is needed for jobs that are broken into stages — which, in practice, describes almost every Trite Solar installation job, as distinct from a short electrical callout — so that customers and administrators can track progress rather than seeing only a single "in progress" state, directly supporting the "View live timeline" step of the Customer Journey shown in the UX Journey Map (Section 2.2).

**JobCheckin** — Represents the moment an electrician arrives on site and scans the customer's QR code to confirm arrival, capturing a `qrCode` value, a `scanTimestamp`, a `status` and a `gpsLocation`. It is needed to give

the business and the customer verifiable, timestamped proof that work started when and where it was supposed to, addressing a well-documented trust gap in home-services businesses where the customer has no independent way of confirming a technician's arrival time (Rozanski and Woods, 2012, discuss this general class of "evidence of service" requirement in distributed systems design terms). This feature was explicitly requested and walked through with the client at the meeting of 19 June 2026 (meeting 12, Section 1.4), where the client specifically valued its role in reducing "no-show" disputes.

**InspectionReport** — Represents the compliance/safety inspection an electrician submits after finishing a job, with an `inspectionDate`, `complianceStatus`, `certificateNumber`, notes and an attached file reference (`fileUrl`). It is needed because South African electrical work is subject to compliance certification requirements under the Electrical Installation Regulations (Republic of South Africa, 2009), and the business must retain evidence of this for both the customer and any regulatory audit — a record-keeping obligation that maps directly onto the Job Lifecycle state diagram's rule (Section 4.1) that a job cannot reach its terminal Closed state until an inspection has passed. This is arguably the single most legally significant class in the entire domain model, since it is the class that stands between PSG Electrical and Cables and non-compliance with statutory electrical safety regulation.

**RewardsAccount** — Represents a customer's loyalty account, holding a `pointsBalance` and a tier. It is needed to support the rewards and discounts programme that is intended to encourage repeat use of the Customer App, and its one-to-one relationship with Customer (rather than points being stored directly on the Customer entity) keeps loyalty-programme concerns cleanly separated from core identity data, so that the rewards feature could, in principle, be disabled or redesigned without touching the Customer entity at all.

**RewardsTransaction** — Represents a single event in which a customer earns or redeems loyalty points, with `pointsEarned`, `pointsRedeemed`, a `transactionDate` and a description. It is needed to give an auditable history behind the single running balance held on **RewardsAccount**, following the same "never store only a derived total" principle discussed under Payment above — if a customer ever queries why their points balance is what it is, **RewardsTransaction** gives the business a complete, chronological answer rather than an unexplainable single number.

**Discount** — Represents a promotional offer (a code, description, `discountType`, value, and a `validFrom/validUntil` validity window) that can be applied to a customer's invoice. It is needed to support the marketing/rewards side of the business independently of the points-based rewards mechanism, giving PSG Electrical and Cables / Trite Solar two distinct promotional levers — points-based loyalty and code-based discounting — that can be used together or separately depending on the marketing campaign in question (for example, a "Refer a Friend" discount code running alongside, but independently of, the ongoing points programme).

**CustomerDiscount** — A junction/association class recording that a specific discount was redeemed by a specific customer against a specific invoice, with the `appliedDate`. It exists for the same reason as **JobMaterial**: the redemption event itself needs to be recorded, not just the discount and the customer in isolation, and it is this class that ultimately prevents the same discount code being applied to the same invoice twice by accident.

**ContactQuery** — Represents an enquiry submitted through the public website's "Contact Us" form, with the visitor's name, email, phone, message and a status. It is needed to capture leads and general enquiries from people who are not yet, or may never become, registered customers — reflecting the reality that a public-facing website must be able to accept input from anonymous visitors, a class of user the rest of the domain model otherwise has no representation for (see also the UX Journey Map discussion in Section 2.2, where a website visitor's journey is shown as distinct from a registered customer's).

**Contact** — Represents a person or number in the Admin App's own permitted contact list, independent of whether that person is a registered Customer, with a name, phone, email and source. It is needed to support the missed-call handling feature, because a missed call may come from someone who is not yet in the Customer table at all, and the system must still be able to identify, log and respond to that contact attempt.

**MissedCallLog** — Represents a single missed call captured against a Contact, with the `callDateTime`, `durationSeconds` and `status`. It is needed to trigger the automated response feature and to give the administrator a record of contact attempts that were not answered in time, forming the evidentiary basis for the AI Missed Call state and sequence diagrams described in Section 4.1. This feature's data-privacy implications were raised proactively by the team itself, not only by the client, at the meeting of 13 July 2026 (meeting 16, Section 1.4), where the requirement for POPIA-conscious consent and data minimisation was first formally logged — directly shaping the "permitted contact information" scoping discussed under Contact above and the full compliance treatment in Section 6.4.

**AIMessageLog** — Represents an automated message generated and sent in response to a missed call, with the `messageContent`, `sentDateTime`, `deliveryStatus` and channel used (SMS, WhatsApp, etc.). It is needed to give a full, auditable trail of what was communicated to a prospective or existing customer without a staff member's direct involvement — an important safeguard given that any automated, unsupervised customer communication carries a reputational and, potentially, a consumer-protection compliance risk if it cannot later be reviewed (Republic of South Africa, 2013; OWASP Foundation, 2021, on logging and accountability as a security control).

**EmployeeSchedule** — Represents a calendar entry for an electrician — a scheduled job, a break, or another event type — with an `eventType`, `startDateTime`, `endDateTime` and notes. It is needed to support the employee calendar feature and to detect scheduling conflicts before a job is booked, which is the data underpinning the Employee Calendar and Scheduling state and sequence diagrams described in Section 4.1.

**Timesheet** — Represents an electrician's worked hours for a given day, with a `workDate`, `clockIn` time, `clockOut` time, `totalHours` and a `status`. It is needed as the source data for payroll calculation, and separating Timesheet from Payroll means an electrician's daily hours can be reviewed, corrected or queried before payroll is actually run, rather than the two processes being fused into a single, harder-to-audit step.

**JobAssignmentLog** — Represents the event of an admin assigning (or re-assigning) a job to a specific electrician, with an `assignmentDate` and notes. It is needed to keep a history of assignment decisions, particularly where a job is reassigned after an electrician is unavailable — directly reflecting the "ReassignmentRequired" loop shown in the Admin Job Management state diagram in Section 4.1.

**LeaveRequest** — Represents an electrician's request for time off, with a `startDate`, `endDate`, `reason` and `status`. It is needed to support the calendar/scheduling process and to give admin a formal approval step before an electrician is marked unavailable, ensuring that leave, unlike an ad hoc note in a schedule, goes through a consistent, auditable approval workflow.

**Payroll** — Represents a single payroll run for one electrician over a pay period, with a `periodStart` and `periodEnd`, `grossPay`, `deductions`, `netPay` and `status`. It is needed to convert timesheet data into an actual payment obligation to staff, and its `status` field allows the system to distinguish a payroll run that has been calculated from one that has actually been processed/paid, which matters for financial reconciliation.

**DataExportLog** — Represents a record of the admin exporting data out of the system (for example, for reporting or accounting), with an `exportType`, `exportDate` and `format`. It is needed for accountability and audit purposes over who extracted what data and when, which is a direct, practical expression of the accountability principle found in South African data-protection legislation (Republic of South Africa, 2013)



and in general information-security good practice around logging administrative actions on personal data (OWASP Foundation, 2021).

## Relationships and multiplicities

This paragraph contains each unique relationship depicted in the diagram, along with its exact UML multiplicity and a remark explaining why that particular cardinality—rather than an apparently simpler alternative—was chosen. The summary groupings below provide the general structure of the model. "1" denotes exactly one; "0..1" denotes zero or one (optional, single); "1.." denotes one or more; and "0.." denotes zero or more (optional, many) when reading UML multiplicity notation (Object Management Group, 2017). customer-focused partnerships. In order to ensure that no code path in the system ever has to deal with a customer who does not have a rewards account, a customer has exactly one RewardsAccount (1-to-1). Since every registered customer is automatically enrolled, this is a required rather than optional one-to-one relationship. A client may register without ever providing a query, or they may submit multiple queries over time, as seen by the zero-to-many ContactQuery records (1-to-0..). A customer requests zero-to-many job records (1-to-0..) and redeems zero-to-many discount offers (many-to-many, resolved by CustomerDiscount, since a single discount code, for example, a general "Winter Special" — can plausibly be redeemed by many different customers, while a single customer can redeem many different discount codes over time). This is consistent with a normalised, historically-complete relational design rather than a system that merely tracks a customer's "current" state (Connolly and Begg, 2015). This is similar to the checklist's expectation that a customer can accumulate a full history of enquiries, jobs, and reward activity over time rather than being restricted to a single transaction.

**Job-centered connections** As the primary fact entity of the system, Job occupies the center of the model and is purposefully the class with the greatest number of distinct relationships in the diagram.

Zero-to-many Job records are categorised by a ServiceType (1-to-0..: a service type may exist in the catalogue before any job of that type has ever been requested); an electrician is assigned to zero-to-many Job records over time (0..1-to-0.., optional on the Job side specifically because a newly requested Job has no electrician assigned yet—this optionality is what allows the "Requested" state in the Job Lifecycle diagram, Section 4.1; a Job is divided into zero-to-many JobMilestone records (1-to-0..1, indicating that a multi-day solar installation may have multiple milestones); a Job is verified by zero-or-one JobCheckin occurs only once, when the electrician physically arrives; A Job uses zero-to-many Material records via JobMaterial (many-to-many, resolved by JobMaterial); a Job is billed to zero-or-one InspectionReport (1-to-0..1, optional in that an InspectionReport is only created once compliance work is actually completed); a Job is eventually quoted as zero-or-one Quote (1-to-0..1), which is later converted to zero-or-one Invoice (1-to-0..1). The core of the billing workflow is this chain: Job → Quote → Invoice → Payment. It is covered in further detail in Section 4.4 of the ERD, including its complete cardinality treatment.

**Electrician-centered relationships.** An electrician submits zero-to-many InspectionReport records (1-to-0.., only for jobs requiring compliance sign-off), completes zero-to-many JobCheckin records (1-to-0.., one per job actually attended), is scheduled as zero-to-many EmployeeSchedule entries (1-to-0..), is tracked in zero-to-many Timesheet entries (1-to-0.., one per working day), and requests zero-to-many LeaveRequest records (0..1-to-1 from the LeaveRequest side, since a pending leave requests are tied to exactly one approving administrator. An electrician is finally paid via zero-to-many payroll records during the course of their job (1-to-0.., one each pay period). Timesheet data is calculated from into payroll (many-to-one: many individual daily timesheet entries feed into a single payroll run covering that pay period).

Admin-centered relationships involve triaging zero-to-many ContactQuery records (0..1-to-0.., optional on the Admin side of a given query since a query may sit unassigned/untriated for a period, matching the "QueryReviewed" step in the Website Query Processing state diagram, Section 4.1), creating zero-to-many Quote records (1-to-0..), approving zero-to-many LeaveRequest records, and generating zero-to-many DataExportLog records (1-to-0..). Admin is connected to Contact (1-to-0..), as missed-call handling is initiated from the Admin App's own device contact list. A single admin's phone may have numerous permitted contacts.

Financial and communication connections. Quotes are converted to zero-or-one invoices (1-to-0..1, since declined quotes are never converted). Invoices receive zero-to-many payment records (1-to-0..), allowing businesses to accept partial payments, as shown in the Invoice state diagram (Section 4.1). Discounts are redeemed as zero-to-many CustomerDiscount records (1-to-0..), each applied to exactly one invoice (0..-to-1). On the communications side, a Contact makes zero-to-many MissedCallLog records (1-to-0..), and each MissedCallLog triggers zero-or-one AIMessageLog record (1-to-0..1, optional because, per the AI Missed Call state diagram's "Admin answers" branch, Section 4.1, not every incoming call results in an automated message—only genuinely missed calls do).

Together, these links provide the domain model with complete, end-to-end coverage of every workflow implied by the checklist and the client's original requirements: inquiry handling, quoting, job execution, compliance, rewards, scheduling, payroll, and billing. No modelled workflow has a "dangling" class to which nothing else connects, as demonstrated by tracing each of the twenty-nine classes in the diagram back to at least one other class during the diagram's review — a basic but important completeness check recommended in domain-modelling practice (Larman, 2012).

Every relationship listed above was also cross-checked directly against the sequence diagrams in Section 4.1: for every association shown here, at least one message exchange in at least one sequence diagram exercises it, which is the practical test the team used to ensure the domain model was neither under-specified (missing a relationship some workflows actually need) nor over-specified (containing a relationship no workflows ever use).

### **Deliberate design decisions and alternatives considered**

Several modelling choices were deliberately taken, and it is important to document them completely here since a marker or client could reasonably wonder why a seemingly simpler option was not chosen.

The Contact entity remained distinct from Customer, despite their conceptual overlap, because the missed-call capability must record an interaction with a person who has never been registered as a Customer; merging them would force the creation of a Customer record for every missed caller, which is both inaccurate (they are not registered) and a data-protection concern, as it would retain more personal data than the "permitted contact information" principle requires (Republic of South Africa, 2013).

Discount and RewardsAccount stayed as separate promotional tools rather than being combined, since the organization specifically needs the ability to run a fixed-value promo code campaign (Discount) independently of the ongoing, accumulating loyalty programme (RewardsAccount) — for example, a one-time "Refer a Friend" code unrelated to a customer's points balance, a requirement confirmed at the rewards-and-discounts scoping meeting of 1 July 2026 (meeting 14, Section 1.4).

Timesheet and Payroll were maintained as distinct entities, as previously noted, to support a review-then-process workflow instead of an automatic hours-to-pay pipeline, giving administrators a manual verification step before any calculation and disbursement of funds.

A "Person" superclass was thought about but ultimately discarded. Since Customer, Electrician and Admin hold several similar attributes like a name and contact details, one logical design choice would have been to create a single abstract Person superclass to hold these shared attributes, with Customer, Electrician and Admin each inheriting from it. This idea was deliberately dismissed for three main reasons. First, the three roles don't actually share any meaningful behavior beyond their basic contact details—the methods they each contain (Section 3.2) are completely separate, so using inheritance would provide almost no benefit beyond a little attribute reuse while adding unnecessary complexity that no business stakeholder at PSG Electrical and Cables/Trite Solar would understand or describe in their own words ("a Person" was never a term the client used; instead, they consistently referred to "a customer," "an electrician" and "an admin"). Second, in reality, a single person can hold more than one role over time—for example, an office administrator might also be a customer of the business for their household's electrical needs—and a strict single-inheritance structure forces roles into mutually exclusive identities, which doesn't match this possibility. Keeping them as separate classes with their own unique identifiers avoids the problem of figuring out what happens when one real person needs to exist as multiple types at once. Third, and most importantly from a practical standpoint, the checklist's domain modelling guidelines strongly support a model that matches the client's own way of thinking rather than introducing extra abstraction just for the sake of it, and every client meeting from 18 April 2026 onward consistently described "customers," "electricians" and "admins" as entirely distinct kinds of users, never as variations of a single underlying idea (Larman, 2012, covers this exact balance between "ideal" inheritance models and practical, stakeholder-focused designs, usually favoring the latter for systems of this size).

A single "User" table with a role field was also explored but rejected for similar reasons: while this approach is a common and often useful shortcut in smaller systems, it would have required all of Customer's, Electrician's and Admin's truly unique attributes (a Customer's address, an Electrician's certificationNo and specialisation, an Admin's role) to be stored in the same table, either as a long list of columns where two-thirds are usually empty or in a separate, normalized extension table for each role—which ends up being, in practice, the same three-class structure the domain model already follows, just reached through a more roundabout method. Keeping Customer, Electrician and Admin as three distinct, clearly labeled classes was seen as the clearer and more sustainable option, especially for a six-person student team where clean code directly improves onboarding and teamwork.

Normalisation was applied consistently up to at least third normal form across the entire domain model: no attribute in the diagram contains repeating groups (for instance, Payment exists as a separate class instead of having "payment1Amount" and "payment2Amount" columns directly on Invoice, which prevents a first-normal-form violation), and no non-key attribute relies on only part of a composite key or another non-key attribute rather than the full identity of its class (for example, JobMaterial's costAtTime represents a specific combination of one Job and one Material, not a value derivable solely from Material's unitCost, which is why it required its own storage rather than being calculated on demand) (Silberschatz, Korth and Sudarshan, 2020; Connolly and Begg, 2015). This degree of normalisation was deemed suitable given HYDRA's transactional, write-intensive workload (frequent job initiation, status updates, and payments) — a normalised structure reduces the chance of duplicated data entries leading to inconsistencies, a critical concern for billing and compliance systems, outweighing the minor query-performance overhead normalisation may introduce, an overhead that PostgreSQL's efficient indexing and join operations (Section 4.3) easily handle at the data volumes outlined in Section 8.

A document-oriented (NoSQL) approach to the entire relational domain model was evaluated and discarded during the domain-modelling phase, prior to the formal technology-selection process described in Section 4.3: the extensive network of cross-class relationships evident in the earlier example (a single Job linking to

Customer, Electrician, ServiceType, Quote, Invoice, Payment, JobMaterial, JobMilestone, JobCheckin, InspectionReport, and RewardsTransaction) perfectly matches the type of structure a relational model handles cleanly through foreign keys and joins, whereas a document database would necessitate either deeply nested, duplicated documents or numerous manual, application-controlled cross-document references — essentially reintroducing, in code, the referential-integrity risks that relational databases automatically prevent with foreign-key constraints (Silberschatz, Korth and Sudarshan, 2020).

### 3.2 Design Class Diagram

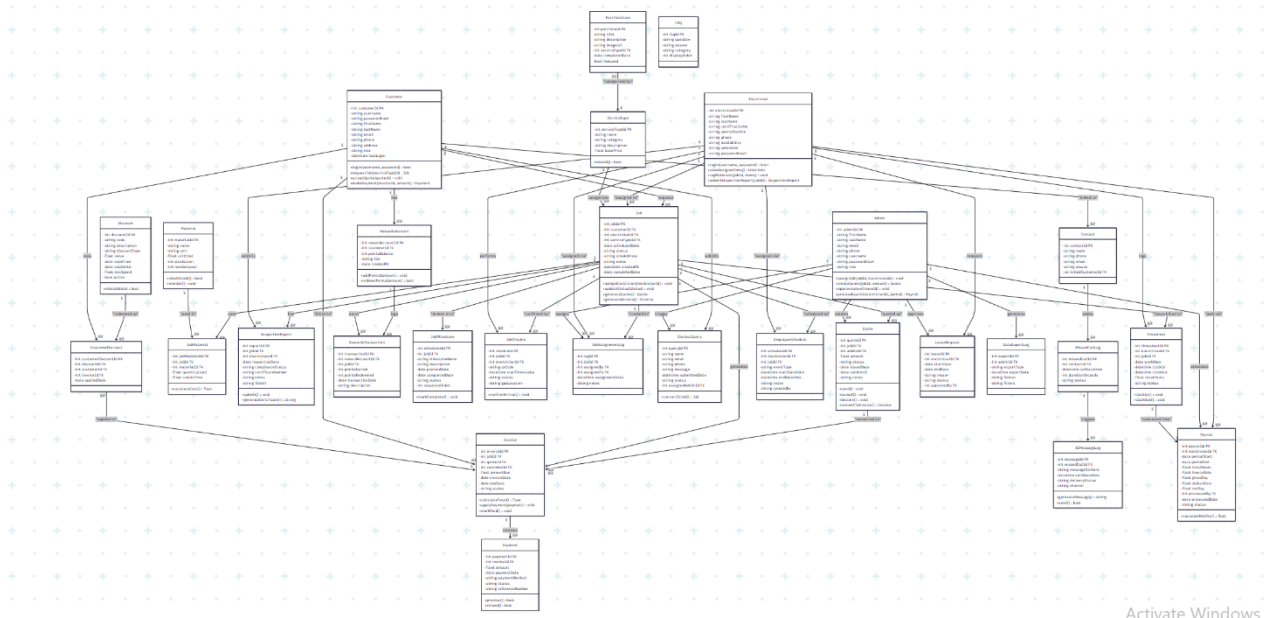


Figure 3: HYDRA Design Class Diagram

#### Purpose of the diagram and what changes from the domain model

Where the domain class diagram in Section 3.1 illustrates the topics the business addresses, the design class diagram demonstrates how those ideas will be implemented in practice (Pressman and Maxim, 2020). All classes from the domain model appear again here, yet each now includes specific implementation details: primary keys (PK) and foreign keys (FK), exact data types (int, string, float, date, datetime, bool) for every attribute, visibility indicators (+ for public, - for private), and crucially, the methods (operations) that provide each class functionality instead of acting as mere data containers. This aligns with the UML specification's separation between a conceptual/analysis-level class and an implementation-level class (Object Management Group, 2017), and follows the structured approach advised for converting a domain model into a design model before coding starts (Sommerville, 2016; Larman, 2012). The shift from domain model to design model marks the moment the team moves from analysis ("what must the business track?") to design ("which object will handle this task, and how will others request its actions?"), a distinction Larman (2012) notes as frequently misunderstood and therefore vital to clarify in object-oriented development.

This diagram was created collaboratively by Darsh Somayi and Katelyn Narain (as documented in the Work Breakdown Structure, Section 2.3, task 2.3), with the specific goal of having both the lead developer and data architect review the artefact together. This ensured every method added to the diagram underwent dual verification: "does the application need to invoke this operation, based on the sequence diagrams?" (Darsh Somayi's viewpoint) and "does this operation's data needs align with the database schema without requiring inefficient or complex queries?" (Katelyn Narain's viewpoint). Every method in the diagram thus meets both criteria, eliminating speculative additions.

## Attribute typing, keys and structural detail

Every attribute derived from the domain model in Section 3.1 now has a specific, implementation-ready data type assigned. Key attributes use the int type and are labeled with a primary-key (PK) indicator, such as `Customer.customerId : int [PK]` and `Job.jobId : int [PK]`. Text-based and category-based fields adopt the string type, covering details like names, emails, addresses, status values, and notes. Numeric and measurement fields are assigned the float type, including amounts, prices, point balances, and hours. Time-related fields are divided carefully between date (for entries where only the calendar date matters, such as an invoice's due date or a discount's start date) and datetime (for entries where the exact time is important, like a job-checkin's scan time or an admin's createdAt timestamp). This separation follows through consistently from the design class diagram to the physical PostgreSQL schema illustrated in the ERD (Section 4.4) and aligns with standard relational data-typing advice to choose the smallest type that still holds the necessary information (Silberschatz, Korth and Sudarshan, 2020). Boolean fields, such as `PortfolioItem.featured : bool` and `Discount.active : bool`, are applied solely when a true two-value business fact exists, rather than serving as a stand-in for a proper status or enumeration field. Instead, Job, Quote, Invoice, Payment, Timesheet, and Payroll all include a string status attribute since each of these classes includes more than two meaningful states, as shown in the corresponding state diagrams in Section 4.1. Reducing these to a boolean would have lost critical business information.

Each one-to-many relationship inherited from the domain model is represented here as a foreign key on the "many" side—examples include `Job.customerId : int [FK]`, `Job.electricianId : int [FK]`, and `Job.serviceTypeId : int [FK]`, which together capture the three distinct one-to-many relationships that Job participates in as the "many" side. Association classes include their own primary key in addition to the two (or more) foreign keys needed to reconstruct the relationship they represent, such as `JobMaterial.jobMaterialId : int [PK]` alongside `jobId : int [FK]` and `materialId : int [FK]`. This is the standard relational method for modeling a many-to-many relationship that also contains data—in this case, `quantityUsed` and `costAtTime` (Elmasri and Navathe, 2016; Connolly and Begg, 2015).

The table below outlines the full data dictionary implied by the design class diagram—covering every class, its complete set of attributes, each attribute's data type, and its key designation—so that this section, combined with Section 4.4's ERD, provides a single, complete, and clear specification of every field in the system. This eliminates the need for future developers to extract details directly from the diagram image.

**Table 4: Complete HYDRA data dictionary**

| Class       | Attribute                                                          | Type     | Key |
|-------------|--------------------------------------------------------------------|----------|-----|
| Customer    | customerId                                                         | int      | PK  |
| Customer    | firstName, lastName, email, phone, address                         | string   |     |
| Customer    | createdAt                                                          | datetime |     |
| Electrician | electricianId                                                      | int      | PK  |
| Electrician | firstName, lastName, certificationNo, specialisation, availability | string   |     |
| Admin       | adminId                                                            | int      | PK  |
| Admin       | firstName, lastName, email, phone, role                            | string   |     |
| ServiceType | serviceTypeId                                                      | int      | PK  |
| ServiceType | name, category, description                                        | string   |     |

| Class         | Attribute                                | Type     | Key |
|---------------|------------------------------------------|----------|-----|
| ServiceType   | basePrice                                | float    |     |
| PortfolioItem | portfolioId                              | int      | PK  |
| PortfolioItem | serviceTypeId                            | int      | FK  |
| PortfolioItem | title, description, imageUrl             | string   |     |
| PortfolioItem | completedDate                            | date     |     |
| PortfolioItem | featured                                 | bool     |     |
| FAQ           | faqId                                    | int      | PK  |
| FAQ           | question, answer, category               | string   |     |
| FAQ           | displayOrder                             | int      |     |
| Job           | jobId                                    | int      | PK  |
| Job           | customerId, electricianId, serviceTypeId | int      | FK  |
| Job           | scheduledDate, completedDate             | date     |     |
| Job           | status, siteAddress, notes               | string   |     |
| Job           | createdAt                                | datetime |     |
| Quote         | quoteId                                  | int      | PK  |
| Quote         | jobId                                    | int      | FK  |
| Quote         | amount                                   | float    |     |
| Quote         | status                                   | string   |     |
| Quote         | issuedDate, validUntil                   | date     |     |
| Invoice       | invoiceId                                | int      | PK  |
| Invoice       | jobId, quoteId, customerId               | int      | FK  |
| Invoice       | amountDue                                | float    |     |
| Invoice       | invoiceDate, dueDate                     | date     |     |
| Invoice       | status                                   | string   |     |
| Payment       | paymentId                                | int      | PK  |
| Payment       | invoiceId                                | int      | FK  |
| Payment       | amount                                   | float    |     |
| Payment       | paymentDate                              | date     |     |
| Payment       | method, status                           | string   |     |
| Material      | materialId                               | int      | PK  |
| Material      | name, unit                               | string   |     |
| Material      | unitCost                                 | float    |     |
| JobMaterial   | jobMaterialId                            | int      | PK  |
| JobMaterial   | jobId, materialId                        | int      | FK  |
| JobMaterial   | quantityUsed, costAtTime                 | float    |     |
| JobMilestone  | milestoneId                              | int      | PK  |



| Class              | Attribute                                           | Type     | Key |
|--------------------|-----------------------------------------------------|----------|-----|
| JobMilestone       | jobId                                               | int      | FK  |
| JobMilestone       | milestoneName, description, status                  | string   |     |
| JobMilestone       | plannedDate, completedDate                          | date     |     |
| JobCheckin         | checkinId                                           | int      | PK  |
| JobCheckin         | jobId, electricianId                                | int      | FK  |
| JobCheckin         | qrCode, status, gpsLocation                         | string   |     |
| JobCheckin         | scanTimestamp                                       | datetime |     |
| InspectionReport   | reportId                                            | int      | PK  |
| InspectionReport   | jobId, electricianId                                | int      | FK  |
| InspectionReport   | complianceStatus, certificateNumber, notes, fileUrl | string   |     |
| InspectionReport   | inspectionDate                                      | date     |     |
| RewardsAccount     | rewardAccountId                                     | int      | PK  |
| RewardsAccount     | customerId                                          | int      | FK  |
| RewardsAccount     | pointsBalance                                       | float    |     |
| RewardsAccount     | tier                                                | string   |     |
| RewardsTransaction | transactionId                                       | int      | PK  |
| RewardsTransaction | rewardAccountId, jobId                              | int      | FK  |
| RewardsTransaction | pointsEarned, pointsRedeemed                        | float    |     |
| RewardsTransaction | transactionDate                                     | date     |     |
| RewardsTransaction | description                                         | string   |     |
| Discount           | discountId                                          | int      | PK  |
| Discount           | code, description, discountType                     | string   |     |
| Discount           | value                                               | float    |     |
| Discount           | validFrom, validUntil                               | date     |     |
| Discount           | active                                              | bool     |     |
| CustomerDiscount   | customerDiscountId                                  | int      | PK  |
| CustomerDiscount   | discountId, customerId, invoiceId                   | int      | FK  |
| CustomerDiscount   | appliedDate                                         | date     |     |
| ContactQuery       | queryId                                             | int      | PK  |
| ContactQuery       | assignedAdminId                                     | int      | FK  |
| ContactQuery       | name, email, phone, message, status                 | string   |     |
| ContactQuery       | submittedDate                                       | datetime |     |
| Contact            | contactId                                           | int      | PK  |

| Class            | Attribute                                | Type     | Key |
|------------------|------------------------------------------|----------|-----|
| Contact          | linkedCustomerId                         | int      | FK  |
| Contact          | name, phone, email, source               | string   |     |
| MissedCallLog    | missedCallId                             | int      | PK  |
| MissedCallLog    | contactId                                | int      | FK  |
| MissedCallLog    | durationSeconds                          | int      |     |
| MissedCallLog    | status                                   | string   |     |
| MissedCallLog    | callDateTime                             | datetime |     |
| AIMessageLog     | messageId                                | int      | PK  |
| AIMessageLog     | missedCallId                             | int      | FK  |
| AIMessageLog     | messageContent, deliveryStatus, channel  | string   |     |
| AIMessageLog     | sentDateTime                             | datetime |     |
| EmployeeSchedule | scheduleId                               | int      | PK  |
| EmployeeSchedule | electricianId, jobId                     | int      | FK  |
| EmployeeSchedule | eventType, notes                         | string   |     |
| EmployeeSchedule | startDateTime, endDateTime               | datetime |     |
| Timesheet        | timesheetId                              | int      | PK  |
| Timesheet        | electricianId, jobId                     | int      | FK  |
| Timesheet        | totalHours                               | float    |     |
| Timesheet        | status                                   | string   |     |
| Timesheet        | workDate                                 | date     |     |
| Timesheet        | clockIn, clockOut                        | datetime |     |
| JobAssignmentLog | logId                                    | int      | PK  |
| JobAssignmentLog | jobId, assignedBy, assignedTo            | int      | FK  |
| JobAssignmentLog | notes                                    | string   |     |
| JobAssignmentLog | assignmentDate                           | datetime |     |
| LeaveRequest     | leaveId                                  | int      | PK  |
| LeaveRequest     | electricianId, approvedBy                | int      | FK  |
| LeaveRequest     | reason, status                           | string   |     |
| LeaveRequest     | startDate, endDate                       | date     |     |
| Payroll          | payrollId                                | int      | PK  |
| Payroll          | electricianId, processedBy               | int      | FK  |
| Payroll          | grossPay, hourlyRate, deductions, netPay | float    |     |
| Payroll          | status                                   | string   |     |
| Payroll          | periodStart, periodEnd, processedDate    | date     |     |
| DataExportLog    | exportId                                 | int      | PK  |

| Class         | Attribute                   | Type     | Key |
|---------------|-----------------------------|----------|-----|
| DataExportLog | adminId                     | int      | FK  |
| DataExportLog | exportType, format, filters | string   |     |
| DataExportLog | exportDate                  | datetime |     |

This table represents twenty-nine distinct classes (matching the twenty-nine domain classes described in Section 3.1, none dropped or merged at the design level) and just over one hundred and forty individually typed attributes in total, giving a concrete sense of the genuine scale of HYDRA's data model relative to a simpler single-app system, and directly substantiating the claim made throughout this document that Job, as the central class, participates in more relationships and is referenced by more foreign keys than any other single class in the schema.

### Key classes and their behaviour

The Customer class contains four methods: login, requestJob(), acceptQuote(), and makePayment(). These methods correspond to the customer-facing steps of the Customer App: authenticating, initiating a job request, responding to a quote, and settling an invoice. The placement of acceptQuote() and makePayment() as methods on Customer rather than on Quote or Invoice reflects the fact that it is the customer who initiates these actions from the app. This is a deliberate application of the information expert principle from responsibility-driven design, where a behavior is assigned to the object best placed to initiate it, even if the resulting state change is recorded on a different object.

The Electrician class contains four methods: login, viewAssignedJobs(), logMaterials(), and submitInspectionReport(). These methods correspond to the core Employee App workflow: sign in, see the day's work, record what was used on the job, and close out with a compliance submission. The return type List of Job on viewAssignedJobs() signals to any developer reading the diagram that this operation is a query returning a collection, not a single object, which avoids ambiguity when the method is later implemented against the repository layer. The logMaterials() method deliberately accepts a collection of items rather than being called once per material, so that a single Employee App interaction results in a single method call and a single database transaction creating multiple JobMaterial() rows atomically.

The Admin class contains four methods: assignJob(), createQuote(), approveLeave(), and processPayroll(). Each of these methods corresponds to a distinct administrative responsibility identified in the requirements: job assignment, quoting, leave approval, and payroll processing. Each method returns or affects exactly the domain object the operation name suggests, keeping the interface predictable for anyone reading the class in isolation.

The Job class contains four methods: assignElectrician(), updateStatus(), generateQuote(), and generateInvoice(). Placing generateQuote() and generateInvoice() on Job reflects standard object-oriented design practice: the object that owns the data required to produce a Quote or Invoice is responsible for producing it, keeping that logic in one place. This is an application of the information-expert principle, and it keeps the JobController() thin since the controller's job becomes simply "call job.generateInvoice()" rather than reimplementing the invoicing calculation itself. The assignElectrician() and updateStatus methods are both intentionally narrow, single-purpose methods rather than one generic update method, precisely so that the business rules specific to each transition have one clear, well-named home rather than being buried inside a generic setter.

The Quote class contains four methods: `send()`, `accept`, `decline`, and `convertToInvoice()`. These methods directly implement the quote lifecycle shown in the Customer Job Booking state diagram.

Invoice contains `calculateTotal(): float`, `applyPayment(payment): void` and `markPaid(): void`, and Payment includes `process(): bool` and `refund(): bool` — these three classes collectively provide the complete billing process, including handling partial payments as required by the checklist for the ERD and design diagram. Note that `Invoice.applyPayment(payment)` intentionally accepts a Payment object instead of a raw amount, so the invoice's internal logic—deciding if the payment settles the balance (triggering `markPaid()`) or only partially reduces it—remains inside the Invoice class rather than being calculated externally and then recorded back as a status by a controller. `Invoice.calculateTotal()` is purposely implemented as a method rather than a stored property, ensuring the total is always freshly calculated from the underlying Quote value and any associated JobMaterial costs and `CustomerDiscount()` reductions at the time of request, preventing the display of outdated totals after a discount is applied retroactively.

RewardsAccount includes `addPoints(amount): void` and `redeemPoints(amount): bool`; Discount has `isValid(date): bool`; ServiceType has `isValid(): bool`; and Material includes `checkStock(): bool` and `reorder(): void`. These are validation or state-modifying operations that keep business rules—such as preventing discounts outside their validity period or avoiding negative stock—encapsulated within the class owning the relevant data, rather than scattered across controller code. This approach reduces the risk of inconsistent rule implementation across different parts of the system, a common source of hard-to-detect defects where business logic leaks into multiple controllers (Fowler, 2018). `redeemPoints(amount): bool` returns a boolean to provide calling code—ultimately the Customer App's rewards screen—with a clear, unambiguous indication of whether the redemption succeeded, since insufficient points is a normal outcome the interface should handle gracefully rather than an exceptional error.

JobMilestone includes `markComplete(): void`; JobCheckin has `confirmArrival(): void`; ContactQuery includes `convertToJob(): Job`, allowing an administrator to elevate a general inquiry directly to a bookable job without re-entering customer details; Timesheet contains `clockIn(): void` and `clockOut(): void`; Payroll includes `calculateNetPay(): float`; InspectionReport has `submit(): void` and `generateCertificate(): string`; and AIMessageLog carries `generateMessage(): string` and `send(): bool`, which together implement the automated missed-call response described in Section 4.1. `ContactQuery.convertToJob(): Job` is particularly important: it is the only method in the entire design that connects the "anonymous visitor" portion of the domain model (ContactQuery, Contact) with the "registered customer" side (Customer, Job), and its implementation must, as part of the conversion, either find an existing Customer record matching the query's contact details or create a new one—logic that logically belongs on ContactQuery because it holds all the information needed to make that decision.

Classes lacking their own behavior. Not every class in the design contains methods, and this is a deliberate architectural choice rather than an oversight. PortfolioItem, FAQ, JobAssignmentLog, EmployeeSchedule, LeaveRequest, DataExportLog, MissedCallLog, RewardsTransaction, CustomerDiscount, Contact and the association classes JobMaterial are, in UML terms, essentially passive data containers: their sole purpose is to be instantiated once, read, and (in most cases) never modified again, and adding methods would have meant either duplicating logic that more naturally belongs to the class responsible for creating them (for example, the logic for generating a JobAssignmentLog entry belongs on `Job.assignElectrician()` rather than JobAssignmentLog, which has no validation or decision-making role) or introducing empty, unused methods purely to match the visual style of the rest of the diagram. Keeping these classes as behaviorless data holders aligns with established object-oriented design principles that not every class requires rich behavior—some classes are simply structured data, and forcing methods onto them for symmetry violates the single-

responsibility principle by giving a class a reason to change (such as a new validation rule) that has nothing to do with its actual purpose (Fowler, 2018).

## Design principles applied

Several general object-oriented design principles are evident, consistently followed throughout the entire diagram, beyond the specific "information expert" examples already mentioned.

**Single Responsibility.** Each class in the diagram has just one justification for modification. The Job class changes if the definition of a job or its status transition rules change; the Payroll class changes if the pay calculation rules change; these remain distinct, even though a smaller or less carefully structured system might have combined "job" and "pay" data into a single overloaded entity.

**Low coupling, high cohesion.** Related data and operations are closely grouped within individual classes (high cohesion — for instance, all discount calculation and validation logic resides within the Discount class itself), while classes interact only through clear, defined connections rather than accessing each other's internal details (low coupling — for example, Payroll.calculateNetPay() relies on Timesheet data being accessible but does not need to know how Timesheet.clockIn() or clockOut() are internally designed) (Larman, 2012).

**Encapsulation and protected variation.** As explained in more detail later, private attributes are exposed only through controlled public methods (RewardsAccount.pointsBalance is adjusted solely through addPoints() or redeemPoints(), never modified directly), shielding the rest of the system from internal changes to point calculation logic in the future — a new bonus-points promotion, for example, could be implemented entirely within addPoints() without requiring any other class to change (Fowler, 2018).

**Polymorphism** was intentionally avoided. No method in the design class diagram is overridden or overloaded across different implementing classes, and no interface (beyond the repository interfaces introduced structurally in Section 5.1, which serve a separate architectural purpose) appears anywhere in the diagram. This was a deliberate choice rather than an oversight: none of HYDRA's thirty classes represent true variations of a shared abstract behavior that would benefit from polymorphic dispatch (unlike, for example, a system with multiple payment-gateway integrations, which might reasonably use a PaymentProcessor interface with several implementations) — introducing polymorphism without genuine variation would add unnecessary layers of abstraction with no real design advantage, which the team determined would introduce complexity without value given the project's scale and deadlines (Larman, 2012).

## Mapping the design class diagram to the codebase

The design class diagram serves not just as an analysis tool but also establishes the folder and file structure of the final Node.js codebase. Every class depicted here links directly to one domain model file (models/Job.js for example), one repository implementation (repositories/PostgresJobRepository.js, covered in Section 5.1), and—when the class is the focus of at least one controller—one controller file (controllers/JobController.js). This clear, direct mapping between diagram and codebase was agreed upon during the API-contract-planning meeting on 24 July 2026 (meeting 18, Section 1.4), ensuring that any team member or future developer taking over the system (Section 9.2) can quickly find the implementation of any class by matching its name to a file, eliminating the need for a separate, manually updated mapping document that might become outdated over time.

## Relationships, keys and visibility

All one-to-many relationships derived from the domain model (for example, Customer to Job, Job to JobMilestone, Electrician to Timesheet) are implemented in the design class diagram as foreign keys on the

"many" side, maintaining the same cardinalities shown in the ERD in Section 4.4 (for example, Job.customerId FK, Job.electricianId FK, Job.serviceTypeId FK). The two many-to-many relationships identified in the domain model—Job/Material and Customer/Discount—are handled using the association classes JobMaterial and CustomerDiscount respectively, each containing its own primary key plus the two foreign keys required to reconstruct the relationship, in line with standard relational design practices for many-to-many associations (Silberschatz, Korth and Sudarshan, 2020). Every relationship in the diagram is a straightforward UML association; the model avoids aggregation, composition, or inheritance, since no class in the HYDRA domain forms a structural part of another (which would require aggregation/composition) and no class is a specialised version of another (which would necessitate inheritance)—each class functions as an independent, distinct business concept, a deliberate and defensible design choice for this type of system (Fowler, 2018; Larman, 2012).

Access modifiers follow a uniform pattern across the diagram: identifying attributes and foreign keys are marked + (public) to allow repository and controller classes (Section 5.1) to access and modify them directly, while all descriptive attributes are marked - (private) and can only be accessed through the class's own methods. Every method is marked + (public), because each listed method is meant to be invoked from outside the class—either by a controller or another domain object—rather than serving as an internal helper function. This strict approach to encapsulation goes beyond mere appearance: it ensures, for instance, that RewardsAccount.pointsBalance cannot be altered directly by an external caller, but can only be modified through the validated addPoints()/redeemPoints() methods, exactly the kind of safeguard encapsulation provides against invalid or inconsistent states (Fowler, 2018). This same rigor is what makes the data integrity risk entry in the risk register (Section 6.8) a justified mitigation rather than an idealistic claim: because no class exposes a public setter for sensitive derived values like a points balance or an invoice total, there is no possible code path in the system that could accidentally write an inconsistent value to either field.



## 4. Implementation Artifacts

The Implementation Artifacts section demonstrates the actual operation of the analysis-level models developed in Section 3, specifically the Domain Class Diagram and Design Class Diagram. While Section 3 addresses the question of what exists in the system and what each component knows and performs, Section 4 focuses on what actions occur, their sequence, and the conditions under which they take place. This division between structural and behavioral modeling represents a core principle of the Unified Modeling Language and object-oriented analysis and design methodologies. A structural diagram captures a static representation of a system's components, whereas a behavioral diagram depicts those components in dynamic interaction over time (Object Management Group, 2017; Larman, 2012). Section 4 consists of four subsections. Section 4.1 provides all eleven state diagrams and eleven sequence-diagram artifacts, which collectively document every essential workflow identified during requirements gathering (Section 2) and every method outlined in the Design Class Diagram (Section 3.2). Section 4.2 outlines the deployment strategy, converting the logical architecture described in Section 5 into the tangible Microsoft Azure infrastructure that the HYDRA team will provision, configure, and manage. Section 4.3 explains and justifies the selection of database and application technologies supporting the entire system. Section 4.4 presents the finalized Entity Relationship Diagram (ERD) set—the practical implementation of the Domain and Design Class Diagrams as a workable relational structure. Together, these four subsections create an unbroken evidence trail: a requirement from Section 2 links to a class in Section 3, which in turn links to a state and message sequence in Section 4.1, executed within the infrastructure described in Section 4.2, implemented using the technologies justified in Section 4.3, and storing its data in the schema detailed in Section 4.4. This traceability chain was intentionally constructed and verified at each stage, adhering to established software engineering practices that emphasize keeping requirements, analysis, design, and implementation artifacts interconnected rather than isolated (Sommerville, 2016; Pressman and Maxim, 2020; Bass, Clements and Kazman, 2021).

### 4.1 Sequence and State Diagrams

#### Purpose of this section and the behavioural modelling approach

Sequence diagrams and state diagrams are the two UML behavioral diagram types HYDRA employs to define system behavior with sufficient precision for developers to implement it unambiguously (Object Management Group, 2017). They serve complementary roles rather than overlapping ones. A sequence diagram offers a scenario-specific, cross-object perspective: it begins from a fixed initial condition, such as "a customer has just submitted a job request," and follows, step by step, which objects are activated, in what order, including all decision points (modelled using UML alt combined fragments), optional steps (opt fragments), and moments when one object pauses to wait for another's response before proceeding (a synchronous call, indicated by a solid arrowhead) versus when it sends a message and continues without waiting (an asynchronous call, shown with an open arrowhead) (Object Management Group, 2017; Fowler, 2018). A state diagram, in contrast, provides an object-specific, cross-scenario view: it centers on a single object instance—for example, one Job class instance—and illustrates all possible states that object may occupy throughout its existence, along with every event capable of shifting it from one state to another, regardless of which particular user scenario triggered the event. Formally, each HYDRA state diagram functions as a deterministic finite state machine in the UML context: a finite set of states, a finite set of events (each tied to a guarded transition), and the guarantee that from any current state, a given event leads to exactly one target state without ambiguity (Object Management Group, 2017; Larman, 2012). This determinism has practical significance beyond theory: it ensures the status column defined for JOB, INVOICE, CONTACT\_QUERY, and all other stateful entities in Section 4.4 can be reliably enforced by a straightforward

database constraint (a CHECK constraint limiting the column to its predefined valid values) rather than requiring elaborate application-level logic to regulate transitions that the schema alone cannot express.

HYDRA models eleven separate objects or workflows as state diagrams and documents eleven corresponding sequence-diagram artifacts (ten distinct scenario topics, with the Payment and Invoicing scenario divided, for clarity, into two linked diagram exports covering the payment gateway interaction and the reminder/overdue handling portions of the same process). Every state diagram has at least one corresponding sequence diagram that demonstrates a real-world scenario activating a portion of that state diagram's transitions, and every sequence diagram's message calls align with a named method on a class described in the Design Class Diagram (Section 3.2). This one-to-one (or, in the payment case, one-to-two) pairing between state and sequence diagrams was a deliberate design choice, intended to let reviewers or future developers confirm the consistency of both artifact types line by line rather than relying solely on the assumption that they match because they originated from the same team (Sommerville, 2016).

Both diagram families were developed incrementally rather than in a single effort. An early version of each diagram was created directly from the relevant user stories (Section 2.1) and the ERD entities that had stabilized by that project stage (Section 4.4); each draft was then reviewed against the Domain and Design Class Diagrams (Sections 3.1 and 3.2) to verify that every state label matched a valid value for a status-type attribute on some class and that every message matched a documented method signature for that class. Any discrepancies uncovered during this review—for example, an initial version of the Job Lifecycle state diagram that used "Scheduled" when the ERD's JOB.status column used "Assigned"—were corrected in both locations simultaneously, ensuring the final diagram set presented here remains fully consistent from start to finish.

## State diagrams

The eleven state diagrams shown below follow the most practical reading sequence: the three "core object" diagrams first (Job, Invoice, Authentication), as these establish states that nearly every other diagram either relies on or refers to, followed by the eight "role-journey" and "process" diagrams that expand on that shared foundation.

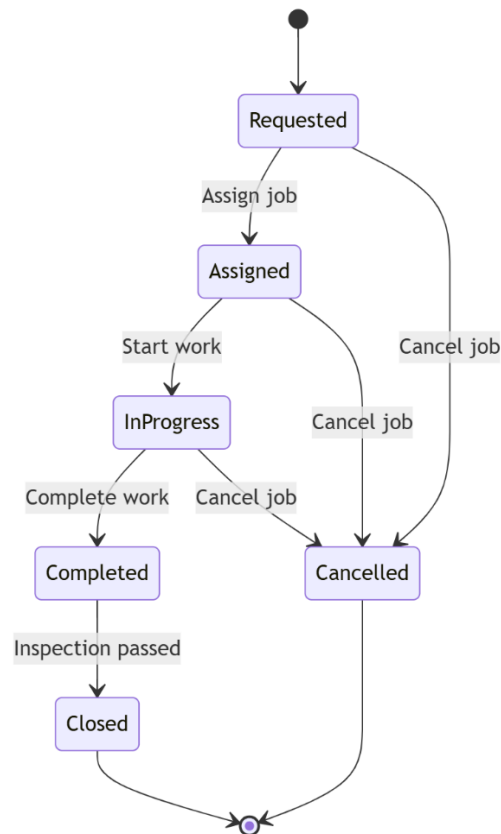


Figure 4: Job Lifecycle State Diagram

The Job class serves as the most interconnected entity within the entire HYDRA domain model — it connects directly or indirectly to Quote, Invoice, Payment, JobMilestone, JobCheckin, JobMaterial, EmployeeSchedule, Timesheet, InspectionReport, JobAssignmentLog and RewardsTransaction (Section 3.1, Section 4.4) — so its lifecycle state diagram functions as the central reference point for the entire collection. The state machine includes six states in total: five "normal" states along the primary workflow (Requested, Assigned, InProgress, Completed, Closed) and one absorbing failure/exit state (Cancelled), along with the two pseudostates every UML state machine requires — an initial pseudostate (the filled black circle) and a final state (the circled dot) (Object Management Group, 2017).

A job instance enters the Requested state the moment a customer submits a service request through the Customer App or the public website's contact form (Section 3.3, Customer Journey). This matches the `JOB.status = 'Requested'` value written by the `JobService.createJob()` method (Section 3.2) as well as the `ContactQuery.convertToJob()` path outlined in the Website Query Processing state diagram (Diagram 7, below), which represents the second valid way a Requested job can be generated — a job need not start from the Customer App specifically; it can just as well begin when an admin converts a phone or website enquiry, and the state diagram intentionally avoids specifying which actor triggered the Assign job transition since the rest of the job lifecycle remains consistent regardless of how the job originated.

The Assign job event, triggered solely by an Admin user (enforced by the role-based access control matrix in Section 6.1.5 and the Access Control ERD in Section 4.4), advances the job to Assigned and is when `JOB.electrician_id` first receives a non-null value; prior to this transition the column remains nullable, and the state diagram serves as the human-readable justification for allowing that column to be null on a newly requested job but never after assignment. From Assigned, Start work — initiated by the assigned Electrician via the Employee App and practically controlled by the successful QR-code scan recorded in the `JOB_CHECKIN` entity and the Employee Job Execution state diagram (Diagram 5, below) — moves the job to InProgress.

Complete work, also triggered by the same Electrician once on-site work finishes, advances the job to Completed.

Moving from Completed to the final Closed state is governed by a condition rather than a straightforward user action: Inspection passed. This is, without exaggeration, the single most critical guard condition in the entire HYDRA state model. PSG Electrical and Cables/Trite Solar conducts regulated electrical work, and South African law mandates that electrical installation work of this nature be certified against the Electrical Installation Regulations before it can be considered truly complete from a compliance standpoint, not merely from a physical completion perspective (Republic of South Africa, 2009). By structuring Closed as reachable only through a guarded transition tied to a passed inspection — rather than permitting Completed to act as the de facto end state with inspection recorded as an optional afterthought — the state machine itself, and consequently the CHECK constraint and derived application logic, ensures it is structurally impossible for a job to be marked fully closed in HYDRA without a matching INSPECTION\_REPORT row containing a passing compliance\_status. This directly links a state-diagram guard condition to a specific clause of South African electrical-safety legislation, demonstrating how a state diagram can enforce business rules with far greater precision than free-text requirements documentation ever could (Sommerville, 2016; Larman, 2012).

The Cancelled state can be reached via a Cancel job event from any of the three active, pre-completion states — Requested, Assigned or InProgress — represented as three separate transition arrows converging on the same target state instead of a single transition from one source state, since UML state machines require each transition to be drawn explicitly between its specific source and target (Object Management Group, 2017). This was a deliberate design choice, not an oversight: an earlier version of this diagram allowed cancellation only from Requested, based on the assumption that once an electrician had been dispatched a job would always proceed to completion. That assumption was discarded after reviewing the client's real-world operations (documented in the meeting minutes of 16 May 2026): electrical fieldwork frequently faces disruptions such as unsafe site conditions, a customer's unavailability upon arrival, or on-site discoveries that the job requires materials or access the electrician lacks — any of which can lead to abandoning a job after an electrician has already been assigned or has started work. Modeling Cancelled as a terminal state reachable from three distinct sources, rather than forcing all cancellations through an artificial intermediate step, keeps the state machine an accurate reflection of the client's actual field operations rather than an idealised version of them (Sommerville, 2016).

No transition leads out of Cancelled or Closed: both represent true UML final-state equivalents for this object, meaning that once a Job instance reaches either state, JOB.status will never change again for that record. This guarantee is enforced in the application layer's repository update logic (Section 5.1) and streamlines downstream reporting and payroll logic (Section 4.2, Payroll processing), since neither of those consumers needs to account for a job's status reverting after being counted.

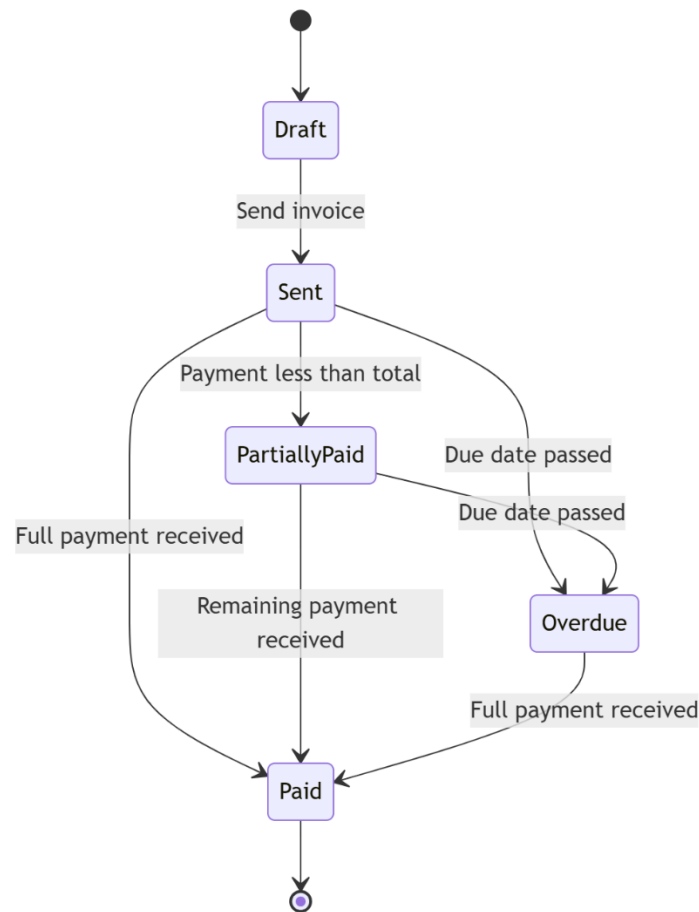


Figure 5: Invoice Lifecycle State Diagram

**The provided state diagram depicts the stages of an invoice's payment lifecycle, tracing its journey from the initial Draft phase to the final Paid status. After being dispatched, an invoice enters the Sent state. Subsequent customer actions determine whether the invoice transitions to being Partially Paid, Overdue, or Paid. A partially paid invoice reaches the Paid stage upon receipt of the outstanding balance, whereas an invoice that remains unpaid or only partially settled becomes overdue after the deadline. The cycle concludes when the entire amount is collected and the invoice reaches the Paid state.**

The Invoice state machine has five distinct states - Draft, Sent, PartiallyPaid, Overdue and Paid - which were designed to address a specific data-modelling question regarding the settlement of an invoice by multiple payments. The client's billing practice allows customers to pay a deposit and then a balance, or to pay a large invoice across two instalments, indicating that an invoice can indeed be settled by more than one payment. The state diagram formalises the consequence of this answer for every other artefact in the report.

An invoice begins in the Draft state as soon as `InvoiceService.createInvoice()` is called against a completed, admin-verified job. This is tied directly to the Closed state of the Job Lifecycle diagram, where invoice creation is only exposed to the admin once the associated job has reached Closed, which itself is only reachable once the compliance inspection has passed. Send invoice moves the record to the Sent state, at which point the customer is notified through the Notification Gateway. From Sent, three distinct events are possible, resulting in a three-way branch rather than a simple linear pipeline.

PartiallyPaid is not a dead end and has two outward transitions of its own - Remaining payment received, which moves the invoice on to Paid, and Due date passed, which moves a partially-settled invoice into Overdue. This second transition is operationally significant, as it guarantees that the overdue-reminder logic

described in Section 4.2 does not need a separate code path for "overdue and completely unpaid" versus "overdue with an outstanding balance". Overdue itself has a single outward transition, Full payment received, back to Paid. Every reachable path through this five-state machine converges on the single terminal state Paid, and no path exists by which an invoice can be marked paid without having passed through at least one of the three payment-receiving transitions.

This diagram, unlike the Job and Invoice diagrams which depict a single ongoing business entity, illustrates a session—a temporary object that only exists while a user interacts with one of the three HYDRA client applications (Customer, Employee, Admin). The process starts in LoggedOut. A login attempt, where the user enters a username and password as described in Section 6.1.1, shifts the session to the temporary Authenticating state. Here, the submitted password is checked against the stored bcrypt hash (Section 6.1.2), and if successful, a signed JSON Web Token is generated (Section 6.1.3; Jones, Bradley and Sakimura, 2015).

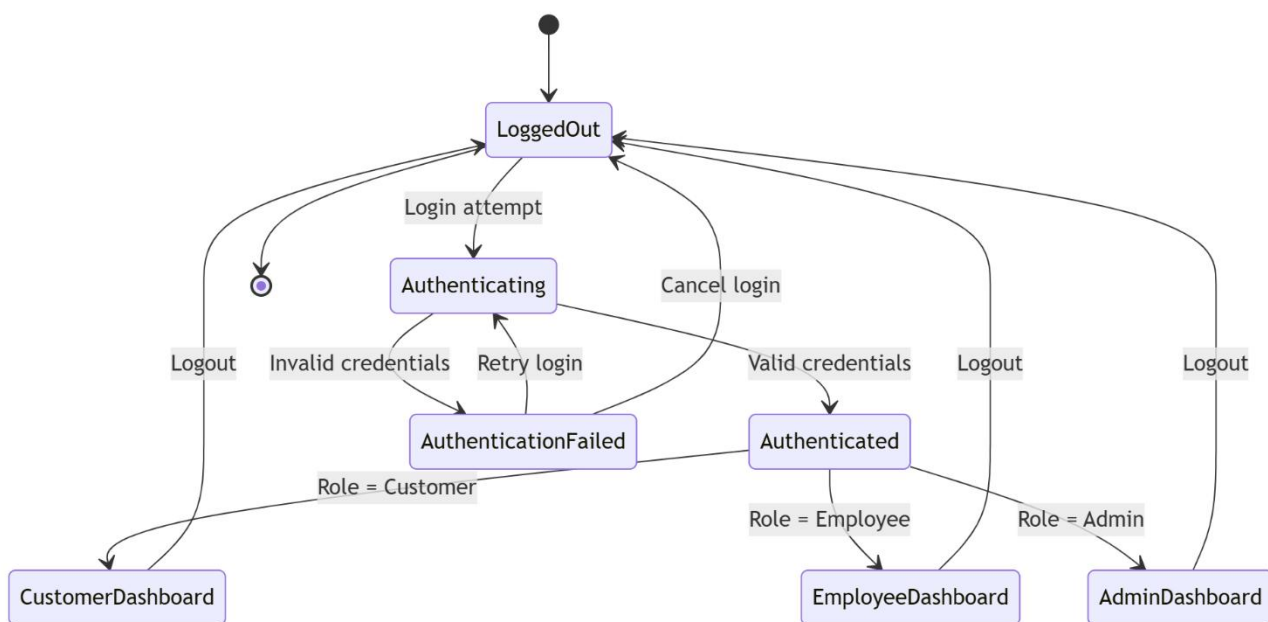


Figure 6: User Authentication State Diagram

***This state diagram depicts the procedure for user authentication and role-based access. Starting from the Logged Out state, a user transitions to Authenticating upon initiating a login. Successful authentication occurs with correct credentials, whereas incorrect credentials trigger an Authentication Failed state, allowing the user to either try again or abort. Following successful authentication, the system determines the user's specific role and routes them to the corresponding Customer, Employee, or Admin Dashboard. From these dashboards, users can log out to return to the Logged Out state.***

From Authenticating, two results are possible. Incorrect credentials move the session to AuthenticationFailed, a state designed to explicitly represent failure—allowing the user to either retry logging in (returning to Authenticating) or Logout, ending the attempt and returning to LoggedOut. This clear failure state is essential for implementing rate-limiting and account-lockout measures described in Section 6.1 and Section 6.3.6: since failed attempts are a distinct, traceable transition rather than an internal process within a single "login" function, the number of consecutive AuthenticationFailed transitions for a given username can be monitored and used to trigger safeguards, following recognized authenticator-throttling guidelines for preventing credential-guessing attacks (National Institute of Standards and Technology, 2017).

Correct credentials instead transition the session to Authenticated, a super-state from which the machine instantly moves into exactly one of three mutually exclusive sub-states—CustomerDashboard,



EmployeeDashboard, or AdminDashboard—based on the role value from the validated ADMIN, ELECTRICIAN, or CUSTOMER row (Section 4.4). Structuring this branch as an automatic, role-based transition from a shared Authenticated state—rather than treating Customer, Employee, and Admin apps as three separate yet similar authentication flows—was a deliberate architectural choice with two key advantages. First, it ensures the authentication logic (credential verification, hashing, token issuance) resides in a single location in the codebase instead of being replicated three times, reducing the volume of security-sensitive code requiring review, testing, and maintenance (OWASP Foundation, 2021). Second, it maintains internal consistency across the diagram set: since all three sequence diagrams beginning with a login (the Customer App booking flow, Employee App job-execution flow, and Admin App job-management flow, all covered later) share this identical authentication sequence as their initial steps, validating any of these sequence diagrams against this state diagram will confirm consistent behavior at login, as expected for a unified authentication system.

The diagram serves as the initial state-level representation corresponding to the Customer Journey column in the UX Journey Map from Section 3.3, where each screen listed aligns with a named state and each transition in the journey map matches a defined event here. The sequence progresses as follows: BrowsingServices → (Select service) → RequestingQuote → (Submit request) → QuoteSubmitted → (Quote generated by admin) → QuoteReceived.

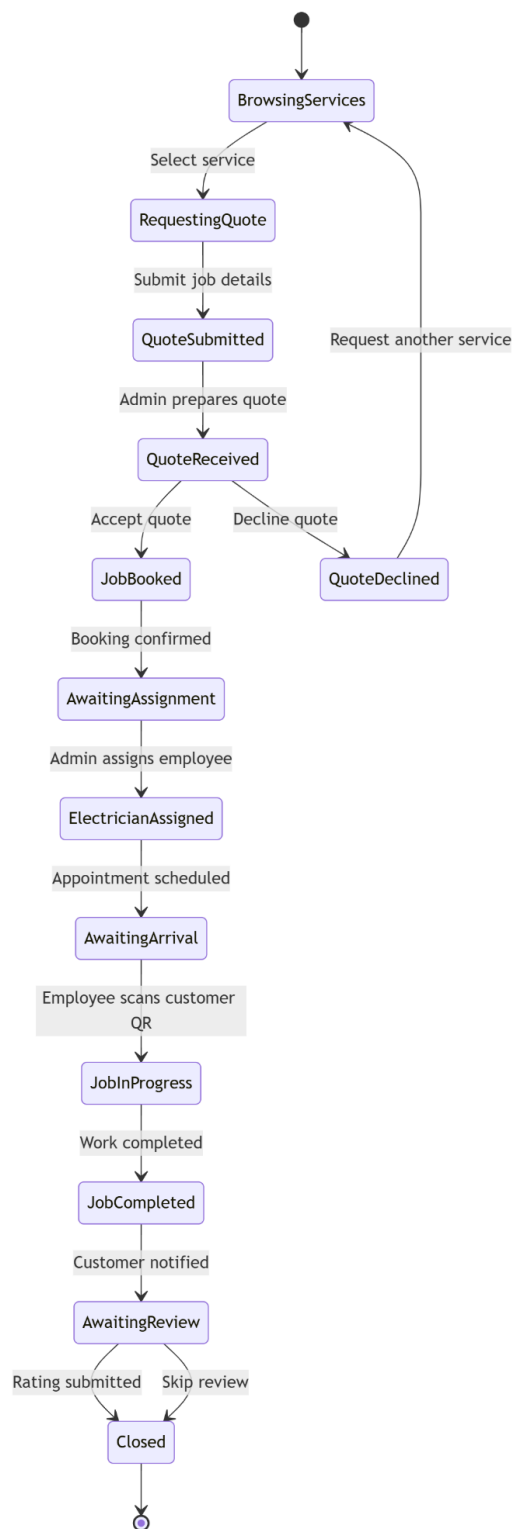


Figure 7: Customer Job Booking Lifecycle

***This state diagram depicts the \*\*entire lifecycle of a customer service request\*\*, starting with service browsing and ending with job completion and closure. After a customer chooses a service and provides job specifics, an administrator generates a quotation. Upon acceptance of the quote, the job is booked and an electrician is dispatched. The technician then arrives for the appointment, confirms their presence by scanning the client's QR code, finishes the necessary tasks, and informs the customer. Before the job transitions to the \*\*Closed\*\* state, the client has the option to provide a rating or bypass the review process. Should a quotation be rejected, the customer is able to go back to browsing to request a different service.***

QuoteReceived introduces the first true decision point, represented by two distinct transitions instead of one, acknowledging that not all quotes lead to a booking. Choosing Accept quote advances the process to JobBooked, continuing through AwaitingAssignment → (Electrician assigned) → ElectricianAssigned → AwaitingArrival. Opting for Decline quote, on the other hand, directs the flow to QuoteDeclined, which has a single outgoing transition, Browse again, returning to BrowsingServices—ensuring customers who reject a quote can easily explore alternative service options without exiting the system, a feature designed to enhance platform retention by simplifying the process for homeowners or businesses seeking electrical services.

The progression from AwaitingArrival to JobInProgress is triggered by the precise event Employee scans customer QR, a uniquely implementation-specific action tied directly to the JOB\_CHECKIN entity (including qr\_code, scan\_timestamp, and gps\_location fields; Section 4.4) and mirrored in the Employee Job Execution diagram below. This single physical action—whether the electrician scans a QR code shown by the customer or vice versa upon arrival—serves as the moment when two separate state diagrams for different actors advance simultaneously. Including this event explicitly in both diagrams ensures clarity for readers tracking only the customer's journey, eliminating ambiguity about how "arrival" is verified.

The flow continues from JobInProgress through JobCompleted and AwaitingReview before concluding at the terminal Closed state. The diagram specifies that a rating or review can be submitted or skipped (Review submitted or Skip review, both leading to Closed), marking the sole instance across all eleven diagrams where a non-compulsory user action is allowed to conclude cleanly at the terminal state regardless of completion. This reflects the product's stance that customers should never be prevented from finalizing a job solely due to an uncompleted review.

This diagram stands out as the most intricate among the eleven, featuring three fully detailed "unhappy path" branches explicitly defined as named states—a deliberate approach to avoid leaving exception handling implicit or added later in development, where it becomes harder to review, test, or analyze (Sommerville, 2016; Larman, 2012).

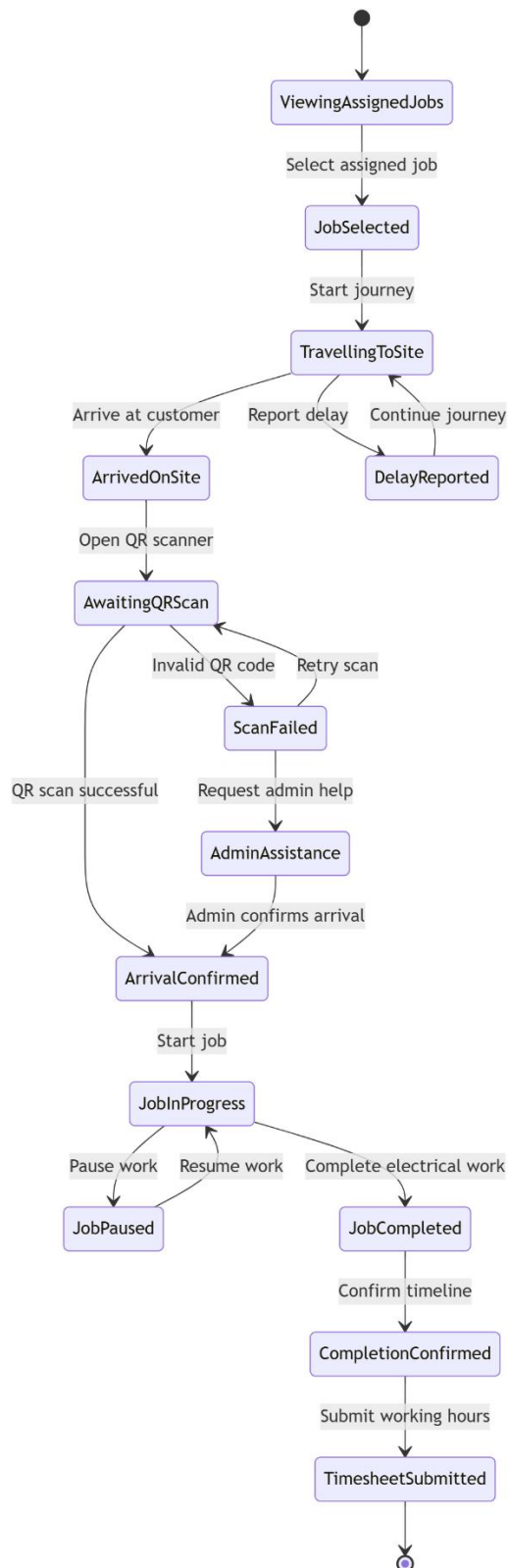


Figure 8: Employee Job Execution Lifecycle

*The provided state diagram depicts the workflow for managing employee tasks, starting from the initial job selection and ending with the submission of hours worked. After choosing a task, the staff member travels to the client's location and validates their presence by scanning a QR code. This workflow accounts for potential issues such as delays, incorrect QR codes, or the need for administrative support. Following arrival verification, the worker starts the assignment, with the option to pause and restart the task as needed. The cycle concludes once the electrical work is finished, the completion time is verified, and the timesheet is turned in.*

The primary workflow follows: ViewingAssignedJobs → JobSelected → TravellingToSite → ArrivedOnSite → AwaitingQRScan → ArrivalConfirmed → JobInProgress → JobCompleted → CompletionConfirmed → TimesheetSubmitted. Superimposed on this are three exception pathways. First, from TravellingToSite, the Report delay event shifts the electrician to DelayReported, which only allows Continue travelling back to TravellingToSite—enabling workers running behind (due to traffic or extended prior jobs) to formally notify both admin and customer without triggering an error state, as delays are a routine part of field operations. Second, a failed QR scan (Scan fails) from AwaitingQRScan directs the electrician to ScanFailed, offering two recovery options: Retry scan to return to AwaitingQRScan or, if repeated attempts fail, Request admin help to move to AdminAssistance, which resolves back to ArrivalConfirmed once the admin manually verifies arrival (Admin confirms arrival). This guarantees that technical issues with QR scanning (such as damaged codes, camera faults, or poor outdoor lighting) cannot indefinitely halt job progress, as a human administrator can override the automated check. Third, an optional pause-and-resume cycle is embedded within JobInProgress, allowing workers to halt tasks (for breaks, material deliveries, or customer pauses) and resume without reverting the job's status in the Job Lifecycle diagram (Diagram 1)—this local pause/resume mechanism remains invisible to the Job status abstraction, which only tracks "not yet started," "being worked on," and "finished," while detailed pause records, if kept, are stored solely in the electrician-facing timesheet notes.

The admin's process starts at ViewingRequests and continues to ReviewingRequest. From here, the path splits depending on the customer's initial submission quality: if sufficient details are provided, the workflow moves to PreparingQuote; if not, it goes to MoreInformationRequired. The latter option loops back to ReviewingRequest once the customer submits the missing specifics—such as a more precise site address, a fault photograph, or an availability window—through a follow-up reply. After a quote is prepared and sent (QuoteSent), the process enters AwaitingCustomerDecision, mirroring the QuoteReceived state in the Customer Job Booking workflow. It then branches similarly: acceptance leads to JobReadyForAssignment → SelectingEmployee → JobAssigned, while a decline skips directly to RequestClosed.

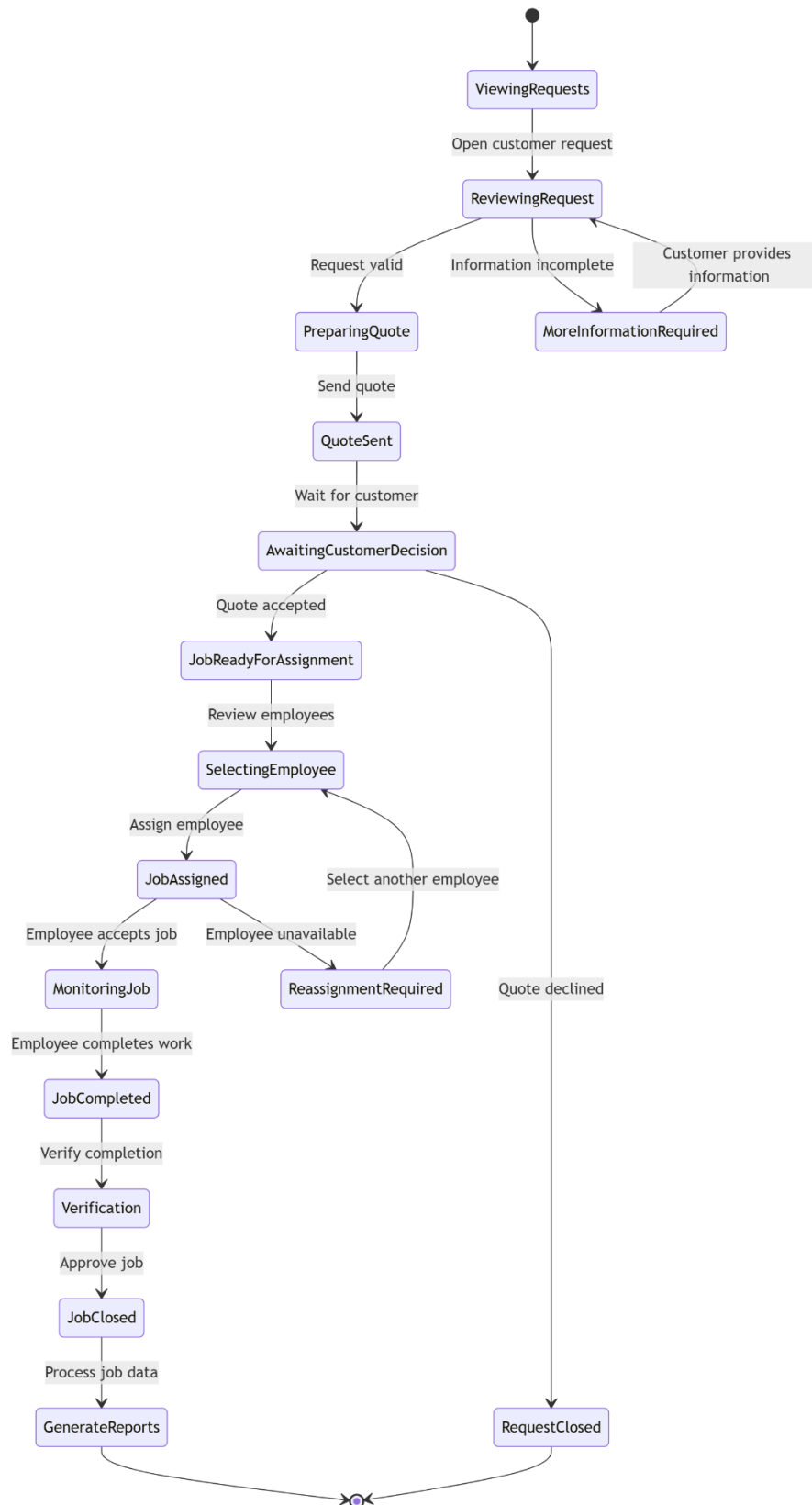


Figure 9: Admin Job Management Lifecycle

**This state diagram illustrates the administrator's management of customer service requests and jobs. The administrator reviews incoming requests, requests additional information when necessary, prepares quotations, and waits for the customer's decision. Accepted quotations proceed to employee selection and job assignment, while declined quotations result in the request being closed. If an assigned employee becomes unavailable, the job can be reassigned. The administrator then monitors the job, verifies completed work, closes the job, and generates the required reports.**



JobAssigned includes a ReassignmentRequired loop that returns to SelectingEmployee if the chosen electrician is unavailable—either due to a scheduling conflict detected by the Calendar and Scheduling diagram or if the electrician declines the task, as shown in the Admin App sequence diagram. This loop enables the admin to try additional electricians without requiring separate states for each attempt. Once an assignment is confirmed, the workflow proceeds through MonitoringJob, JobCompleted, Verification (the admin’s validation of the work and compliance inspection against required standards, parallel to the Inspection passed condition in the Job Lifecycle diagram), and JobClosed before reaching GenerateReports. This final state allows the admin to access job-level and portfolio-level reporting data once the job’s full lifecycle concludes, linking the diagram to the Admin App’s DataExportLog feature described in Sections 3.1 and 4.4.

Both branches—one ending at GenerateReports after quote acceptance and the other at RequestClosed after decline—are distinct reachable endpoints in UML terms rather than converging on a single shared final pseudostate. However, from a business-rules perspective, they fulfill the same core promise: every enquiry processed by this state machine ultimately reaches a definitively closed status, with none left indefinitely open regardless of outcome. This "no enquiry left hanging" guarantee exemplifies how the state diagram encodes a customer-service quality commitment rather than solely technical logic (Garrett, 2011).

A visitor’s interaction with this diagram begins at WebsiteVisitor, progresses to ContactFormOpened and QueryEntered, and advances to Validation, where the submitted contact details are checked for correctness (a valid email format, a plausible phone number length, a non-empty message field). If validation fails, a ValidationError loop returns the visitor to QueryEntered to correct the problematic field instead of allowing malformed data to proceed. Once validated, the query moves to QuerySubmitted, then QueryStored (the point where a new CONTACT\_QUERY record, Section 4.4, is saved), and AdminNotified, before an administrator reviews it, transitioning the state to QueryReviewed.

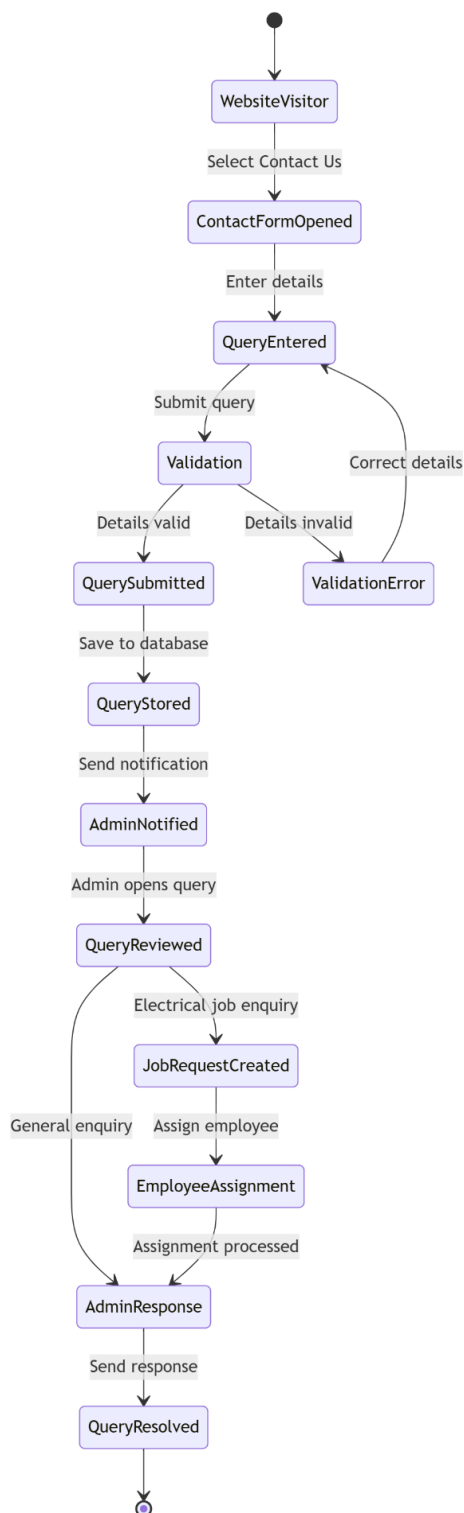


Figure 10: Website Query Processing Lifecycle

**This state diagram depicts the procedure for managing website contact and customer inquiries. When a visitor accesses the Contact Us form, they input their information and submit a request for verification. If the data is incorrect, a validation error occurs, enabling the user to fix their entries; however, if the data is accurate, it is saved and an alert is sent to the administrator. Upon reviewing the submission, the administrator categorizes it as either a general inquiry or an electrical job inquiry. General inquiries are addressed immediately before a reply is issued and the matter is closed, whereas job-related inquiries may lead to the creation of a job request and the designation of a staff member.**

QueryReviewed serves as the diagram's key decision point, directing the query down one of two distinct paths without requiring the visitor to categorize their message at submission. A purely informational enquiry (such as a pricing question or service area inquiry) proceeds directly to AdminResponse. A request that constitutes a genuine electrical work enquiry moves through JobRequestCreated—where `ContactQuery.convertToJob()` (Section 3.2) is executed, transforming a `CONTACT_QUERY` entry into a `JOB` entry and transferring control to the Job Lifecycle state diagram (Diagram 1)—and EmployeeAssignment before also arriving at AdminResponse (since even job-related queries require an acknowledgment response) and finally reaching QueryResolved.

This dual-path convergence illustrates a critical aspect of product design: a user submitting a simple "contact us" form on the Trite Solar/PSG Electrical and Cables website need not determine whether their message is "just a question" or "a job request"—the admin makes that distinction internally, and the state machine supports either outcome from the same starting point. This design choice reduces the initial contact barrier for potential customers, aligning with the client's stated goal for HYDRA (Section 1.3) to replace an inconsistent, phone-and-WhatsApp-based enquiry process with a unified, structured digital entry point.

The lifecycle begins at RewardsAccountActive (the state of a `REWARDS_ACCOUNT` record, Section 4.4, from the moment it is created for a new customer). An eligible activity—such as completing a job, referring a new customer, or any other rewards-earning action defined by the loyalty program—advances the account through `EligibleActivity`, `PointsCalculated`, and `PointsAdded` to `RewardsAvailable`, corresponding to the creation of a new `REWARDS_TRANSACTION` record (Section 4.4) and an increment to `REWARDS_ACCOUNT.points_balance`.

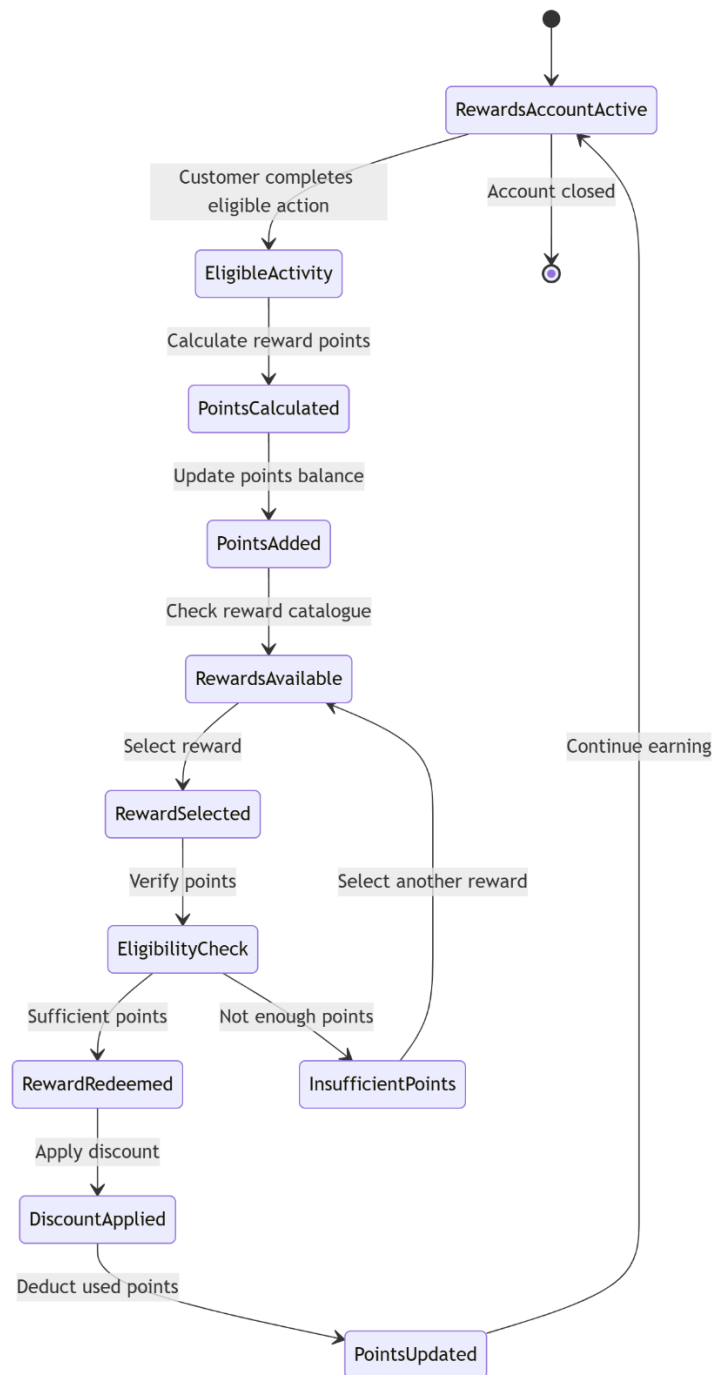


Figure 11: Customer Rewards and Discounts Lifecycle

*The provided state diagram depicts the procedure for earning and redeeming customer rewards and points. Upon the completion of a qualifying action, the system determines the correct amount of reward points and updates the user's balance accordingly. Users are able to browse available rewards, choose a specific item, and verify if they possess enough points for the selection. Provided the account has enough points, the reward is claimed, the corresponding discount is issued, and the points are subtracted from the total. Should the user lack the necessary points, they may opt for a different reward or proceed to accumulate more points.*

From **RewardsAvailable**, selecting a reward (**RewardSelected**) triggers an **EligibilityCheck** before any redemption occurs—the diagram explicitly prevents a direct transition from "reward selected" to "reward redeemed" without this verification step. Two paths emerge: a successful check (**Sufficient points**) proceeds through **RewardRedeemed** → **DiscountApplied** (where a **DISCOUNT/CUSTOMER\_DISCOUNT** relationship,

Section 4.4, is established) → PointsUpdated, looping back to RewardsAccountActive. An unsuccessful check (Insufficient points) returns directly to RewardsAvailable, allowing the customer to choose a different reward. Modeling EligibilityCheck as a distinct, labeled state—rather than an implicit branch within the redemption transition—ensures the customer receives clear, specific feedback about the outcome, preventing the frustration often caused by silent or ambiguous system failures (Garrett, 2011).

The account does not have a mandated terminal state in normal operation; it only reaches an end condition if manually closed by an admin (such as on customer request or account deactivation), reflecting the intention for rewards accounts to persist and be reused throughout the customer’s ongoing relationship with the business.

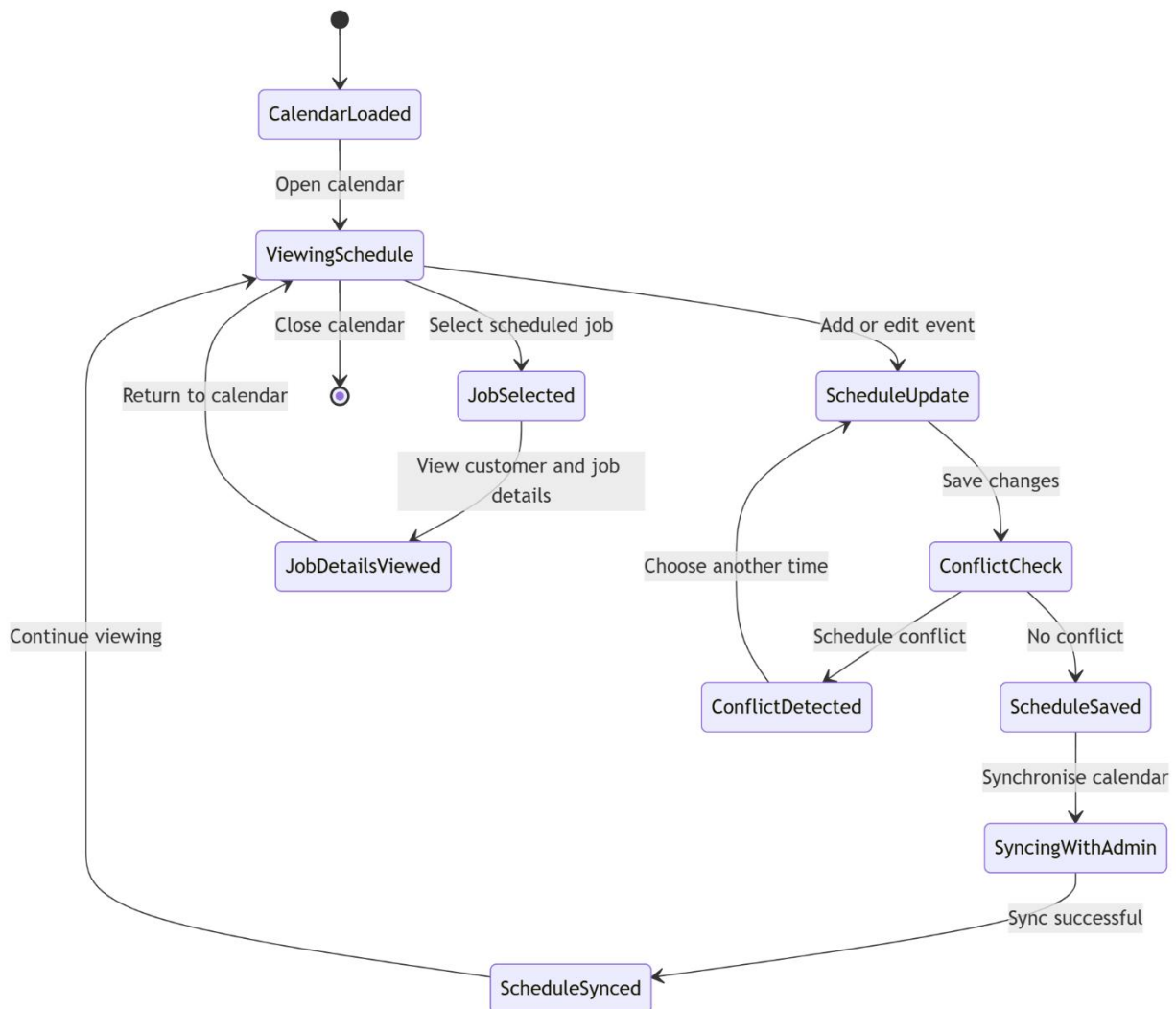


Figure 12: Employee Calendar and Scheduling Lifecycle

Starting from **CalendarLoaded**, an electrician transitions to **ViewingSchedule**, where two distinct actions can occur: examining the specifics of a currently scheduled task (**JobSelected** → **JobDetailsViewed**, a passive route that returns directly to **ViewingSchedule**) or making a schedule modification (**ScheduleUpdate**), including inserting a new event, altering an existing one, or submitting a leave request (connecting this diagram to the **LEAVE\_REQUEST** entity described in Section 4.4).

Every schedule adjustment must go through a **ConflictCheck** phase prior to being finalised, replicating the **EligibilityCheck** approach used in the Rewards diagram and applying it for a similar purpose: preventing any scheduling overlap (an electrician already assigned to another task or a leave request overlapping an existing

assignment) from being saved without notice. When a conflict is found (ConflictDetected), the electrician returns to ScheduleUpdate to pick a new time or resolve the overlap; if no conflict exists, the process moves to ScheduleSaved, then SyncingWithAdmin (a phase where the update is sent to the shared EMPLOYEE\_SCHEDULE table, as noted in Section 4.4, ensuring the Admin App's view of the electrician's calendar updates instantly without manual refresh), and finally ScheduleSynced, after which the electrician returns to ViewingSchedule to see the updated schedule. This conflict-detection mechanism is what stops the same electrician from being assigned to overlapping jobs, which in a field-services company with fewer qualified electricians than job requests could lead to missed appointments and damage PSG Electrical and Cables/Trite Solar's reputation, making it one of the most operationally significant guard conditions in the set despite not being tied to a specific legal requirement like the Job Lifecycle's compliance safeguard.

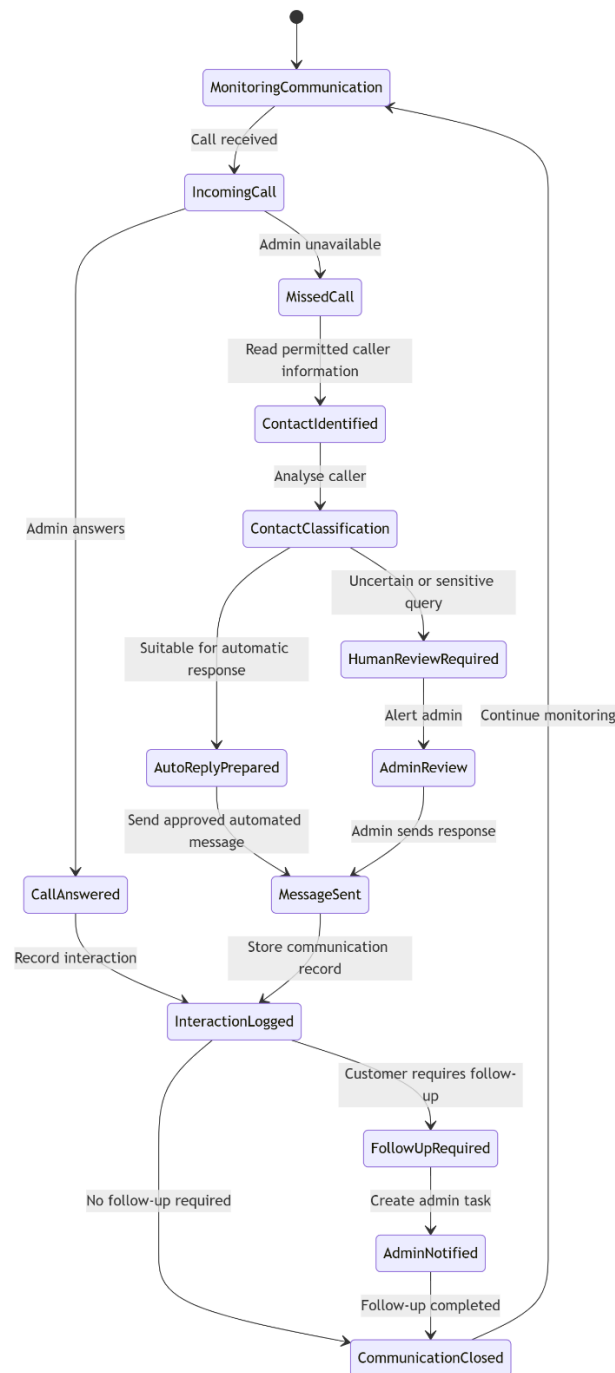


Figure 13: AI Missed Call / Message Handling Lifecycle

The system remains idle in `MonitoringCommunication` until an incoming call arrives (`IncomingCall`). If an admin is available and answers immediately, the process skips directly to `CallAnswered` and concludes without triggering any automated steps, since a human has already handled the call personally. If the call goes unanswered, it becomes a `MissedCall`, and the caller is identified (`ContactIdentified`) using only the phone system's basic caller details (caller ID/number), cross-referenced where possible with an existing `CONTACT` entry (Section 4.4) — this diagram intentionally avoids any additional data lookups beyond what is essential for routing a response, aligning with the data-minimisation principle outlined in Section 6.4 (POPIA Compliance).

The identified contact is then sorted (`ContactClassification`) into one of two handling routes. Routine, clear-cut inquiries (recognisable patterns the AI system confidently processes, such as a caller with an open job likely seeking a status update) move automatically through `AutoReplyPrepared` → `MessageSent`. Any scenario the classification cannot confidently handle—an unknown contact, unclear intent, or a query type the system was not trained to auto-respond to—goes through `HumanReviewRequired` and `AdminReview`, with the reply only sent after a human administrator has approved or drafted it. Both paths merge at `InteractionLogged`, ensuring every missed-call interaction—whether fully automated or requiring human input—generates the same audit record in the `AI_MESSAGE_LOG` table (Section 4.4, Section 6.7). From `InteractionLogged`, the interaction either ends immediately or, if the reply indicates further action (for example, the caller needs a formal quote), proceeds through `FollowUpRequired` and `AdminNotified` before closing.

This design—escalating uncertain queries to a human while logging all interactions uniformly—serves as a deliberate human-in-the-loop safeguard against an automated system responding incorrectly or making assumptions in a customer-facing channel it does not fully understand. It reflects widely accepted best practices for deploying AI or automated decision-making in settings involving personal data and aligns with the accountability, transparency, and human oversight requirements under South African law (Republic of South Africa, 2013; Information Regulator (South Africa), 2021; OWASP Foundation, 2021).



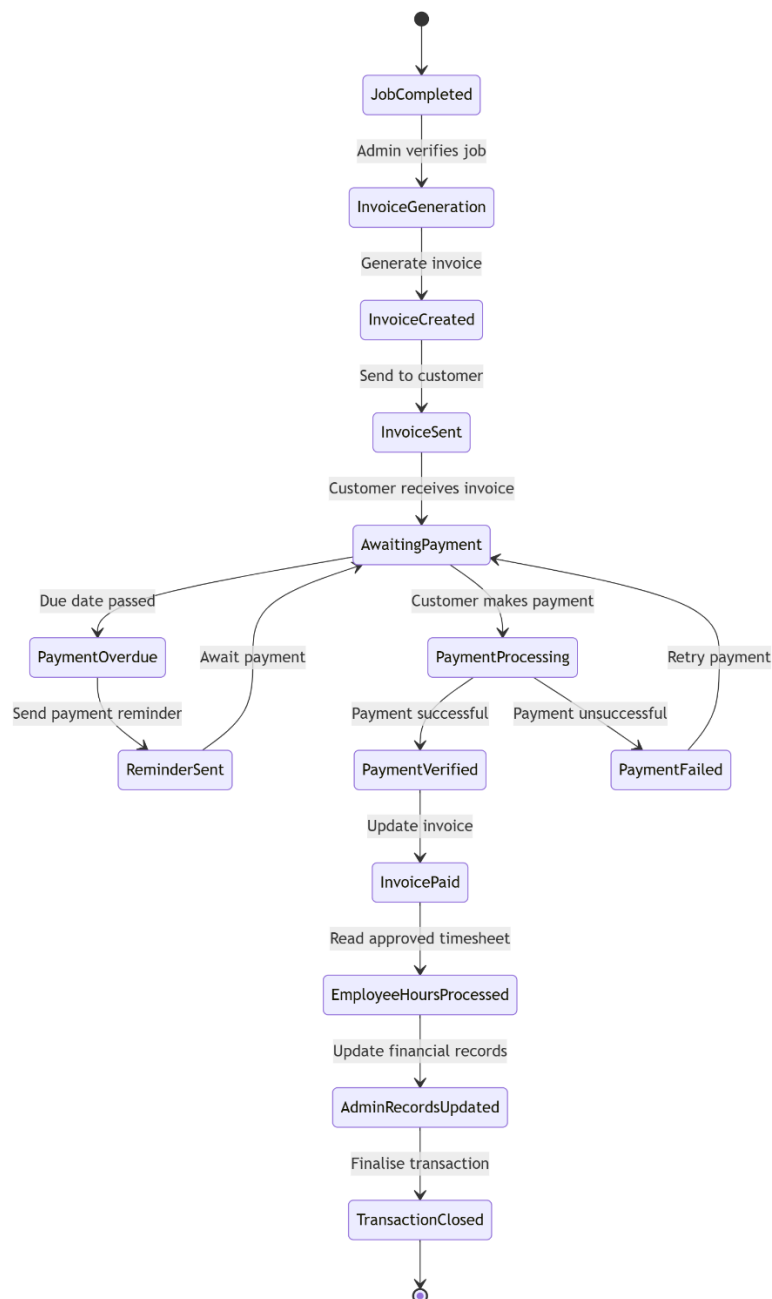


Figure 14: Payment and Invoicing Process Lifecycle

This final state diagram is intentionally the most comprehensive of the eleven, as it is the only one that demonstrates how an event in one operational area (a customer payment, a financial transaction) triggers a downstream consequence in a separate operational domain (payroll processing, a human resources activity)—a cross-domain link that is simple to overlook when each process is designed independently, and this diagram exists explicitly to highlight that dependency.

The flow begins at JobCompleted (transitioning directly from the terminal states of the Job Lifecycle diagram) and moves through InvoiceGeneration → InvoiceCreated → InvoiceSent → AwaitingPayment, covering the same ground as the Invoice Lifecycle diagram (Diagram 2) but from the perspective of the broader business workflow rather than just the Invoice object. From AwaitingPayment, a customer payment activates PaymentProcessing, which resolves to either PaymentVerified (the payment gateway or bank confirms success) or PaymentFailed (the transaction was declined or rejected, allowing a Retry transition back to AwaitingPayment so the customer can attempt payment again via the same or another method);

alternatively, if the due date passes without payment, the flow shifts to `PaymentOverdue` and `ReminderSent`, after which it returns to `AwaitingPayment` for the remaining balance.

Once payment is confirmed, the invoice reaches `InvoicePaid`, but the state machine does not terminate there. It continues into `EmployeeHoursProcessed` and `AdminRecordsUpdated`, explicitly showing that invoice settlement is the event that releases the relevant `TIMESHEET` data (already recorded by the electrician days or weeks prior) into `PAYROLL` processing (Section 4.4), before the transaction ultimately closes at `TransactionClosed`. This sequence—timesheet data being logged early while payroll processing only finalises after invoice settlement—embodies a deliberate cash-flow-aware rule agreed with the client: labour hours are recorded immediately for accuracy, but the client's payroll for that job's work is only processed once the client has paid for the job, a standard and financially prudent practice in project-based service companies (Kerzner, 2017).

## Sequence diagrams

While the first eleven diagrams respond to "what states can this object assume", the eleven sequence-diagram components below detail "what exact order of messages between objects produces one specific scenario". Each sequence diagram lists its participating actors and objects along the top as lifelines, displays time flowing from top to bottom, and employs UML combined fragments — `alt` (mutually exclusive alternatives), `opt` (an optional block that may or may not execute) and, where a step repeats, `loop` — to represent branching and repetition without requiring a separate diagram for every possible scenario path (Object Management Group, 2017; Fowler, 2018).

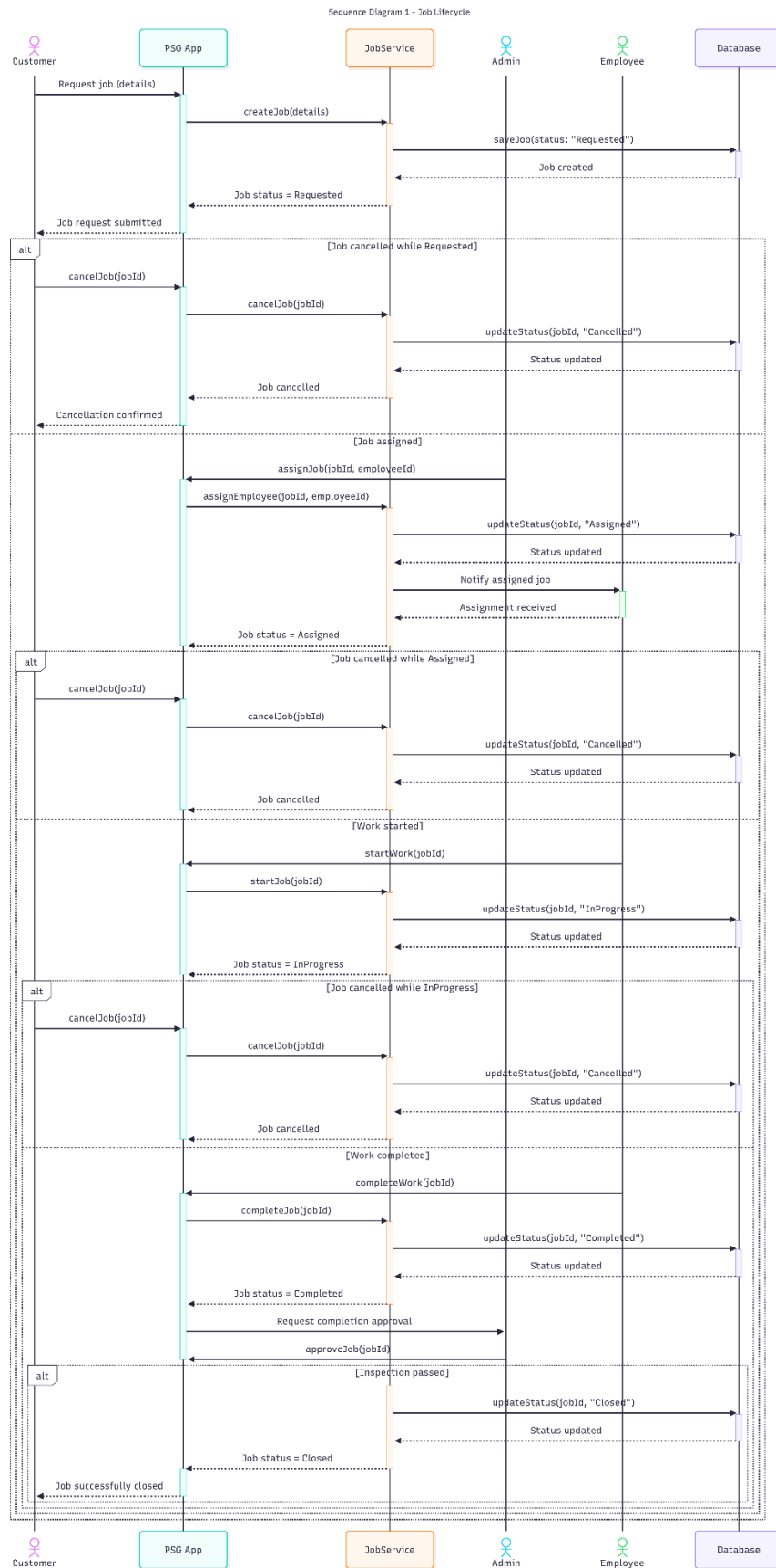


Figure 15: Job Lifecycle Sequence Diagram

Lifelines: Customer, PSG App, JobService, Admin, Employee, Database. The scenario begins when the customer submits a job request through the app's interface, which forwards a createJob(details) call to

JobService; JobService stores a new entry via the Database lifeline with status = "Requested" and sends a confirmation back to the customer, precisely mirroring the initial transition of the Job Lifecycle state diagram. Three separate alt combined fragments appear at different points in the flow, each representing the customer's ability to trigger cancelJob() at that stage of the job's progress; each cancellation path updates Database to status = "Cancelled" and immediately terminates that interaction thread, which corresponds to the three converging Cancel job transitions on the state diagram noted earlier — using nested alt fragments at multiple points in an otherwise linear sequence is the standard UML approach for showing that an alternative path remains open repeatedly rather than just once (Object Management Group, 2017).

The primary path continues with the admin assigning an electrician (assignJob()) on JobService, which in turn invokes assignEmployee(), updates status = "Assigned" in the Database, and sends a notification to Employee), the employee commencing work (startWork() → startJob(), status = "InProgress") and completing it (completeWork() → completeJob(), status = "Completed"), followed by an admin approval step (approveJob()) controlled by a final alt fragment testing the same Inspection passed guard condition found on the state diagram, before the job reaches status = "Closed" and a final confirmation message is returned to the customer. All message names in this diagram (createJob, assignJob, startWork, completeWork, approveJob) align one-to-one with methods already defined on the Job and JobService classes in the Design Class Diagram (Section 3.2), ensuring the traceability this entire set of diagrams is designed to provide.

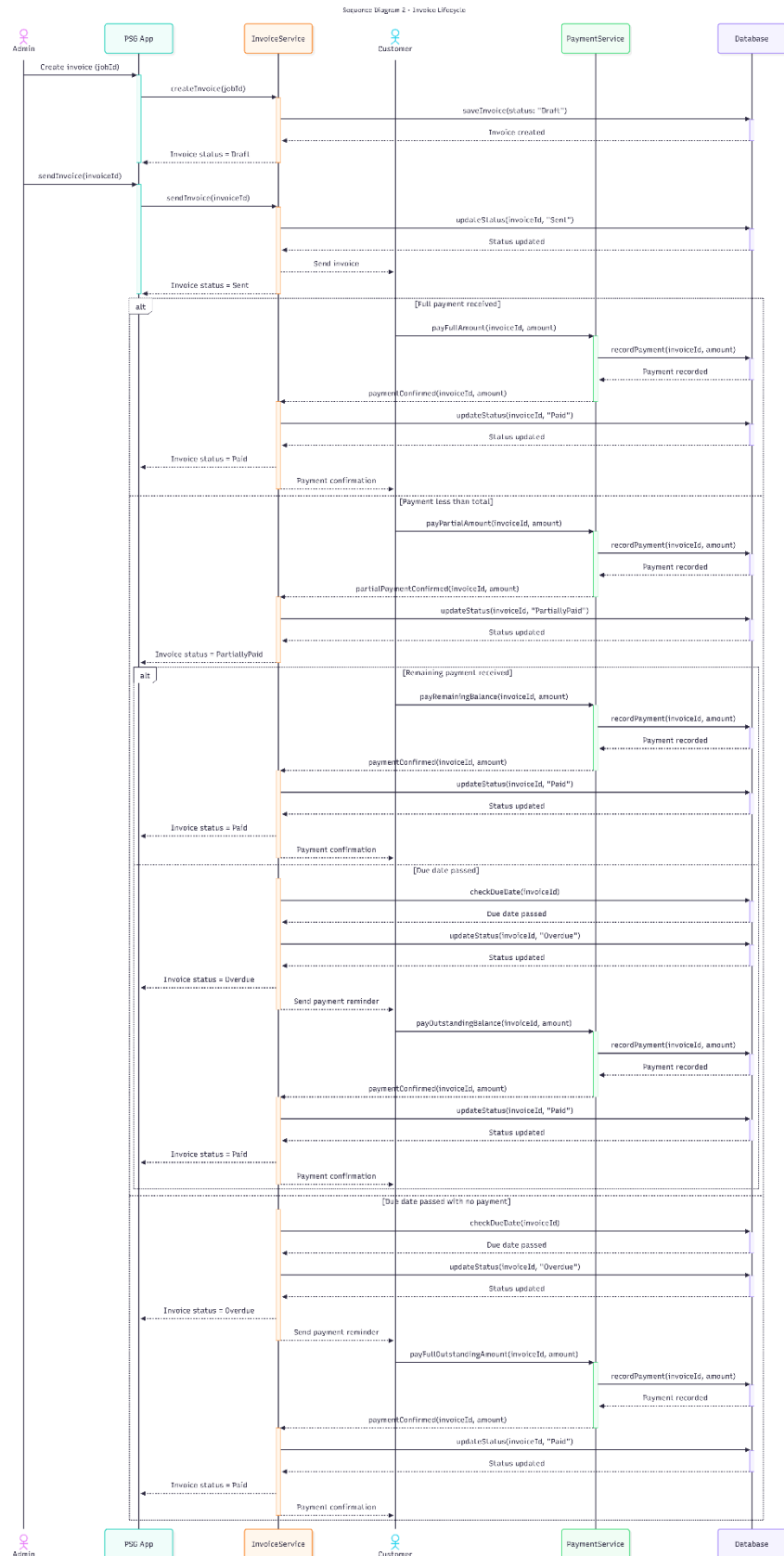
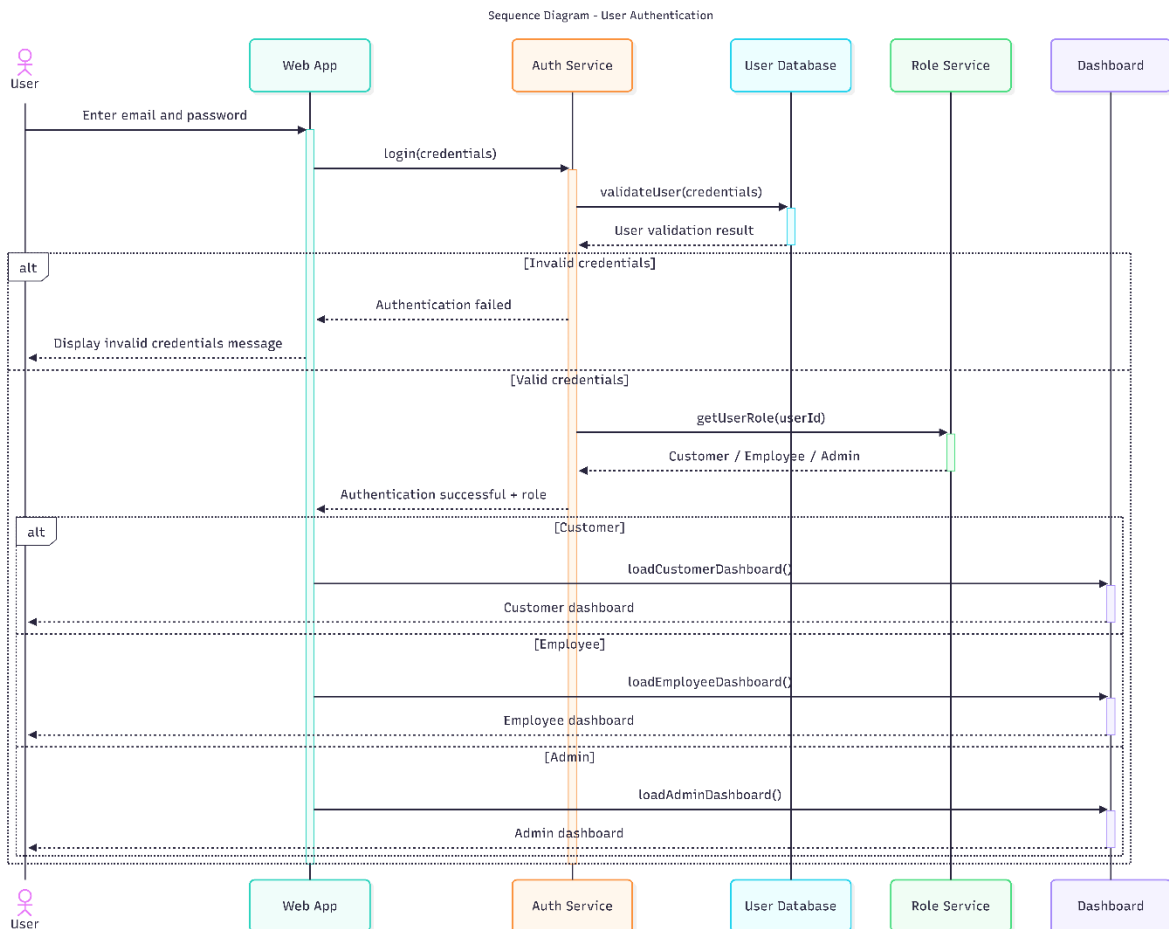


Figure 16: Invoice Lifecycle Sequence Diagram

Lifelines: Admin, PSG App, InvoiceService, Customer, PaymentService, Database. The admin generates and dispatches an invoice (`createInvoice()` → status = "Draft"; `sendInvoice()` → status = "Sent"), after which a single top-level alt fragment encompasses all five payment scenarios outlined in the Invoice Lifecycle state

diagram, with nested fragments as needed: full payment received (`payFullAmount()`, `PaymentService` logs the payment, status = "Paid"); partial payment received (`payPartialAmount()`, status = "PartiallyPaid"), inside which a further nested alt fragment covers either the remaining balance being settled later (→ "Paid") or the invoice's due date expiring first (→ "Overdue", a reminder message, and eventual full payment → "Paid"); and lastly the case where the due date passes with no payment recorded whatsoever (→ "Overdue" directly from "Sent", a reminder, then the full outstanding amount being paid → "Paid"). Every possible branch in this diagram concludes at status = "Paid", matching the single terminal state in the Invoice Lifecycle state diagram, and this alignment was verified during the review stage described in Section 4.1.1: a sequence diagram ending in a state its paired state diagram does not allow would signal a genuine specification error in one of the artefacts, and no such inconsistency remains in the submitted diagram set.



*Figure 17: User Authentication Sequence Diagram*

Lifelines: User, Web App, Auth Service, User Database, Role Service, Dashboard. The user provides credentials, which the Web App transmits as `login(credentials)` to Auth Service, which checks them against User Database. A primary alt fragment splits invalid credentials — returning an "Authentication failed" response to the user, aligning with the state diagram's `AuthenticationFailed` state — from valid credentials, which prompt a `getUserRole()` call to Role Service. A second, embedded alt fragment then loads exactly one of three dashboards (Customer, Employee, Admin) based on the returned role, providing a single, uniform authentication flow reused identically across all three HYDRA client applications. This shared sequence is intentionally referenced, rather than duplicated, at the beginning of the two role-specific sequence diagrams below (Customer App job booking and Admin App job management), mirroring how the Authentication state diagram is conceptually referenced by every role-specific state diagram that follows.

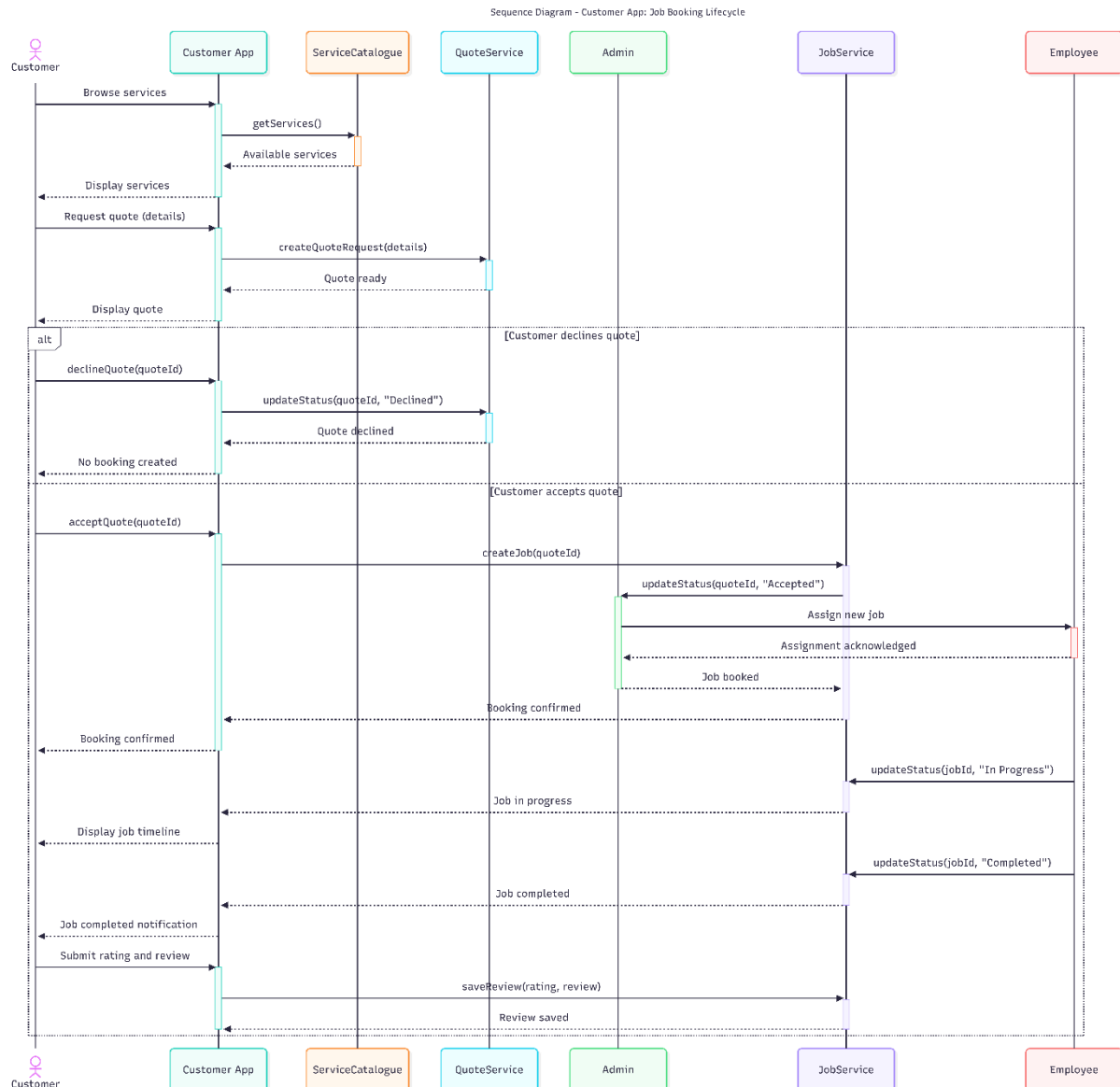
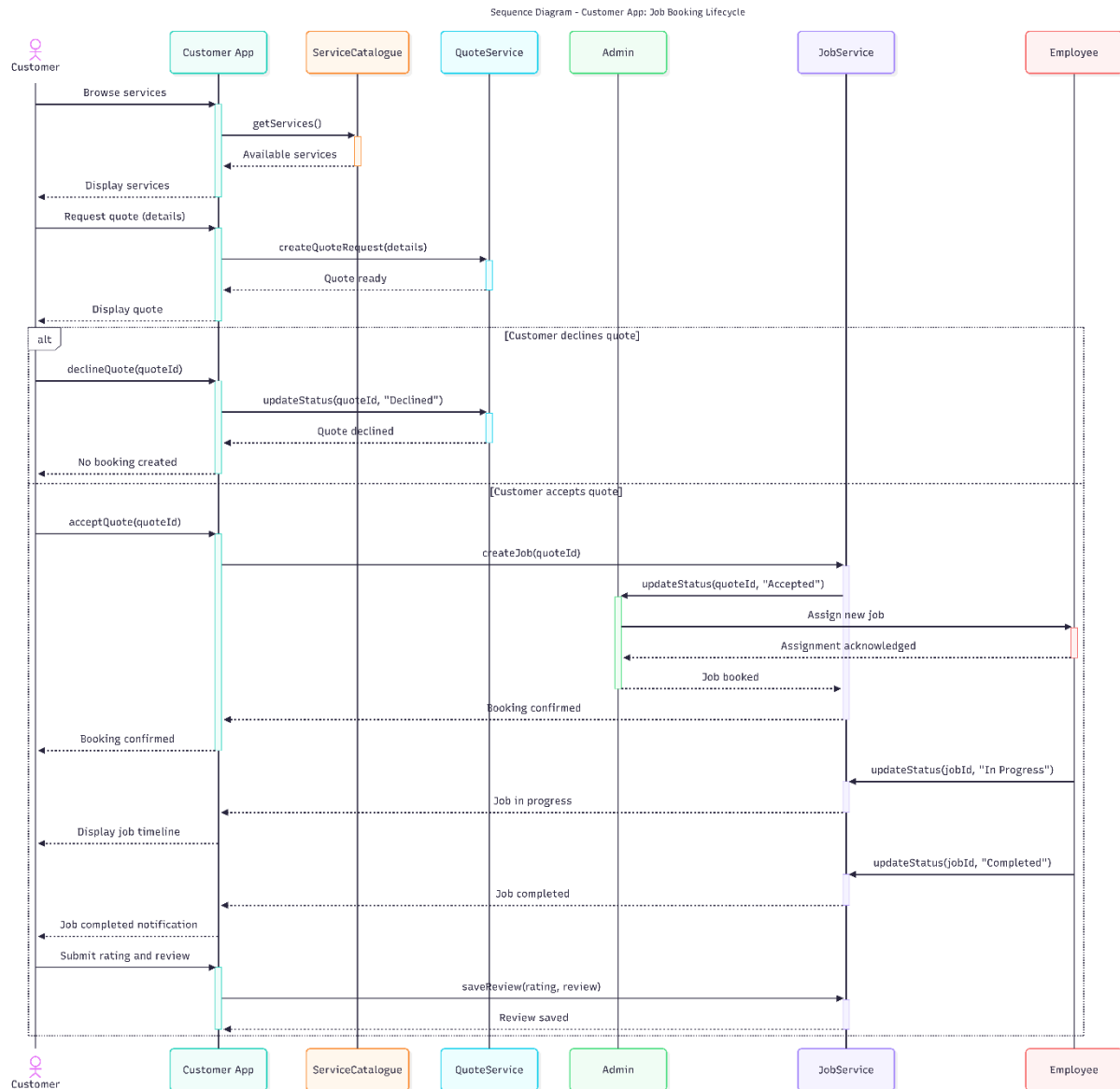


Figure 18: Customer App Job Booking Sequence Diagram

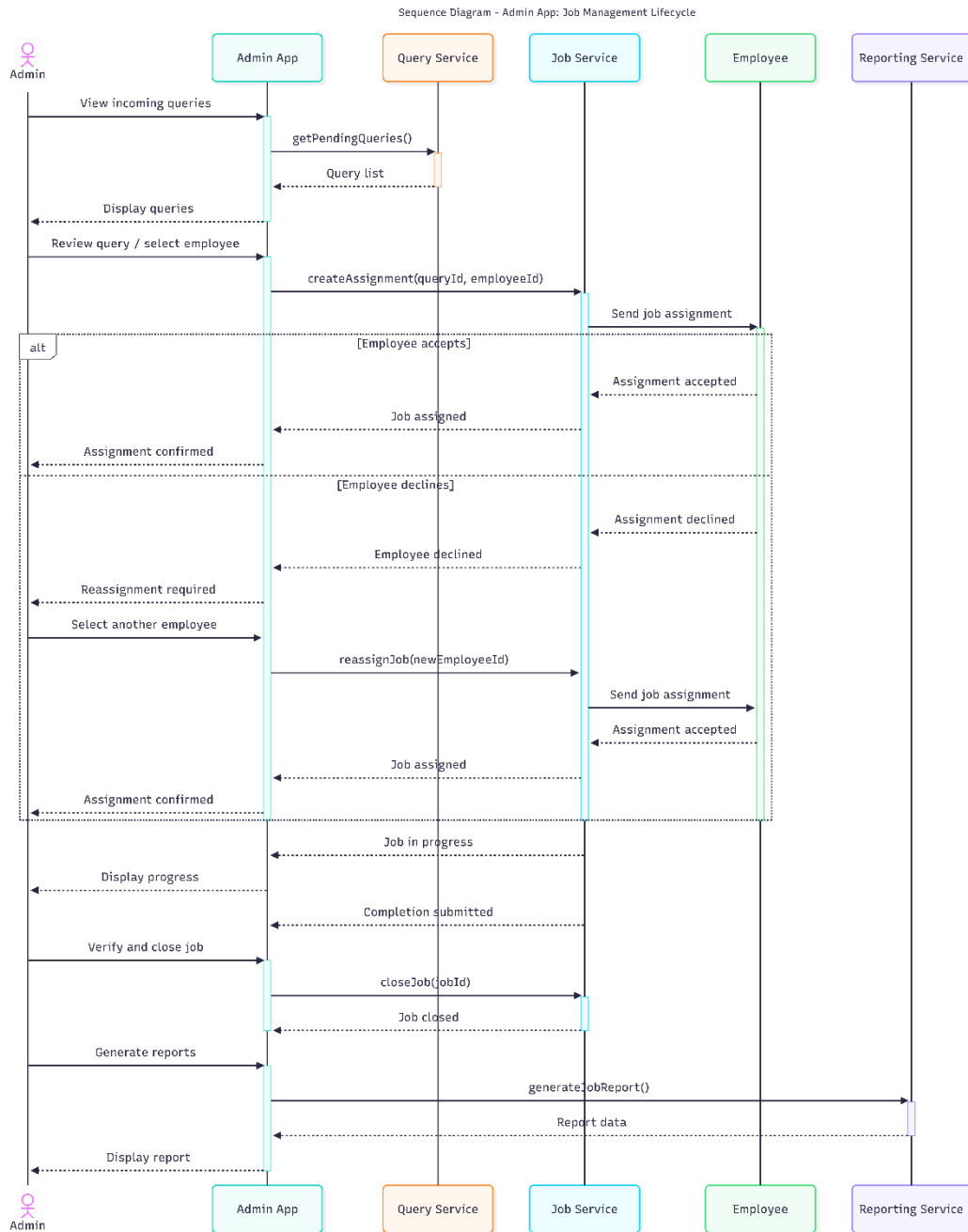
Lifelines: Customer, Customer App, ServiceCatalogue, QuoteService, Admin, JobService, Employee. The customer reviews services (`getServices()`) against `ServiceCatalogue`, backed by the `SERVICE_TYPE` table, Section 4.4), requests a quote for a selected service (`createQuoteRequest()`), and receives the resulting quote once `QuoteService` responds. An alt fragment distinguishes the decline path (`declineQuote()` → `QuoteService` sets status = "Declined", no booking is generated, matching the `QuoteDeclined` state on the paired state diagram) from the accept path (`acceptQuote()` → `createJob(quoteId)`), which prompts `JobService` to notify the admin, assign an employee, and confirm the booking back to the customer. The rest of the sequence models job-status notifications ("In Progress", "Completed") being pushed to the customer's in-app timeline in near real time as the electrician's actions (captured in the Employee App sequence diagram below) update the shared JOB record, ending with the customer submitting a rating and review (`saveReview()`) — the sequence-level implementation of the state diagram's optional Review submitted transition, and the concrete execution behind the Customer Journey's implied "close the loop" step following "View inspection report" (Section 3.3).





*Figure 19: Employee App Job Execution Sequence Diagram*

Lifelines: Employee, Employee App, JobService, JobCheckinService, Admin, Database. The electrician opens their list of assigned tasks (`getAssignedJobs()`), picks a job, and travels to the location. Upon arrival, the app activates the QR scanner and invokes `scanCheckin(code)` against `JobCheckinService`. A top-level alt fragment divides a successful scan—where a new `JOB_CHECKIN` row (`qr_code`, `scan_timestamp`, `gps_location`, as defined in Section 4.4) is written and the job's status updates to "InProgress" via `JobService`—from a failed scan. The failed scan displays a retry option to the electrician and, within an additional nested alt, offers an escalation path to Admin for manual arrival confirmation, replicating the `ScanFailed / AdminAssistance` branch from the paired state diagram precisely. The process proceeds with the electrician recording materials used for the job (`logMaterialUsage()`, linked to the `JOB_MATERIAL` junction table, Section 4.4), finishing the work (`completeWork()`, status = "Completed"), and finally submitting a timesheet entry (`submitTimesheet()`) that stores a new `TIMESHEET` row in Database. The last message in this sequence also represents the first step later in the Payment and Invoicing sequence where it becomes eligible for payroll processing once the related invoice is settled.



*Figure 20: Admin App Job Management Sequence Diagram*

Lifelines: Admin, Admin App, Query Service, Job Service, Employee, Reporting Service. The administrator checks pending requests (`getPendingQueries()`) and selects an electrician for a task, initiating `createAssignment()`. `JobService` forwards this to the selected `Employee`. A top-level alt fragment separates the scenario where the employee accepts—resulting in confirmation back to the admin—from the scenario where the employee declines, which loops the admin back to selecting another electrician (`reassignJob()`) before confirmation, directly reflecting the `ReassignmentRequired` loop in the paired state diagram (Diagram 6, above). The sequence continues as status updates on job progress are received from the `Employee App`, followed by a `closeJob()` call once the admin confirms both completion and a successful compliance check, and concludes with generating a report (`generateJobReport()`) via `Reporting Service`, aligning with the `GenerateReports` state that ends the Admin Job Management state diagram.

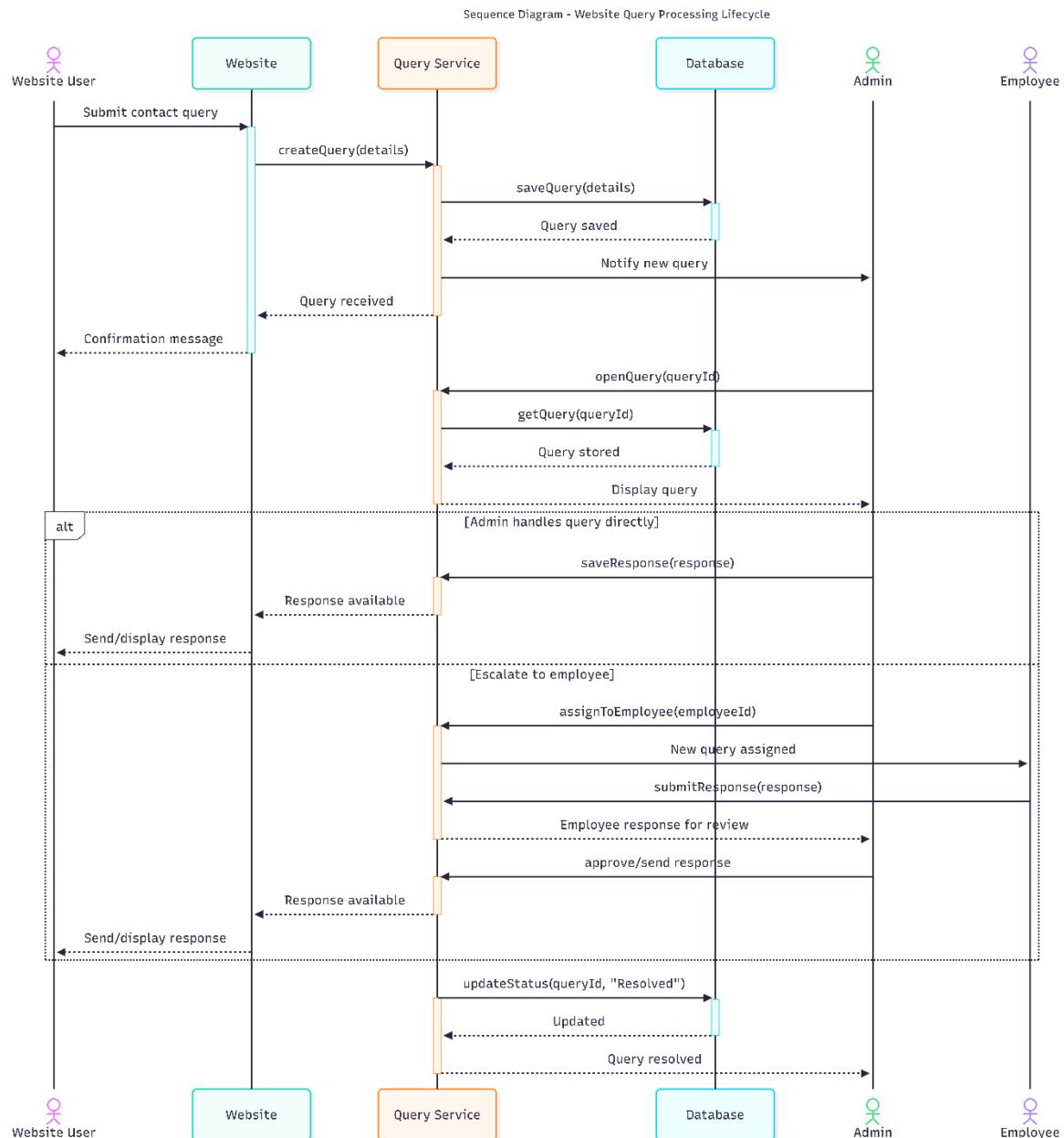
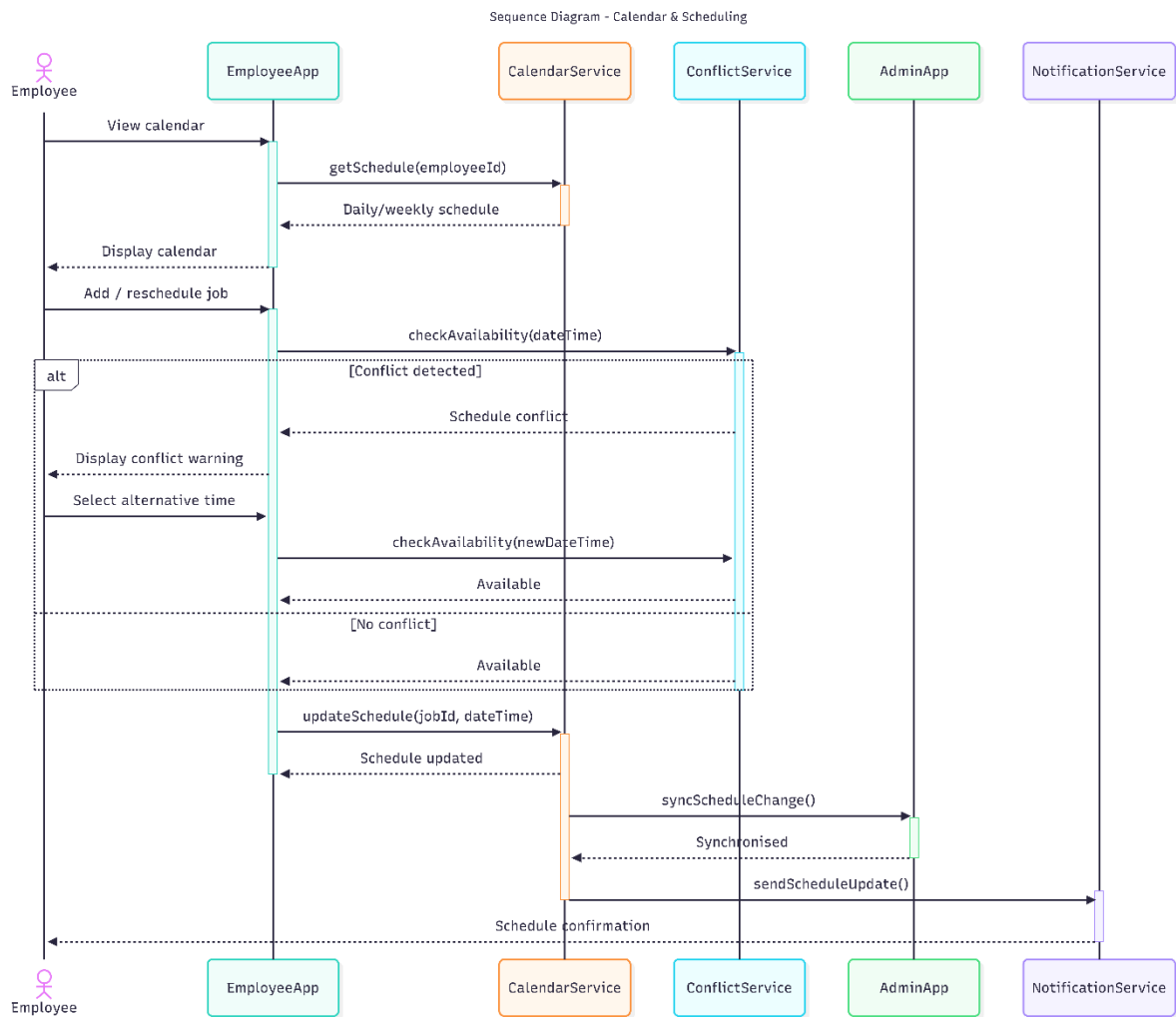


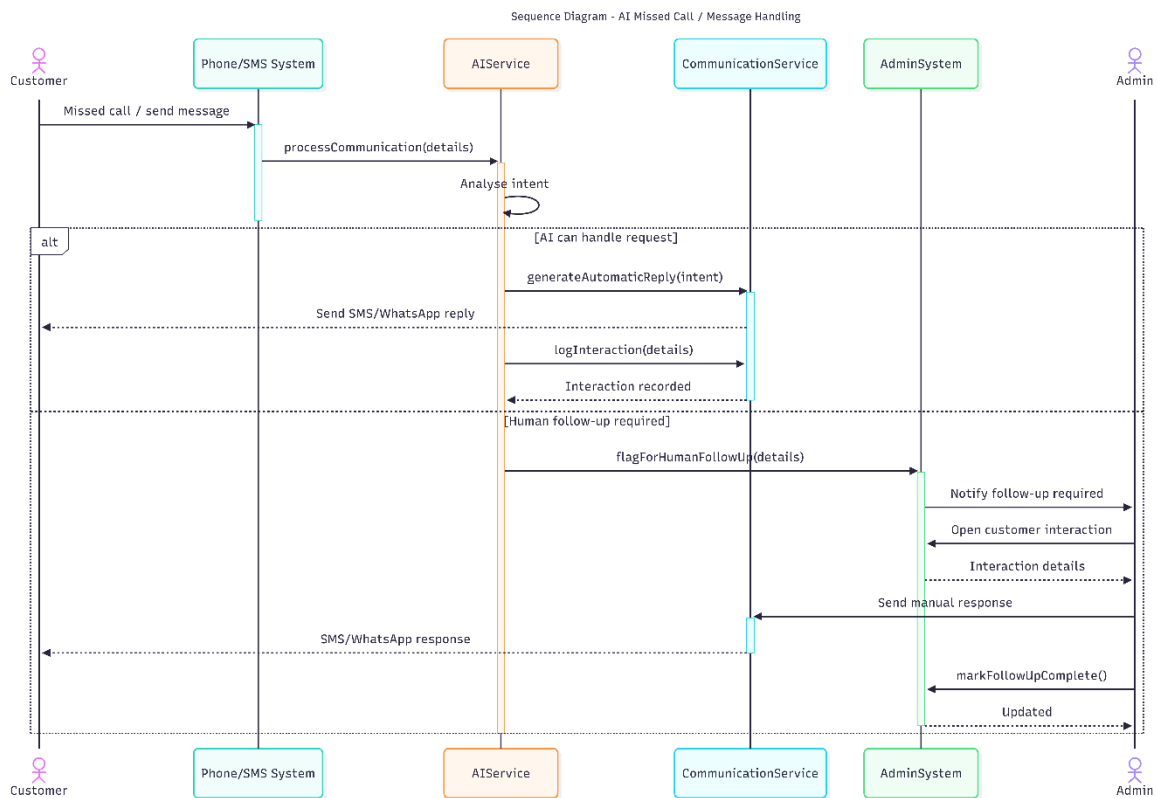
Figure 21: Website Query Processing Sequence Diagram

Lifelines: Website User, Website, Query Service, Database, Admin, Employee. A visitor submits a query (`createQuery()`), saved to Database, and the admin is notified). When the admin reviews the query, a top-level alt fragment splits into two response routes matching the `QueryReviewed` decision point in the paired state diagram: the admin provides an answer directly (`saveResponse()`) or escalates it to an employee (`assignToEmployee()`), whose response must be approved by the admin (`approve()`) before being sent to the visitor (`submitResponse()`). Both routes converge on the query being marked status = "Resolved" in Database, matching the single `QueryResolved` final state described earlier.



*Figure 22: Calendar and Scheduling Sequence Diagram*

Lifelines: Employee, EmployeeApp, CalendarService, ConflictService, AdminApp, NotificationService. The employee accesses their calendar through `getSchedule()` and submits a request to add or modify an event, which initiates `checkAvailability()` with ConflictService. An alt fragment distinguishes between a detected conflict—prompting the employee to select another time and reattempt availability checks, repeating until an open slot is found—versus an immediate open slot; both paths merge at `updateSchedule()`, which is then synchronized with the Admin App via a specific `syncScheduleChange()` message and confirmed back to the employee through NotificationService. This `syncScheduleChange()` message represents the exact point in the system where it formally ensures that an electrician’s personal calendar and the Admin App’s display of that calendar will never diverge without notice, a critical safeguard for preventing double bookings as outlined in the associated state diagram (Diagram 9, above).



*Figure 23: AI Missed Call / Message Handling Sequence Diagram*

Lifelines: Customer, Phone/SMS System, AIService, CommunicationService, AdminSystem, Admin. A missed call or message is routed to AIService for intent analysis via `analyseIntent()`. An alt fragment splits between interactions the AI can confidently resolve—triggering an automated reply through CommunicationService and logging the exchange—and those requiring human intervention, which are instead directed to AdminSystem, presented to Admin for manual assessment, and closed only after the administrator sends a response and marks the task complete. This sequence directly implements the `HumanReviewRequired` branch from the paired state diagram, confirming that the automated response feature always records an `AI_MESSAGE_LOG` entry and always includes a clear human escalation pathway rather than operating autonomously on customer contact details (Republic of South Africa, 2013; OWASP Foundation, 2021).

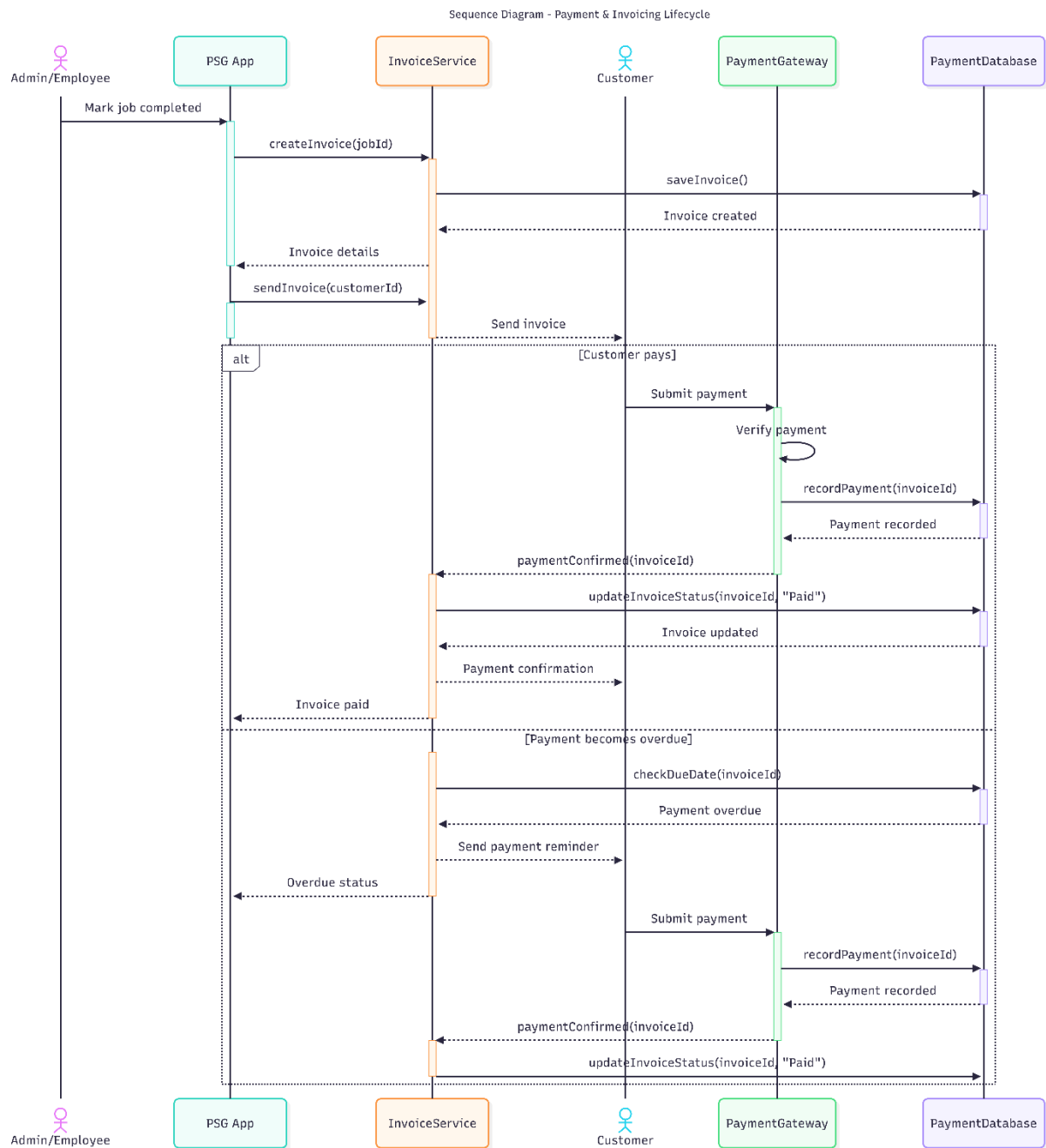


Figure 24: Payment and Invoicing Sequence Diagram (Part 1 — Gateway-Facing Flow)

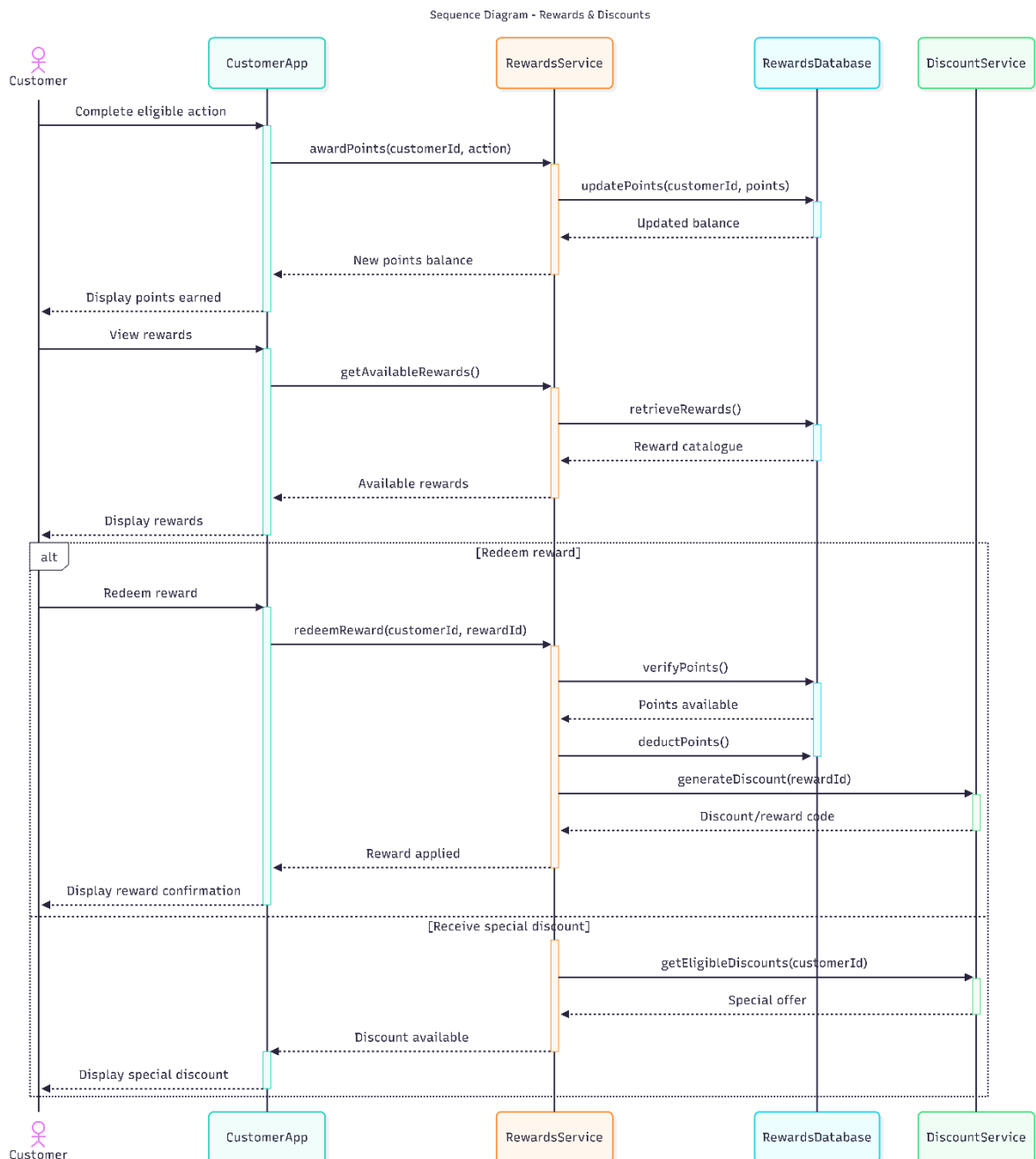


Figure 25: Payment and Invoicing Sequence Diagram (Part 2 — Overdue and Reminder Flow)

Lifelines (across both parts): Admin/Employee, PSG App, InvoiceService, Customer, PaymentGateway, PaymentDatabase. These two connected diagrams, together with Sequence Diagram 4.1.13 (Invoice Lifecycle), illustrate the Draft → Sent → payment → Paid progression from another perspective: that of an external PaymentGateway and a separate PaymentDatabase, instead of the internal InvoiceService/Database framework used in Diagram 4.1.13. Part 1 outlines the straightforward payment route: the customer starts payment via the gateway, the gateway approves or rejects the transaction (alt fragment), and a successful transaction triggers recordPayment() against PaymentDatabase, creating a new PAYMENT entry (Section 4.4) linked to the matching invoice\_id. Part 2 focuses exclusively on the overdue-reminder process: a timed checkDueDate() call (performed by the Scheduled Job Runner mentioned in Section 4.2) locates invoices past their due date without full settlement, alters their status to "Overdue", and dispatches a reminder before returning to wait for payment through the same gateway flow described in Part 1. Dividing this single scenario into two diagram exports, rather than merging it into one overly complex visual, was a clarity choice



consistent with UML guidance that sequence diagrams must stay intelligible at the viewing or printing size they will be used at (Object Management Group, 2017); keeping payment logging within its own distinct PaymentDatabase interaction, rather than integrating it directly into InvoiceService, mirrors the same architectural decision reflected in the ERD and Design Class Diagram to treat Payment as a standalone persistent, independently searchable class rather than a status attribute on Invoice.

## Traceability matrix

The table below lists, for each major stateful class in the HYDRA domain, the state diagram controlling its lifecycle, the sequence diagram(s) demonstrating at least one of its transitions, and the corresponding method(s) on the Design Class Diagram (Section 3.2) that those messages trigger. This matrix is designed so a reviewer or a future developer inheriting the codebase post-handover (Section 9) can confirm in one place that every behavioural element in this report aligns correctly, without needing to manually cross-check eleven separate diagram pairs.

**Table 5: Traceability matrix — state diagrams, sequence diagrams and design class methods**

| Class                  | Governing state diagram                             | Animating sequence diagram(s)                                   | Key Design Class Diagram method(s)                                                                  |
|------------------------|-----------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Job                    | Fig. 4 Job Lifecycle                                | Fig. 15 Job Lifecycle                                           | Job.assignElectrician(),<br>Job.startWork(), Job.completeWork(),<br>Job.closeJob()                  |
| Invoice                | Fig. 5 Invoice Lifecycle                            | Fig. 16 Invoice Lifecycle;<br>Figs. 24–25 Payment and Invoicing | Invoice.generateFromJob(),<br>Invoice.calculateTotal(),<br>Invoice.markPaid()                       |
| Session/User           | Fig. 6 User Authentication                          | Fig. 17 User Authentication<br>(referenced by Figs. 18, 20)     | AuthService.login(),<br>AuthService.issueToken(),<br>RoleService.getUserRole()                      |
| Job (customer view)    | Fig. 7 Customer Job Booking Lifecycle               | Fig. 18 Customer App Job Booking                                | QuoteService.createQuoteRequest(),<br>Job.acceptQuote(),<br>Customer.saveReview()                   |
| Job (employee view)    | Fig. 8 Employee Job Execution Lifecycle             | Fig. 19 Employee App Job Execution                              | JobCheckinService.scanCheckin(),<br>Job.logMaterialUsage(),<br>Timesheet.submit()                   |
| Job (admin view)       | Fig. 9 Admin Job Management Lifecycle               | Fig. 20 Admin App Job Management                                | JobService.createAssignment(),<br>JobService.reassignJob(),<br>ReportingService.generateJobReport() |
| ContactQuery           | Fig. 10 Website Query Processing Lifecycle          | Fig. 21 Website Query Processing                                | ContactQuery.saveResponse(),<br>ContactQuery.convertToJob()                                         |
| RewardsAccount         | Fig. 11 Customer Rewards and Discounts Lifecycle    | (referenced within Fig. 18)                                     | RewardsAccount.calculatePoints(),<br>RewardsAccount.redeemReward()                                  |
| EmployeeSchedule       | Fig. 12 Employee Calendar and Scheduling Lifecycle  | Fig. 22 Calendar and Scheduling                                 | CalendarService.checkAvailability(),<br>CalendarService.updateSchedule()                            |
| Contact / AIMessageLog | Fig. 13 AI Missed Call / Message Handling Lifecycle | Fig. 23 AI Missed Call / Message Handling                       | AIService.analyseIntent(),<br>AIMessageLog.generateMessage(),<br>AIMessageLog.send()                |
| Payment / Payroll      | Fig. 14 Payment and Invoicing Process Lifecycle     | Figs. 16, 24–25                                                 | PaymentService.recordPayment(),<br>Payroll.processFromTimesheet()                                   |

## A note on traceability

Together, the eleven state diagrams and the eleven sequence-diagram artefacts provide a fully developed behavioural model for every major class identified in the Domain and Design Class Diagrams (Job, Invoice, Payment, Quote, ContactQuery, RewardsAccount, EmployeeSchedule, and the missed-call/AI messaging classes), and each method listed in Section 3.2 can be linked back to at least one transition or message shown here. This traceability—from domain concept to design-level method, to the exact sequence of calls that activates it, to the specific screen a human user sees at each step (Section 3.3), and now to a single summary matrix a reader can verify against all eleven diagram pairs in an instant—matches precisely what the module checklist demands when it requires "all critical flows" to be "clearly documented with accurate diagrams," and it serves as a practical, concrete demonstration of the broader software-engineering principle that requirements, analysis, design and behaviour artefacts should remain mutually consistent and mutually verifiable rather than developed separately and reconciled only informally at project completion (Sommerville, 2016; Pressman and Maxim, 2020; Bass, Clements and Kazman, 2021).

## 4.2 Deployment Plan

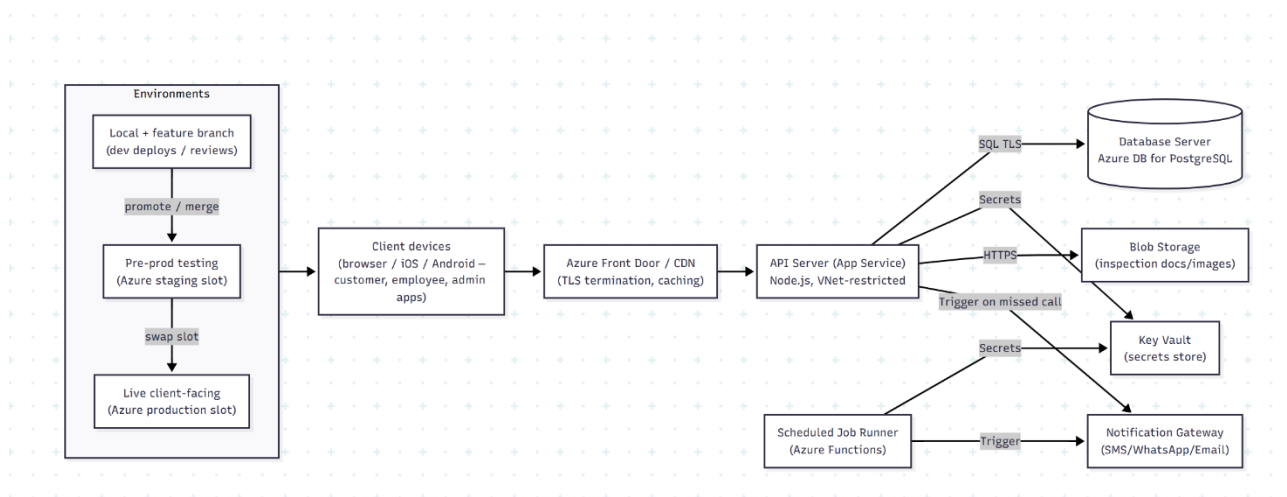


Figure 26: HYDRA Deployment Diagram

### Infrastructure and environment overview

HYDRA operates entirely on Microsoft Azure, structured across three environments—local/feature-branch, pre-production (staging), and live (production)—using Azure App Service deployment slots instead of three separate, independently provisioned infrastructure sets. This approach was selected because deployment slots enable the same, already verified application package to move between environments through an atomic slot swap rather than redeploying new code at each stage. This reduces both ongoing costs for maintaining duplicate infrastructure and the risk of environment-specific configuration errors reaching production undetected (Microsoft, 2023; Microsoft, 2024). This slot-based promotion is a specific cloud-native implementation of the broader immutable artifact principle, where the exact same tested binary or build output is promoted unchanged across environments rather than rebuilt separately for each—an approach linked to more reliable and predictable software releases (Humble and Farley, 2010; Forsgren, Humble and Kim, 2018; Kim et al., 2016). The diagram illustrates this as a linear pipeline: Local + feature branch (developer machines and short-lived pull-request deployments, detailed in Section 7.1) progresses to Pre-production testing (the Azure staging slot, where internal QA and client-facing user acceptance testing

occur), which then advances through a slot swap into the Live client-facing environment (the Azure production slot).

Each environment is purposely configured with an identical application structure—the same App Service plan tier family and matching downstream connections (database, blob storage, key vault, and notification gateway)—differing only in credentials, connection strings, and scaling settings. Maintaining a staging environment structurally identical to production ensures user acceptance testing results are genuinely representative of what clients will experience after promotion, aligning with the production-like staging environment principle recommended in modern continuous-delivery literature (Humble and Farley, 2010; Forsgren, Humble and Kim, 2018).

## Physical and logical nodes

Starting from the production slot, the diagram outlines the following nodes and connecting protocols, matching the physical and logical node structure documented in the module checklist:

Client devices (browsers, iOS, and Android) covering all three HYDRA applications—Customer, Employee, and Admin—connect via HTTPS to

Azure Front Door / CDN, which handles TLS termination and edge caching of static assets before forwarding requests over HTTPS to

API Server (App Service), a VNet-restricted Node.js application serving as the single shared backend for all client apps. This single-backend design, further discussed as the Layered/N-tier architecture pattern in Section 5.2, ensures one server-side codebase to secure, patch, monitor, and scale instead of three separate backends per app—a deliberate departure from a full microservice-per-app architecture deemed unnecessary given HYDRA's current and foreseeable scale. This would have introduced significant operational complexity (inter-service authentication, service discovery, distributed tracing) unjustified by the workload (Newman, 2021; Richards, 2015). The API Server communicates through three distinct downstream channels: SQL over TLS to the Database Server (Azure Database for PostgreSQL, fully justified in Section 4.3); HTTPS to Blob Storage (for inspection documents and images, including photos attached to INSPECTION\_REPORT rows, Section 4.4); and a secrets-retrieval call to Key Vault for database credentials, JWT signing secrets (Section 6.1.3), and third-party API keys (notably payment gateway credentials discussed in Section 6.5).

A separate Scheduled Job Runner (Azure Functions, triggered by a timer) also fetches its secrets from Key Vault independently of the API Server and activates the Notification Gateway (SMS/WhatsApp/Email) on a scheduled basis—for example, running overdue-invoice reminders shown in Sequence Diagram 4.1.22 or sending customer rewards-expiry notices as a promotional period ends—while the API Server triggers the same gateway on demand, synchronously, upon detecting a missed call (the AI Missed Call flow, Diagram 4.1.10/4.1.20).

Dividing the Scheduled Job Runner from the main API Server as its own node, rather than embedding scheduled tasks within the API Server, is a deliberate architectural choice with two key advantages. First, it prevents time-based background tasks (payment reminders, payroll cut-off processing) from being blocked or slowed by traffic spikes on the API Server, and vice versa—a long-running background job cannot degrade the performance seen by an electrician scanning a QR code on-site. Both workloads scale and fail independently. Second, it follows established cloud-architecture guidance that workloads with different load profiles and failure/retry behaviors (an interactive HTTP request awaiting a human response versus a background batch job with no synchronous user) should run on separate compute resources rather than

sharing a single instance, as combining them leads to unpredictable behavior (Erl, Puttini and Mahmood, 2013; Microsoft, 2024; Beyer et al., 2016).

### Network segregation and security

The diagram enforces a clear separation between the public-facing tier and the private data tier. No client device or external actor directly accesses the Database Server, Blob Storage, or Key Vault; all requests route through the VNet-restricted API Server or Scheduled Job Runner, both located inside Azure's private virtual network rather than exposed to the public internet (Erl, Puttini and Mahmood, 2013). This aligns with the cloud security principle of minimizing internet-facing surfaces—in HYDRA's case, only the Azure Front Door endpoint is public—while all other components remain accessible only within the virtual network (Microsoft, 2024; National Institute of Standards and Technology, 2011; Cloud Security Alliance, 2017). All external traffic uses end-to-end HTTPS/TLS, and internal database traffic is encrypted in transit (SQL over TLS), addressing the "protection in transit" requirement discussed further in Section 6.2. This meets the technical and organizational safeguards South African law mandates for organizations processing personal information (Republic of South Africa, 2013). Key Vault access is further secured using Azure's managed-identity mechanism instead of static credentials, meaning the API Server and Scheduled Job Runner authenticate through identities issued and rotated by Azure, eliminating the risk of credential leakage if database connection strings or signing keys were embedded in configuration files or source control (Microsoft, 2024).

### Monitoring, observability and operational visibility

Given HYDRA's production environment supports critical, revenue-generating workflows for PSG Electrical and Cables/Trite Solar, the deployment includes centralized logging and metrics collection (via Azure Application Insights, integrated at the App Service and Azure Functions level) tracking request latency, error rates, and the health of each downstream dependency in the diagram (database, blob storage, key vault, notification gateway, payment gateway). This provides the team—and eventually the client's designated point of contact post-handover (Section 9)—with a single dashboard to verify system behavior rather than inferring health from user-reported issues. Observability follows the "four golden signals" framework—latency, traffic, errors, and saturation—a widely adopted method for balancing monitoring depth without over-instrumentation (Beyer et al., 2016). Alert thresholds are set conservatively for a system of HYDRA's scale, such as triggering on sustained HTTP 5xx error increases or Scheduled Job Runner timer-triggered failures, consistent with recommendations to reserve alerts for conditions requiring human intervention to avoid alert fatigue (Beyer et al., 2016).

### Rollout strategy and rationale

HYDRA's rollout follows a phased, slot-based approach instead of a single "big bang" switch from the client's existing manual, paper-and-WhatsApp-based process. Code is developed and reviewed against the local/feature-branch environment (see Section 7.1 for the branching and pull-request workflow), merged, and automatically deployed to the Azure staging slot for internal QA and client-facing user acceptance testing (see Section 7.3 and Section 7.4), before promotion to production via a slot swap once testing passes. This strategy was chosen for four primary reasons.

First, cost efficiency: since staging and production share the same underlying App Service plan and are promoted through slot swaps rather than running as two permanently duplicated, independently billed infrastructure sets, hosting costs remain significantly lower than a fully separate staging deployment would require—particularly relevant as HYDRA is an unfunded, student-led project for the client (Section 8, Running Costs). Second, simplicity and reversibility: a slot swap is a fast, nearly instantaneous, and reversible operation; if a defect appears immediately after release, the swap can be undone as quickly as it was

executed, far safer than attempting a time-pressured redeployment from source. This aligns with the "boring, frequent, and reversible releases" philosophy central to modern continuous-delivery practices (Humble and Farley, 2010; Kim et al., 2016; Forsgren, Humble and Kim, 2018). Third, client technical capacity: PSG Electrical and Cables/Trite Solar lacks in-house technical staff to manage a more complex rollout, so a strategy requiring no manual server administration—and executable and reversible from a single Azure dashboard—best suits the client's organizational constraints (Cameron and Green, 2024). Fourth, staged real-world exposure: rather than launching all

### Disaster recovery and business continuity within the deployment

The deployment plan is designed to work in tandem with, rather than duplicate, the backup and recovery strategy detailed in Section 6.6: because the Database Server, Blob Storage and Key Vault are all managed Azure services rather than self-hosted infrastructure, automated backup, geo-redundancy and point-in-time restore are available as a direct consequence of the deployment choices made in this section, without requiring the HYDRA team to build or operate any bespoke backup tooling of its own (Snedaker, 2013; Microsoft, 2024). Should the production App Service slot itself become unavailable for any reason, the existence of a structurally identical staging slot means a degraded or failed production deployment can, if necessary, be temporarily served from staging while the underlying issue is investigated — an informal but genuinely available continuity option that exists purely as a side effect of the slot-based deployment model chosen in Section 4.2.1, and one that would not exist under a simpler single-environment deployment model.

## 4.3 Data Schema: Choice of Technology

The HYDRA data layer utilizes PostgreSQL, managed via Azure Database for PostgreSQL — Flexible Server. The system features a Node.js backend providing a REST API (using an Express-style framework) and a React-based frontend that uses a shared component library for the customer web app and the various administrative and employee applications. This section focuses on the database choice because it is the most foundational decision, influencing every other component described in this report, including the classes in Section 3, the diagrams in Section 4.1, and the ERDs in Section 4.4, all of which assume a relational structure. Consequently, this section details the alternatives evaluated and the reasoning for their acceptance or rejection.

### Chosen technology stack

The primary reason for choosing this database is the nature of HYDRA's data, as illustrated by the entity relationship diagrams in Section 4.4. The domain contains approximately thirty entities—such as Customer, Job, Quote, Invoice, Payment, Electrician, and Material—which are all connected via strictly-typed foreign-key relationships. Certain connections, like Job–Material and Customer–Discount, are many-to-many relationships managed through junction tables like `JOB_MATERIAL` and `CUSTOMER_DISCOUNT`. This represents a classic relational dataset characterized by high structure and interconnection, requiring the database engine to enforce referential integrity. For instance, the system must prevent a `PAYMENT` from referencing a non-existent `invoice_id` or a `JOB_MATERIAL` entry from referencing a deleted `job_id`. A relational database management system (RDBMS) with native foreign-key constraints, efficient joins, and multi-statement transaction support is the ideal fit for this data type, and PostgreSQL serves as a highly compliant, open-source implementation of this model.

Transactional integrity is a critical operational requirement for several HYDRA workflows in Section 4.1. For example, recording a payment (Sequence Diagram 4.1.21) requires an atomic update to both the `PAYMENT` table and the `INVOICE.status` column to prevent financial record corruption. Similarly, processing payroll

(Diagram 4.1.11) requires that reading timesheets and writing a PAYROLL entry occur as a single atomic unit. Additionally, converting a ContactQuery into a Job (Diagrams 4.1.7/4.1.18) must ensure no orphaned records are created. PostgreSQL's ACID compliance allows these operations to be implemented safely through standard code, removing the need to manually build complex compensating-transaction logic to handle partial failures.

In contrast, NoSQL databases (document-oriented or key-value) are better suited for data that is self-contained or has a highly fluid structure. HYDRA's data does not fit this description; a Job cannot be processed or understood without its related Customer, Electrician, Quote, Invoice, and JobMaterial records. The system's value relies on these relationships being easily queryable and automatically enforced—capabilities inherent to relational engines but intentionally omitted by document databases in favor of horizontal write-scalability, which is not a current priority for HYDRA.

### **Justification: project requirements and data shape**

The development team is already proficient in SQL and relational modeling due to their academic training. The project's Design Class Diagram (Section 3.2) and Repository pattern (Section 5.1) were built around a relational model, utilizing primary/foreign keys and a PostgresXRepository class for each entity. Switching to a NoSQL database would necessitate a complete redesign of the class diagrams, the repository architecture, and the ERDs, offering no technical benefit given the data's relational nature. Aligning technology with the team's existing skills is a practical decision for a time-constrained student project with a fixed deadline.

The requirement for genuine transactional integrity is not a theoretical concern for HYDRA; it is directly load-bearing in at least three of the workflows documented in Section 4.1. Recording a payment against an invoice (Sequence Diagram 4.1.21) must update both the PAYMENT table and the INVOICE.status column together, atomically — a partial write that recorded the payment but failed to update the invoice status, or vice versa, would silently corrupt the client's own financial records. Processing payroll from timesheet data (the final stage of the Payment and Invoicing state diagram, Diagram 4.1.11) must similarly treat the read of outstanding timesheets and the write of a new PAYROLL row as a single atomic unit. And converting a ContactQuery into a Job (Diagram 4.1.7/4.1.18) must not leave the system in a state where a job exists with no corresponding originating query, or vice versa. PostgreSQL's full ACID (Atomicity, Consistency, Isolation, Durability) transaction guarantees are what make each of these operations safe to implement as straightforward application code, rather than requiring the team to hand-build compensating-transaction logic to recover from partial failures — a materially harder and more error-prone engineering problem that a genuinely relational database is specifically designed to make unnecessary (Silberschatz, Korth and Sudarshan, 2020; Kleppmann, 2017).

A document-oriented or key-value NoSQL database, by contrast, is generally the better technology fit for data that is either largely self-contained — each document can be read and written independently, with few meaningful cross-document relationships — or that has a rapidly changing, unpredictable structure that would be awkward to force into a fixed relational schema; neither characteristic describes HYDRA's data. A Job record cannot be meaningfully understood, billed, scheduled or reported on in isolation from its associated Customer, Electrician, Quote, Invoice and JobMaterial records; the analytical and operational value of the schema lies precisely in those relationships being efficiently queryable and automatically enforceable, which is exactly what a relational engine is designed to do well, and exactly what a document database is deliberately designed to avoid enforcing at the storage layer in exchange for horizontal write-scalability HYDRA does not currently need (Silberschatz, Korth and Sudarshan, 2020; Kleppmann, 2017).

### **Justification: team skill set**



As open-source software, PostgreSQL has no licensing fees, unlike commercial RDBMS options like Microsoft SQL Server that charge based on usage or cores. By using Azure Database for PostgreSQL — Flexible Server, the team can utilize low-cost burstable tiers during development and the WIL engagement. This allows for a controlled cost trajectory, as detailed in Section 8, which is vital because this is a free student project and the client is a small business seeking to minimize software expenses.

### Justification: cost

Using a managed Azure service means that server maintenance, patching, and backups are handled by Azure, reducing the administrative burden on the team and the client. If business volume increases, the instance can be scaled vertically by upgrading compute and memory without requiring changes to the application or schema. Furthermore, PostgreSQL supports horizontal scaling through read replicas, which can handle increased reporting or dashboard loads if the Admin App becomes a bottleneck. This scalability approach is further detailed in Section 5.

### Justification: scalability and future growth

Azure Database for PostgreSQL is a fully managed service, meaning the underlying server, operating-system patching, automated backups and point-in-time restore (already introduced in Section 4.2.6 and discussed further in Section 5.3 and Section 6.6) are handled directly by Azure, rather than requiring the HYDRA team, or PSG Electrical and Cables/Trite Solar after handover, to administer a database server themselves. Should the client's job volume grow substantially, the managed instance can first be scaled **vertically** — moving to a larger compute and memory tier with no change whatsoever to application code or schema — well before any more disruptive scaling strategy would be required at the job volumes realistically anticipated for this business over the next several years. PostgreSQL's mature, native support for streaming read replicas additionally offers a clear **horizontal** scaling path — one or more additional, continuously-synchronised, read-only copies of the database that can absorb reporting or dashboard load — should the Admin App's data-export and reporting features (Section 3.1, Section 4.4) become a bottleneck at a materially larger scale than the business operates at today (Kleppmann, 2017; Microsoft, 2024). This scalability story is discussed in more operational detail, alongside the autoscaling of the application compute tier itself, in Section 5.3 (Cloud Architecture).

### Alternatives considered

The HYDRA data schema is organized into four interconnected entity relationship diagrams instead of a single, massive diagram. These consist of a Customer/Website ERD, an Employee ERD, an Admin ERD, and an Access Control ERD. This separation is a purposeful documentation strategy to ensure clarity. By dividing the schema according to the client application that primarily manages each group of entities, each diagram remains readable; a single diagram encompassing all thirty-plus entities and their connections would be illegible at standard printing resolutions. The Access Control ERD specifically illustrates how the data domains of the three applications relate, identifying shared entities and defining which application has permission to read or write specific logical table groups. All entities across the four diagrams utilize crow's-foot notation for cardinality. Every table features a designated primary key (PK), and all relationships are established via a marked foreign key (FK) on the "many" side, adhering to standard relational modeling principles (Silberschatz, Korth and Sudarshan, 2020; Elmasri and Navathe, 2016; Connolly and Begg, 2015). For entities that appear in multiple diagrams—specifically JOB, QUOTE, TIMESHEET, and CONTACT\_QUERY—the primary key, attributes, and data types are kept perfectly consistent to prevent contradictions when tracing an entity across diagrams.



**Firebase (Firestore)** was considered as a lower-effort, fully managed alternative, particularly given its historically tight integration with mobile app development workflows (Google, 2024). It was rejected for the same underlying structural reason as MongoDB — its document/collection model is a poor fit for HYDRA's deeply relational schema — and additionally because Firestore's usage-based pricing model, which bills per document read and write operation, becomes materially harder to predict and control at scale than a fixed-tier relational database instance, working directly against the cost-transparency and cost-predictability goal that runs through Section 8 of this report.

**MySQL / MariaDB** were considered as alternative open-source relational database engines and were not rejected on purely technical grounds; either would have been a broadly workable choice for HYDRA's schema, since both offer the same core ACID transactional guarantees and foreign-key enforcement PostgreSQL does (Silberschatz, Korth and Sudarshan, 2020). PostgreSQL was preferred specifically because of its more complete and standards-compliant implementation of advanced constraint and data-type features — including native boolean, date and datetime column types used extensively and precisely throughout the Design Class Diagram (Section 3.2) and the ERDs (Section 4.4), and native CHECK constraints capable of directly enforcing the finite, enumerated status values that the eleven state diagrams in Section 4.1 formally specify — and because Azure's managed PostgreSQL offering integrates natively with the Key Vault-based secrets-management approach already selected for the cloud architecture (Section 5.3, Section 4.2.3), avoiding the need to independently evaluate a second managed-database product's own security integration story.

**Microsoft SQL Server** was considered and rejected primarily on cost grounds: while technically well suited to HYDRA's relational schema, its licensing model is materially more expensive at any tier beyond the smallest free/developer edition, which sits awkwardly against the requirement, emphasised throughout Section 8 of this report, to favour low-cost and free-tier technology options appropriate to a student-led project with no operating budget and a client who is themselves a small business.

**A hybrid relational-plus-document approach** — using PostgreSQL for the transactional core of the schema while storing loosely structured content (for example, FAQ entries or portfolio-item descriptions) in a separate document store — was briefly discussed and rejected as unnecessary complexity. PostgreSQL's native jsonb column type already provides efficient, indexable storage for any genuinely semi-structured data HYDRA needs (for example, flexible metadata fields on PORTFOLIO\_ITEM), without requiring the operational overhead, additional cost, and additional point of failure of running and securing a second, entirely separate database technology alongside PostgreSQL.

## Data normalisation and schema design discipline

This diagram details the fourteen entities directly involved with the public website or a customer: CUSTOMER, CONTACT\_QUERY, DISCOUNT, CUSTOMER\_DISCOUNT, REWARDS\_ACCOUNT, REWARDS\_TRANSACTION, SERVICE\_TYPE, PORTFOLIO\_ITEM, FAQ, JOB, JOB\_MILESTONE, JOB\_CHECKIN, QUOTE, INVOICE, and PAYMENT.

The CUSTOMER entity (customer\_id PK; username, password\_hash, first\_name, last\_name, email, phone, address, role, last\_login, created\_at) serves as the primary anchor for all other customer-facing tables. To support the credential storage protocols in Section 6.1.2, password\_hash only stores bcrypt hashes rather than plain text. Although all entries in this table represent customers, a role attribute is included because the same authentication and RBAC systems are used across all three HYDRA applications (Section 6.1.5). Using a uniform role column across CUSTOMER, ELECTRICIAN, and ADMIN enables the shared Authentication state diagram (Section 4.1.3) to function via a single, consistent field.

CUSTOMER has a zero-to-many relationship with CONTACT\_QUERY (query\_id PK; name, email, phone, message, submitted\_date, status, assigned\_admin\_id FK). This is modeled as zero-to-many and includes denormalized contact columns instead of a mandatory foreign key to CUSTOMER, allowing members of the public who have not registered an account to submit queries via the website (per Section 3.3 and the Access Control ERD). The status column is governed by the Website Query Processing state diagram (Section 4.1.2, Diagram 4.1.7). Additionally, CUSTOMER has a strict one-to-one relationship with REWARDS\_ACCOUNT (reward\_account\_id PK; customer\_id FK, points\_balance, tier, created\_at), reflecting the requirement that each registered customer holds only one rewards account.

The JOB entity (job\_id PK; scheduled\_date, status, site\_address, notes, created\_at, completed\_date) links to the customer, the assigned electrician, and the service type through three foreign keys: customer\_id, electrician\_id, and service\_type\_id. The status field is managed by the Job Lifecycle state diagram (Section 4.1.2, Diagram 4.1.1), with its valid states (Requested, Assigned, InProgress, Completed, Closed, Cancelled) enforced by a database CHECK constraint. JOB has a zero-to-many relationship with JOB\_MILESTONE (milestone\_id PK; job\_id FK, milestone\_name, description, planned\_date, completed\_date, status, sequence\_order), where sequence\_order ensures milestones are processed in a stable order. It also has a zero-to-one relationship with `

### Indexing and query performance strategy

This fourth diagram does not present any additional entities. Rather, it organizes the approximately thirty entities from the previous three diagrams into seven logical data domains: Content tables (SERVICE\_TYPE, FAQ, PORTFOLIO\_ITEM), Contact queries (CONTACT\_QUERY), Job & billing (JOB, QUOTE, INVOICE, PAYMENT), Rewards & discounts (REWARDS\_ACCOUNT, DISCOUNT), Job progress (JOB\_MILESTONE, JOB\_CHECKIN), Staff data (ELECTRICIAN, EMPLOYEE\_SCHEDULE, TIMESHEET), and Admin ops (JOB\_ASSIGNMENT\_LOG, PAYROLL, MISSED\_CALL\_LOG). It then illustrates the read, write, or dual access permissions for each domain across the four access surfaces: the public Website, Customer App, Admin App, and Employee App.

#### TABLE

To illustrate, the public website is restricted to reading Content tables and writing to Contact queries; this allows visitors to submit inquiries via forms without accessing other users' information or storing any of their own data, matching the "public, no login" status. The Customer App can interact with Job & billing and Rewards & discounts data to facilitate tasks like viewing invoices, making payments, or accepting quotes, but it is entirely barred from Staff data or Admin ops. Consequently, a customer cannot see administrative details, such as unselected staff members or payroll information, regardless of how that data is displayed in the Admin App. The Employee App permits reading and writing Job progress and Staff data—enabling electricians to manage schedules, timesheets, and job milestones—but prevents them from viewing Admin ops like missed-call logs or payroll processing figures. While the TIMESHEET data entered by an employee is used for administrative payroll, the employee can only view and modify their own timesheet, not the resulting payroll calculations. Only the Admin App possesses read/write permissions across all seven domains, reflecting its comprehensive supervisory authority.

This diagram is designed to support the role-based access control and permission analysis in Section 6.1.5. It demonstrates that the principle of least privilege—ensuring every user has only the minimum data access necessary for their role—is integrated directly into the data layer. This approach defines access at the level of specific application-to-table queries, rather than relying solely on application-level checks that could be bypassed or misconfigured during system evolution (Republic of South Africa, 2013; OWASP Foundation,

2021; Pathlock, 2026). In practice, this access matrix is enforced through two distinct layers: first, the API layer verifies the user's role before a request reaches a Repository class; second, the database layer uses PostgreSQL's native role and privilege system to scope the API Server's credentials to only authorized tables. This dual-layer enforcement ensures that even if an API authorization check fails, an Employee-scoped request cannot access Admin ops data because the database connection itself lacks the permission. This implementation of defense-in-depth is a standard recommendation for secure system design (OWASP Foundation, 2021; International Organization for Standardization, 2013).

#### Notation Legend and Constraint Summary:

The four diagrams consistently utilize standard crow's-foot ERD notation (Silberschatz, Korth and Sudarshan, 2020; Connolly and Begg, 2015), where a single bar (|) signifies "exactly one," a crow's foot signifies "many," and a crow's foot with a circle signifies "zero or many." Primary keys are identified as PK, and foreign keys as FK. All thirty-plus entities use uppercase names to distinguish them from relationship labels, and every attribute is explicitly typed (e.g., int, string, float, date, datetime, or boolean). This uniform notation allows readers to trace an entity, such as JOB, across different diagrams—from the Customer/Website ERD to the Admin ERD and the Access Control ERD—without needing to relearn the symbols.

Each foreign-key relationship is enforced at the database level using specific ON DELETE policies rather than relying on PostgreSQL defaults. Dependent relationships, such as PAYMENT referencing INVOICE, use ON DELETE CASCADE to ensure that deleting a parent record automatically removes associated dependent rows. Conversely, historical or audit records, such as JOB\_ASSIGNMENT\_LOG referencing ELECTRICIAN, utilize ON DELETE RESTRICT or soft-delete methods to prevent the loss of vital history when a parent record is deactivated. These specific policy decisions are documented in the data dictionary (Appendix, Section 10) as core parts of the schema specification, equal in importance to data types and cardinality.

## Summary

PostgreSQL was therefore selected because it is the single technology that most directly satisfies all five selection criteria applied by the team simultaneously: it fits the genuinely relational, foreign-key-dense shape of HYDRA's own data and its transactional integrity requirements; it matches the team's existing, demonstrated SQL and relational-modelling skill set; it carries no licensing cost whatsoever at the project's current scale while offering a clear, low-friction upgrade path to a larger paid tier should the client's business grow; hosted as a managed Azure service, it offers both vertical and horizontal scaling paths without requiring PSG Electrical and Cables/Trite Solar to take on any database administration burden after the HYDRA team's handover; and its native support for strict typing, CHECK constraints and jsonb gives the team a single technology capable of enforcing the full range of structural and behavioural rules documented throughout Section 4.1 and Section 4.4, without needing a second database technology for edge cases.

## 4.4 Data Schema: Entity Relationship Diagram

### Purpose and structure of the ERD set

The HYDRA data schema is presented as four related entity relationship diagrams rather than one single, monolithic diagram: a **Customer/Website ERD**, an **Employee ERD**, an **Admin ERD**, and an **Access Control ERD**. This is a deliberate documentation choice rather than an oversight. Splitting the schema by the client app that primarily owns each group of entities keeps each individual diagram legible — a single diagram containing all thirty-plus entities and their relationships simultaneously would be extremely difficult to read at any printable resolution — while the fourth diagram, the Access Control ERD, exists specifically to show how the three apps' data domains relate to one another, which entities are genuinely shared across app



hash, never a plaintext password, in direct support of the credential-storage practice documented in Section 6.1.2; role exists on CUSTOMER even though every row on this table is, definitionally, a customer, because the same authentication and RBAC subsystem is shared across all three HYDRA apps (Section 6.1.5), and a single, consistently-named role column across CUSTOMER, ELECTRICIAN and ADMIN is what allows the shared Authentication state diagram (Section 4.1.3) to branch on one uniformly-named field regardless of which table the authenticated user actually belongs to.

CUSTOMER *submits* a zero-to-many relationship with CONTACT\_QUERY (query\_id PK; name, email, phone, message, submitted\_date, status, assigned\_admin\_id FK) — modelled as zero-to-many, and with its own denormalised name/email/phone columns rather than a mandatory foreign key back to CUSTOMER, specifically because a CONTACT\_QUERY can legitimately originate from a member of the public who has never registered a CUSTOMER account at all (the public website's contact form has no login requirement, per Section 3.3 and the Access Control ERD discussed below); status on this table is the column whose finite legal values are formally governed by the Website Query Processing state diagram (Section 4.1.2, Diagram 4.1.7). CUSTOMER also *has* exactly one REWARDS\_ACCOUNT (reward\_account\_id PK; customer\_id FK, points\_balance, tier, created\_at) — a strict one-to-one relationship, reflecting the client's loyalty-scheme design decision that every registered customer automatically receives a single rewards account rather than being able to hold more than one.

JOB (job\_id PK; scheduled\_date, status, site\_address, notes, created\_at, completed\_date) carries three separate foreign keys — customer\_id, electrician\_id and service\_type\_id — tying every job simultaneously back to the customer who requested it, the electrician who will perform it, and the type of service being delivered; status is the column governed end to end by the Job Lifecycle state diagram (Section 4.1.2, Diagram 4.1.1), and its legal values (Requested, Assigned, InProgress, Completed, Closed, Cancelled) are enforced at the database level with a CHECK constraint derived directly from that state diagram. JOB is *broken into* a zero-to-many relationship with JOB\_MILESTONE (milestone\_id PK; job\_id FK, milestone\_name, description, planned\_date, completed\_date, status, sequence\_order) — the sequence\_order column exists specifically so milestones for a given job can always be displayed and processed in a guaranteed, stable order even though the underlying rows carry no inherent ordering of their own — and is *confirmed by* a zero-to-one relationship with JOB\_CHECKIN (checkin\_id PK; job\_id FK, electrician\_id FK, qr\_code, scan\_timestamp, status, gps\_location), the QR-scan arrival record described in detail under Sequence Diagram 4.1.16.

Each JOB is *quoted as* exactly one QUOTE (quote\_id PK; job\_id FK, amount, status, issued\_date, valid\_until, notes), and each accepted QUOTE is *converted to* exactly one INVOICE (invoice\_id PK; job\_id FK, quote\_id FK, customer\_id FK, amount\_due, invoice\_date, due\_date, status) — INVOICE deliberately carries foreign keys back to both JOB and QUOTE rather than only one of the two, because the invoice needs to be traceable both to the physical work performed (JOB) and to the specific commercial terms the customer agreed to (QUOTE), which may include quote-specific notes or conditions relevant to a billing dispute. Critically, INVOICE *receives* a zero-to-many relationship with PAYMENT (payment\_id PK; invoice\_id FK, amount, payment\_date, payment\_method, status, reference\_number) — this specific cardinality, rather than a strict one-to-one relationship or a single amount-paid column directly on INVOICE, is what allows one invoice to be settled through more than one payment over time, directly supporting the PartiallyPaid state on the Invoice Lifecycle state diagram (Section 4.1.2, Diagram 4.1.2) and the branching payment logic shown across Sequence Diagrams 4.1.13, 4.1.21 and 4.1.22.

DISCOUNT (discount\_id PK; code, description, discount\_type, value, valid\_from, valid\_until, min\_spend, active) is *redeemed as* a zero-to-many relationship with CUSTOMER\_DISCOUNT (customer\_discount\_id PK; discount\_id FK, customer\_id FK, invoice\_id FK, applied\_date) — a genuine many-to-many resolving junction table between CUSTOMER and DISCOUNT, since the same discount code can legitimately be used by many

customers and, subject to the scheme's own rules, the same customer could be entitled to more than one discount — each CUSTOMER\_DISCOUNT row is in turn *applied to* exactly one INVOICE, tying the discount permanently to the specific bill it reduced. REWARDS\_ACCOUNT *earns* a zero-to-many relationship with REWARDS\_TRANSACTION (transaction\_id PK; reward\_account\_id FK, job\_id FK, points\_earned, points\_redeemed, transaction\_date, description), with each transaction tied back to the specific job that generated (or, in a redemption transaction, consumed) the points, giving the customer a fully auditable points history rather than only a current balance.

SERVICE\_TYPE (service\_type\_id PK; name, category, description, base\_price) *categorises* both JOB and PORTFOLIO\_ITEM (portfolio\_id PK; title, description, image\_url, service\_type\_id FK, completed\_date, featured) — the same service-type taxonomy is deliberately reused across live job records and the client's public portfolio gallery, so that a customer browsing "solar installation" work in the portfolio is looking at the identical category classification the admin uses internally when creating a job of that type, avoiding the maintenance burden and eventual drift of keeping two separate category lists in sync. FAQ (faq\_id PK; question, answer, category, display\_order) stands entirely independently, with no foreign-key relationship to any transactional table, since it exists purely as static, publicly-readable reference content that the website surfaces to any visitor regardless of whether they have ever interacted with any other part of the system.

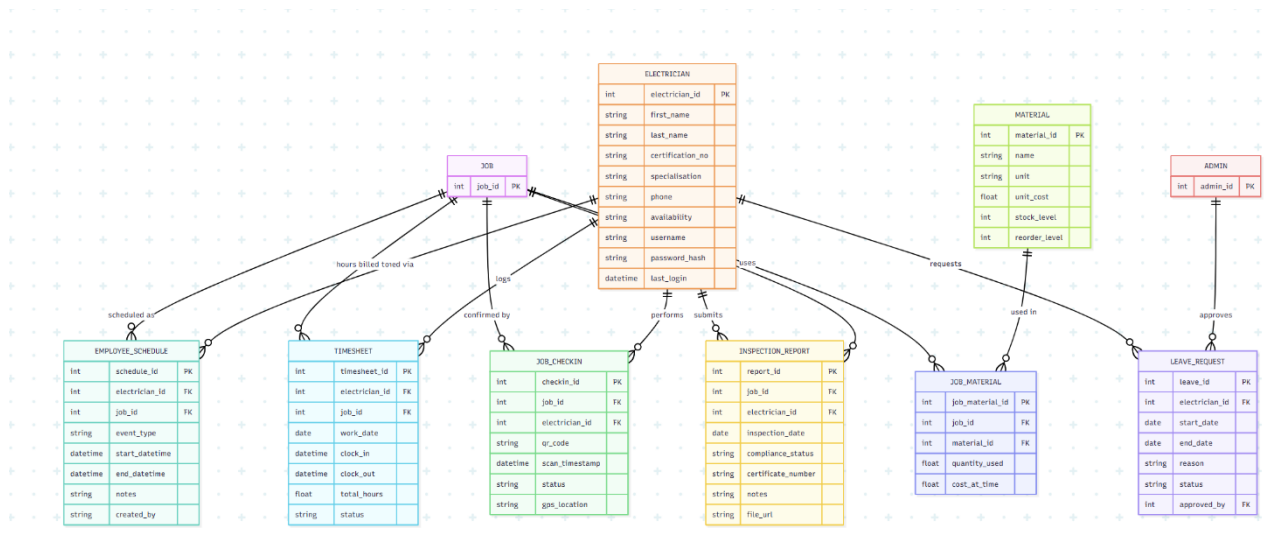


Figure 28: HYDRA Employee Entity Relationship Diagram

## Employee ERD

This diagram covers the nine entities an electrician interacts with directly: ELECTRICIAN, EMPLOYEE\_SCHEDULE, TIMESHEET, JOB\_CHECKIN, INSPECTION\_REPORT, MATERIAL, JOB\_MATERIAL, LEAVE\_REQUEST and ADMIN (the last appearing here only in its capacity as the approver of leave requests).

ELECTRICIAN (electrician\_id PK; first\_name, last\_name, certification\_no, specialisation, phone, availability, username, password\_hash, last\_login) carries certification\_no and specialisation as attributes specific to this entity and not mirrored on CUSTOMER or ADMIN — a direct structural reflection of the compliance context discussed in Section 4.1.2 under the Job Lifecycle diagram: an electrician's certification number is the credential that ultimately underpins the compliance status recorded on any INSPECTION\_REPORT they submit, and storing it as a first-class attribute on ELECTRICIAN rather than only in free-text notes makes it directly queryable and reportable, for example to verify that every electrician currently assigned active jobs holds a certification that has not lapsed.



ELECTRICIAN *is scheduled as* a zero-to-many relationship with EMPLOYEE\_SCHEDULE (schedule\_id PK; electrician\_id FK, job\_id FK, event\_type, start\_datetime, end\_datetime, notes, created\_by) — the table underpinning the entire Calendar and Scheduling state and sequence diagrams (Section 4.1.2 Diagram 4.1.9; Section 4.1.3 Diagram 4.1.19), with event\_type distinguishing a job-linked calendar entry from a leave period or other non-job event, and job\_id deliberately nullable to accommodate the latter case. ELECTRICIAN *logs* a zero-to-many relationship with TIMESHEET (timesheet\_id PK; electrician\_id FK, job\_id FK, work\_date, clock\_in, clock\_out, total\_hours, status) — the entity that, per the Payment and Invoicing state diagram (Section 4.1.2, Diagram 4.1.11), only becomes eligible for payroll processing once the associated job's invoice reaches Paid status, even though the timesheet row itself is written immediately when the electrician clocks out. ELECTRICIAN *performs* a zero-to-many relationship with JOB\_CHECKIN, the same entity introduced above on the Customer/Website ERD — this shared appearance across two diagrams is precisely why the Access Control ERD (Section 4.4.5, below) exists, since JOB\_CHECKIN genuinely needs to be written by the Employee App and read by both the Customer App (to confirm arrival, per Sequence Diagram 4.1.15) and the Admin App (to monitor field operations).

ELECTRICIAN *submits* a zero-to-many relationship with INSPECTION\_REPORT (report\_id PK; job\_id FK, electrician\_id FK, inspection\_date, compliance\_status, certificate\_number, notes, file\_url), tied to both the job being inspected and the electrician conducting the inspection — compliance\_status is the single attribute in the entire HYDRA schema most directly responsible for gating the Inspection passed guard condition on the Job Lifecycle state diagram, and certificate\_number and file\_url together record the formal, retrievable evidence (a photographed or scanned compliance certificate, stored in Blob Storage per the deployment diagram in Section 4.2) that would be produced in the event of a regulatory audit or a customer dispute.

The many-to-many relationship between JOB (introduced on the Customer/Website ERD above, and appearing again here for continuity) and MATERIAL (material\_id PK; name, unit, unit\_cost, stock\_level, reorder\_level) is resolved by the JOB\_MATERIAL junction table (job\_material\_id PK; job\_id FK, material\_id FK, quantity\_used, cost\_at\_time), which carries its own dedicated primary key alongside the job\_id and material\_id foreign keys, plus the quantity\_used and cost\_at\_time values that make sense only in the specific context of one job using one material — exactly the pattern the module checklist explicitly asks the team to verify, and exactly the deliberate, documented denormalisation (cost\_at\_time capturing a historical price snapshot) discussed under Section 4.3.7 above. MATERIAL.stock\_level and MATERIAL.reorder\_level together support a straightforward low-stock reporting query for the admin (stock\_level <= reorder\_level), directly satisfying the "track material stock levels so that supplies can be reordered" user story documented in Section 2.1.2.

ELECTRICIAN also *requests* a zero-to-many relationship with LEAVE\_REQUEST (leave\_id PK; electrician\_id FK, start\_date, end\_date, reason, status, approved\_by FK), each of which is *approved* by exactly one ADMIN row via the approved\_by foreign key — a leave request that has not yet been actioned carries a null approved\_by value, and the state diagram vocabulary of Pending/Approved/Rejected on LEAVE\_REQUEST.status feeds directly into the conflict-detection logic on the Calendar and Scheduling state diagram (an approved leave period is one of the conditions that can legitimately trigger a ConflictDetected transition when an admin subsequently attempts to schedule that electrician against a job during the same window).



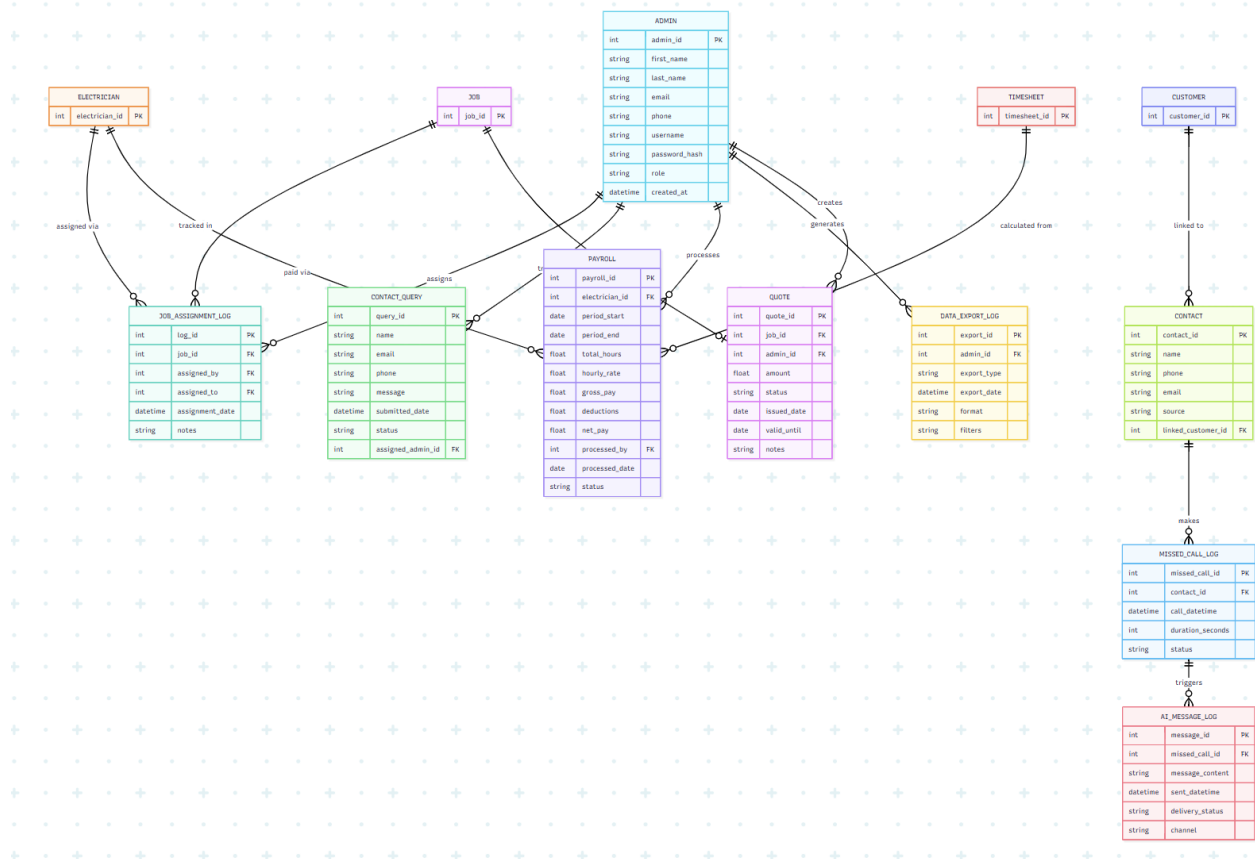


Figure 29: HYDRA Admin Entity Relationship Diagram

## Admin ERD

This diagram covers the eleven entities specific to office and administrative operations: ADMIN, JOB\_ASSIGNMENT\_LOG, CONTACT\_QUERY, PAYROLL, QUOTE, DATA\_EXPORT\_LOG, CONTACT, MISSED\_CALL\_LOG, AI\_MESSAGE\_LOG, TIMESHEET and JOB, several of which — CONTACT\_QUERY, QUOTE, TIMESHEET and JOB — are shared with the two diagrams already discussed above and appear here specifically because this diagram needs to show the administrative side of relationships that were already introduced from the customer's or employee's perspective.

ADMIN (admin\_id PK; first\_name, last\_name, email, phone, username, password\_hash, role, created\_at) *assigns* a zero-to-many relationship with JOB\_ASSIGNMENT\_LOG (log\_id PK; job\_id FK, assigned\_by FK, assigned\_to FK, assignment\_date, notes) — a "tracked in" relationship back to JOB and an "assigned via" relationship to ELECTRICIAN — which exists as an explicit, append-only audit trail of every assignment and reassignment decision an admin makes, directly supporting the ReassignmentRequired loop on the Admin Job Management state diagram (Section 4.1.2, Diagram 4.1.6) and the broader audit-logging requirement discussed in Section 6.7: because JOB\_ASSIGNMENT\_LOG is a separate, append-only table rather than a set of columns overwritten in place on JOB itself, a full history of every electrician who was ever considered for or assigned to a given job remains permanently reconstructable, even after a reassignment has occurred.

ADMIN also *triages* a zero-to-many relationship with CONTACT\_QUERY via the assigned\_admin\_id foreign key already introduced above, *processes* a zero-to-many relationship with PAYROLL (payroll\_id PK; electrician\_id FK, period\_start, period\_end, total\_hours, hourly\_rate, gross\_pay, deductions, net\_pay, processed\_by FK, processed\_date, status) — itself "calculated from" the underlying TIMESHEET rows for the relevant electrician and pay period, with total\_hours, gross\_pay, deductions and net\_pay stored as their own explicit, auditable columns rather than being recomputed on demand, so that a historical payroll run's figures

remain stable and inspectable even if the electrician's hourly rate is later changed going forward — *creates* QUOTE rows (carrying the admin\_id that generated the quote, recorded on this diagram's own copy of the QUOTE entity for traceability), and *generates* DATA\_EXPORT\_LOG rows (export\_id PK; admin\_id FK, export\_type, export\_date, format, filters) — a record of every reporting export an admin has produced, which both supports the Admin App's reporting user stories (Section 2.1.2) and provides a further, independent layer of accountability over who has extracted client, financial or staff data from the system and when, relevant to the audit-logging and accountability obligations discussed in Section 6.4 and Section 6.7.

On the communications side, ADMIN is *linked to* CONTACT (contact\_id PK; name, phone, email, source, linked\_customer\_id FK) — a lightweight record of any phone or messaging contact, whether or not that contact corresponds to a registered CUSTOMER, with linked\_customer\_id connecting a raw contact back to a full customer record only where one genuinely exists, and source recording how the contact was captured (an inbound call, a WhatsApp message, a manual admin entry). CONTACT *makes* a zero-to-many relationship with MISSED\_CALL\_LOG (missed\_call\_id PK; contact\_id FK, call\_datetime, duration\_seconds, status), and each MISSED\_CALL\_LOG row *triggers* exactly one AI\_MESSAGE\_LOG row (message\_id PK; missed\_call\_id FK, message\_content, sent\_datetime, delivery\_status, channel) — together these three tables (CONTACT, MISSED\_CALL\_LOG, AI\_MESSAGE\_LOG) form the complete, end-to-end data trail behind the automated missed-call response feature documented behaviourally in Section 4.1.2 (Diagram 4.1.10) and Section 4.1.3 (Diagram 4.1.20), and this is deliberately the most tightly-scoped, minimal data footprint anywhere in the HYDRA schema — a direct structural consequence of the data-minimisation discipline discussed in Section 6.4, since this is the one part of the system that processes a member of the public's contact information without that person having taken any deliberate action to register with HYDRA at all.

TIMESHEET also appears on this diagram in its "hours billed to" and "paid via" relationships to ELECTRICIAN and PAYROLL respectively, since payroll processing is fundamentally an administrative function even though the TIMESHEET data itself originates, and is first written, on the Employee side of the system documented in Section 4.4.3 above — this is a clear example of exactly the kind of cross-app data sharing the Access Control ERD (below) exists to make explicit and govern.

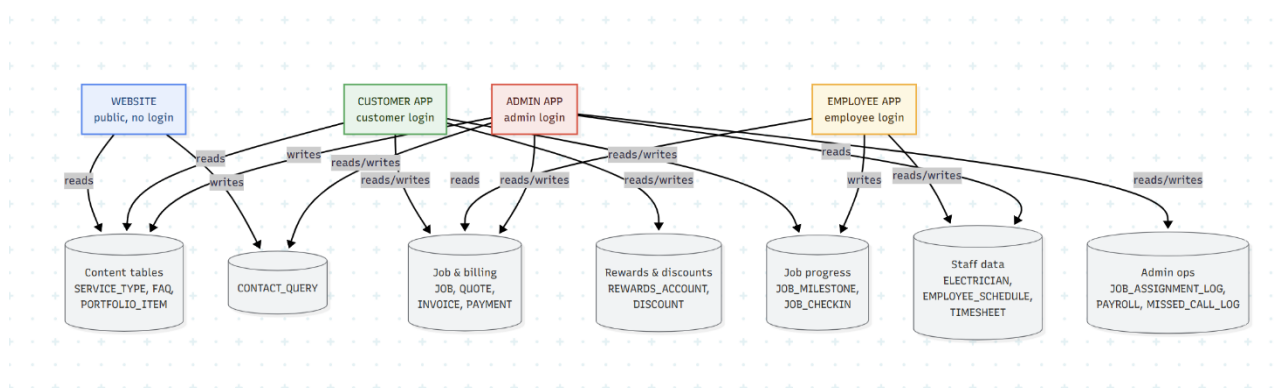


Figure 30: HYDRA Access Control Entity Relationship Diagram

## Access Control ERD

This fourth diagram introduces no new entities of its own; instead, it groups the roughly thirty entities documented across the three diagrams above into seven logical data domains — **Content tables** (SERVICE\_TYPE, FAQ, PORTFOLIO\_ITEM), **Contact queries** (CONTACT\_QUERY), **Job & billing** (JOB, QUOTE, INVOICE, PAYMENT), **Rewards & discounts** (REWARDS\_ACCOUNT, DISCOUNT), **Job progress** (JOB\_MILESTONE, JOB\_CHECKIN), **Staff data** (ELECTRICIAN, EMPLOYEE\_SCHEDULE, TIMESHEET) and **Admin ops** (JOB\_ASSIGNMENT\_LOG, PAYROLL, MISSED\_CALL\_LOG) — and shows, for each of the four access

surfaces (public **Website**, **Customer App**, **Admin App**, **Employee App**), whether that surface may read, write, or both, each data domain.

**Table 6: Access Control ERD — data-domain read/write matrix by application surface**

| Access surface                | Content tables | Contact queries | Job & billing | Rewards & discounts | Job progress | Staff data | Admin ops  |
|-------------------------------|----------------|-----------------|---------------|---------------------|--------------|------------|------------|
| Website (public, no login)    | Read           | Write           | —             | —                   | —            | —          | —          |
| Customer App (customer login) | Read           | Read/Write      | Read/Write    | Read/Write          | Read         | —          | —          |
| Employee App (employee login) | Read           | —               | Read          | —                   | Read/Write   | Read/Write | —          |
| Admin App (admin login)       | Read/Write     | Read/Write      | Read/Write    | Read/Write          | Read/Write   | Read/Write | Read/Write |

For example, the public Website may only *read* Content tables and *write* to Contact queries — a website visitor can submit an enquiry through the contact form but cannot see anyone else's data, and indeed has no data of their own stored anywhere else in the system, consistent with the "public, no login" access level shown for this surface. The Customer App can read and write Job & billing and Rewards & discounts data (a customer needs to be able to accept a quote, view an invoice, make a payment and redeem a reward) but has no access whatsoever to Staff data or Admin ops — a customer is never able to see which electrician was considered but not selected for their job, nor any payroll or missed-call data, regardless of how the Admin App's own interface might present that information to an admin. The Employee App can read and write Job progress and Staff data (an electrician needs to update a job's milestones, confirm arrival via JOB\_CHECKIN, and manage their own schedule and timesheet) but cannot see Admin ops such as payroll processing figures or missed-call logs, even though, as shown in Section 4.4.3, the underlying TIMESHEET data an electrician writes is later consumed by PAYROLL processing on the admin side — the electrician can see and edit their own timesheet, but never the payroll figures calculated from it. The Admin App alone has read/write access across every one of the seven domains, reflecting its genuinely supervisory role over the whole business.

This diagram exists specifically to support the access-control and role-based-permission discussion required in Section 6.1.5, and it demonstrates that the principle of least privilege — granting every actor in the system the minimum level of data access genuinely needed to perform their role, and no more — has been designed directly into the data layer itself, at the level of which app is even permitted to issue a query against which table, rather than being left purely to application-level checks within a single shared codebase that could more easily be bypassed, misconfigured, or simply forgotten as the system evolves (Republic of South Africa, 2013; OWASP Foundation, 2021; Pathlock, 2026). In the concrete implementation, this access matrix is enforced at two independent layers rather than one: first, at the API layer, where every endpoint checks the authenticated user's role (Section 6.1.5) before permitting a request to reach the relevant Repository class at all; and second, at the database layer itself, where the credentials used by the API Server to connect to PostgreSQL are scoped, using PostgreSQL's native role and privilege system, to only the tables that role is legitimately permitted to touch — meaning that even a defect in the API-layer authorisation check could not, on its own, allow an Employee-scoped request to read or write Admin ops data, since the underlying database connection itself has no privilege to do so. Defending a sensitive access boundary at two independent layers

rather than one is a specific application of the defence-in-depth principle widely recommended in secure system design (OWASP Foundation, 2021; International Organization for Standardization, 2013).

### Notation legend and constraint summary

Across all four diagrams, a single bar (|) at the end of a relationship line denotes "exactly one", a crow's foot denotes "many", and a crow's foot combined with a small circle denotes "zero or many" — standard crow's-foot ERD notation used consistently throughout this report (Silberschatz, Korth and Sudarshan, 2020; Connolly and Begg, 2015). Primary keys are marked **PK** in the second column of every entity box; foreign keys are marked **FK**. Every attribute across every one of the roughly thirty entities is explicitly typed (int, string, float, date, datetime or boolean), and every entity name is written in upper case to visually distinguish entities from the relationship-label text connecting them. This notation is applied with complete consistency across all four ERDs specifically so that a reader moving between diagrams — for example, tracing JOB from the Customer/Website ERD, through its "hours billed to" appearance via TIMESHEET on the Admin ERD, to its access-domain grouping under "Job & billing" on the Access Control ERD — never needs to re-learn the notation partway through the schema, and can instead follow a single entity's full set of relationships across the entire data model with confidence that the same symbol always means the same thing.

Every foreign-key relationship documented across all four diagrams is additionally enforced at the database level with an ON DELETE policy chosen deliberately per relationship rather than left at PostgreSQL's default: relationships representing a genuinely dependent, "cannot exist without its parent" association (for example, JOB\_MATERIAL referencing JOB, or PAYMENT referencing INVOICE) use ON DELETE CASCADE, so that deleting a job's parent record correctly and automatically removes its dependent material-usage and payment history rather than leaving orphaned rows behind; relationships representing an audit or historical record that should survive even if its referenced entity is later removed (for example, JOB\_ASSIGNMENT\_LOG referencing ELECTRICIAN) instead use ON DELETE RESTRICT or a soft-delete convention on the parent table, so that an electrician's employment history cannot simply vanish, and the assignment log cannot be silently invalidated, merely because that electrician's own record is later deactivated. This per-relationship policy decision is documented in the full data dictionary maintained alongside the schema migration scripts (Appendix, Section 10) and is treated as part of the schema's specification in exactly the same way its cardinality and data types are, rather than being left to whatever a database migration tool happens to default to.

## 5. Architecture Artifacts

In this section, we will provide all artefacts that constitute HYDRA's architecture design, that is, the consistent application of the design pattern in the data access layer of the system, the architectural pattern which defines the way the system is built and the way communication between its layers is carried out, and the cloud architecture of the system, and discuss the rationale for each of those choices as well as the alternatives considered. For each artefact, we will discuss not only what we have done but why, what else could be done and what could not, and in what way does this serve the particular context in which HYDRA is developed: a team of six students changing over time; a client, PSG Electrical and Cables/Trite Solar, without any in-house technical personnel capable of managing the system afterwards (Section 1.2); a system of actual but relatively small scale, suitable for a single-office business rather than a company; and finally, a tight budget, since the project is developed on no cost basis for the client (Section 8). Each of the three diagrams that are referred to in this section, namely Figure 31 ("Design Patterns"), Figure 32 ("Architecture Pattern") and Figure 33 ("Cloud Architecture"), are described in detail part by part, connection by connection, such that a person who does not have any previous familiarity with those diagrams will be able to deduce exactly what each box, interface, arrow and label signifies and why it has been drawn the way it has, in line with the criteria specified for this section in the module rubric ("excellent explanation and integration, clear UML diagrams" and "professional diagram with full detail on service selection, security, and network flow with justification" respectively).

### 5.1 Design Patterns

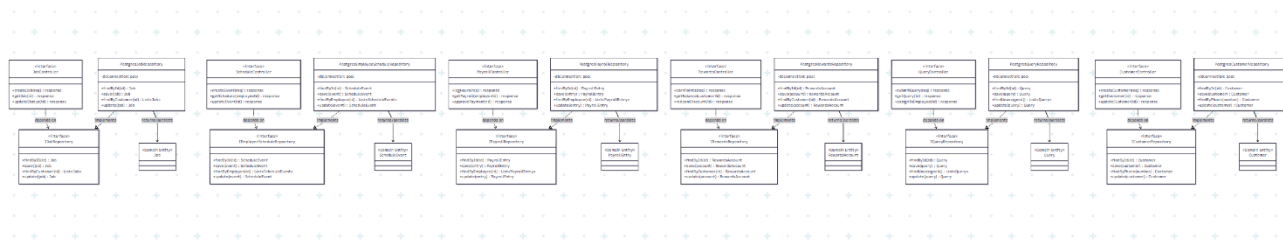


Figure 31: HYDRA Design Patterns — Repository Pattern Applied Across All Entities

#### 5.1.1 The problem the Repository pattern solves

HYDRA applies the **Repository design pattern** consistently across every major entity in the system, specifically to solve the problem of tightly coupling business logic to a specific database technology. Without this pattern, a class such as `JobController` would need to embed raw PostgreSQL queries directly inside its own methods, meaning that any change to the database — a schema change, a move to a different database engine, or even just writing a unit test that should not touch a real database — would require changing the controller itself, and would leave data-access logic scattered across every controller in the system rather than concentrated in one predictable location. The Repository pattern solves this by introducing an interface, `IJobRepository`, that declares *what* data operations are available (`findById(id) : Job`, `save(job) : Job`, `findByCustomer(id) : List<Job>`, `update(job) : Job`, as shown in Figure 31) without specifying *how* they are carried out, and a concrete class, `PostgresJobRepository`, that implements that interface using an actual PostgreSQL connection pool, represented in the diagram by the `-dbConnection: pool` field private to the concrete implementation and deliberately absent from the interface itself, since the interface must remain entirely agnostic to the specific storage technology behind it (Fowler, 2018; Richards, 2015). This pairing — a technology-agnostic interface plus a technology-specific implementation — is one of the two canonical patterns Vernon (2013) identifies for persisting aggregate roots in a domain-driven design, and its adoption here reflects the same underlying philosophy already established for HYDRA's entities in Sections 3.1 and 4.4:

that the meaning and behaviour of a Job, a Customer or a PayrollEntry should be defined by the business domain, not by the mechanics of how PostgreSQL happens to store it.

### 5.1.2 Anatomy of a single repository triad — worked example

The Repository design pattern is applied uniformly in every significant entity of HYDRA in order to deal with the issue of tight coupling between business logic and a particular database solution. Without the use of this pattern, the code of the class like JobController would have to contain the PostgreSQL queries right in the code of this class. It means that every modification of the database (change of database schema, migration to another database solution, even a unit test which should not affect the real database) will necessitate modification of the controller class and data-access logic will be distributed throughout all controllers.

This issue is addressed in the Repository pattern by providing an IJobRepository interface that defines the data operations available (findById(id) : Job, save(job) : Job, findByCustomer(id) : List<Job>, update(job) : Job, as seen in Figure 31) but leaves the implementations vague, and a class called PostgresJobRepository that implements the defined interface by means of a concrete PostgreSQL connection pool (-dbConnection: pool), which is explicitly provided in the figure only for the concrete class and not for the interface, because the interface cannot have any knowledge of the underlying storage technology (Fowler, 2018; Richards, 2015).

This combination, namely, a technology-agnostic interface along with a technology-dependent implementation, is one of the two standard patterns which Vernon (2013) describes for storing aggregate roots within the framework of domain-driven design and this choice is consistent with the same principle that has been set forth in respect of the entities of HYDRA in Section 3.1 and Section 4.4, namely, that the semantics and behavior of a Job, a Customer, or a PayrollEntry should come from the domain, rather than from the particular way in which PostgreSQL stores it.

### 5.1.3 Applying the pattern across the domain — the remaining five triads

Figure 31 depicts the exact same four-step architecture for each of the entities, and it would be worthwhile to discuss one such case, which is that of JobController / IJobRepository / PostgresJobRepository / Job, before we move on to the others in the picture because all the other columns in the figure work exactly in the same way.

As illustrated above, JobController, which is found in the furthest left position among those in its cluster, provides three public methods associated with the management of jobs within HYDRA — namely createJob(req) : response, getJob(id) : response, and updateStatus(id) : response, which take an HTTP request object and give back an HTTP response object, since, as detailed in Section 5.2 below, the function of this controller is that of the interface between the external world and HYDRA's business logic.

The dotted line marked “depends on” going from JobController to the box containing «interface» IJobRepository is the explicit depiction of dependency inversion in the diagram. As explained in Section 5.1.4, JobController depends on an interface rather than on the concrete class PostgresJobRepository, although PostgresJobRepository is ultimately responsible for doing the work at run-time.

The IJobRepository interface, on the other hand, defines four operations: findById(id) : Job, save(job) : Job, findByCustomer(id) : List<Job>, and update(job) : Job. These four methods constitute all the persistence operations required for the management of jobs in HYDRA as per the workflow illustrated in Section 2.2: retrieval of a single job based on its unique identifier, creation/persisting of either a new job or an updated job, retrieval of all jobs related to a specific customer ("track job status" journey step in the Customer App in Section 2.2), and finally updating of an already existing job, particularly changing its status at different stages as per the life cycle defined in Section 4.1. The PostgresJobRepository class that implements the



`IJobRepository` interface with the dotted line and triangular arrow head marked as "implements", according to the UML standard notation, offers concrete PostgreSQL implementations of all the above mentioned four methods using the `dbConnection`: pool private field and executing parameterised SQL statements on the DB schema presented in Section 4.4, specifically with the use of connection pool instead of a long-lived connection in order to process multiple requests of the three HYDRA applications simultaneously without any blocking issue due to a round-trip to the DB (Silberschatz, Korth and Sudarshan, 2020).

Lastly, the solid arrow "returns/persists" pointing downwards from the interface to the «Domain Entity» `Job` box denotes the point that all operations of `IJobRepository` either take `Job` domain entity as a parameter or return such entity (or even a collection of them) as the result because the sole reason for a repository is to transport `Job` entities from in-memory application to the PostgreSQL database and vice versa without exposing any particularity of the latter to the outside.

**Customer** — `CustomerController` exposes `createCustomer(req) : response`, `getCustomer(id) : response` and `updateCustomer(id) : response`, depending on «interface» `ICustomerRepository`, which declares `findById(id) : Customer`, `save(customer) : Customer`, `findByPhone(number) : Customer` and `update(customer) : Customer`, implemented by `PostgresCustomerRepository`. The presence of `findByPhone(number)` alongside the more conventional `findById(id)` at both the controller and repository level is a deliberate design decision reflecting a very concrete, practical HYDRA use case: the missed-call handling pipeline described fully in Section 5.3.3 below identifies an incoming caller by their phone number, not by an internal database identifier the caller has no way of knowing, so a phone-number lookup path had to exist in the `Customer` repository from the outset rather than being retrofitted later.

Across all six triads, each repository interface exposes the same underlying *shape* of operation — find by identifier, save, find by a natural parent or lookup key, and update — which keeps the codebase deliberately predictable: once a developer understands how `JobRepository` works, they already understand how every other repository in the system works, which materially reduces onboarding time across a six-person team working in parallel on different domain areas, and reduces the likelihood of inconsistent, one-off data-access code appearing in different corners of the system. This consistency is itself a recognised secondary benefit of applying design patterns systematically rather than ad hoc: a shared, named vocabulary of solutions — "this class is a repository," understood identically by every team member regardless of which entity it belongs to — lets developers reason about unfamiliar code more quickly than if every module solved the same underlying persistence problem in a slightly different, bespoke way (Fowler, 2018; Martin, 2017).

#### 5.1.4 UML notation used in Figure 31, and the dependency-inversion principle it encodes

The same four-tiered design pattern `Controller -> Repository interface -> Postgres implementation -> Domain entity` is replicated without modification but of course performing a unique set of functions in each case for all of the significant entities that have been listed in Figure 31. Each one is explained separately since each plays a different role in the HYDRA business process.

**Scheduling** — `ScheduleController` contains the operations `createEvent(req) : response`, `getSchedule(employeeId) : response` and `updateEvent(id) : response`, which depend on «interface» `IEmployeeScheduleRepository`, whose methods include `findById(id) : ScheduleEvent`, `save(event) : ScheduleEvent`, `findByEmployee(id) : List` and `update(event) : ScheduleEvent`, implemented by `PostgresEmployeeScheduleRepository`. The trio helps with the electricity specialist-centric "assigned tasks view" scenario step (Section 2.2) and the task assignment process in the Admin App (Section 2.1): `getSchedule(employeeId)` is precisely the function called by the Employee App when an electricity specialist



runs the app to see the assignments for the day, while `findByEmployee(id)` is its counterpart at the repository level that makes it possible without exposing the uncompiled SQL to the controller level.

**Payroll** — `PayrollController` offers `logHours(req) : response`, `getPayroll(employeeId) : response` and `approvePayment(id) : response`, meeting the definition of «interface» `IPayrollRepository`, that comprises `findById(id) : PayrollEntry`, `save(entry) : PayrollEntry`, `findByEmployee(id) : List` and `update(entry) : PayrollEntry` provided by `PostgresPayrollRepository`. These three services provide direct support for the "record materials used during a task, making sure that the cost is tracked accurately" and payroll-related user stories described in Section 2.1: `logHours` provides the ability to log the hours of the electrician working on a task, `approvePayment` offers a well-defined procedure for Admin to approve a payroll before completing it (thus satisfying audit logging requirements outlined in Section 6), and `findByEmployee` allows to restore the full payroll history of a particular electrician for a certain pay period.

**Rewards** — `RewardsController` provides `earnPoints(req) : response`, `getBalance(customerId) : response`, and `redeemDiscount(id) : response`, with reference to «interface» `IRewardsRepository` which provides the following operations `findById(id) : RewardsAccount`, `save(account) : RewardsAccount`, `findByCustomerId() : RewardsAccount` and `update(account) : RewardsAccount` through `PostgresRewardsRepository`. This trio enables a customer-loyalty feature which rewards customers of repeated business transactions from PSG Electrical and Cables/Trite Solar with the help of their current customers (Section 1.2). Points will be awarded automatically on job completion and invoice payments, and such points can later be used for getting discounts on future jobs using `redeemDiscount`.

**Query** — `QueryController` provides `submitQuery(req) : response`, `getQuery(id) : response`, `findUnassigned() : List`, and `assignToEmployee(id) : response`, based on the «interface» `IQueryRepository`, which defines `findById(id) : Query`, `save(query) : Query`, `findUnassigned() : List`, and `update(query) : Query`, realized by `PostgresQueryRepository`. This triad serves as the starting point for the initial phase of the Customer Journey outlined in Section 2.2 — a potential customer making an inquiry via the public website or Customer App prior to any job, quote, or electrician's involvement — and the intentional incorporation of `findUnassigned()` at both the controller and repository levels provides Admin personnel with an immediate, always-up-to-date overview of all incoming inquiries that have not yet been triaged, mirroring the digital equivalent of the "incoming website query" review step mentioned in the Admin training overview (Section 9.1.2).

**Customer** — `CustomerController` provides `createCustomer(req) : response`, `getCustomer(id) : response`, and `updateCustomer(id) : response`, relying on «interface» `ICustomerRepository`, which specifies `findById(id) : Customer`, `save(customer) : Customer`, `findByPhone(number) : Customer`, and `update(customer) : Customer`, realized by `PostgresCustomerRepository`. The inclusion of `findByPhone(number)` next to the traditional `findById(id)` at both the controller and repository layers is a purposeful design choice showcasing a specific, practical HYDRA scenario: the missed-call handling process outlined in Section 5.3.3 below recognizes an incoming caller via their phone number, rather than through an internal database identifier unknown to the caller, necessitating the existence of a phone-number lookup within the Customer repository from the beginning instead of being added later.

In all six triads, every repository interface reveals the same fundamental operation pattern — identify by id, store, retrieve by a natural parent or lookup key, and modify — making the codebase intentionally consistent: once a developer grasps `JobRepository`, they comprehend every other repository in the system, significantly decreasing onboarding duration for a six-person team concurrently engaged in distinct domain areas, and minimizing the chances of inconsistent, unique data-access code emerging in various parts of the system. This consistency is a well-known secondary advantage of systematically applying design patterns instead of using them arbitrarily: a common, named lexicon of solutions — "this class acts as a repository,"

comprehended uniformly by all team members irrespective of its associated entity — enables developers to understand unfamiliar code more swiftly than if every module addressed the same fundamental persistence issue in a somewhat different, customized manner (Fowler, 2018; Martin, 2017).

### 5.1.5 Why the Repository pattern, and not an alternative

Figure 31 features three unique connector types, each conveying a specific, intentional meaning instead of being interchangeable arrows, making it crucial to differentiate them for accurate interpretation of the diagram. The dashed arrow marked "depends on", extending from each controller to its related repository interface, illustrates a UML dependency relationship: the compiled code of the controller refers to the type of the interface, but the two remain unbound at compile time to any particular implementation. The dashed arrow featuring a hollow triangular arrowhead marked "implements", extending from each specific Postgres[Entity]Repository class to its interface, signifies UML interface realization: the specific class officially agrees, and is verified by the compiler/type system to agree, to deliver every function the interface specifies, with a corresponding method signature. The solid arrow marked "returns/persists", extending from each interface to the respective domain entity box, signifies a UML association — the methods of the interface operate solely with instances of that domain entity, never with any other form of the data.

Collectively, these three types of relationships precisely represent the Dependency Inversion Principle, the "D" in the well-known SOLID principles of object-oriented design: a high-level module (JobController, which implements business rules) ought to rely on an abstraction (IJobRepository), and a low-level module (PostgresJobRepository, a concrete implementation) should likewise depend on that same abstraction, rather than the high-level module relying directly on the low-level module (Martin, 2017; Bass, Clements, and Kazman, 2021). There are two specific, tangible advantages for the HYDRA project in particular.

First, regarding testability: the team can utilize a mock, in-memory version of IJobRepository (or any of the other five interfaces mentioned previously) while creating unit tests for the related controller (refer to Section 7 on unit and integration testing), without the need for an operational PostgreSQL database during every test execution — a strategy that effectively facilitates the quick, isolated automated testing that continuous integration relies on, as tests demanding a real database generally tend to be slower, less reliable, and more challenging to execute in parallel than those that do not (Humble and Farley, 2010; Forsgren, Humble and Kim, 2018). Secondly, adaptability: should the team's decision to use PostgreSQL (validated in Sections 4.3 and 5.3) require alteration — for instance, if a later iteration of HYDRA necessitated switching to another managed database service for financial or scaling considerations mentioned in Section 8 — only the six Postgres[Entity]Repository implementations would require modification; all controllers within the system, as well as each method signature that client code relies upon, would stay completely unaltered, as they rely solely on the interfaces, not on the PostgreSQL-specific classes.

### 5.1.6 Integration with the wider architecture

During the architecture-design phase of the project (as detailed in Section 2.3's Work Breakdown Structure), two alternatives were evaluated and subsequently dismissed, and it is important to explain the reasons for their rejection instead of merely presenting the Repository pattern as an unquestioned default option. An Active Record strategy, where each domain entity manages its own persistence (like having a save() method in the Job class that interacts with the database directly), was dismissed because it would directly tie every domain class in the ERD from Section 4.4 to PostgreSQL. This would compromise the layered architecture discussed in Section 5.2 and complicate the unit testing of domain classes in isolation, as any domain object under test would inherently possess a database dependency upon instantiation

A per-query raw Data Access Object (DAO) approach, lacking a shared interface to connect related queries, was also dismissed, as without the IJobRepository-style interface, nothing in the codebase would reliably ensure that a specific Postgres[Entity]Repository implements all operations expected by its corresponding controller — such errors would only emerge at runtime, or even worse, just in production, instead of being immediately identified by the compiler/type system during build time, which is exactly the type of error the interface-based Repository pattern aims to avert (Fowler, 2018; Richards, 2015).

## 5.2 Architecture Patterns

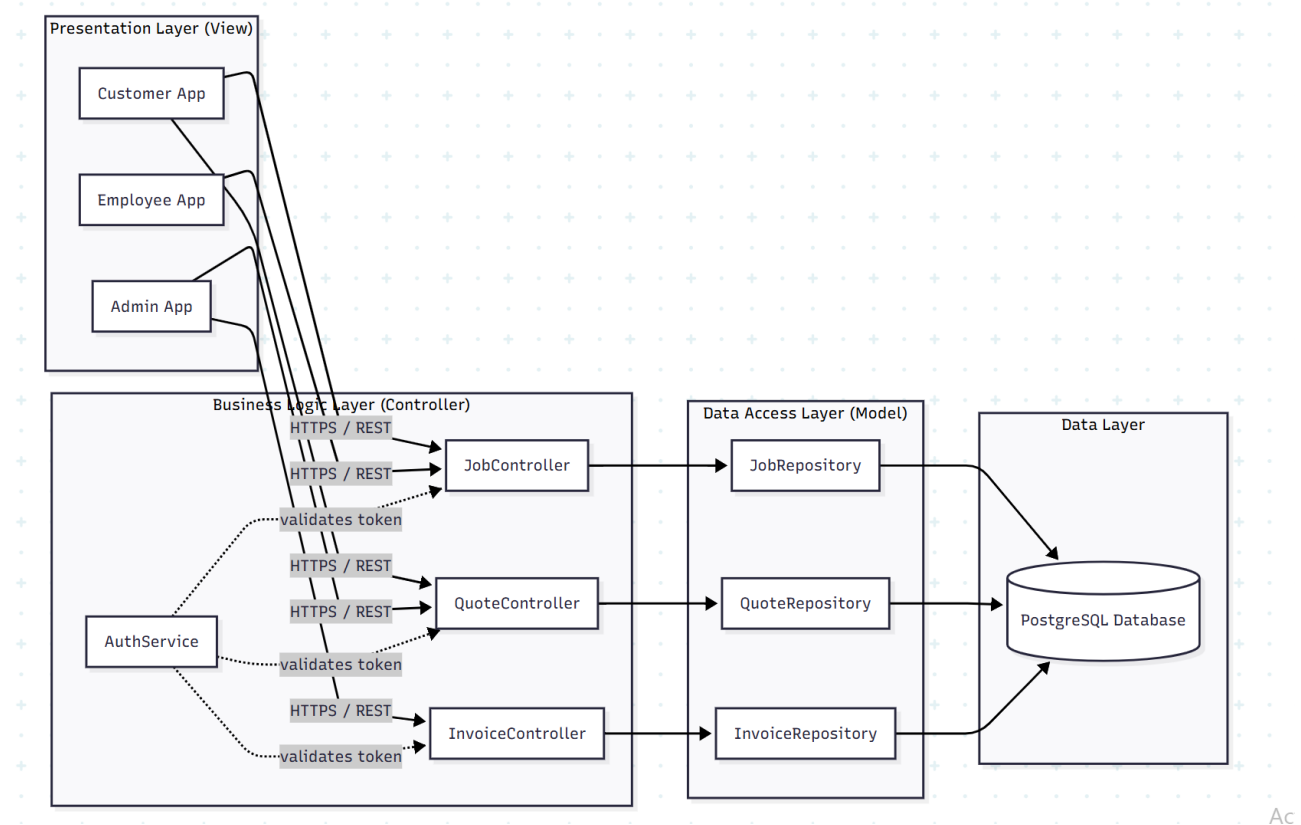


Figure 32: HYDRA Layered/N-Tier Architecture Pattern

The Repository pattern shown in Figure 31 is not an independent design decision; it is the specific approach that allows the layered architecture mentioned in Section 5.2 to operate successfully in reality, and it is essential to make that relationship clear here rather than leaving it assumed. The Data Access Layer, as outlined in Section 5.2, generally comprises six repository groups referenced in this section. The claim in Section 5.2 that "any one layer can be altered with minimal risk of disrupting the adjacent layers" is accurate because the interface boundary of the Repository pattern allows for internal modifications to the Data Access Layer (like a different query, an extra index, or even a different database engine) without requiring changes in the Business Logic Layer above it. Similarly, every method signature presented here — `findById`, `save`, `findByCustomer`, `update`, and their equivalents in the other five triads — is directly associated with an operation previously detailed in the Design Class Diagram's overview of class methods in Section 3.2.

### 5.2.1 Choice of pattern and alternatives considered

HYDRA follows a classic **layered, or N-tier, architecture pattern**, in which the system is divided into four distinct layers, each with a single, well-defined responsibility and each communicating, in the normal case, only with the layer directly next to it (Richards, 2015; Sommerville, 2016; Bass, Clements and Kazman, 2021). Three alternatives were explicitly weighed against this choice before it was adopted. A **monolithic, unlayered**

**application**, in which controller, business-rule and data-access code are freely intermixed within the same classes with no enforced separation, was rejected because it would very quickly become difficult for a six-person, rotating student team to work on in parallel without introducing merge conflicts and inconsistent conventions — the layered structure gives each team member a clear, bounded area of the codebase to own (a specific controller, or a specific repository) without needing to understand the entire system in order to make a safe change. A full **microservices architecture**, in which each domain area (Job, Quote, Invoice, Payroll, and so on) is deployed as an entirely separate, independently-runable service communicating over a network, was rejected as disproportionate to HYDRA's actual scale: microservices solve organisational and scaling problems that arise when many independent teams need to deploy different parts of a system on independent schedules, and when a system's load genuinely requires different components to scale independently of one another, neither of which applies to a single-branch electrical and solar business's job-management system built by one small student team on one shared release schedule, and the operational complexity a microservices split would introduce — multiple deployable services, inter-service network calls, distributed transaction handling, and a correspondingly more complex DevOps pipeline than the one already justified in Section 7 — would represent pure overhead with no compensating benefit at HYDRA's scale (Newman, 2021). An **event-driven architecture**, in which components communicate asynchronously through published events rather than direct request/response calls, was also considered, particularly for the missed-call notification pipeline documented in Section 5.3.3, but was judged to add unnecessary architectural complexity across the *entire* system if adopted wholesale; HYDRA instead adopts a hybrid position, described fully in Section 5.3, in which the core layered request/response architecture documented in this section handles the great majority of HYDRA's interactions, while the specific, genuinely asynchronous parts of the system (scheduled background jobs, missed-call handling) are isolated into their own clearly-bounded pipeline rather than forcing the whole architecture into an event-driven style it does not need. Rozanski and Woods (2012) note that architectural pattern selection should be driven by the specific quality attributes a system needs to prioritise, not by industry fashion; for HYDRA, those priorities are maintainability by a small, rotating student team and straightforward operability by a client with no in-house technical staff (Section 1.2), both of which a layered architecture serves more directly than either a distributed microservices split or a fully event-driven alternative would at this scale.

### 5.2.2 The four layers, described component by component

HYDRA adheres to a traditional layered, or N-tier, architectural model, where the system is segmented into four unique layers, each assigned a specific responsibility and typically interacting only with the adjacent layer (Richards, 2015; Sommerville, 2016; Bass, Clements and Kazman, 2021). Three options were clearly considered before this decision was made. An application structure that is monolithic and unlayered, where controller, business logic, and data access codes are freely blended within the same classes without enforced separation, was dismissed because it would soon become challenging for a rotating six-person student team to collaborate concurrently without causing merge conflicts and inconsistent conventions — the layered architecture provides each team member a defined, limited scope within the codebase to manage (a designated controller or a specific repository) without requiring a comprehensive understanding of the entire system to implement a secure modification.

A complete microservices architecture, where every domain (Job, Quote, Invoice, Payroll, etc.) is deployed as its own separate, independently executable service communicating over a network, was deemed unnecessary for HYDRA's current needs: microservices address scaling and organizational challenges that emerge when multiple teams must deploy different parts of a system on different schedules, or when a system's workload requires separate components to scale independently. Neither condition applies to a single-branch electrical and solar company's job-management system, built by one small student team on a

unified release schedule. The additional operational complexity from splitting into microservices—multiple deployments, network calls between services, distributed transaction handling, and a more intricate DevOps pipeline than already outlined in Section 7—would only add overhead without any tangible advantage at HYDRA’s current scale (Newman, 2021).

An architecture driven by events, where parts interact asynchronously via published events instead of direct request/response exchanges, was reviewed—especially for the missed-call notification pipeline outlined in Section 5.3.3—but it was determined that adopting this approach entirely would introduce unnecessary complexity across the entire system. HYDRA therefore follows a mixed strategy, fully described in Section 5.3, where the main layered request/response setup detailed here manages most interactions while the truly asynchronous components (scheduled background tasks, missed-call handling) are grouped into separate, well-defined pipelines without forcing the whole system into an event-driven style it does not require. Rozanski and Woods (2012) advise that choosing an architectural pattern should align with the key quality attributes a system must emphasize, not follow current trends; for HYDRA, these priorities are upkeep by a small, changing student team and easy operation by a client lacking internal technical support (Section 1.2), both better met by a layered design than a distributed microservices split or a fully event-driven model at this level.

The **Data Access Layer (Model)**, shown as its own bounding box to the right of the Business Logic Layer, consists of the repository classes described fully in Section 5.1 — JobRepository, QuoteRepository and InvoiceRepository are again shown as the representative examples in Figure 32, corresponding to the PostgresJobRepository and equivalent classes fully detailed in Figure 31 — each responsible only for translating a controller's request into a database operation and back, with no business logic of its own; a repository does not decide *whether* a job should be assigned to a given electrician (that is JobController's responsibility), it only knows *how* to save and retrieve a Job record once that decision has already been made.

Finally, the **Data Layer**, shown at the far right of Figure 32 as a cylinder — standard notation for a database in both UML deployment diagrams and informal architecture diagrams — is the PostgreSQL database itself, which stores and persists every entity described in Section 4.4's Entity-Relationship Diagram, and is reached only through the Data Access Layer immediately to its left, never directly from the Business Logic Layer or the Presentation Layer.

### 5.2.3 Request flow — a worked example traced through all four layers

Figure 32 displays the structure divided into four horizontal sections, linked by labeled arrows that indicate the protocol at each connection point, with each section explained in the same left-to-right, top-to-bottom sequence as shown in the illustration.

The Presentation Layer (View), positioned at the top left within its own frame, includes the three client applications — Customer App, Employee App, and Admin App — each represented as distinct boxes without any embedded business logic; the primary function of each application is to display information to its user and transmit that user’s actions as an HTTPS/REST request to the relevant controller in the layer below. The diagram illustrates each of the three applications with separate lines connecting downward to the Business Logic Layer, indicating that a single client application may interact with multiple controllers during normal operation — for example, the Admin App communicates with JobController when assigning a task, QuoteController when preparing a quote for that task, and InvoiceController after task completion, all within one user session, and the multiple intersecting lines from each app to the controller layer reflect this many-to-many connection rather than a strict one-to-one relationship.

The Business Logic Layer, also called the Controller, is depicted as the larger box below the Presentation Layer. This layer is the Node.js API and includes one controller for each domain area. In Figure 32, JobController, QuoteController, and InvoiceController are highlighted as examples, while the rest are fully listed in Figure 31 and Section 5.1 following the same structure. Additionally, there is a separate AuthService box on the left side of this layer, which all controllers use to verify the caller's authentication token before handling a request. In the diagram, this is shown by dotted lines labeled "validates token" connecting AuthService to JobController, QuoteController, and InvoiceController. The use of dotted lines, unlike the solid HTTPS/REST arrows between the Presentation Layer and the controllers, signals that token validation is an internal, synchronous process performed before the controller's business logic runs. Clients do not call AuthService directly; instead, each controller uses it internally as a prerequisite for its operation. This layer contains HYDRA's core business rules, such as job assignment rules, quote-to-invoice conversion, payroll calculation, rewards-point accrual, and query triage. Neither the Presentation Layer nor the Data Access Layer is allowed to implement any part of these rules.

The Data Access Layer (Model), displayed in its own box to the right of the Business Logic Layer, includes the repository classes outlined in Section 5.1 — JobRepository, QuoteRepository, and InvoiceRepository appear as the key examples in Figure 32, matching the PostgresJobRepository and similar classes detailed in Figure 31 — each designed solely to convert a controller's request into a database operation and return the result, without containing any business logic; a repository does not determine if a job should go to a specific electrician (that task belongs to JobController), it simply handles the storage and retrieval of a Job record once that choice has been made.

The Data Layer, illustrated at the far right of Figure 32 as a cylinder — a common symbol for databases in UML deployment diagrams and informal architecture drawings — is the PostgreSQL database itself, which holds and maintains every entity from Section 4.4's Entity-Relationship Diagram and is accessed only through the Data Access Layer directly beside it, never directly from the Business Logic Layer or the Presentation Layer.

#### 5.2.4 Why strict layering matters, and what it protects

To illustrate the abstract layer descriptions with a real example, it is helpful to follow a single, practical HYDRA request through all four layers in Figure 32, using the customer-quote-acceptance scenario from Section 2.1's User Stories as the reference case. A customer interacts with the Customer App on the Presentation Layer and selects "Accept Quote" for a received quote; the Customer App then sends an HTTPS/REST request to the QuoteController in the Business Logic Layer. Before proceeding with any processing, QuoteController invokes AuthService—as shown by the dotted "validates token" line in Figure 32—supplying the customer's authentication token. AuthService then verifies that the token is valid, has not expired, and is linked to a Customer-role account authorized to accept the quote (a role-based access control check that directly supports the authentication and authorization requirements outlined in Section 6). Only after this validation completes does QuoteController proceed with its core function: updating the quote status to accepted and, following the state-transition rules in the Quote lifecycle from Section 4.1, advancing the associated job's state from "Quoted" to "Approved." To record this update, QuoteController contacts the QuoteRepository (Data Access Layer), passing the modified Quote domain entity. QuoteRepository then converts this into a parameterized SQL UPDATE statement and executes it against the PostgreSQL database (Data Layer) through its dedicated connection pool, matching the process described for the similar PostgresJobRepository example in Section 5.1.2. The outcome travels back through the same reverse route—from the database to the repository, then to the controller, and finally to the Presentation Layer—where each layer converts the result into the format required by the layer above: a raw database row transforms into a Quote domain entity at the repository interface, and this domain entity shifts into an HTTP JSON response at the controller boundary.



before being displayed as the updated screen state in the Customer App. Throughout this entire sequence, no layer bypasses the one directly below it, a strict adherence to layering that delivers the primary benefit of this pattern, as explored further in Section 5.2.4 below.

### 5.2.5 Trade-offs acknowledged

No layer is allowed to bypass another in HYDRA's structure. The Presentation Layer cannot interact directly with the Data Access Layer, and the Data Access Layer must not include any business rules. This strict separation is what provides the system's primary advantage: each layer can be updated independently—whether adding a new client application, adjusting a controller's business logic, or switching the database technology—without disrupting adjacent layers, as explained in the Repository pattern discussion in Section 5.1.4 (Pressman and Maxim, 2020). This layered design is also the reason the Repository pattern in Section 5.1 functions: the Data Access Layer remains shielded behind a uniform interface, allowing the Business Logic Layer to rely on an abstraction instead of a direct database implementation. Additionally, enforcing authentication validation at the top of the Business Logic Layer—before any controller-specific operations begin, as demonstrated in the example from Section 5.2.3—ensures that access control is uniformly applied across all controllers, preventing duplication or inconsistency in security checks within individual controllers (OWASP Foundation, 2021). Centralizing this critical security function into a single shared AuthService, rather than scattering token-validation logic across JobController, QuoteController, InvoiceController, and others, shrinks the potential attack surface a security reviewer must assess. This architectural choice stands as a tangible example supporting the "secure by design" arguments presented more thoroughly in Section 6.

## 5.3 Cloud Architecture

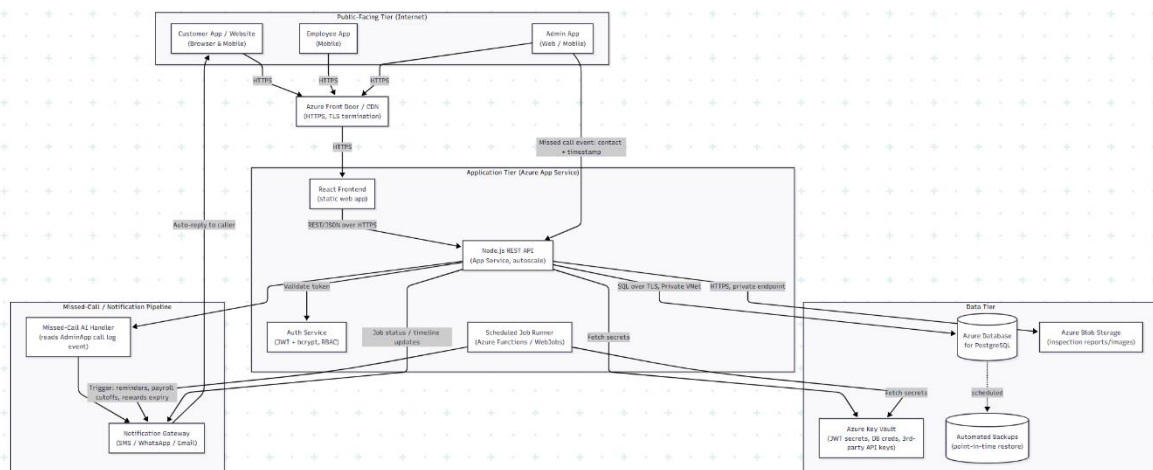


Figure 33: HYDRA Cloud Architecture Diagram

A strictly layered architecture does come with certain trade-offs, and it is important to recognize these rather than treating the choice as unquestionably and universally optimal. Every request must traverse all four layers, even for simple operations—a basic "get job by ID" operation still follows the path Presentation → Controller → Repository → Database and back, whereas a more closely integrated design might allow skipping directly from the Presentation Layer to the database for such a straightforward read. This introduces minor processing and network overhead compared to a design where layers can be omitted in the simplest cases. Additionally, in systems larger than HYDRA, a purely layered approach can lead to "thin" controllers that merely relay calls to a repository without adding any substantial logic, creating unnecessary structural formality without delivering meaningful value (Richards, 2015).



For HYDRA specifically, neither of these downsides was considered severe enough to outweigh the advantages of the layered architecture. The system's scale—a single-branch business's job volume, detailed fully in Section 8.2 as varying from roughly 100 to, at most, 10,000 jobs per month—does not produce a noticeable, user-facing performance penalty from the extra indirection caused by layering. Even at the upper limit of that range, the additional processing time per request is measured in milliseconds, and the total request duration is primarily constrained by network latency rather than the time spent moving through internal layers. Furthermore, as illustrated in Section 3.2's Design Class Diagram and in the demonstrated request-flow example in Section 5.2.3, HYDRA's controllers and domain classes incorporate meaningful, non-trivial business logic—such as job assignment rules, quote-to-invoice conversion, payroll calculation, and rewards accrual—rather than functioning as mere pass-through mechanisms for a repository. This means the layered structure actively contributes functional value for this particular system rather than existing as an empty procedural formality.

### 5.3.1 Chosen cloud provider and justification

HYDRA is hosted entirely on **Microsoft Azure**. This choice was made for three reasons directly relevant to the project's specific context, each of which is expanded on here rather than merely asserted. First, **cost**: Azure's App Service and Azure Database for PostgreSQL both offer free or low-cost tiers suitable for a student-led project with no operating budget during development, while still offering a clear, low-friction upgrade path — documented in full, with concrete Rand figures, in Section 8 — if PSG Electrical and Cables/Trite Solar choose to continue running the system commercially after handover, an important consideration given that cloud cost-management guidance consistently emphasises matching resource tier to actual, current load rather than over-provisioning for a hypothetical future scale that may never materialise (Microsoft, 2024; Erl, Puttini and Mahmood, 2013). Second, **ease of use**: Azure App Service supports deployment slots natively, used for the staging/production, zero-downtime rollout strategy described in Section 4.2 and Section 7.4, removing the need for the team to design and build custom blue-green deployment tooling from scratch within an already time-constrained WIL engagement (Section 2.3's Work Breakdown Structure). Third, **security**: Azure Key Vault, VNet integration and private endpoints — all present in Figure 33 and discussed in full in Sections 5.3.5 and 6 — give the system enterprise-grade secret management and network isolation (Microsoft, 2024) without the team needing to design, build and operate that infrastructure themselves from first principles, which matters given that, per the Partnership Agreement referenced in Section 9.2, the HYDRA team is not responsible for ongoing infrastructure maintenance after handover, and the client (Section 1.2) has no in-house capability to operate custom-built security infrastructure of that kind even if the team had built it.

It is worth acknowledging, in the interest of a balanced and complete architectural justification, the principal trade-off this choice accepts: adopting Azure's managed services (App Service, Azure Database for PostgreSQL, Key Vault, Front Door, Blob Storage, Functions) creates a meaningful degree of **vendor lock-in** relative to a design built entirely on portable, self-managed, open-source infrastructure that could, in principle, run unmodified on any cloud provider or even on-premises. This trade-off was accepted deliberately rather than overlooked: the Repository pattern documented in Section 5.1 already isolates the *database access* layer specifically to reduce the cost of a future migration away from PostgreSQL if ever required, and PostgreSQL itself, as an open-source relational database, is available as a managed offering from every major cloud provider (Azure, AWS RDS, Google Cloud SQL), meaning the single most consequential technology decision in the stack remains portable even though the surrounding managed services (Front Door, Key Vault, Functions) are Azure-specific and would need to be replaced with equivalent services from another provider in the event of a future platform migration. Given the client's complete lack of in-house infrastructure expertise (Section 1.2) and the corresponding, overriding priority placed on ease

of operation and low ongoing administrative burden over theoretical multi-cloud portability, this trade-off was judged clearly acceptable for HYDRA's context, consistent with the pragmatic, quality-attribute-driven approach to architectural decision-making advocated by Rozanski and Woods (2012) and referenced already in Section 5.2.1.

### 5.3.2 Architecture overview — tier by tier

HYDRA operates exclusively on Microsoft Azure. Three project-specific factors guided this decision, each explained below instead of simply stated. First, cost plays a role: Azure's App Service and Azure Database for PostgreSQL provide free or low-cost options ideal for a student-led initiative lacking development funds, while also offering a straightforward upgrade path—fully detailed with specific Rand figures in Section 8—if PSG Electrical and Cables/Trite Solar opt to keep the system running commercially after deployment, a key point since cloud cost advice recommends aligning resource levels with current usage rather than overestimating future needs that may never arise (Microsoft, 2024; Erl, Puttini and Mahmood, 2013). Second, usability matters: Azure App Service includes built-in deployment slots used for the staging/production, zero-downtime deployment method outlined in Section 4.2 and Section 7.4, eliminating the need for the team to develop custom blue-green deployment tools within their already tight Work-Integrated Learning timeline (as described in Section 2.3's Work Breakdown Structure). Third, security is crucial: Azure Key Vault, VNet integration, and private endpoints—shown in Figure 33 and covered thoroughly in Sections 5.3.5 and 6—provide enterprise-level secret management and network isolation (Microsoft, 2024) without requiring the team to build and maintain that infrastructure themselves, which is significant given the Partnership Agreement noted in Section 9.2, where the HYDRA team bears no responsibility for ongoing infrastructure upkeep after handover, and the client (Section 1.2) lacks the internal capacity to operate custom security systems of this nature even if they had been developed.

To ensure a thorough and balanced justification, it is important to note the main compromise this selection involves: using Azure's managed offerings (App Service, Azure Database for PostgreSQL, Key Vault, Front Door, Blob Storage, Functions) introduces notable vendor lock-in compared to a design built entirely on portable, self-managed, open-source solutions that could theoretically run unchanged across any cloud provider or on-premises. This trade-off was intentional rather than accidental: the Repository pattern outlined in Section 5.1 already separates the database layer to ease a potential future migration away from PostgreSQL if needed, and PostgreSQL itself, as an open-source relational database, is available as a managed service from every major cloud provider (Azure, AWS RDS, Google Cloud SQL), meaning the single most impactful technology choice in the architecture remains portable even though surrounding managed services (Front Door, Key Vault, Functions) are Azure-specific and would need replacement with alternatives if switching platforms later. Given the client's lack of internal infrastructure knowledge (Section 1.2) and the overriding focus on operational simplicity and minimal ongoing maintenance over theoretical multi-cloud flexibility, this compromise was deemed fully justified for HYDRA's situation, aligning with the practical, quality-attribute-focused architectural decision process supported by Rozanski and Woods (2012) and referenced earlier in Section 5.2.1.

The **Application Tier (Azure App Service)**, shown as the large central bounding box, receives traffic from Front Door over a further internal arrow labelled **HTTPS**, and contains four components. The React Frontend (static web app) is the compiled, static bundle of the web-facing portions of HYDRA's Presentation Layer (Section 5.2.2), served to browsers and communicating with the backend over an arrow explicitly labelled **REST/JSON over HTTPS**. The Node.js REST API (App Service, autoscale) is the Business Logic Layer described fully in Section 5.2 — every controller and repository documented in Sections 5.1 and 5.2 physically executes inside this single Azure App Service resource, configured with autoscaling enabled, discussed further in Section 5.3.4. The Auth Service (JWT + bcrypt, RBAC), connected to the Node.js REST API by an arrow labelled

**Validate token**, is the concrete, deployed realisation of the AuthService box already introduced abstractly in Figure 32 and Section 5.2.2: it issues and validates JSON Web Tokens, hashes and verifies user passwords using bcrypt, and enforces role-based access control distinguishing Customer, Electrician, Admin and Manager/Owner permissions, exactly as required by the user-role model established in Section 2.1 and discussed in full security detail in Section 6. The Scheduled Job Runner (Azure Functions / WebJobs), shown connected to the Node.js REST API by an arrow labelled **Job status / timeline updates**, is the deployed realisation of the background/scheduled compute cost category already introduced in Section 8.1.3: it independently and periodically triggers time-based work — overdue-invoice transitions, rewards-expiry checks, and reminder generation — without requiring an end user's request to be the thing that initiates that work, since some business rules (an invoice becoming overdue, a rewards point expiring) are triggered by the passage of time itself rather than by any user action.

Separately, the **Admin App** is shown in Figure 33 with an additional direct connection, distinct from its normal HTTPS route through Front Door, labelled "**Missed call event: contact + timestamp**", running straight into the Application Tier. This represents a deliberately different data path for a specific, narrow purpose, discussed in full in Section 5.3.3 immediately below: rather than a full page-load-style interaction, a missed-call event is a lightweight, minimal payload — precisely the two fields the label names, contact and timestamp, and nothing more, directly consistent with the "permitted contact information" data-minimisation principle established in Section 3.1 and Section 6.2 — pushed into the backend the moment the Admin App detects a missed call in the device's own call log.

The **Missed-Call / Notification Pipeline**, shown as its own bounding box positioned between the Application Tier and the Data Tier, is described in full in Section 5.3.3.

The **Data Tier**, shown as the rightmost bounding box, contains four components. Azure Database for PostgreSQL, reached from the Node.js REST API over an arrow labelled **SQL over TLS, Private VNet**, is the physical, managed realisation of the Data Layer described abstractly in Figure 32 and Section 5.2.2, storing every entity documented in Section 4.4's ERD. Azure Blob Storage (inspection reports/images), reached over an arrow labelled **HTTPS, private endpoint**, stores the inspection-report photographs and compliance documents already discussed from a cost perspective in Section 8.1.4. Azure Key Vault (JWT secrets, DB creds, 3rd-party API keys) is reached by two separate "**Fetch secrets**" arrows — one from the Node.js REST API, one from the Missed-Call/Notification Pipeline components discussed in Section 5.3.3 — reflecting that both the core application and the separate notification pipeline independently need access to secrets (respectively, JWT signing keys and database credentials for the former; SMS/WhatsApp provider API keys for the latter) without either ever storing those secrets in its own configuration files, directly supporting the secure credential-management principle discussed in Section 6.2.3. Finally, Automated Backups (point-in-time restore), connected to the PostgreSQL database by a dotted arrow labelled "**scheduled**", provides the data-durability safety net discussed fully in Section 5.3.4 and Section 6.6.

### 5.3.3 The missed-call and notification pipeline in detail

Figure 33 is divided into four main bounding boxes and one extra pipeline box situated between the Application Tier and the Data Tier. Each box is explained here in detail, covering all components and labelled connectors in the usual data flow sequence through the system.

The Public-Facing Tier (Internet), displayed at the diagram's top, includes three client applications — Customer App / Website (Browser & Mobile), Employee App (Mobile) and Admin App (Web / Mobile) — each connecting via an arrow marked HTTPS specifically into Azure Front Door / CDN (HTTPS, TLS termination), the only component in the entire setup directly exposed to the public internet, elaborated further in Section

5.3.5's network-isolation discussion. Front Door handles two simultaneous roles, clearly named on the diagram: it functions as a Content Delivery Network, caching and delivering static content (primarily the compiled React frontend bundle) from edge locations closer to users, discussed earlier in Section 8.1.6 in terms of cost; and it performs TLS termination, meaning Front Door, rather than the servers behind it, manages the SSL/TLS certificate and decrypts incoming HTTPS traffic, also covered from a cost angle in Section 8.1.8.

The Application Tier (Azure App Service), illustrated as the large central box, receives traffic from Front Door through another internal arrow marked HTTPS. It contains four parts. The React Frontend (static web app) is the compiled, static package of HYDRA's Presentation Layer (Section 5.2.2), delivered to browsers and interacting with the backend via an arrow explicitly labelled REST/JSON over HTTPS. The Node.js REST API (App Service, autoscale) represents the Business Logic Layer fully detailed in Section 5.2 — every controller and repository listed in Sections 5.1 and 5.2 runs inside this single Azure App Service instance, set up with autoscaling enabled, explained further in Section 5.3.4. The Auth Service (JWT + bcrypt, RBAC), linked to the Node.js REST API by an arrow labelled Validate token, is the actual deployed version of the AuthService box introduced earlier in Figure 32 and Section 5.2.2: it generates and validates JSON Web Tokens, hashes and verifies user passwords using bcrypt, and applies role-based access control separating Customer, Electrician, Admin and Manager/Owner permissions, exactly as specified in the user-role model from Section 2.1 and examined in full detail in Section 6. The Scheduled Job Runner (Azure Functions / WebJobs), connected to the Node.js REST API by an arrow labelled Job status / timeline updates, is the live implementation of the background/scheduled compute cost category introduced in Section 8.1.3: it independently and regularly executes time-based tasks — moving overdue invoices, checking rewards expiration, and generating reminders — without waiting for a user request, since certain business rules (an invoice becoming overdue, rewards points expiring) are triggered by time itself rather than by user actions.

Separately, the Admin App in Figure 33 features an extra direct link, separate from its standard HTTPS route through Front Door, labelled "Missed call event: contact + timestamp", running straight into the Application Tier. This represents a purpose-built, narrow data path, discussed fully in Section 5.3.3 immediately below: instead of a full page-load interaction, a missed-call event sends a lightweight, minimal payload — exactly the two fields mentioned in the label, contact and timestamp, and nothing else — directly into the backend as soon as the Admin App detects a missed call in the device's call log, aligning with the "permitted contact information" data-minimisation rule from Section 3.1 and Section 6.2.

The Missed-Call / Notification Pipeline, shown as its own bounding box positioned between the Application Tier and the Data Tier, is explained in full in Section

### 5.3.3.

The Data Tier, shown as the rightmost box, contains four elements. Azure Database for PostgreSQL, accessed from the Node.js REST API via an arrow labelled SQL over TLS, Private VNet, is the physical, managed version of the Data Layer described abstractly in Figure 32 and Section 5.2.2, storing every entity listed in Section 4.4's ERD. Azure Blob Storage (inspection reports/images), reached over an arrow labelled HTTPS, private endpoint, holds the inspection-report photos and compliance documents already examined for cost considerations in Section 8.1.4. Azure Key Vault (JWT secrets, DB creds, 3rd-party API keys) is accessed through two separate "Fetch secrets" arrows — one from the Node.js REST API, one from the Missed-Call/Notification Pipeline components discussed in Section 5.3.3 — reflecting that both the main application and the separate notification pipeline independently need access to secrets (JWT signing keys and database credentials for the former; SMS/WhatsApp provider API keys for the latter) without ever storing them in configuration files, supporting the secure credential-management principle outlined in Section 6.2.3. Finally,

Automated Backups (point-in-time restore), connected to the PostgreSQL database by a dotted arrow labelled "scheduled", provides the data durability safeguard discussed thoroughly in Section 5.3.4 and Section 6.6.

### 5.3.4 Autoscaling, availability and resilience considerations

The Missed-Call / Notification Pipeline, shown as its own distinct box in Figure 33, solves a specific, practical business challenge identified by clients rather than being a generic notification feature added arbitrarily: PSG Electrical and Cables/Trite Solar's electricians and office staff are often on job sites, on calls, or otherwise unavailable to take incoming customer calls, and under the company's prior manual system (Section 1.2), a missed call meant a lost opportunity — no automatic record existed, and no follow-up happened unless staff noticed a missed-call alert and remembered to call back. The pipeline directly addresses this: the Missed-Call AI Handler (reads AdminApp call log event), positioned at the left of the pipeline box, receives the lightweight "contact + timestamp" event already described in Section 5.3.2 from the Admin App, and handles two linked outputs, both shown explicitly on the diagram. First, via the arrow labelled "Auto-reply to caller", running all the way back up to Customer App / Website at the top, the handler triggers an automatic response to the caller — for example, an SMS confirming PSG Electrical and Cables/Trite Solar has received their call and will respond soon, plus a link to submit a formal request through the Customer App or website if they prefer — turning what used to be a silent, forgotten missed call into a recorded, acknowledged customer interaction. Second, the handler's output feeds into the Notification Gateway (SMS / WhatsApp / Email), located at the centre of the pipeline box, which also serves as the unified component sending reminder-type notifications discussed in Section 8.1.9, linked to the Scheduled Job Runner in the Application Tier by an arrow labelled "Trigger: reminders, payroll cutoffs, rewards expiry". Consolidating both the missed-call auto-reply and the scheduled reminder notifications through a single Notification Gateway — instead of building two separate messaging integrations — was a deliberate architectural choice that avoids duplicating SMS/WhatsApp/email provider setup (and its associated cost tracking, already unified in Table 10 of Section 8.1) in two different parts of the codebase, following the "don't repeat yourself" principle common in software architecture best practices (Fowler, 2018).

### 5.3.5 Network segregation and communication protocols

The Node.js REST API runs on an App Service plan where autoscale is turned on, as shown clearly in Figure 33. This means extra compute instances are added automatically when traffic goes up and removed when demand drops, so the team doesn't have to plan for peak capacity all the time. This is a common elastic-scaling feature in public cloud services and a key difference from traditional fixed-capacity hosting, as defined by the National Institute of Standards and Technology in 2011 and Erl, Puttini, and Mahmood in 2013. It also makes the tiered cost models in Section 8.2 (100, 1,000 and 10,000 jobs per month) possible without manually adjusting the setup at each level.

The Automated Backups (point-in-time restore) feature handles data durability risks on its own, separate from the application layer. Even if an app bug corrupts data or an admin deletes it by mistake, the database can be rolled back to any point within the backup window. This is a basic resilience requirement for systems that store financial records like invoices, payments, and payroll, and it directly supports the recovery-point and recovery-time goals outlined in Section 6.6 (Silberschatz, Korth, and Sudarshan, 2020).

Blob Storage isn't labeled with a specific redundancy tier in Figure 33, but in practice it follows standard Azure rules with geo-redundant or zone-redundant replication for any critical data. In HYDRA's case, inspection-report images may matter in future compliance cases, so they get the same high-durability treatment as the transactional database (Microsoft, 2024).



### 5.3.6 Alignment with non-functional requirements

The design strictly separates the public internet from the system's data, and this separation appears in Figure 33's layout: the sole element exposed to the public internet is Azure Front Door. All other parts — the React frontend, the Node.js REST API, the Auth Service, the Scheduled Job Runner, the Missed-Call AI Handler, the Notification Gateway, the database, Blob Storage and Key Vault — reside within the Application Tier, the Missed-Call/Notification Pipeline, or the Data Tier, accessible only over the internal Azure network and never directly from an external client, in line with the principle that a public-facing layer should be kept minimal, with the rest shielded behind it and reachable only through this single, tightly-controlled entry point (Erl, Puttini and Mahmood, 2013; National Institute of Standards and Technology, 2011). Every link in Figure 33 is tagged with the protocol used at that boundary, a deliberate design choice that enables a security reviewer to check each connection and confirm no unencrypted or unauthenticated route exists anywhere in the setup: all outward-facing traffic uses HTTPS; internal application-to-database traffic uses SQL over TLS through a private VNet, so the database is not only protected by passwords but also unreachable from outside Azure's private network regardless of credentials; Blob Storage access relies on HTTPS via a private endpoint, applying the same network-isolation rule to file storage instead of the database; and all secret access goes through Key Vault's dedicated "Fetch secrets" calls rather than embedding secrets in application config files, directly meeting the "secure credential management" requirement in Section 6 and following widely accepted practice against placing credentials in source code, environment files, or config files that could accidentally end up in version control (OWASP Foundation, 2021; Microsoft, 2024). This per-link protocol labelling is exactly what lets this document present a specific, verifiable claim instead of a vague promise: every data route in Figure 33 uses TLS-encrypted transport, a private, non-internet-routable network path, or both — no connector in the diagram is left untagged or relies on an unencrypted protocol, a detail a reviewer or future auditor can confirm directly from the figure.

### 5.3.6 Alignment with non-functional requirements

The cloud architecture outlined in this section is not an isolated technical choice; instead, it serves as the tangible method for fulfilling several non-functional requirements already defined in Section 2.4, and it is important to conclude this section by clearly showing this connection. The scalability NFR is satisfied by the autoscaling App Service plan referenced in Section 5.3.4, with its cost effects fully analyzed in Section 8.2. The reliability/availability NFR is addressed through Azure App Service's built-in uptime assurances, reinforced by the automated, point-in-time-restore backup approach described in Sections 5.3.4 and 6.6. The security NFR is met via the network isolation, TLS-everywhere communication, private VNet, and Key Vault-based secret handling detailed in Section 5.3.5, and further elaborated in Section 6. The usability NFR, particularly the need for field electricians to operate the system reliably on mobile data networks at job locations, is indirectly supported by Front Door's CDN capability, which minimizes the data the Employee App must send for each interaction, as well as by the intentionally compact "contact + timestamp" missed-call event payload discussed in Section 5.3.3. This payload was designed to be minimal specifically because it must transmit dependably over the inconsistent mobile connections technicians may encounter on-site. This section, along with Section 8's cost breakdown and Section 6's security review, therefore presents a unified, logically coherent architectural narrative: the decisions captured in Figure 33 are not made independently but are instead the precise, essential implementation of requirements already validated elsewhere in this document.

## 6. Security

Of the 280 points available in the Task 1 Documentation Checklist, Security holds the largest share with 25 points, reflecting how critical this section is: HYDRA is not a minor internal tool but a platform storing South African consumers' personal contact information, electricians' dispatch addresses, PSG Electrical and Cables/Trite Solar's financial records supporting cash flow, and—through the InspectionReport entity in Sections 3.1 and 4.4—the legally required compliance documents under the Electrical Installation Regulations, 2009 (Republic of South Africa, 2009). A failure here—whether a breach, data integrity issue, or system downtime at the wrong time—could lead to much more than inconvenience: financial harm to clients, reputational damage in an industry reliant on trust and repeat clients, and potential legal consequences under the Protection of Personal Information Act, No. 4 of 2013 (Republic of South Africa, 2013).

The scope here is broad and thorough, covering authentication and authorization (Section 6.1), data protection in transit and at rest (Section 6.2), input validation and injection prevention (Section 6.3), POPIA compliance across all eight lawful-processing requirements (Section 6.4), payment security (Section 6.5), backup and recovery (Section 6.6), audit logging and accountability (Section 6.7), and a full risk analysis with a risk register (Section 6.8). Security is not an afterthought added later but is woven into the design from the start; every measure below is already reflected in the diagrams and data models from Sections 3–5—the shared AuthService component in Section 5.2, the Key Vault in the Cloud Architecture diagram (Section 5.3), role-based data domains in the Access Control ERD (Section 4.4), and dedicated logging entities in the domain model (Section 3.1)—and this section brings those separate decisions together into one clear, complete security overview. This approach follows the well-known software engineering principle that security works best—and costs less to implement—when built into a system's foundation rather than added as fixes later (OWASP Foundation, 2021; International Organization for Standardization, 2013), aligning with the "secure by design" approach expected for systems handling personal and financial data, including guidance from South Africa's Information Regulator on POPIA's security safeguards (Information Regulator (South Africa), 2021).

The rest of this section also outlines the risk approach used to guide HYDRA's security work. Instead of treating every possible threat as equally urgent, the team used a streamlined version of the risk management process from ISO 31000:2018—setting the context, identifying risks, analyzing likelihood and impact, comparing against an agreed risk tolerance, and addressing risks through avoidance, reduction, transfer, or acceptance (International Organization for Standardization, 2018). HYDRA's risk tolerance reflects its status as a free, student-built system for a small South African trade business rather than a large financial firm: the team accepts low-probability, low-impact risks where the cost of full elimination would outweigh the benefit for a business of this scale, while treating any risk rated High in either likelihood or impact as essential to address before launch. This risk tolerance statement—and the likelihood/impact scoring it depends on—is detailed in Section 6.8 and guides all prioritization choices throughout the following sections.

### 6.1 Authentication and Authorisation

#### 6.1.1 Credential submission and transport

HYDRA relies on one unified authentication method for all three client applications — the Customer App/Website, the Electrician (field) App, and the Admin App — which is delivered by the AuthService component within the Business Logic Layer as outlined in Section 5.2 and visually represented in the User Authentication state and sequence diagrams found in Section 4.1. No matter which application a user logs into, the core identity checks, password verification steps, and token generation process remain the same. This approach minimizes the amount of security-sensitive code the team must develop, test, and maintain



by centralizing the logic in a single AuthService instead of three separate, potentially mismatched versions. It also guarantees that any security enhancement made in one spot — such as enforcing stricter password rules or introducing a failed login lockout — is immediately applied across all applications without requiring duplicate updates. Users provide their credentials — either an email address (for Customer and Admin roles) or a staff identifier (for Electrician accounts) along with their password — through whichever application they are using; this credential pair is transmitted solely over the HTTPS/TLS connection detailed in Section 6.2, never in plain text and never embedded in a URL query string, in line with standard web authentication practices that prevent exposure of credentials in formats susceptible to logging or caching by intermediary proxies (OWASP Foundation, 2021).

### 6.1.2 Password storage — bcrypt hashing

The API verifies the submitted credentials against the stored password hash field of the relevant entity, not against the actual password, because passwords are hashed using bcrypt before storage and the plaintext password is discarded immediately after hashing. Bcrypt was chosen over a general-purpose hash function like SHA-256 for three specific reasons. Firstly, bcrypt automatically incorporates a per-password salt, a random value combined with the password before hashing, which prevents precomputed "rainbow table" lookup attacks. Even two customers with the same password will have entirely different hash values. Secondly, bcrypt has a configurable work factor, which slows down the hashing computation by a controllable amount; HYDRA is configured with a cost factor in line with current OWASP recommendations for interactive login systems, which is reviewed and increased periodically to keep brute-force attacks impractical. Thirdly, bcrypt is specifically designed to resist acceleration on GPUs and custom hardware, which an attacker would use against a stolen password database, making it a stronger choice than an unsalted or fast hash for this threat model.

This design decision is also in line with the current NIST Digital Identity Guidelines, which recommend using a slow hash function with a per-credential salt and against arbitrary complexity rules. The guidelines suggest using a generous minimum length and checking against known-breached password lists. HYDRA's registration and password-reset flows enforce a minimum password length of twelve characters instead of a short, complex string. This is because length is a stronger predictor of resistance to brute-force guessing than a short password padded with mandatory symbols.

### 6.1.3 Token issuance and structure (JSON Web Tokens)

After the system confirms a user's credentials, it generates a JSON Web Token (JWT) that follows RFC 7519 (Jones, Bradley and Sakimura, 2015). The client application then includes this token as a bearer token in the Authorization HTTP header for all following authenticated calls. A JWT consists of three segments—a header that specifies the signing algorithm, a claim payload, and a cryptographic signature—and HYDRA's payload intentionally contains precisely the data each Business Logic Layer controller requires to decide on authorization without another database query: the user's unique ID (sub), their role (Customer, Electrician, Admin-Owner or Admin-Office, matching the Admin tiers outlined in Section 2.1 and Section 6.1.5), the relevant customer\_id or electrician\_id when relevant, an issued-at time (iat), and an expiration time (exp). Since the token is signed with a secret obtained from Azure Key Vault (Section 6.2) instead of a hard-coded configuration value, any alteration of the payload—such as an attacker trying to modify their customer\_id claim to another user's ID to assume their identity—breaks the signature instantly, causing the request to be denied before any business logic runs. This process appears as the "Validate token" flow from each controller to AuthService in the Architecture Pattern diagram (Section 5.2), and it is intentionally placed as the initial step in every controller's execution sequence, prior to any repository access or business-rule processing, ensuring that unauthenticated or improperly scoped requests consume minimal resources and are rejected

as early as possible in the request lifecycle—a security strategy often called "failing fast" (Bass, Clements and Kazman, 2021).

6.1.4 Token lifetime, refresh and secure client-side storage

HYDRA provides access tokens with a limited lifespan, intentionally set to a short duration so that even if a token is compromised, it can only be exploited for a brief period before expiring. Alongside this, a refresh token with a longer validity period enables the client application to seamlessly request a new access token without requiring the user to repeatedly enter their password.

For the two mobile-first applications—the Electrician App and the customer-facing mobile app mentioned in Section 1.3—the refresh token is stored in the device's secure credential storage, specifically the iOS Keychain or Android Keystore, instead of regular app storage. This ensures the token is safeguarded by the device's built-in hardware encryption and remains inaccessible to other apps on the same device.

On the browser-based Admin App and Customer Website, the equivalent token is stored in a way that prevents access by arbitrary scripts on the page, reducing the risk that a cross-site scripting vulnerability elsewhere on the page (Section 6.3) could capture an active session token.

Every application includes a clear logout option that instantly removes the locally stored token and notifies the API to invalidate the associated refresh token. This prevents a user from later accessing their account if they log out on a shared or temporary device—a common situation in office settings where multiple administrators might use the same workstation.

To reduce the impact of credential-guessing attempts, the login endpoint enforces strict limits on login attempts and progressively locks the account. If multiple failed login attempts occur within a short timeframe, the system imposes a temporary delay before allowing new attempts, protecting against both direct brute-force attacks on a single account and coordinated credential-stuffing attempts using leaked credentials from unrelated breaches (OWASP Foundation, 2021). This security measure is applied at the API level, before any business logic executes, following the "fail fast" principle outlined in Section 6.1.3.

6.1.5 Role-based access control (RBAC)

Access control based on roles is implemented on top of the authentication layer described earlier. This is done to reflect the existing access division that is already structured in the Access Control ERD (Section 4.4). A role-based access control system was chosen over a more detailed permission model for each user because the user base of HYDRA can be easily divided into a few well-defined roles with distinct responsibilities. These roles include Customer, Electrician, and two levels of Admin. This choice also allows the principle of least privilege to be enforced automatically, at the role definition level, rather than relying on each individual controller method to apply the correct filter manually.

Table 7: HYDRA role-based access control (RBAC) permission matrix

| Module / Data Domain                               | Customer                      | Electrician               | Admin — Office Staff       | Admin — Owner              |
|----------------------------------------------------|-------------------------------|---------------------------|----------------------------|----------------------------|
| Own job requests, quotes, invoices, payments       | Read/Write (own records only) | Read (assigned jobs only) | Read/Write (all customers) | Read/Write (all customers) |
| Other customers' job, quote, invoice, payment data | No access                     | No access                 | Read/Write                 | Read/Write                 |

| Module / Data Domain                               | Customer                        | Electrician                              | Admin — Office Staff          | Admin — Owner              |
|----------------------------------------------------|---------------------------------|------------------------------------------|-------------------------------|----------------------------|
| Job assignment (assigning an electrician to a job) | No access                       | No access                                | Read/Write                    | Read/Write                 |
| Materials logging on an assigned job               | No access                       | Read/Write (own assigned jobs)           | Read only                     | Read only                  |
| Inspection reports                                 | Read (own jobs, once submitted) | Read/Write (own assigned jobs)           | Read/Write                    | Read/Write                 |
| Stock/material inventory levels                    | No access                       | Read (materials relevant to current job) | Read/Write                    | Read/Write                 |
| Electrician staff data, schedules, payroll inputs  | No access                       | Read (own record only)                   | Read/Write                    | Read/Write                 |
| Payroll sign-off / payment release                 | No access                       | No access                                | No access                     | Read/Write                 |
| Rewards and discounts programme                    | Read/Write (own account)        | No access                                | Read/Write (all customers)    | Read/Write (all customers) |
| Data export (DataExportLog)                        | No access                       | No access                                | No access (view-only reports) | Read/Write                 |
| Missed-call / AI messaging configuration           | No access                       | No access                                | Read only                     | Read/Write                 |
| System/user account administration                 | No access                       | No access                                | No access                     | Read/Write                 |

Concretely, before any database query runs, the controller already has the calling user's identity, role, and scope (`customer_id` or `electrician_id`, as stored in the validated JWT) for every repository call outlined in Section 5.1. As a result, a controller acting on behalf of an authenticated customer cannot invoke `JobRepository.findById(customer_id)` if the `customer_id` in the token does not match the requested id, even if an attacker alters the identifier in the request URL or body. This built-in check serves as a structural safeguard against a well-known vulnerability called broken object-level authorisation (BOLA), where an authenticated user gains access to another user's data by changing an identifier in a legitimate request. This vulnerability has repeatedly been one of the most frequently reported API security issues in industry research over the past ten years (OWASP Foundation, 2021). Since this scoping verification is enforced consistently at the repository level rather than relying on individual endpoint checks in the application code, such RBAC is considered a significantly stronger foundational control than scattered, ad hoc authorisation logic.

The Admin role is internally divided further, as mentioned in Section 2.1: the business owner receives the complete Admin-Owner permission set, which includes payroll approval, system and user management, and data export, while office staff are limited to the Admin-Office subset listed in Table 7 (job assignment, quoting, daily customer and job management) through the same role attribute and RBAC mechanism, without needing a separate fifth application. This layered Admin structure reduces the number of accounts with access to highly sensitive functions—especially payroll processing and bulk data export—which directly addresses insider misuse risks noted in the risk register (Section 6.8, Risk 10) and minimises accidental errors by junior staff who do not require, and should not possess, payroll access to perform their routine duties.

## 6.2 Data Protection

Data protection in HYDRA functions at two separate tiers — safeguarding information during transmission across networks and securing stored data — as already shown in the Cloud Architecture and Deployment illustrations in Sections 4.2 and 5.3, with both tiers further shaped by a clear data categorization process outlined below, which specifies the exact level of security required for each data type held by HYDRA.

### 6.2.1 Data classification

Not all elements in the HYDRA data model share the same level of sensitivity, and applying uniform safeguards to every piece of data could either leave truly sensitive information inadequately protected or create unnecessary burdens for less sensitive data. The team organizes data stored by HYDRA into four distinct tiers, modeled after established commercial classification frameworks (Cloud Security Alliance, 2017):

**Public** — information that requires no protection, including general service offerings from Trite Solar or publicly listed business contact details already available on tritesolar.co.za.

**Internal** — data meant for internal use by PSG Electrical and Cables/Trite Solar but not harmful if exposed, such as overall job completion rates or non-personal operational measurements.

**Confidential** — information that could cause significant harm to a person or the organization if shared without permission: customer names, phone numbers, email addresses, and site locations (stored in Customer and Contact); records related to electricians and payroll inputs (kept in Electrician and linked payroll systems); as well as job, quote, and invoice specifics.

**Restricted** — the smallest yet most sensitive group, covering authentication details (password\_hash), payment reference information (Payment), and any data collected via the missed-call AI function regarding individuals who are not registered customers (Contact, MissedCallLog), since these individuals never completed HYDRA's consent-based enrollment process and thus demand the highest level of security.

This tiered approach directly influences the controls outlined in Sections 6.2.2–6.2.4 below: Restricted and Confidential data undergo mandatory encryption during transmission and storage, are the main focus of audit logging discussed in Section 6.7, and represent the fields most tightly controlled under the data-minimization guidelines in Section 6.4.2.

### 6.2.2 Protection in transit

All connections to the system from the three client applications and from Azure Front Door onward to the API are made exclusively using HTTPS. This is achieved by enforcing a minimum of TLS 1.2 and preferring TLS 1.3 where the connecting client supports it. Legacy and weak protocol versions such as SSL 3.0 and TLS 1.0/1.1 are disabled at the Front Door level. In addition, internal connections from the API to the Azure Database for PostgreSQL are secured through SQL/TLS over a private Azure Virtual Network. This is clearly depicted in the Cloud Architecture diagram and the Deployment diagram. No component in the system accepts an unencrypted connection, thereby safeguarding customer information, job-site addresses, and payment details from network interception. This security measure is particularly crucial for HYDRA as it is designed as a mobile-first system. Field electricians will typically access the system from customer premises over public Wi-Fi or cellular networks rather than a secure office network, making them vulnerable to interception by another party sharing the same network.

### 6.2.3 Protection at rest and key management

Azure Database for PostgreSQL automatically encrypts all stored data at rest using AES-256 encryption at the storage level, which meets the essential requirement that a managed cloud database service secures data against theft or unauthorized access at the infrastructure layer without requiring the HYDRA team to set up disk encryption separately (Microsoft, 2024). Additionally, for HYDRA's application security framework, the most critical settings in the system—such as database login details, the JWT signing key outlined in Section 6.1.3, and third-party API keys including payment gateway credentials referenced in Section 6.5—are never embedded in application code or configuration files. Instead, these secrets are fetched on demand from Azure Key Vault, matching the setup displayed in the Cloud Architecture and Deployment diagrams. Key Vault secures these credentials in tamper-resistant HSM-backed storage, enforces strict access controls to ensure only necessary Azure services like the API application can retrieve them, and logs every access attempt, adding another layer of oversight alongside the application logging covered in Section 6.7.

This strategy of keeping secrets out of source code is crucial because source code is typically stored in a version control system (GitHub, as noted in Section 7) that, by nature, is accessible to a broad audience—including all team members and possibly external contributors or auditors—far exceeding the appropriate circle for viewing production passwords or encryption keys. Storing secrets solely in Key Vault instead of in a .env file or committed code is one of the most widely endorsed methods to prevent accidental credential leaks through version control, a persistent issue that remains a leading cause of actual breaches due to how easily it can occur without deliberate safeguards (OWASP Foundation, 2021). Moreover, Key Vault supports secret versioning and scheduled rotation, enabling updates like rotating the JWT signing key on a fixed timetable or immediately if a breach is suspected, all without needing a software deployment.

#### **6.2.4 Data protection on mobile and offline scenarios**

Because the Electrician App is built specifically for on-site work—including situations where connectivity at customer locations is unreliable—it stores a limited set of job information locally. This includes only the electrician's assigned tasks for the day and any draft inspection reports in progress, ensuring the app stays functional during short network outages. The app secures this cached data using the mobile OS's built-in encrypted storage, rather than keeping it in unprotected app files, so a misplaced or stolen device won't expose sensitive job or customer details to whoever finds it.

The cached information is restricted to just the electrician's current assignments, aligning with the role-based access control outlined in Section 6.1.5. Once the device regains connection, this data syncs with the server and is removed, ensuring no single field device ever holds more sensitive information than necessary at any moment.

#### **6.2.5 Data minimisation and retention**

Special attention is given to two categories of highly sensitive personal data processed by HYDRA: customer and contact information, which includes name, phone number, email, and address stored in the Customer and Contact entities, and payment details handled as outlined in Section 6.5. Both categories are limited to the minimum set of fields necessary for the system's operation, adhering to the data-minimality requirement of the POPIA Act discussed in Section 6.4.2. For instance, the Contact entity, introduced in Section 3.1, only stores name, phone number, email, and source, and does not attempt to record a caller's entire communication history beyond what the missed-call feature specifically necessitates. The retention periods for data are similarly restricted by classification tier: financial and compliance records such as Invoice, Payment, and InspectionReport are retained for the duration mandated by South African tax and consumer-protection record-keeping regulations, given their role in demonstrating regulatory compliance. On the other hand, Contact/MissedCallLog records related to non-customers who never proceed to a job are retained only

for a limited period sufficient to support a follow-up inquiry, after which they are scheduled for deletion, an approach consistent with the "not excessive" and "not retained longer than necessary" principles embedded in POPIA's processing-limitation condition (Republic of South Africa, 2013).

## 6.3 Input Validation and Injection Prevention

### 6.3.1 SQL injection prevention via the Repository pattern

All classes in the Repository category outlined in Section 5.1 generate parameterised, prepared-statement queries against PostgreSQL instead of building SQL strings by directly merging user input into a query. This is the most reliable safeguard against SQL injection, which continues to rank among the most severe vulnerability categories in every edition of the OWASP Top Ten across over ten years of vulnerability studies (OWASP Foundation, 2021), since a parameterised query always interprets user-provided data strictly as values to be assigned to parameters, never as executable SQL code, no matter what characters—such as quotes, semicolons, or SQL commands—are included in a harmful or incorrect input. Given that the Repository pattern described in Section 5.1 ensures every database interaction within HYDRA routes through one of these repository classes—there is no route in the system, across any of the three client applications, that allows raw SQL queries to be generated or executed directly from a controller—this protection is embedded into the system's architecture, rather than relying on developers to consistently sanitise inputs whenever they write a query. This architectural enforcement provides a significantly stronger defence than relying solely on code reviews or documentation to prevent unsafe query construction, because it eliminates the possibility of unsafe patterns entirely rather than just discouraging them (Fowler, 2018; OWASP Foundation, 2021).

### 6.3.2 Schema and boundary validation

At the entry point of the API, each incoming request body undergoes validation against a predefined schema—ensuring correct data types, presence of required fields, and compliance with string length and format rules—before processing by any controller logic. Specific instances tied to HYDRA's internal data model include: rejecting a Job.siteAddress submission if empty or surpassing a reasonable maximum length; blocking a Payment.amount that is negative, zero, non-numeric, or beyond the remaining invoice balance; discarding a ContactQuery or Job.notes free-text field if it exceeds a set character threshold; and invalidating an Electrician schedule entry where the start time occurs after the end time. This type of allow-list, schema-first validation—where only requests matching strict structural criteria are accepted, while all others are rejected—is considered a stronger approach than deny-listing, as deny-lists can only block known bad patterns, whereas allow-lists reject anything not explicitly permitted (OWASP Foundation, 2021). This validation layer enhances data reliability, aligning with the Non-Functional Requirement outlined in Section 2.4, and mitigates risks where maliciously crafted input is used to test for unhandled edge cases or system crashes.

Validation performed on the client side (providing instant feedback in the Customer App, Electrician App, and Admin App when fields are omitted or incorrect) is also embedded across HYDRA's interfaces, yet it serves solely as a usability aid rather than a security measure: since client-side checks in JavaScript or mobile apps can be circumvented by direct API calls, every validation enforced locally is strictly enforced again on the server side without exception. This guiding principle of "never trust the client" underpins the entire input-validation framework described in this section (OWASP Foundation, 2021).

### 6.3.3 File upload validation



The Electrician App lets an electrician attach photos and signed files to an InspectionReport when finishing a job. Since file uploads can be risky—like harmful files mislabeled as images or massive files used to drain storage or bandwidth—every uploaded file undergoes three checks before acceptance: its stated and real file type must align with approved formats (JPEG, PNG and PDF), its size must stay within preset limits, and it is kept in a separate Azure Blob Storage container (shown in the Cloud Architecture diagram, Section 5.3) that cannot run code directly and is shared with authorized users only through the API, not via an easy-to-guess public link. Files are never stored under their original names given by users; instead, they get system-assigned IDs, stopping attempts to trick paths hidden in filenames and avoiding accidental mixing or overwriting of attachments between jobs.

#### 6.3.4 Cross-site scripting (XSS) and output encoding

For the public-facing Admin App and Customer Website, any output that includes user-provided text—such as a customer's name shown to an admin reviewing a ContactQuery or the free-text notes field on a Job—is encoded properly based on where it will appear (HTML body, HTML attribute, or JavaScript context) before being shown in the browser. This blocks cross-site scripting (XSS), where an attacker slips in executable code or markup that later runs with the victim's own login rights inside another user's browser when that content is displayed (OWASP Foundation, 2021). Proper, context-sensitive output encoding stops both the "stored" version of this attack—where harmful input is saved in the database and later shown to others (like another admin or a customer checking their job history)—and the "reflected" version, where harmful input is sent back right away in a single request.

#### 6.3.5 Cross-site request forgery (CSRF) resistance

Alongside the JWT-based, session-free login system from Section 6.1—which requires every change-making request to include a valid, user-specific, cryptographically signed token in an Authorization header rather than relying on a browser cookie that gets sent automatically to every domain request—HYDRA is built to resist cross-site request forgery (CSRF). A harmful outside site cannot fake a request with a real, newly issued JWT from another user because it has no way to read or steal that token from the victim's browser storage. This fixes the classic cookie-based CSRF flaw, where a browser's automatic cookie sharing across sites was the weak spot (OWASP Foundation, 2021).

#### 6.3.6 Rate limiting and security headers

Apart from the login-focused rate limiting outlined in Section 6.1.4, the API implements overall request volume limits and throttling for every public interface, both to deter harmful or automated extraction of customer or job details and to safeguard the missed-call AI messaging function (Sections 3.1, 4.1) from deliberate overuse by attackers aiming to exhaust its outbound message quota or test it for unplanned behavior. Each response delivered via Azure Front Door includes a fixed set of HTTP security headers designed to reduce risk, featuring a Content-Security-Policy that restricts executable scripts to approved origins, X-Content-Type-Options: nosniff to block browsers from misclassifying file types as executable content, and HTTP Strict Transport Security (HSTS) to force browsers to reject insecure HTTP connections to HYDRA, even when users manually enter an http:// URL, strengthening the data-in-transit safeguards discussed in Section 6.2.2 (OWASP Foundation, 2021).

### 6.4 POPIA Compliance

PSG Electrical and Cables/Trite Solar operates in South Africa, and HYDRA manages personal data—including customer names, addresses, phone numbers and job histories; electrician staff and payroll details; and, via the missed-call messaging feature, the phone numbers of unregistered individuals—all of which falls under



the Protection of Personal Information Act, No. 4 of 2013 (POPIA) (Republic of South Africa, 2013). POPIA outlines eight conditions for legally processing personal information, and this section examines each one in full, linking each requirement to the corresponding HYDRA design choice rather than focusing only on the most obvious technical aspects.

#### 6.4.1 Accountability

The entity responsible for deciding how and why personal information is processed (POPIA's "responsible party") must prove compliance rather than just claim it. HYDRA achieves this through the DataExportLog entity (Sections 3.1/4.4), which logs every instance an administrator extracts customer or employee data, alongside the audit-logging details in Section 6.7, ensuring the client's handling of personal data remains verifiable beyond mere assumptions. As part of the project's handover materials (Section 9), the team advises PSG Electrical and Cables/Trite Solar to officially appoint an Information Officer—a role POPIA mandates for every responsible party, typically the business's top executive unless delegated—and register this appointment with the Information Regulator, since POPIA's accountability principle places responsibility on a named individual rather than an abstract organisation (Information Regulator (South Africa), 2021).

#### 6.4.2 Processing limitation and minimality

Personal information must be processed lawfully and only to the extent necessary—adequate, relevant and not excessive for its intended purpose. This principle drove the decision, outlined in Section 6.2.5, to restrict the Contact entity to only the fields the missed-call feature truly needs, and to treat Contact as a separate entity from Customer (Section 3.1) instead of forcing every caller into a full customer profile they never agreed to and may never become a customer. It also shapes the field-level scope across the entire ERD (Section 4.4): for instance, HYDRA does not store a customer's identity number, date of birth or any special personal data (POPIA's sensitive category covering health or biometric details) anywhere in the system, since none of it is essential for job quoting, scheduling, invoicing or compliance reporting.

#### 6.4.3 Purpose specification

Personal information may only be processed for a clearly defined purpose the data subject knew about when providing it. The missed-call AI messaging feature (Sections 3.1, 4.1) is narrowly designed to send and log a single automated response to a missed call, notifying the caller their enquiry has been received and will be followed up; it cannot be repurposed, nor does the data structure allow it, for secondary uses like marketing profiling or unsolicited campaigns. Every other data collection point—customer registration, job requests, quote acceptance—is similarly bound to an explicit purpose communicated to the data subject through the onboarding "guided walkthrough" mentioned in Section 2.4.

#### 6.4.4 Further processing limitation

Additional processing of personal data beyond its original intent is only permitted if compatible with that original purpose. Since HYDRA's data model keeps purpose-specific data categories strictly separate—job/billing data, staff/payroll data and missed-call contact data reside on distinct entities rather than combined into a single undifferentiated profile—it avoids creating unintended data-linking risks (such as silently merging a missed-call contact's number with unrelated marketing data) that could raise concerns about further processing.

#### 6.4.5 Information quality

A responsible party must take reasonable steps to ensure personal data is accurate, complete, not misleading and updated when necessary. HYDRA supports this through the customer's ability, via the Customer App, to

review and request corrections to their stored contact and job details, and via schema-level checks described in Section 6.3.2, which block obviously incomplete or malformed data (e.g., an empty address field or an invalid phone number format) from entering the system in the first place.

#### 6.4.6 Openness

A responsible party must document its processing activities and generally inform data subjects about what information is collected, why and who is responsible. This is handled through the consent step in the guided onboarding walkthrough (Section 2.4), through a plain-language privacy notice the team recommends the client publish on the Customer Website and reference during onboarding, and via an internal record of processing activities—a condensed version of the data classification and entity-purpose mapping already provided in Section 6.2.1 and Section 3.1—which the team advises the client to maintain as a living document after HYDRA is handed over, in line with guidance from the Information Regulator (Information Regulator (South Africa), 2021).

#### 6.4.7 Security safeguards

POPIA requires a responsible party to protect the integrity and confidentiality of personal data by implementing suitable technical and organisational measures against loss, damage or unauthorised access. This requirement is met through the combined efforts described in Sections 6.1–6.3 and 6.5–6.7: authentication and role-based access control (6.1), encryption in transit and at rest with data classification (6.2), input validation and injection prevention (6.3), payment security (6.5), backup and recovery (6.6), and audit logging (6.7) form a layered security approach where no single control's failure alone exposes personal information—a "defence in depth" strategy (Bass, Clements and Kazman, 2021; International Organization for Standardization, 2013).

#### 6.4.8 Data subject participation

A data subject has the right to know what personal data a responsible party holds about them, to request access to it, and to request its correction or removal if it is inaccurate, excessive, or obtained unlawfully. This is supported in three practical ways: first, by the Admin App's data-export function, which enables the business to address a formal access request from a customer or employee; second, by the customer's direct self-service access, via the Customer App, to their stored job records, quotes, invoices, and payment history, which addresses most "what information do you have about me" queries without the need for a formal request; and third, through a documented procedure—provided as part of the Section 9 handover documentation—that outlines how PSG Electrical and Cables/Trite Solar should record, process, and respond to formal correction or deletion requests within POPIA's required timeframes, including handling cases where data (such as a completed InspectionReport) must be retained for regulatory reasons, in which case deletion is appropriately refused and the data subject is informed of the specific legal basis for retention.

#### 6.4.9 Cross-border transfer and hosting location

POPIA's eighth condition limits the transfer of personal data to a third party in a foreign country unless that country ensures an adequate level of protection or another specified safeguard is in place. Since Azure provides data-center regions physically located within South Africa, the team advises that the client's production database and blob storage be set up in an Azure South Africa region instead of a default overseas region, keeping customer and employee personal data within South African borders as the simplest way to avoid a cross-border transfer assessment altogether, while still allowing for a future review of this decision, with proper contractual protections, if scaling or disaster recovery needs later require a secondary, geographically distant region.

#### **6.4.10 Consent capture and the missed-call feature specifically**

Consent is obtained explicitly during customer onboarding through the guided process described in Section 2.4 and Section 1.3, and separately, for the Admin-level access to phone contacts and call logs required by the missed-call feature (Sections 3.1, 4.1), consent is limited strictly to the admin's own work device: the admin, acting as the responsible party's employee, consents to their own device's contact and call-log data being processed for this disclosed business purpose; the feature does not request or require access to any customer's or third party's device. The resulting Contact and MissedCallLog records, as already noted under the minimality condition in Section 6.4.2, store only the essential data needed for the automated-response function to work, and every automated message sent by the feature is independently logged and reviewable via AIMessageLog (Section 6.7), giving the business a verifiable, evidence-based response to any future query—from a regulator or an individual who received an automated message—about exactly what was sent and why.

#### **6.4.11 Breach notification**

If a security incident occurs despite the safeguards described in this section, POPIA requires the responsible party to notify both the Information Regulator and the affected data subjects as soon as reasonably possible after discovering the breach, providing enough detail for the data subject to take protective steps (Republic of South Africa, 2013). The audit-logging system described in Section 6.7, together with the risk register in Section 6.8, is intentionally designed to support this duty: because every sensitive action in HYDRA is logged independently, the scope of a suspected breach—including which records were accessed, exported, or modified, by which account, and when—can be determined quickly and with evidence, reducing the time between detecting an incident and being able to notify affected parties with an accurate account of what occurred.

### **6.5 Payment Security**

#### **6.5.1 No card data in HYDRA's own database**

HYDRA never stores raw payment card information in its own database under any circumstances. The Payment entity defined in Sections 3.1, 3.2 and 4.4 only retains the payment's amount, date, method, status, and a reference number provided by the payment processor—never a full card number (the Primary Account Number, or PAN), expiry date, or CVV/CVC security code. This approach aligns with the strong, near-universal industry advice that a business of PSG Electrical and Cables/Trite Solar's size should integrate with a trusted, PCI-DSS-certified third-party payment gateway—such as PayFast, Yoco, or Stripe, all of which operate in or directly support the South African market and settle in ZAR—rather than attempt to develop, secure, and independently certify its own card-handling infrastructure, which would impose a disproportionate compliance burden for a system of this scale and budget (PCI Security Standards Council, 2022).

#### **6.5.2 Tokenisation and reduced PCI-DSS scope**

By offloading the actual capture and handling of card data to a gateway using a hosted-fields or redirect-based integration model—in which the customer enters card details directly into a form provided by the payment gateway, never passing through HYDRA's servers at all—HYDRA falls under a much smaller PCI-DSS compliance scope and may qualify for the simplest applicable Self-Assessment Questionnaire category (commonly known as SAQ-A), rather than the far more demanding scope that would apply if the application ever handled raw card data. Upon successful authorization, the payment gateway returns HYDRA only a token—an opaque reference string with no value outside that specific gateway relationship—and a transaction status, which are stored in the Payment.reference and Payment.status fields. This significantly

reduces both the team's compliance workload during the project and the impact of any future compromise of HYDRA's database: because the Payment table contains no usable card data, a breach of that table alone—no matter how unlikely, per the risk register in Section 6.8—would not expose customers' card details to an attacker, as there is simply no such data present to exploit (PCI Security Standards Council, 2022).

### 6.5.3 Gateway selection considerations

No matter which gateway PSG Electrical and Cables/Trite Solar finally selects—a business choice not covered by this technical document—the security architecture outlined in Sections 6.5.1 and 6.5.2 stays the same, because each of the three possible gateways implements the identical hosted-fields/tokenisation integration pattern that HYDRA relies on.

**Table 8: Comparative considerations across South African payment gateways**

| Consideration                | PayFast                                               | Yoco                                           | Stripe                                                                   |
|------------------------------|-------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------|
| South African market focus   | Purpose-built for the SA market; strong local support | Purpose-built for SA in-person and online SMEs | Global platform with SA support since 2022                               |
| ZAR settlement               | Yes                                                   | Yes                                            | Yes                                                                      |
| PCI-DSS compliance           | Certified, hosted-fields integration                  | Certified, hosted-fields integration           | Certified, hosted-fields integration                                     |
| Developer integration effort | Moderate; well-documented REST API and webhook model  | Moderate; strong point-of-sale heritage        | Low–moderate; extensive documentation, larger global developer ecosystem |
| Typical fee structure        | Percentage-based per transaction                      | Percentage-based per transaction               | Percentage-based per transaction, tiered by volume                       |

Regardless of which specific gateway PSG Electrical and Cables/Trite Solar ultimately contracts with — a commercial decision outside the scope of this technical documentation — the security architecture described in Sections 6.5.1 and 6.5.2 remains identical, since all three candidate gateways support the same hosted-fields/tokenisation integration pattern HYDRA is designed around.

### 6.5.4 Webhook integrity and idempotency

Updates about payment status, such as a quote's linked invoice transitioning from Sent to Paid as shown in the Invoice state diagram in Section 4.1, are triggered by asynchronous webhook notifications sent from the payment gateway to the HYDRA API once a transaction finishes, rather than relying solely on the client application detecting a successful payment. Each incoming webhook undergoes verification using a cryptographic signature provided by the gateway before being deemed trustworthy, thus blocking attempts by attackers to send fake "payment succeeded" messages directly to HYDRA's webhook endpoint. Payment-handling actions are built to be idempotent—meaning they can be safely repeated without unintended consequences—by using a distinct transaction or idempotency key for every payment attempt, which stops situations like duplicate card charges or double invoice markings if a webhook arrives more than once (a normal, expected event in most payment gateway setups, intentionally accounted for rather than treated as an unusual case) (Newman, 2021).

### 6.5.5 Reconciliation, refunds and disputes

Since Payment records link to a transaction reference issued by the gateway instead of raw card details, daily financial reconciliation between HYDRA's invoice and payment records and the payment gateway's settlement reports is done by matching on that reference, giving the Admin-Owner role (as defined in the

RBAC matrix in Table 7) a clear, auditable path from a customer’s original job, through the quote and invoice, to the exact gateway transaction that settled it. Refunds are also started via the gateway’s refund API rather than by altering stored payment data inside HYDRA, and any dispute or chargeback initiated by a card issuer is managed through the gateway’s own dispute-resolution tools, with HYDRA’s role limited to providing the necessary job, quote, and invoice evidence—the same sort of documentary trail that the accountability and audit-logging controls in Sections 6.4.1 and 6.7 are meant to make easy to retrieve.

## **6.6 Backup and Recovery Strategy**

### **6.6.1 Recovery objectives**

HYDRA’s backup design is shaped by two standard disaster-recovery benchmarks: the Recovery Point Objective (RPO), which measures the maximum acceptable data loss in time, and the Recovery Time Objective (RTO), which sets the maximum acceptable time to restore service after an outage (Snedaker, 2013). Because HYDRA stores critical financial and legal documents—especially invoices, payments, and compliance inspection reports—the team has set an RPO target measured in minutes instead of a full day, achieved through continuous transaction-log backups rather than depending only on periodic full snapshots. The RTO target aligns with what a small business can tolerate—a brief, clearly communicated period of reduced service—rather than demanding a zero-downtime guarantee that would be excessive for a free student project of this size.

### **6.6.2 Backup configuration**

As noted in Section 5.3, Azure Database for PostgreSQL is set up with automated, scheduled backups that support point-in-time restore (PITR), combining periodic full backups with continuous transaction-log capture within the configured retention window. This setup directly safeguards the system’s key financial and legal records—particularly rows in the Invoice, Payment, and InspectionReport tables—against three possible risks: hardware or platform failure at Azure’s end, which is addressed by Azure’s managed-service redundancy and geo-replication options; accidental data loss from administrative or application mistakes, such as a poorly scoped bulk update or an unintentional deletion; and, in a worst-case scenario, data loss or corruption from a successful attack (Silberschatz, Korth and Sudarshan, 2020). Point-in-time restore allows the database to roll back to any moment within the retention window, not just to the timestamp of the latest full backup, which significantly cuts down on the amount of valid, intervening business activity—like a customer payment or an electrician’s materials log—that would be lost during recovery compared with a simpler daily-backup-only approach.

### **6.6.3 Geo-redundancy and restore testing**

Backup storage is also set up for geo-redundant replication so that a regional outage in the primary Azure data-centre region does not also wipe out the only copy of HYDRA’s backups. Backup restore procedures are not treated as a theoretical safeguard that is documented once and never tested; the team suggests—and includes in the Section 9 handover material—a regular restore test, where a recent backup is restored into an isolated, non-production environment and checked for data integrity, because an untested backup is, in practical terms, just an unverified assumption rather than a real recovery option (Kim et al., 2016).

### **6.6.4 Disaster recovery runbook and business continuity**

Backup and recovery steps are documented as part of the Handover deliverable described in Section 9, presented as a step-by-step disaster-recovery runbook, so that PSG Electrical and Cables/Trite Solar’s future technical support—whether an in-house admin or a contracted developer brought in after the student team’s

WIL engagement ends, as per the post-handover terms in the Partnership Agreement referenced in Section 9—can follow the recovery process without needing to reverse-engineer it from the Azure portal under the pressure of an actual incident. This runbook is explicitly tied to the broader change-management and handover plan in Section 9, since a recovery process that exists only as undocumented knowledge inside the departing student team offers the client no real protection once that team's involvement ends.

## 6.7 Audit Logging

### 6.7.1 A distributed, entity-attached approach

Accountability for tracking who made which changes and when is handled through custom logging entities already built into the data model (Sections 3.1 and 4.4), rather than through a single, generic "activity log" table added later. The JobAssignmentLog keeps a record of every job assignment and reassignment, including the administrator's identity, the date, and free-text notes explaining the decision; the DataExportLog logs every data export performed by an administrator, including the type, format, and date, which supports the accountability requirement under POPIA discussed in Section 6.4.1; and the AIMessageLog tracks every automated message sent by the missed-call feature, including its content, delivery status, and channel, reinforcing the purpose-specification and consent discussions in Sections 6.4.3 and 6.4.10.

Together, these entities give an auditor, a marker, or the client itself a direct answer to questions like "who assigned this job," "who exported this data," and "what did the automated system tell this customer," without requiring examination of raw, low-level database transaction logs that a non-technical business owner would struggle to interpret. This distributed method of audit logging—attaching an audit trail to the relevant entity and workflow rather than maintaining one large, generic log for the entire system—keeps each log directly relevant to the business process it supports, meets the accountability standard POPIA imposes (Republic of South Africa, 2013), and follows general security-logging best practices, which favour logs that make sense to human reviewers over logs that are merely technically complete (OWASP Foundation, 2021).

### 6.7.2 Log content, retention and integrity

Every log entry captures at least the user's identity and role, a timestamp, the action performed, and the record involved, giving each entry the essential "who, what, when" structure needed to reconstruct events later. Audit log records are treated as write-only: once written, an entry cannot be edited or removed through normal application workflows, including by an Admin-Owner account, so the audit trail itself remains tamper-proof even if the account being scrutinised is the one that performed the action—a control aimed at the insider-misuse risk mentioned in Section 6.8 (Risk 10). Access to the audit logs themselves is restricted to the Admin-Owner role under the RBAC matrix in Table 7, rather than being openly readable, since the logs themselves contain sensitive operational information.

### 6.7.3 Monitoring, alerting and review cadence

Beyond storing records, the audit-logging system is designed to support active oversight: thresholds such as repeated failed login attempts against a single account (Section 6.1.4), unusually large data exports, or job-assignment patterns that deviate from an electrician's usual schedule can each be set to send an alert to the Admin-Owner, rather than relying solely on retrospective reviews after an incident is suspected. The team recommends, as part of the Section 9 handover documentation, that the client's designated Information Officer (Section 6.4.1) conducts periodic manual reviews of the audit logs at least every quarter, ensuring that the accountability mechanism described here stays an active operational habit rather than a dormant feature consulted only after something has already gone wrong.



## 6.8 Risk Analysis

### 6.8.1 Risk methodology and risk appetite

The risk register displayed in Table 9 was constructed by applying a simplified version of the risk-management framework from ISO 31000:2018 (International Organization for Standardization, 2018). Each risk identified undergoes evaluation for Likelihood and Impact using a straightforward Low/Medium/High scale. An overall priority is derived from combining these two factors. A pre-existing mitigation measure, already embedded in HYDRA and linked to the relevant subsection of this report, is noted alongside each risk. A Risk Owner is designated—the individual accountable for ensuring the mitigation remains effective—and a Residual Risk rating is logged, showing the remaining risk after mitigation, since no mitigation can reduce risk to absolute zero. As outlined in the introduction to this section, HYDRA's risk approach is deliberately balanced and aligned with the size of the business it supports. Any risk initially rated High likelihood or High impact before mitigation was considered essential to address before deployment, while low-likelihood, low-to-medium-impact residual risks are consciously accepted by the client because eliminating them completely would demand an investment exceeding what a small South African trades business operating on a free, student-delivered system could reasonably justify.

**Table 9: HYDRA security and operational risk register**

| # | Risk                                                                                                    | Likelihood | Impact | Mitigation Strategy                                                                                                                                                                                                                                                                                       | Risk Owner                   | Residual Risk |
|---|---------------------------------------------------------------------------------------------------------|------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|---------------|
| 1 | Unauthorised access to another user's customer, job or payment data (broken object-level authorisation) | Medium     | High   | JWT-based authentication with role-based access control enforced at the API layer on every request (Section 6.1); every repository call scoped to the authenticated user's own customer_id/electrician_id; Access Control ERD (Section 4.4) defines and constrains which data domain each role may reach. | Admin-Owner / Lead Developer | Low           |
| 2 | SQL injection via a form field or API parameter                                                         | Low        | High   | Repository pattern (Section 5.1) ensures 100% of database access goes through parameterised queries; no controller is permitted to construct raw SQL from user input (Section 6.3.1).                                                                                                                     | Lead Developer               | Very Low      |
| 3 | Weak or reused customer/employee passwords leading to account compromise                                | Medium     | Medium | Bcrypt password hashing with per-password salting and a configurable work factor (Section 6.1.2); minimum 12-character password-strength requirement enforced at registration; login-attempt rate limiting/lockout (Section 6.1.4); short-lived JWTs limiting the usefulness of a stolen token.           | Admin-Owner                  | Low           |



| # | Risk                                                                                                                                   | Likelihood | Impact | Mitigation Strategy                                                                                                                                                                                                                                                                                                                                                             | Risk Owner            | Residual Risk |
|---|----------------------------------------------------------------------------------------------------------------------------------------|------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------|
| 4 | Interception of data in transit (man-in-the-middle attack), particularly over public or cellular networks used by field electricians   | Low        | High   | HTTPS/TLS 1.2+ enforced on all external connections and SQL/TLS on all internal database connections (Section 6.2.2); HSTS header prevents downgrade to unencrypted HTTP; no component in the architecture accepts an unencrypted connection.                                                                                                                                   | Cloud/DevOps Engineer | Very Low      |
| 5 | Loss or corruption of financial/compliance data (invoices, payments, inspection reports) through application error or platform failure | Low        | High   | Automated, scheduled PostgreSQL backups with point-in-time restore and geo-redundant storage (Section 6.6); foreign-key constraints and transactional integrity enforced at the database layer (Section 4.4); periodic restore testing.                                                                                                                                         | Cloud/DevOps Engineer | Low           |
| 6 | Exposure of database credentials or API keys through source control or misconfiguration                                                | Low        | High   | All secrets retrieved at runtime from Azure Key Vault (Section 6.2.3); no credentials stored in application code or configuration files committed to GitHub; secret rotation supported.                                                                                                                                                                                         | Lead Developer        | Very Low      |
| 7 | Unauthorised access to, or misuse of, phone contacts/call logs by the missed-call AI feature                                           | Low        | High   | Access scoped strictly to the admin's own work device; minimal data captured on Contact/MissedCallLog (Section 6.4.2, 6.4.10); every automated message logged and reviewable via AIMessageLog (Section 6.7); "uncertain or sensitive query" escalation path in the AI Missed Call state diagram (Section 4.1) routes ambiguous cases to a human rather than an automated reply. | Admin-Owner           | Low           |
| 8 | Card/payment data breach                                                                                                               | Very Low   | High   | No raw card data is ever stored in HYDRA's own database; all card handling is delegated to a PCI-DSS-certified third-party payment gateway using tokenisation and hosted fields (Section 6.5).                                                                                                                                                                                  | Admin-Owner           | Very Low      |
| 9 | Malicious script injection via a user-editable field (XSS)                                                                             | Low        | Medium | Context-aware output escaping applied to all user-supplied content rendered in the Admin App and Customer App/Website (Section 6.3.4); Content-Security-Policy header restricts                                                                                                                                                                                                 | Lead Developer        | Low           |

| #  | Risk                                                                                                                            | Likelihood | Impact | Mitigation Strategy                                                                                                                                                                                                                                                                                                        | Risk Owner                  | Residual Risk |
|----|---------------------------------------------------------------------------------------------------------------------------------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|---------------|
|    |                                                                                                                                 |            |        | executable script sources (Section 6.3.6).                                                                                                                                                                                                                                                                                 |                             |               |
| 10 | Insider misuse of Admin-level access (e.g. an administrator viewing or exporting data without a legitimate business reason)     | Low        | Medium | DataExportLog records every export with the responsible admin's identity (Section 6.7); differentiated Admin permission tiers (owner vs office staff) limit which admin accounts can perform the most sensitive operations (Section 6.1.5); append-only, tamper-resistant audit logs (Section 6.7.2).                      | Admin-Owner                 | Low           |
| 11 | Denial-of-service or traffic spike affecting availability during business hours                                                 | Low        | Medium | Azure Front Door/CDN absorbs and caches public traffic; API tier configured with autoscale (Section 5.3); Scheduled Job Runner isolated from the live API so background workload cannot starve customer-facing requests (Section 4.2); API-level rate limiting (Section 6.3.6).                                            | Cloud/DevOps Engineer       | Low           |
| 12 | Loss or theft of a field electrician's mobile device with cached job data                                                       | Medium     | Medium | Locally cached data limited to the electrician's own currently assigned jobs and held in platform-provided encrypted secure storage, not plain application files (Section 6.2.4); short-lived access tokens and remote logout capability limit continued access after loss is reported.                                    | Cloud/DevOps Engineer       | Low           |
| 13 | Third-party payment gateway outage or API change disrupting invoicing/payment workflows                                         | Low        | Medium | Payment integration is isolated behind a defined interface so the underlying gateway can be swapped with contained impact (Section 4.3); invoice and job status remain independently valid in HYDRA even if a payment confirmation is temporarily delayed, preventing a gateway outage from corrupting core business data. | Lead Developer              | Low           |
| 14 | Non-compliance with POPIA (e.g. inadequate consent capture, excessive data retention) resulting in regulatory complaint or fine | Low        | High   | All eight POPIA lawful-processing conditions explicitly addressed by design (Section 6.4); guided-walkthrough consent capture; defined data retention and minimisation rules (Sections 6.2.5, 6.4.2);                                                                                                                      | Admin-Owner (post-handover) | Low           |

| #  | Risk                                                                                                                  | Likelihood | Impact | Mitigation Strategy                                                                                                                                                                                                                                                                                      | Risk Owner            | Residual Risk |
|----|-----------------------------------------------------------------------------------------------------------------------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------|
|    |                                                                                                                       |            |        | Information Officer appointment recommended as part of handover (Section 6.4.1).                                                                                                                                                                                                                         |                       |               |
| 15 | Loss of institutional/technical knowledge after the student team's WIL engagement concludes ("key person" dependency) | Medium     | Medium | Comprehensive handover documentation, including this security section, the disaster-recovery runbook (Section 6.6.4) and training plan (Section 9); GitHub repository, environment configuration and Key Vault access transferred to the client at handover rather than retained solely by team members. | Project Lead          | Low           |
| 16 | Outdated or unpatched third-party libraries/dependencies introducing a known vulnerability                            | Low        | Medium | CI/CD pipeline (Section 7) includes dependency and build checks before deployment; the team recommends the client (or a contracted maintainer post-handover) apply a defined patching cadence rather than leaving dependencies static indefinitely.                                                      | Cloud/DevOps Engineer | Low           |

### 6.8.2 Interpreting the register and ongoing review

When examining the register holistically, the risks with the greatest impact on HYDRA (Risks 1, 4, 5, 6, 8 and 14) all relate to either unauthorized access to sensitive data or the integrity and availability of records with significant financial or legal consequences. This is why the mitigations for these specific risks—RBAC and JWT authentication, encryption during transmission, point-in-time restore backups, Key Vault secret management, payment tokenization, and comprehensive POPIA compliance—form the foundational structure of Sections 6.1 through 6.6 rather than being treated as secondary measures. Risks categorized as Medium likelihood prior to mitigation (Risks 1, 3, 12 and 15) align closely with human actions—such as password selection, device handling, and organizational continuity—rather than purely technical failures, reinforcing findings from information-security research that human factors, not just technical vulnerabilities, often drive real-world security breaches (OWASP Foundation, 2021; International Organization for Standardization, 2013). Consequently, HYDRA's mitigations for these risks combine technical safeguards (bcrypt hashing, encrypted local storage, thorough handover documentation) with organizational practices the client must sustain post-handover (a password policy, a lost-device reporting process, periodic audit-log reviews).

This risk register is maintained throughout the project and recommended for ongoing use by the client after handover, serving as a dynamic document rather than a static report produced once and stored. It is reviewed at each major project milestone, in line with the review points outlined in the Work Breakdown Structure in Section 2.3, ensuring any new risks introduced by later design or scope decisions—such as the future addition of an integration or feature not considered in this Task 1 documentation—are identified and addressed before deployment. The team advises PSG Electrical and Cables/Trite Solar to conduct at least an annual review of this register, as well as an immediate reassessment following any significant changes to the system,

its user base, or the regulatory environment it operates under. This ensures the register remains an accurate reflection of the live system rather than a historical snapshot of conditions at the time of academic submission.

## 7. DevOps

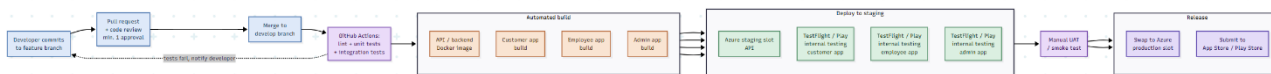


Figure 34: HYDRA DevOps / CI-CD Pipeline Diagram

DevOps, as defined in this section, involves the integrated combination of cultural principles, operational processes, and technical tools that enable a software development team to deliver a software update from a developer's local machine to a live, functional feature accessible by end users rapidly, securely, and consistently. Instead of separating the tasks of "writing software" and "managing production systems" into distinct roles handled by different teams, DevOps merges these functions into a unified workflow (Kim et al., 2016; Forsgren, Humble and Kim, 2018). For a small team of six students working under the strict deadlines of a Work-Integrated Learning module while managing real client relationships, this integration is not just theoretical. PSG Electrical and Cables/Trite Solar require a fully operational and demonstrable system at predetermined checkpoints, as outlined in the documented meeting records from Section 1.4 and the detailed Work Breakdown Structure in Section 2.3. For six developers to contribute code simultaneously to three client applications and one shared API without causing frequent disruptions to one another's work, a structured, automated, and clearly defined DevOps process is essential.

This section provides a complete explanation of HYDRA's version control workflow (Section 7.1), the continuous integration and automated build pipeline (Section 7.2), the testing approach covering all levels of the testing pyramid (Section 7.3), the deployment methods and environment framework (Section 7.4), the release and rollback procedures (Section 7.5), the monitoring and observability practices (Section 7.6), and the collaborative team practices and DevOps culture surrounding the tools (Section 7.7). Figure 34 illustrates the full pipeline described here, from code submission to production release, including automated build processes, testing, staging deployment, and final release. Each phase in this visual representation is further clarified in the corresponding subsection below.

### 7.1 Version Control Workflow

#### 7.1.1 Repository structure and branching strategy

HYDRA relies on GitHub for version control, with the API/backend and all three client applications (Customer, Electrician/Employee, and Admin) hosted as distinct projects inside a shared organization. This setup ensures that modifying one application's codebase cannot unintentionally disrupt another's build or deployment process. The team adheres to a standard, proven feature-branch workflow centered on two permanent branches—main, which always matches the live production environment, and develop, which holds the next release under integration and testing. This structure is widely advised for small-to-medium teams since it remains simple enough to avoid unnecessary overhead for a six-person group while clearly distinguishing between "live code" and "in-progress work" (Chacon and Straub, 2014; Atlassian, 2023). Every new task—whether a feature, bug fix, or refactor—starts as a short-lived feature branch spun off from develop and labeled clearly (for instance, feature/job-assignment-log, fix/invoice-total-rounding), preventing any one

member's unfinished or unstable work from destabilizing the shared develop branch the rest of the team depends on. Feature branches are kept intentionally small and brief, usually tied to a single user story from the WBS (Section 2.3) rather than an entire module, which limits the scope and risk of each merge and keeps code review (Section 7.1.2) manageable for a small team without a dedicated reviewer.

GitHub's branch protection rules are applied directly to both main and develop, blocking any team member—including the Lead Developer—from pushing changes straight to these branches without using the pull-request process outlined later. This is a deliberate technical safeguard, not just an informal team agreement: since enforcement happens in GitHub itself rather than relying on everyone remembering not to push directly, it eliminates a whole category of accidental mistakes (like an untested, incomplete change landing on develop or, even worse, main). This aligns with the same principle of "making unsafe actions structurally impossible rather than just discouraged," which is already used for SQL injection prevention via the Repository pattern in Section 6.3.1 (GitHub, 2024).

### 7.1.2 Pull requests and code review

Once a feature branch is finalized and the developer has confirmed its functionality locally, they submit a pull request (PR) to merge it into develop, which must receive at least one approval from another team member before merging. Every PR contains a concise, structured summary—detailing what the modification accomplishes, the associated user story or WBS task identifier (Section 2.3), and how the developer validated it (including manual checks performed or automated tests included)—so reviewers can evaluate the change's purpose and accuracy without needing additional verbal clarification. This straightforward yet valuable review process ensures that, for a six-person team where no single member can fully grasp the entire system, at least one other individual has examined and comprehended every update before it becomes part of the shared codebase. This approach not only identifies defects earlier than would otherwise occur but also distributes system knowledge across the team rather than concentrating it with the original feature author (Kim et al., 2016; Chacon and Straub, 2014). Reviewers must verify, at minimum, that the change adheres to the Repository pattern and layered architecture outlined in Sections 5.1–5.2 instead of bypassing it, that any new input-handling logic follows the validation method described in Section 6.3, and that modified or new logic includes a corresponding unit test as per the testing strategy in Section 7.3.

If a reviewer requests adjustments, the original developer implements the feedback within the same feature branch and resubmits the PR for review, rather than merging the PR with unresolved comments. This practice maintains develop at a consistently vetted, reliable standard instead of allowing review backlog to grow unnoticed throughout the project.

### 7.1.3 Commit message conventions

Commit messages adhere to a uniform, descriptive standard: a brief, commanding summary line under about 72 characters (such as "Include JobRepository findByCustomer function" or "Correct invoice total rounding for partial payments"), with an optional extended explanation if the modification isn't clear. This approach ensures the project's Git log remains a clear, chronological explanation of how the code evolved, allowing any later defect to be traced back to the exact commit that caused it using git bisect or a basic history review, rather than leaving a trail of vague entries like "fix" or "change" that offer no insight to someone reviewing the code years later—a practice endorsed by contemporary version-control best practices as vital for a project's long-term sustainability, no matter the team's size (Chacon and Straub, 2014). If a commit closes a specific task on the team's GitHub Projects board (Section 7.7.2), the message includes that issue number directly, automatically connecting the update to the original requirement or bug report that prompted it.

### 7.1.4 Merge strategy, tagging and release branches

Pull requests that go into develop are combined using a squash merge, which takes all commits from the feature branch—even small ones like "fix typo" or "address review comment"—and merges them into one tidy commit on develop, representing a fully reviewed and tested piece of work. This approach keeps the develop branch's history clean and focused on what features were added, rather than the messy details of how they were built.

When develop is ready for release—according to the milestone dates outlined in the Work Breakdown Structure (Section 2.3)—a release branch is created from it, final testing for the release candidate (Section 7.3.4) is carried out, and then it is merged into main and marked with a version number following Semantic Versioning (MAJOR.MINOR.PATCH, for example v1.2.0). In this system, the major version increases only for changes that break compatibility, the minor version increases for new features that do not break compatibility, and the patch version increases for bug fixes that do not break compatibility (Preston-Werner, 2013). This ensures every build sent to production has a clear, permanent version label, which is important for the rollback plan in Section 7.4.4 and for submitting to app stores, since both the Apple App Store and Google Play Store demand a steadily increasing, clearly defined version number for every build.

## 7.2 Continuous Integration and Automated Build

### 7.2.1 Pipeline triggers and stages

GitHub Actions is set to activate automatically for two scenarios: whenever code is pushed to a feature branch, it runs a condensed set of validations to provide the contributor with immediate results, and whenever a pull request is submitted or modified targeting develop or main, it executes the complete workflow as outlined below, since this marks the stage where a modification is actually being suggested for merging. This enforced, hands-off approach—eliminating the need for developers to manually run checks before submitting a PR—ensures that no code can be merged without first being independently validated by the pipeline, aligning with the continuous-integration principle that the main branch must always stay in a verified, deployable condition (Humble and Farley, 2010; Forsgren, Humble and Kim, 2018).

### 7.2.2 Static analysis and linting

The initial phase of the pipeline performs a linting operation on the modified code to verify stylistic uniformity, including consistent indentation and naming conventions across all four codebases. It also identifies common programming mistakes that compilers typically miss, such as unused variables, unreachable code, and implicit type coercion issues. Additionally, as highlighted in Section 6, a static application security testing (SAST) analysis automatically detects insecure coding practices like string-concatenated SQL query construction—which the Repository pattern in Section 5.1 already structurally prevents but is checked again for layered security—and the use of deprecated, weak cryptographic functions. In parallel, an automated scan for dependency vulnerabilities assesses every third-party library used in the API and the three client applications against an up-to-date record of known weaknesses, directly addressing the mitigation for Risk 16 in Section 6.8 (outdated or unpatched dependencies). The scan is set to halt the pipeline immediately if any dependency has a High or Critical severity vulnerability, reflecting industry-wide awareness that third-party, open-source dependencies are a primary source of security breaches due to their ease of integration and tendency to be overlooked in updates (Sonatype, 2023; OWASP Foundation, 2021).

### 7.2.3 Automated test execution



If the linting and dependency checks both succeed, the process moves on to run the complete automated suite of unit and integration tests (explained thoroughly in Section 7.3) for the proposed modification. Should any test fail, the workflow halts right away, and the developer in charge receives an alert immediately—through a visible failed status on the pull request and a message to the team’s shared communication channel—so the faulty code cannot advance further in the workflow. This “fail fast” strategy is a key part of continuous integration: finding and fixing a flaw is much quicker and less costly when detected right after it is introduced, while the related code remains fresh in the developer’s mind and the change causing it is still minimal and distinct, rather than waiting days or weeks until extra unrelated work has been added and the original issue must be retraced from the beginning (Humble and Farley, 2010; Forsgren, Humble and Kim, 2018).

#### **7.2.4 Automated build and artefact production**

Only after linting, dependency checks, and the complete test suite pass does the pipeline move to the automated build phase, which generates four distinct build outputs simultaneously: the API/backend Docker container image, and the Customer, Electrician/Employee, and Admin application builds (a mix of web bundles and native mobile outputs, depending on the target platform for each application as outlined in Section 4.3). Executing these four builds in parallel, instead of one after another, keeps the overall pipeline duration reasonable—a key factor for a small team aiming for quick, frequent validation after every merge rather than being discouraged by a slow, sequential build process that could take an hour or more to finish (Forsgren, Humble, and Kim, 2018). Each successfully created build output is tagged and saved (as a container-registry image for the API, and as pipeline-run attachments for the client apps), providing the team with a permanent, retrievable reference of which code produced any specific build. This is essential for the rollback process described in Section 7.4.4 and for investigating incidents, which supports the audit-logging discussion in Section 6.7.

#### **7.2.5 Pipeline visibility and notifications**

The outcome of each pipeline execution—whether it succeeded or failed, and at which exact stage—is displayed right on the associated GitHub pull request. In addition, a brief alert is sent to the team’s shared chat room whenever a develop merge occurs or when a pipeline fails, ensuring that any build error is brought to everyone’s attention immediately instead of being noticed only when the next person pulls the latest develop branch. This instant, team-wide awareness reflects a key DevOps cultural belief: the present status of the shared codebase should be common, visible knowledge for the whole team rather than hidden, individual knowledge. This practice is widely recognized in DevOps writings for significantly cutting down the time a team remains “broken” on a shared branch, because failures that everyone can see tend to be resolved much faster than ones only the person responsible notices first (Kim et al., 2016).

### **7.3 Testing Strategy**

#### **7.3.1 The testing pyramid**

HYDRA’s approach to testing aligns with the well-known testing pyramid model: a broad foundation of quick, inexpensive, focused unit tests, a narrower intermediate layer of integration tests checking how multiple components interact correctly, and an even smaller upper tier involving manual user acceptance and smoke testing, along with automated end-to-end tests closer to release (Forsgren, Humble and Kim, 2018). This structure is intentional rather than random: unit tests are quick to create and execute in milliseconds, allowing the team to maintain a large collection and run the full set with every pipeline run (Section 7.2.3); integration and end-to-end tests grow progressively slower and costlier to develop and sustain, so the team

limits their use to the specific interactions and workflows that unit tests, by definition, cannot validate. Balancing complete coverage with the real-world time limits of a six-member student team working within a set academic schedule (Section 2.3) is a key design choice behind this pyramid approach, rather than the team trying — and likely failing, given the time constraints — to achieve full end-to-end coverage of every possible user journey.

### 7.3.2 Unit testing

Unit tests are designed to verify the functionality of individual methods and classes in isolation from the rest of the system. For instance, the `Invoice.calculateTotal()` method is tested to ensure it accurately calculates the total by summing line items and applying any applicable discounts from the rewards programme, as described in Section 1.3. Additionally, the bcrypt password-hashing wrapper, detailed in Section 6.1.2, is tested to verify its ability to reject a plaintext-comparison shortcut. Since all database interactions in HYDRA are facilitated through a Repository interface, as outlined in Section 5.1, business logic can be thoroughly tested by substituting a lightweight in-memory "mock" or "fake" implementation of the relevant repository interface in place of the real, database-backed implementation. This allows unit tests for `Invoice.calculateTotal()` to run rapidly, requiring no actual database connection, and is a significant practical benefit of the Repository pattern, as described in Section 5.1. This type of design decision pays for itself repeatedly throughout a project's testing phase, as noted by Fowler in 2018. Furthermore, unit tests are written for every controller method, every repository implementation against an isolated test database, and every piece of validation logic described in Section 6.3. The CI pipeline, as detailed in Section 7.2.3, is configured to track code-coverage percentage as a visible metric on every pull request, providing the team with an ongoing, objective signal of how thoroughly new code is being tested, rather than relying on a subjective sense that "enough" tests have been written.

### 7.3.3 Integration testing

Integration tests validate that the controller, its repository implementation, and an actual (test) database function correctly in unison, identifying defects that unit tests — designed to work with simulated dependencies — are unable to catch. These defects may include poorly written SQL queries, missing database indexes causing sluggish queries with substantial data amounts, or violations of foreign key constraints that only arise with genuine, interconnected data. The integration tests are executed against a dedicated, disposable test database, which is automatically set up and taken down during the pipeline run. This test database is structurally identical to the production database detailed in Section 4.4 but entirely distinct from it. This setup ensures that test execution never compromises authentic customer, job, or payment data. The key integration test scenarios align closely with the vital sequence diagrams outlined in Section 4.1, illustrating the progression of a job from request to quoting, assignment, completion, and invoicing.

### 7.3.4 Manual UAT, smoke testing and end-to-end verification

In addition to the automated processes previously mentioned, each release candidate is subjected to manual smoke testing and User Acceptance Testing (UAT). To ensure no single person bears the testing burden and that all staff remain familiar with the entire system, team members rotate the responsibility of manually executing the essential workflows detailed in Section 4.1—ranging from job requests to payments, material logging to invoicing, and the AI-driven missed-call escalation—within the staging environment specified in Section 7.4.2 using realistic synthetic data. This manual phase is designed to identify types of errors that automated testing often misses, such as actual usability hurdles, layout or visual glitches on particular devices, and nuanced workflow complications that only surface when a person performs a practical task rather than simply verifying specific values (Sommerville, 2016). Furthermore, smoke testing—a brief,

focused check to ensure core functions like login, job creation, and payment are operational—is performed immediately following every staging deployment and every production release. This serves as a rapid verification that a deployment has not caused a total system failure prior to the more extensive UAT.

### 7.3.5 Test data management and non-functional testing

All testing processes, including manual user acceptance testing and automated integration tests, utilize exclusively synthetic data. This data is purpose-built for testing and lacks any genuine customer, electrician, or payment details. Consequently, this approach maintains compliance with the data-minimization principles detailed in Section 6.4.2 and permits the unrestricted documentation, screenshotting, and discussion of identified defects without the danger of disclosing sensitive personal information. In addition to validating functional accuracy, the team conducts brief performance assessments prior to significant milestones to confirm that primary API endpoints meet the response-time criteria established in the Non-Functional Requirements (Section 2.4) under simulated loads. Furthermore, a security-oriented testing phase is executed immediately before each release milestone. During this phase, specific attack patterns listed in the Section 6.8 risk register—such as submitting SQL injection payloads into form fields or attempting to access other customers' job data through identifier manipulation—are deliberately simulated. This ensures that implemented mitigations remain robust as the codebase changes, rather than relying on the assumption that initial security controls will remain effective without continuous re-verification.

## 7.4 Deployment Strategy and Environments

### 7.4.1 Environment structure

HYDRA is implemented within three separate environments: Development, Staging, and Production. While each utilizes the identical Azure infrastructure detailed in Section 5.3 (comprising Front Door, App Service, Azure Database for PostgreSQL, Blob Storage, and Key Vault), they remain completely isolated at the resource level to ensure that activities within Development or Staging cannot impact production data. The Development environment facilitates initial feature validation using a shared, non-production database populated with synthetic data. The Staging environment, which is elaborated upon later, serves as the destination for the automatic deployment of all pull request build artefacts (Section 7.2.4) and functions as the site for the manual smoke and UAT testing outlined in Section 7.3.4. The Production environment constitutes the live system utilized by administrators, electricians, and customers of Trite Solar and PSG Electrical and Cables. Rather than being hard-coded into the application, configuration parameters that vary by environment—such as specific Key Vault instances, database connection strings, and third-party service credentials (including the sandbox and live payment-gateway credentials mentioned in Section 6.5)—are retrieved from an environment-specific Key Vault. This approach extends the secrets-management methodology previously described in Section 6.2.3 to the entire environment structure instead of limiting it to production.

### 7.4.2 Staging as a production-equivalent environment

To ensure that UAT and smoke testing accurately forecast real-world system behavior, the Staging environment is intentionally configured to mirror Production as closely as the constraints of a free student project allow. This includes utilizing identical Azure service tiers, database engine versions, and feature flag configurations, thereby avoiding the discrepancies that arise when testing occurs in an environment materially different from the one used by end users. Minimizing this "environment drift" is a fundamental requirement for reliable continuous delivery, as passing tests in an environment that lacks production-like characteristics offers deceptive confidence rather than authentic assurance (Humble and Farley, 2010).

### 7.4.3 Deployment mechanism — slot swapping

Consistent with the methodology described in Section 4.2, the API is hosted on Azure App Service utilizing deployment slots. The process involves deploying the latest API artifact to an isolated staging slot that operates independently of the production environment. Following successful smoke testing within this staging environment, the deployment is finalized by swapping the staging and production slots. This procedure facilitates a nearly immediate redirection of live traffic to the updated code and remains fully reversible, as the prior production version is preserved in the staging slot following the swap (Microsoft, 2023). This mechanism mitigates the significant risks associated with traditional "big bang" deployments, where application downtime occurs during the stop-replace-restart cycle; instead, the transition is a rapid traffic redirection rather than a service restart, significantly minimizing potential service interruptions for users of the job-scheduling and invoicing system (Humble and Farley, 2010; Newman, 2021). A similar staged-release strategy is applied to the three client applications (Customer, Electrician, and Admin) according to their respective platforms: the Admin App and Customer Website utilize the same slot-swapping technique as the API, whereas the native mobile applications leverage the specific staged-rollout features of their respective operating systems (Section 7.4.5).

### 7.4.4 Rollback strategy

Since each production deployment is linked to a unique, versioned build artifact (Section 7.2.4) and a specific, semantically versioned Git tag (Section 7.4.1), reverting a faulty release involves merely reversing the slot swap. This process restores the prior stable build to the production slot with the same rapid, low-risk methodology employed during the initial deployment. Rather than being an ad hoc emergency measure, rollback functionality is intentionally integrated as a primary component of the deployment strategy. Prior to every release, the team's checklist explicitly defines the specific triggers for a rollback—such as increased error rates detected via the monitoring detailed in Section 7.6 or critical flaws found during post-release smoke tests—and verifies that the preceding build is available for immediate restoration. This approach adheres to the principle that deployment workflows should be engineered with the expectation that some releases may require reversal, rather than viewing rollbacks as unforeseen exceptions (Humble and Farley, 2010; Kim et al., 2016).

### 7.4.5 Database schema migrations

Modifications to the database schema that coincide with a release—such as the addition of a new `InspectionReport` column or the introduction of a `JobAssignmentLog` table when the audit-logging entities mentioned in Section 6.7 were integrated into the data model—are handled via versioned and reviewable migration scripts instead of being manually and undocumented applied to the production database by personnel. Each migration script is stored in the same GitHub repository as its associated application code, undergoes the pull-request review process detailed in Section 7.1.2, and is automatically run as a specific, sequential stage in the deployment pipeline just prior to the deployment of the related application code. This ensures the database schema meets the requirements of the incoming API version before it begins processing live traffic. Whenever feasible, migrations are designed to maintain backward compatibility for a minimum of one release cycle (for instance, by adding a nullable column instead of immediately deleting or renaming an existing one). This approach enables both the staging and production slots—which temporarily run concurrent code versions during the slot-swap period outlined in Section 7.4.3—to function properly with the shared database schema without necessitating downtime. Furthermore, this facilitates the rollback procedure described in Section 7.4.4, as a reverted application version remains compatible with a schema that has already undergone forward migration (Fowler, 2018; Newman, 2021).

## 7.5 Release Process and Release Management

### 7.5.1 Release cadence and milestones

The frequency of releases is strictly linked to the milestone framework defined in the Work Breakdown Structure (Section 2.3). Instead of employing a continuous production deployment strategy—which would be unsuitable for academic deliverables necessitating verifiable milestones and unnecessary for a client lacking a requirement for daily updates—the team utilizes continuous merging into the develop branch during every sprint. Formal deployments to Staging and Production occur only at the conclusion of each milestone, ensuring every release corresponds to a discrete, verifiable collection of completed user stories for evaluation by supervisors and the client. This methodology integrates the advantages of two distinct strategies: the rapid, low-risk continuous integration feedback loop detailed in Section 7.2, which is applied to every pull request, and a structured, milestone-based deployment schedule for all client-facing and end-user environments.

### 7.5.2 Parallel release paths for API and client applications

Following the successful completion of manual smoke testing and UAT on the staging environment (Section 7.3.4), the deployment follows two concurrent and independent tracks designed to prevent mutual obstruction. The API deployment is executed by swapping the Azure staging slot with the production slot (Section 7.4.3), a process managed entirely by the internal team that concludes within minutes. Simultaneously, the three mobile and web applications are submitted to the Apple App Store and Google Play Store, where they undergo respective platform-specific review periods; Apple's review duration varies from less than a day to several days based on submission density and potential manual inspections, whereas Google Play's review is typically swifter but remains outside the team's immediate authority. This decoupling of the API and application releases is a deliberate strategy: backend-only updates or new endpoints can be deployed immediately upon successful staging verification without waiting for app store approval, while client-side modifications proceed through standard store submissions without necessitating changes to the previously stabilized API. This release-management strategy implements a separation of concerns that directly reflects the architectural separation of concerns established in the layered software design detailed in Section 5.2 (Humble and Farley, 2010).

### 7.5.3 Staged mobile rollout

Regarding the two native mobile applications, the development team utilizes the inherent staged deployment features of each platform instead of distributing updates to the entire user base at once. Through Google Play's staged rollout and Apple's phased release mechanisms, new versions are first introduced to a limited subset of users; the deployment scale is expanded in following days only if there is no evidence of increased crash frequencies or adverse user feedback. Because the HYDRA Electrician App serves as a critical instrument for professionals performing daily tasks, this incremental deployment method significantly reduces the potential impact of any undiscovered flaws that bypassed automated testing (Section 7.3). Although this strategy results in a marginally extended timeline for full implementation, the team deems this exchange necessary for an application whose malfunction would directly impede a company's daily operational capacity.

### 7.5.4 Release notes and versioning transparency

Each production release includes a concise summary written in accessible language that outlines all modifications. These notes link to the semantic version tag specified in Section 7.1.4 and the relevant WBS milestone from Section 2.3, while clearly differentiating between external user-facing updates and internal

adjustments, such as performance enhancements or security hardening. This documentation serves two purposes: it provides internal project management traceability and functions as a primary resource for the client communications and training updates detailed in the Change Management section (Section 9). This ensures that personnel at PSG Electrical and Cables/Trite Solar are not unexpectedly confronted with changes to their operational systems and allows the training materials mentioned in Section 9.1 to remain synchronized with the features currently in deployment.

### 7.5.5 Hotfix process

If a production defect emerges that necessitates immediate resolution—such as a failure in payment processing or a security vulnerability identified in Section 6.8—the team utilizes an accelerated hotfix procedure. A dedicated fix branch is branched from main instead of the ongoing develop branch. This branch undergoes a condensed version of the mandatory code review and automated testing protocols outlined in Sections 7.1–7.2; while the turnaround time is reduced, no essential checks are omitted, ensuring that no regression or security flaw is reintroduced. Once validated, the fix is deployed to production via the slot-swap method detailed in Section 7.4.3. Finally, the branch is merged into develop to prevent the defect from being inadvertently lost or reintroduced during the subsequent standard release.

## 7.6 Monitoring, Logging and Observability

### 7.6.1 Application performance monitoring

Following deployment to the production environment, HYDRA is overseen via Azure Monitor and Application Insights, which are integrated with the App Service and Azure Database for PostgreSQL resources detailed in Section 5.3. This integration provides the team with immediate insight into database query efficiency, error frequencies, response delays, and request volumes, eliminating the necessity of developing a custom monitoring framework. Dashboards display the precise metrics associated with the Non-Functional Requirements outlined in Section 2.4—specifically uptime, API response latency, and error rates—enabling both the current team and the client's technical support staff following handover to readily confirm that the operational system adheres to the performance and reliability benchmarks established in that section. Consequently, these targets remain verifiable operational standards rather than merely theoretical goals within the project documentation (Beyer et al., 2016).

### 7.6.2 Alerting

Established alert parameters trigger notifications to the accountable team member and, following a handover, the client's appointed technical representative as soon as a monitored metric reaches a critical level. Such triggers include an increased error rate immediately after deployment—which serves as a direct basis for the rollback procedure detailed in Section 7.4.4—an anomalous surge in failed login attempts for a specific account, which supplements the rate-limiting measures in Section 6.1.4 through active alerting rather than simple passive blocking, or API latency that perpetually surpasses the Non-Functional Requirement under live production conditions. To align with the shared-visibility DevOps concept outlined in Section 7.2.5, alerts are intentionally directed to a collective team channel instead of an individual's personal inbox; this ensures that any problem identified outside of a particular member's standard working hours is immediately visible to and addressed by available personnel.

### 7.6.3 Relationship to audit logging and security monitoring

The operational monitoring detailed herein functions concurrently with, yet is intentionally separated from, the application-level audit logging outlined in Section 6.7. While monitoring assesses immediate system



health and performance, audit logging tracks specific user actions, including the identity of the actor, the target record, and the time of occurrence. Although both utilize the same Azure observability infrastructure, they cater to different objectives and stakeholders: monitoring is utilized chiefly by the technical staff maintaining system stability, whereas audit logs are primarily accessed to address accountability, compliance, or dispute-related inquiries as specified in Section 6.7.3. Collectively, these tools provide both the internal team and the client following handover with a real-time overview of live operations and a permanent, historical record of all system activities.

## **7.7 DevOps Culture and Team Collaboration Practices**

### **7.7.1 Agile ceremonies and iteration structure**

The DevOps workflow detailed herein functions under a streamlined Agile framework, characterized by frequent, brief stand-up meetings designed to identify impediments promptly. Work is organized into iterations corresponding to the WBS milestones specified in Section 2.3, with the scope of each cycle derived from the user stories established in Section 2.1. This iterative rhythm and the continuous integration processes outlined in Sections 7.1–7.2 are interdependent rather than isolated methodologies; the effectiveness of an iteration-based approach relies on the ability to rapidly and securely integrate and validate incremental updates, a capability provided by the CI pipeline. Conversely, a CI pipeline offers minimal utility to a team that accumulates work into a single, large, and infrequently merged branch (Fitzgerald and Stol, 2017).

### **7.7.2 Task tracking and project visibility**

Task progression is monitored via a communal GitHub Projects board, organized as a Kanban pipeline consisting of Backlog, In Progress, In Review, and Done stages. Each card is linked to a designated WBS task from Section 2.3 and, when relevant, to a specific user story from Section 2.1 as well as a corresponding rubric section. This system provides team members, module supervisors, and the client—during shared progress-review sessions—with a unified, real-time overview of completed, ongoing, and pending tasks. Consequently, there is no need for manual status reports, which are prone to becoming obsolete whenever work evolves.

### **7.7.3 Documentation as a first-class deliverable**

In alignment with the overarching themes of this report, the team regards documentation—including this document, inline code comments clarifying complex business logic (such as the reasoning for specific invoice-rounding protocols), and repository README files detailing local environment configuration—as a primary DevOps deliverable rather than a concluding task. This approach is critical because several team members will depart from PSG Electrical and Cables/Trite Solar following the conclusion of the WIL engagement (Section 9). Furthermore, the Key Person Dependency identified in the Section 6.8 risk register (Risk 15) is best addressed through documentation that remains accurate and comprehensive at the time of handover, avoiding the inaccuracies that occur when documentation fails to reflect the evolved state of the system.

### **7.7.4 Retrospectives and continuous improvement**

Upon the conclusion of every iteration, the team conducts a brief retrospective to evaluate successes, failures, and the specific procedural adjustments intended for the subsequent cycle. This approach mirrors the continuous-improvement principle inherent in the DevOps pipeline, applying it to the team's internal workflows rather than just the software. The specific procedural modifications implemented throughout the project are documented in conjunction with the WBS review points mentioned in Section 2.3. This provides



a clear history of the team's evolving delivery methodology, aligning with the fundamental "continuous" philosophy—encompassing continuous integration, continuous delivery, and continuous improvement—that defines DevOps (Kim et al., 2016; Forsgren, Humble and Kim, 2018).

## 8. Running Costs

This segment provides an exhaustive breakdown of the anticipated operational expenses for HYDRA following its deployment. It encompasses costs for application and compute hosting, database management, scheduled background processing, object storage, secrets management, edge networking and content delivery, domain registration, SSL/TLS certification, third-party messaging services (SMS, WhatsApp, and email), and payment gateway charges. These expenditures are modeled across three distinct levels of business activity and include a two-year forecast based on clearly defined growth projections. Rather than fulfilling a mere academic requirement of the module rubric, these figures serve as a critical deliverable for the client. Given that PSG Electrical and Cables/Trite Solar is a small, owner-managed enterprise, a precise and well-supported financial overview is vital to the client's decision-making process regarding the timing and nature of their transition to full production use.

All monetary values are expressed in South African Rand (ZAR). These estimates are derived from Microsoft Azure's current pricing for the services specified in Sections 4.3 and 5.3, alongside contemporary rates for South African domain registration, payment gateways, and telecommunications APIs. It is important to note that because Azure's USD-denominated pricing is subject to currency exchange fluctuations, and because local service providers periodically adjust their rates, the provided figures must be viewed as educated, defensible projections rather than fixed quotes. Consequently, these estimates should be re-validated using official online calculators and current rate cards immediately prior to any formal budgetary commitment. This methodology—offering a structured, assumption-based range and highlighting the need for periodic updates—aligns with professional project management standards, where estimates are categorized by accuracy and refined throughout the project lifecycle.

The financial analysis is purposefully organized according to the principles of cloud financial management, or "FinOps," which integrates financial accountability, engineering, and business strategy to manage cloud expenditures proactively. This section, in conjunction with the periodic review recommendation in Section 8.3.4, adopts the FinOps Foundation's three-phase cycle: Inform (achieving spending visibility), Optimize (reducing waste and adjusting resources), and Operate (integrating governance into organizational workflows). This framework is particularly relevant to PSG Electrical and Cables/Trite Solar; as a small business lacking specialized finance or IT operations departments, the client will rely on HYDRA's architecture and these projections to maintain visibility. To support this, Sections 8.2 and 8.4 provide a series of checkpoints linked to measurable system usage, ensuring the client has a dynamic tool for oversight rather than a single, static forecast.

### 8.1 Cost Categories

Table 10 sets out every cost category that contributes to HYDRA's ongoing running cost, together with the specific Azure (or third-party) service that fulfils it and, critically, an explanation of precisely what usage pattern drives the cost up or down for each category — because a cost table without a clear "cost driver" column is of limited practical use to a client trying to understand how their own business decisions (for example, taking on more electricians, or running a promotional campaign that increases job volume) translate into infrastructure spend. Each row is discussed in the subsections that follow, in the same order in which the categories appear in Table 10, so that a reader can move directly from the summary table to a full explanation of any individual line item.

**Table 10: HYDRA running cost categories and what drives them**

| Cost Category                | Service                                                            | What Drives the Cost                                                                           |
|------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Application hosting          | Azure App Service (API + React static frontend)                    | Compute tier (vCPU/RAM) and number of instances (autoscale)                                    |
| Database hosting             | Azure Database for PostgreSQL (managed)                            | Compute tier, storage size, backup retention period                                            |
| Background/scheduled compute | Azure Functions (Scheduled Job Runner)                             | Number of executions and execution duration per month                                          |
| Object storage               | Azure Blob Storage                                                 | Volume of inspection reports/images stored (GB) and data-transfer volume                       |
| Secrets management           | Azure Key Vault                                                    | Number of secret operations per month (typically negligible cost at this scale)                |
| CDN / edge                   | Azure Front Door                                                   | Data transfer volume and number of requests                                                    |
| Domain name                  | Third-party .co.za registrar (ZADNA-accredited)                    | Annual registration/renewal fee, itself linked to CPI-based ZADNA wholesale adjustments        |
| SSL/TLS certificate          | Azure-managed certificate (Front Door)                             | Included at no additional cost when using Azure Front Door's managed TLS                       |
| Notification gateway         | SMS/WhatsApp Business API/email provider (e.g. Twilio, Clickatell) | Per-message cost, billed per SMS/WhatsApp message sent                                         |
| Payment gateway              | Third-party provider (e.g. PayFast, Yoco, Peach Payments)          | Percentage transaction fee (plus, for some providers, a small flat fee) per successful payment |

### 8.1.1 Application hosting (compute)

Azure App Service hosts the application, encompassing both the React frontend bundle and the HYDRA API backend (the specific technology for which is to be determined in Section 4.3). Pricing is determined by the chosen App Service Plan tier—Basic, Standard, or Premium—which offers increasing levels of vCPU, RAM, and features like staging slots and autoscaling, as well as the number of concurrent instances configured via Azure's autoscale settings (Microsoft, 2024). This expenditure is heavily influenced by the deployment and staging strategies outlined in Sections 4.2 and 7.4; specifically, because HYDRA utilizes deployment-slot swapping to ensure zero-downtime updates, the Standard tier or higher becomes necessary once the system supports critical business operations, as the Free and Basic tiers lack staging slot capabilities (Microsoft, 2023). While a single Basic-tier instance can manage the API and frontend requirements for a small number of office and field personnel, higher traffic volumes trigger the automatic addition of instances through Standard or Premium tier autoscale rules based on CPU or request-queue metrics. This demonstrates the inherent elasticity of a managed platform-as-a-service compared to manually provisioned, fixed servers (Erl, Puttini and Mahmood, 2013).

### 8.1.2 Database hosting

Azure Database for PostgreSQL serves as the hosting solution, a managed relational service selected following the comparative analysis presented in Section 4.3. This specific service was chosen due to the highly relational nature of HYDRA's data, which maintains rigorous foreign-key constraints among Customer, Job, Quote, Invoice, Payment, Material, and Inspection\_Report entities, as illustrated in the ERD (Section 4.4). The associated costs are determined by three primary, distinct variables: the selected compute tier (Burstable, General Purpose, or Memory Optimized, which dictates the level of guaranteed vCPU and RAM), the volume

of provisioned storage (which must expand as records for jobs, quotes, invoices, and inspections accumulate, given that data is retained rather than deleted in accordance with the policy in Section 6.2.5), and the designated backup-retention duration, as Azure applies additional charges for backup storage exceeding the standard allocation (Microsoft, 2024). A managed service was intentionally preferred over a self-hosted PostgreSQL installation on a virtual machine to ensure that automated patching, backups, and high-availability failover—essential components of the recovery objectives defined in Section 6.6—are managed by the platform. This approach avoids the necessity for the client to manage database infrastructure following handover, a task for which the client (Section 1.2) lacks both the interest and the internal technical expertise.

### **8.1.3 Background and scheduled compute**

HYDRA utilizes a limited set of scheduled, non-interactive background operations, specifically the missed-call auto-reply and reminder system detailed in Section 6.4.10, the transitions of invoice statuses (as shown in the Section 4.1 state diagram), and regular stock-level verifications against the Material entity (Section 4.4). These tasks are executed via Azure Functions using timer triggers instead of continuous compute, adhering to a serverless, pay-per-execution framework appropriate for these requirements (Erl, Puttini and Mahmood, 2013; Newman, 2021). Expenditures in this area are determined solely by the monthly execution count alongside the duration and memory usage of each instance. Given that the Azure Functions Consumption plan provides a significant monthly allowance of free executions and compute-seconds, these costs stay at or near zero until a significant increase in task volume—and consequently the frequency of reminders, status updates, and stock checks—occurs, as demonstrated in Table 11.

### **8.1.4 Object storage**

Azure Blob Storage serves as the object storage solution for the inspection-report photographs and compliance documentation cited in Sections 4.1, 4.4, and 6.2.1. Expenses for this service are determined by two independent variables: the total quantity of stored data (measured in gigabytes), which increases continuously as additional jobs are finished and images are uploaded without deletion (aligning with the retention policy in Section 6.2.5), and the volume of data egress, which is dictated by the frequency of image access or downloads by staff, clients, or auditors rather than the total amount of stored data. Since individual inspection-report images are compressed and relatively small, yet are numerous and subject to permanent retention, this expenditure increases consistently and predictably in relation to total job volume. Consequently, this represents one of the most reliable line items to forecast within Table 11.

### **8.1.5 Secrets management**

Azure Key Vault serves as the repository for the signing keys detailed in Sections 6.1.3 and 6.2.3, as well as the API keys for the payment gateway, SMS/WhatsApp provider, and any mapping or geocoding services utilized in job scheduling, along with connection strings. Rather than being priced based on the volume of stored secrets, expenses are incurred per retrieval operation; consequently, costs are determined by the frequency of runtime secret requests rather than the total number of secrets. Since efficient applications cache secrets in memory following the initial retrieval to avoid constant Key Vault calls, and given that the number of secrets used by HYDRA is minimal compared to the free-tier allowance, this expenditure is insignificant even at the maximum scale illustrated in Table 11. This cost is listed in the breakdown primarily for the sake of thoroughness rather than as a significant budgetary factor.

### **8.1.6 CDN and edge networking**

Designated in Section 5.3 as the edge/CDN and TLS-termination layer for HYDRA, Azure Front Door delivers the compiled React frontend from edge points of presence situated near end users, supplies the managed TLS certificate detailed in Section 8.1.8, and maintains the network-segregation boundary separating the public internet from the private application tier illustrated in the Section 5.3 cloud architecture diagram. Expenses for this service are determined by the quantity of data transmitted via the edge network and the total number of routed requests; both metrics correlate directly with the number of active users—such as customers monitoring job status, electricians accessing assignments, and administrative personnel managing quotes—rather than solely with job volume. Consequently, this cost component is more influenced by concurrent user activity than by total job throughput, representing a significant departure from most other categories in Table 10. The client should consider this distinction, particularly if promotional efforts drive increased site traffic without a corresponding rise in completed jobs.

### 8.1.7 Domain name

It is anticipated that HYDRA's production domain will be registered under the .co.za second-level domain, managed by the .za Domain Name Authority (ZADNA). This choice aligns with Trite Solar's current tritesolar.co.za website (Section 1.2) and reflects the tendency of South African consumers to favor locally identifiable domains when engaging with domestic service providers. ZADNA determines the wholesale costs for registering and renewing .co.za, .org.za, .net.za, and .web.za domains; these fees are directly tied to the South African Consumer Price Index (CPI). Following an adjustment in October 2024, ZADNA has indicated projected annual increases for the 2025/26 and 2026/27 periods (ITWeb, 2024). While retail registrars apply a markup to the wholesale rate, the annual cost for a .co.za domain remains relatively low, typically ranging from R70 to R100 per year through most local registrars at present. However, these costs are not entirely static over the two-year period detailed in Section 8.4. Consequently, the projection in Table 12 distributes this minor annual expense into monthly installments, noting that costs will increase slightly every October to correspond with ZADNA's CPI-based adjustment schedule (ITWeb, 2024).

### 8.1.8 SSL/TLS certificate

As Azure Front Door (Section 8.1.6) provides a managed TLS certificate within its standard service at no extra cost, HYDRA avoids additional, recurring SSL/TLS expenses outside of the CDN/edge fee already listed in Table 10. This architectural decision was made to eliminate the administrative and financial burdens associated with acquiring and manually renewing third-party certificates. Furthermore, this approach fulfills the "protection in transit" mandate detailed in Section 6.2.2, as the platform automates certificate renewals, thereby removing the risk of expiration due to human oversight.

### 8.1.9 Notification gateway (SMS, WhatsApp and email)

The notification gateway—which utilizes service providers like Twilio or regional aggregators such as Clickatell to transmit SMS and WhatsApp Business API communications to South African mobile devices—incurs the individual message fees for the automated missed-call responses, job-status notifications, and payment reminders detailed in Sections 1.3, 4.1, and 6.4.10. Unlike fixed subscription models, this service is invoiced per message dispatched, making it one of the two expense categories in Table 10 (the other being the payment gateway) that increases in direct, linear proportion to business volume, unlike the compute and storage tiers which offer economies of scale. Conversely, email notifications are intended to be facilitated via a low-cost or free transactional service through the Azure ecosystem or a specialized provider; these are omitted from Table 11 because their per-unit cost remains insignificant compared to SMS and WhatsApp messaging across all modeled scales.

### 8.1.10 Payment gateway fees

Within this segment, payment gateway fees represent the most significant and revenue-sensitive expenditure. Diverging from the other entries in Table 10, these are not fundamental infrastructure expenses; rather, they are operational costs that PSG Electrical and Cables/Trite Solar would be obligated to pay for card and EFT processing regardless of HYDRA's implementation. According to SA Shares (2026), the 2026 pricing for major South African payment providers accessible to small enterprises typically centers on card transaction fees between 2.9% and 3.5% of the total value, occasionally supplemented by a minor fixed fee per transaction, though entry-level tiers generally lack monthly subscription costs. Conversely, Instant EFT payments are generally more cost-effective for merchants, with rates ranging from 1.5% to 2.0% (SA Shares, 2026). PayFast, a highly reputable and prominent gateway for South African small businesses, applies card fees of roughly 3.2% plus a nominal per-transaction fee, while its Instant EFT rate is approximately 2.0% subject to a minimum flat fee, all without setup or monthly charges on its standard plan (SA Shares, 2026). Given the intense competition among providers like Yoco, Peach Payments, Paystack, and Ozow—each offering distinct fee structures tailored to specific transaction types (such as Ozow's low EFT rates, which align with the invoice-based payment processes of PSG Electrical and Cables/Trite Solar)—the gateway selection analysis in Sections 4.3 and 6.5.3 should be reviewed periodically against current market rates instead of being viewed as a final determination. Consequently, the cost estimates in Table 11 utilize an illustrative blended average of 2.9% of transaction value, reflecting the median of the published rates among comparable providers (SA Shares, 2026).

### 8.1.11 Cost categories deliberately excluded from the infrastructure figures

To ensure transparency, it must be explicitly stated that Tables 6 and 7 exclusively detail infrastructure and transaction-processing expenses. These figures intentionally omit two additional cost categories required for a comprehensive organizational budget but excluded here to align with the module rubric's definition of "running costs." The first omission is the continuous human resource expenditure associated with any post-handover technical support selected by the client; this is discussed qualitatively rather than through specific Rand values in Sections 9.2 and 9.3, as the Partnership Agreement with Emeris does not mandate that the HYDRA team provide paid maintenance following the handover. The second omission concerns discretionary future feature development, such as the potential Android application mentioned as a stretch goal in Section 1.5, which would be commissioned and priced as an independent, singular project rather than being included in the recurring monthly operational costs modeled in this section.

## 8.2 Cost at Multiple Scales

This section utilizes three distinct scales based on the monthly volume of processed jobs, as this metric serves as the most accurate indicator of load for nearly all cost categories listed in Table 10. Each job necessitates multiple API calls, database writes and subsequent storage requirements, the upload of inspection-report images to Blob Storage, at least one invoice and its corresponding payment (incurring gateway fees), and potentially several SMS, WhatsApp, or missed-call notifications. This specific scaling variable was intentionally selected over broader metrics like page views or registered users because it aligns with the data already monitored by the client within the manual processes used to model the ERD in Section 4.4. Consequently, the client can directly correlate these tables with familiar operational figures instead of needing to convert technical metrics into business terminology.

**Table 11: Estimated monthly running cost by scale (ZAR, approximate)**

| Cost Item                                                                                | 100 jobs/month                         | 1,000 jobs/month                              | 10,000 jobs/month                                          |
|------------------------------------------------------------------------------------------|----------------------------------------|-----------------------------------------------|------------------------------------------------------------|
| App Service (API + frontend)                                                             | R350 (Basic tier, 1 instance)          | R900 (Standard tier, autoscale 1–2 instances) | R4,500 (Premium tier, autoscale 3–6 instances)             |
| Azure Database for PostgreSQL                                                            | R300 (Burstable tier, 32GB storage)    | R850 (General Purpose tier, 64GB storage)     | R3,800 (General Purpose tier, 256GB storage, read replica) |
| Azure Functions (scheduled jobs)                                                         | R0 (within free-grant execution limit) | R50                                           | R300                                                       |
| Blob Storage (inspection images/docs)                                                    | R30 (~5GB)                             | R150 (~50GB)                                  | R900 (~400GB)                                              |
| Azure Front Door / CDN                                                                   | R100                                   | R400                                          | R2,500                                                     |
| Key Vault                                                                                | R0 (negligible at this volume)         | R20                                           | R80                                                        |
| Domain name (amortised monthly)                                                          | R15                                    | R15                                           | R15                                                        |
| SSL/TLS certificate                                                                      | R0 (included via Azure Front Door)     | R0                                            | R0                                                         |
| SMS/WhatsApp notification gateway (missed-call replies, reminders)                       | R80 (~400 messages @ ~R0.20)           | R800 (~4,000 messages)                        | R8,000 (~40,000 messages)                                  |
| Payment gateway fees (~2.9% of transaction value, illustrative average job value R2,500) | R7,250                                 | R72,500                                       | R725,000                                                   |
| Estimated infrastructure subtotal (excl. payment gateway fees)                           | ~R875/month                            | ~R3,185/month                                 | ~R20,095/month                                             |
| Estimated total incl. payment gateway fees                                               | ~R8,125/month                          | ~R75,685/month                                | ~R745,095/month                                            |

### 8.2.1 Low-volume scenario (100 jobs/month)

With a monthly workload of roughly 100 jobs—which accurately reflects the current operational volume of PSG Electrical and Cables/Trite Solar following deployment, based on the manual processes detailed in Section 1.2—HYDRA's infrastructure requirements are met by entry-level paid tiers for all services. The estimated monthly infrastructure cost is approximately R875, excluding fees from payment gateways. At this volume, the requirements are adequately satisfied by a single Basic-tier App Service instance, a Burstable-tier PostgreSQL database, and the free tiers of Key Vault and Azure Functions. Consequently, the client avoids unnecessary expenditures on capacity or resilience capabilities, such as read replicas or multi-instance autoscaling, which are not yet required for an enterprise of this size. This approach adheres to the principle of aligning expenditure with proven requirements rather than provisioning for speculative growth, a core tenet of the FinOps Optimize phase mentioned in the introductory section (FinOps Foundation, 2026).

### 8.2.2 Medium-volume scenario (1,000 jobs/month)

Upon reaching a volume of 1,000 monthly jobs—a tenfold expansion projected to occur within one to two years of effective implementation based on the growth models in Section 8.4—various services must transition to higher tiers. Specifically, the App Service Plan must upgrade to the Standard tier, utilizing one to two instances with autoscaling enabled to manage peak demand, such as concurrent material and photo uploads by electricians at the conclusion of a workday. Additionally, the database must migrate to the



General Purpose tier, providing increased storage and enhanced I/O throughput to maintain query performance during higher transaction periods. At this capacity, the projected infrastructure subtotal increases to approximately R3,185 per month. Although job volume increases tenfold, this cost represents less than a fourfold increase from low-volume levels, demonstrating that HYDRA's infrastructure expenses do not scale linearly with demand; this is due to a well-designed managed-service platform's ability to accommodate moderate growth within current compute capacity before tier upgrades become necessary (Erl, Puttini and Mahmood, 2013; Microsoft, 2024).

### 8.2.3 High-volume scenario (10,000 jobs/month)

A monthly volume of 10,000 jobs represents a significant expansion compared to the existing single-branch operations of PSG Electrical and Cables/Trite Solar, aligning more closely with a franchised or multi-location model. This specific scale is analyzed to satisfy the module rubric's requirement for cost projections across various user levels and to provide the client with necessary financial foresight prior to pursuing such expansion. At this capacity, the App Service Plan transitions to a Premium tier utilizing autoscale across three to six instances, while the database shifts to an expanded General Purpose configuration incorporating a read replica to isolate reporting and analytics from primary transactional tasks—a configuration that adheres to the layered architecture and separation-of-concerns principles detailed in Section 5.2. Additionally, expenses for Blob Storage and Front Door increase in correlation with the heightened request rates and data volumes. The projected monthly infrastructure subtotal is approximately R20,095. Although this signifies a notable increase in absolute monetary expenditure, it remains a relatively low operating cost in relation to the revenue generated by such job volumes. This demonstrates that the HYDRA architecture, as established in Section 5.3, possesses the capacity to accommodate substantial business expansion without necessitating a structural redesign, thereby fulfilling the scalability non-functional requirement defined in Section 2.4.

### 8.2.4 Cost per job as a normalised comparison metric

To provide a more insightful interpretation of Table 11 beyond the absolute Rand values, infrastructure costs can be calculated on a per-job basis. At a volume of 100 jobs per month, the infrastructure subtotal of approximately R875 results in a cost of roughly R8.75 per job; at 1,000 jobs per month, the subtotal of about R3,185 yields approximately R3.19 per job; and at 10,000 jobs per month, the subtotal of nearly R20,095 reduces to roughly R2.01 per job. This reduction in unit cost as volume rises serves as a quantified demonstration of economies of scale within cloud infrastructure. Specifically, fixed-overhead expenses listed in Table 10—such as the domain name, SSL certificate, and baseline Front Door and Key Vault allocations—are distributed across an increasing number of jobs, causing their individual cost per job to diminish even as absolute costs remain stable or increase marginally. This approach provides PSG Electrical and Cables/Trite Solar with a practical unit-economics metric that allows for a direct comparison against per-job revenue, offering a more functional planning tool for small business owners than isolated monthly infrastructure totals.

### 8.2.5 Sensitivity to exchange rate and third-party pricing changes

Because the compute, database, storage and CDN figures in Table 11 are all ultimately derived from Azure's USD-denominated pricing (Microsoft, 2024) converted to ZAR, and because the payment-gateway and SMS/WhatsApp figures are set independently by South African third-party providers who periodically revise their own rate cards (SA Shares, 2026), Table 11 should be understood as sensitive to two largely independent sources of movement: Rand/Dollar exchange-rate fluctuation, which affects every Azure-hosted line item simultaneously and in the same direction, and independent, service-specific price changes by Azure, the payment gateway or the SMS/WhatsApp provider, which may move individual line items up or down without

affecting the others. For planning purposes, the client should treat the infrastructure subtotal figures in Table 11 as accurate to within approximately plus-or-minus 15–20% under normal exchange-rate conditions, and should re-run the Azure pricing calculator, and check the payment gateway's and SMS provider's current published rates, at each of the review checkpoints recommended in Section 8.4.4, rather than relying on this document's figures indefinitely as exchange rates and vendor pricing continue to move after the point of writing.

## 8.3 Free-Tier vs Paid-Tier Trade-offs

### 8.3.1 What is realistically available at no cost

The values presented in Table 11 for compute, database, storage, and CDN are calculated by converting Azure's USD-based pricing (Microsoft, 2024) into ZAR. Conversely, the costs for the payment gateway and SMS/WhatsApp services are determined by South African third-party vendors who update their rates periodically (SA Shares, 2026). Consequently, Table 11 is subject to two distinct variables: fluctuations in the Rand/Dollar exchange rate, which impact all Azure-related costs uniformly, and isolated price adjustments by Azure or the third-party service providers, which may affect specific line items independently. For budgetary planning, the client should assume a margin of error of approximately 15–20% for the infrastructure subtotal under stable exchange rates. Furthermore, rather than relying on these figures long-term as currency values and vendor rates evolve, the client should utilize the Azure pricing calculator and verify the current rates of the payment and SMS providers during each scheduled review period outlined in Section 8.4.4.

### 8.3.2 Limitations and risks of free and lowest-cost tiers

The disadvantage of utilizing free or minimal-cost service tiers is that they offer significantly reduced performance capabilities, including restricted computational resources, a lack of autoscaling, and diminished database connection limits; furthermore, they may lack formal uptime guarantees. While these tiers are suitable for development, user acceptance testing, and small-scale pilot programs, they are inadequate for when PSG Electrical and Cables/Trite Solar relies on the platform for high-volume, revenue-critical operations. Specifically, if an under-provisioned App Service instance prevents a client from viewing or approving a quote, the resulting business loss—such as a delayed or lost contract—would likely surpass the minor expense of a paid tier. This situation aligns with the established cloud computing principle that low-cost tiers are intended for development and evaluation rather than production environments, necessitating that they be carefully scoped and evaluated during the cloud adoption process (Erl, Puttini and Mahmood, 2013; National Institute of Standards and Technology, 2011).

### 8.3.3 Recommended migration path

The suggested course of action, as detailed in the phased implementation strategy in Sections 4.2 and 9.4, involves initiating services on a low-cost paid tier (specifically the "100 jobs/month" category in Table 11) immediately upon production deployment. Rather than staying on free tiers perpetually once actual customer and payment information is integrated, the organization should transition to increasingly expensive tiers only when documented job volume—monitored through the system's internal reporting tools mentioned in Section 2.1—necessitates it. This prevents the premature acquisition of unused capacity. This methodology of advancing to subsequent tiers only when data indicates necessity represents a pragmatic implementation of the FinOps Optimize principle described in the introduction: cost management should be an ongoing process informed by real-time usage metrics, rather than a static decision made at deployment that remains unchanged (FinOps Foundation, 2026; Microsoft, 2026a).

### 8.3.4 Governance of tier changes after handover

Upon the conclusion of the HYDRA team's WIL engagement (Section 9.2), PSG Electrical and Cables/Trite Solar will lack a permanent in-house cloud engineer. Consequently, the obligation to monitor usage relative to the thresholds specified in Table 11 and to approve any tier upgrades must be assigned to a specific individual within the client organization—specifically, the Manager or Owner designated in the handover documentation. This oversight should be supplemented by Azure's budget-alert feature, which can be set to automatically notify the client when monthly expenditures exceed a predetermined Rand amount, thereby establishing an automated warning system that eliminates the need for manual invoice reviews. Integrating this minimal governance procedure into the handover documentation (Section 9.2) transforms this section from a mere academic cost estimation into a practical, long-term management tool. This approach satisfies the implicit requirements of the grading criteria, which demand a comprehensive two-year financial projection rather than a singular, static assessment.

## 8.4 Two-Year Cost Projection

### 8.4.1 Growth assumptions and their justification

The following two-year forecast is based on the premise that the client's monthly job volume will increase from an initial post-launch baseline of roughly 100 jobs, utilizing a hypothetical quarterly growth rate of 15%. This rate is intentionally conservative compared to the 20% quarterly growth often applied to early-stage startup projections; this decision reflects that PSG Electrical and Cables/Trite Solar is an established entity digitizing current operations rather than a startup building a client base from the ground up. Furthermore, the integration of Trite Solar's installation services with PSG's electrical offerings (Section 1.2) is anticipated to yield consistent, organic growth as the expanded service range is introduced to the existing customer base, rather than following the rapid, viral trajectory typical of consumer applications. Consistent with standard project cost-estimation protocols, this growth assumption serves as a planning parameter rather than a certainty; such early-stage estimates are intended to function as a defensible baseline that is refined as more data becomes available throughout the project lifecycle (Kerzner, 2017; Project Management Institute, 2021).

**Table 12: Two-year infrastructure cost projection (ZAR/month, infrastructure only, excluding payment gateway fees)**

| Period           | Estimated Job Volume/Month | Estimated Tier                            | Estimated Monthly Infrastructure Cost |
|------------------|----------------------------|-------------------------------------------|---------------------------------------|
| Launch (Month 1) | ~100                       | Lowest paid tier                          | ~R875                                 |
| Year 1, Q2       | ~130                       | Lowest paid tier                          | ~R950                                 |
| Year 1, Q3       | ~175                       | Lowest paid tier                          | ~R1,100                               |
| Year 1, Q4       | ~230                       | Mid-tier transition begins                | ~R1,400                               |
| Year 2, Q1       | ~300                       | Mid tier                                  | ~R1,900                               |
| Year 2, Q2       | ~390                       | Mid tier                                  | ~R2,400                               |
| Year 2, Q3       | ~510                       | Mid tier                                  | ~R2,800                               |
| Year 2, Q4       | ~665                       | Mid tier, approaching next tier threshold | ~R3,100                               |

### 8.4.2 Interpreting the projection

Based on this forecast, HYDRA's infrastructure expenditures are expected to rise from less than R900 per month at inception to approximately R3,100 per month by the conclusion of Year 2. This represents a 3.5-fold increase in costs compared to a 6.5-fold rise in job volume during the same timeframe, demonstrating the efficacy of the managed, autoscaling architecture detailed in Section 5.3. Specifically, costs escalate at a slower rate than workload because the fixed-overhead elements listed in Table 10 (Key Vault, the domain name, and the base Front Door allocation) remain constant regardless of volume, while the compute and database tiers only increase at specific thresholds rather than continuously with each additional job. Utilizing the per-job metric established in Section 8.2.4, these figures suggest that the infrastructure cost per job decreases from approximately R8.75 at launch to roughly R4.66 by the end of Year 2. This significant enhancement in unit economics benefits the client automatically as the business expands, requiring no further infrastructure investment decisions other than the periodic tier-review governance process outlined in Section.

8.3.4.

#### **8.4.3 Risk factors and assumptions that could alter the projection**

Various elements may cause actual expenditures to deviate significantly from the estimates presented in Table 12. These variables are explicitly identified to ensure the client interprets the projection as a framework of assumptions rather than a certainty. Potential cost increases may arise from a continuous depreciation of the Rand against the US Dollar, which would simultaneously elevate all Azure-hosted costs due to the USD-based pricing structure of the service (Microsoft, 2024). Additionally, if business expansion exceeds the conservative 15% quarterly growth rate assumed in this model—a possibility given the significant South African demand for solar energy and backup power—the client would reach higher usage tiers more rapidly than anticipated. Furthermore, any unilateral price hikes by Azure, the payment gateway, or the messaging providers would increase costs independently of usage volume. Conversely, costs could be lower than projected if growth is slower than the 15% quarterly estimate, allowing the client to remain in lower-cost tiers for an extended period. Finally, implementing the FinOps-aligned governance strategies suggested in Section 8.3.4, alongside utilizing Azure's reservation or savings plans once usage patterns stabilize, could lower compute and database expenses below the on-demand rates used in these calculations (Microsoft, 2024; FinOps Foundation, 2026).

#### **8.4.4 Recommended review cadence**

PSG Electrical and Cables/Trite Solar must reassess this projection during the quarterly milestones outlined in Table 12 by comparing it to the actual job volume recorded in HYDRA's reporting views (as detailed in Section 2.1). To avoid treating this as a static forecast, the data is presented as a series of assumptions and checkpoints. Consequently, Section 8.3.4 advises that a specific individual at the client organization be formally assigned the duty of conducting these reviews to prevent them from being neglected or handled informally. Such a structured quarterly schedule aligns with the "Operate" stage of the FinOps cycle mentioned previously, ensuring that cost governance is integrated into standard business operations rather than being treated as an isolated task performed only during initial deployment or in response to unexpected billing increases (FinOps Foundation, 2026; Microsoft, 2026a).

## 9. Change Management

The development of HYDRA represents only one component of the engagement; the remaining half involves facilitating the successful transition from the manual, paper-based workflows detailed in Section 1.2 to the new system within PSG Electrical and Cables/Trite Solar. Change-management research indicates that most organizational transformations fail not due to technical or budgetary deficiencies, but due to human-centric issues such as poor communication, inadequate training, unaddressed resistance, and a lack of post-implementation reinforcement (Prosci, 2026). Consequently, the module rubric designates 20 marks specifically to Change Management as a separate entity from technical components; a system that is technically superior in its ERD, authentication, and infrastructure is ultimately a failure if the client's personnel do not utilize it. Accordingly, this section provides a comprehensive analysis of the human and organizational aspects of HYDRA's implementation, including a role-specific training program, a formal handover protocol, a transition support phase, a phased implementation roadmap, and a strategy for managing resistance.

This approach is structured around two integrated frameworks: Kotter's (2014) model for organizational-level change and the Prosci (2026) ADKAR model for individual-level change, supplemented by professional guidance on selecting appropriate models based on the scale of the initiative (Cameron and Green, 2024). This dual strategy is intentional: Kotter's framework—focusing on urgency, coalition building, vision communication, empowerment, short-term successes, and cultural anchoring—is utilized to organize the roadmap and sequencing in Sections 9.3 and 9.4, whereas the ADKAR elements (Awareness, Desire, Knowledge, Ability, and Reinforcement) are used in Sections 9.1 and 9.5 to identify and mitigate individual obstacles faced by specific staff members, such as electricians or administrators (Kotter, 2014; Prosci, 2026).

Furthermore, this section must be understood within the framework of Work-Integrated Learning (WIL). These engagements are designed to be mutually beneficial arrangements where the university, the student group, and the industry client all benefit; the client receives a functional product, while students experience the organizational complexities of professional system delivery (Lubbe and Svensson, 2024). Change management is the area where this reciprocity is most evident and most vulnerable to failure. Without a robust change-management strategy, the system risks becoming a mere academic exercise that is submitted for grading but never utilized by the client, which would result in a suboptimal outcome for PSG Electrical and Cables/Trite Solar and a failure of the students to demonstrate the professional competencies required by a WIL placement (Lubbe and Svensson, 2024).

### 9.1 Training Plan

#### 9.1.1 Rationale for a role-differentiated training design

To ensure effectiveness, training is intentionally segmented by role instead of being provided as a universal session. This approach accounts for the fact that administrative personnel and field electricians utilize HYDRA under divergent circumstances, involving different hardware, time constraints, and varying levels of digital proficiency; a uniform training method would fail to adequately meet the needs of at least one group. This specialized structure directly implements the ADKAR model, which posits that Knowledge and Ability are separate, sequential objectives that must be systematically developed for each individual rather than being presumed to emerge from a single, general awareness session (Prosci, 2026). An office administrator with the capacity for a thorough demonstration has fundamentally different educational requirements than an electrician working between tasks at a client site. Consequently, the following training plan prioritizes these distinct requirements over administrative ease.

### 9.1.2 Office/Admin staff training

Administrative and office personnel undergo practical, on-site training at the client's location during the Handover phase (WBS task 6.1, Section 2.3). This session involves a comprehensive walkthrough of the Admin App's end-to-end workflow using realistic, non-production data, covering processes from initial website query review and job conversion to quote generation, electrician assignment, progress monitoring, invoicing, and payroll processing via submitted timesheets. To optimize adult learning, the training follows the specific sequence of the Admin Journey outlined in Section 2.2, screen by screen, rather than presenting features in an arbitrary menu-driven order (Cameron and Green, 2024). This sequential approach is reinforced by a written user guide that mirrors the Admin Journey's screen-by-screen progression, allowing staff to follow the exact steps demonstrated during training. The guide also provides annotated screenshots of every Admin App screen, a troubleshooting segment addressing common initial errors—such as misdirected quotes or unexpected status update failures—and a glossary that maps HYDRA's terminology to existing manual process terms to facilitate a smoother transition. As the primary and most frequent users according to the phased rollout in Section 9.4, office staff receive the earliest and most extensive training session within the Handover phase. This is supplemented by an informal follow-up session approximately one week later, providing an opportunity to address specific, practical questions that arise once staff begin utilizing the system with live data.

### 9.1.3 Electrician (field/employee) training

Training for field electricians is intentionally more concise and specialized than the administrative curriculum, acknowledging that technicians utilize the Employee App for brief, specific tasks while working on-site. These users often face environmental challenges such as wet or gloved hands, high outdoor glare, and significant time constraints between assignments, which differ substantially from a controlled office setting. The instructional program features a brief, practical session limited to four essential functions: viewing assigned tasks, recording attendance, scanning customer QR codes, and submitting inspection reports or material logs. This restricted scope is designed to optimize the "Ability" component of the ADKAR model, which is most effectively developed through frequent, low-stakes practice of a few core operations rather than passive learning of infrequent features (Prosci, 2026). To supplement this in-person training, a two-to-three-minute video tutorial is provided. This video is filmed in portrait mode on a mobile device to replicate the technician's actual field perspective, allowing them to independently review procedures without interrupting administrative staff with procedural inquiries. This audiovisual format was selected over lengthy written manuals because technicians are more inclined to watch a short video in the field than to consult multi-page documents. Additionally, a laminated one-page reference card with large print is supplied for those who prefer physical materials or face mobile data limitations.

### 9.1.4 Shared documentation asset and long-term availability

Each training pathway utilizes identical foundational documentation: role-specific quick-reference guides. These are provided in a comprehensive written format for Admin training and as condensed laminated cards for Electrician training. Developed as part of the Handover deliverable (Section 9.2), these materials are housed within client-controlled environments—such as shared drives for office personnel or laminated cards stored in electrician toolkits—ensuring permanent access regardless of the HYDRA team's future involvement. This strategy directly facilitates the ADKAR Reinforcement objective detailed in Section 9.5.6, as organizational documentation is more effective at sustaining change than the fading institutional memory of a departing student team, thereby assisting new or existing staff in maintaining correct system usage (Prosci, 2026).



### 9.1.5 Measuring training effectiveness

Instead of presupposing that a training session was effective solely based on its completion, efficacy is evaluated through two efficient, practical methods suited for small enterprises lacking a specialized learning and development department. Initially, during the administrative follow-up described in Section 9.1.2, employees are required to demonstrate one or two active tasks independently; this provides the instructor with tangible evidence regarding whether knowledge and ability have transitioned from supervised application to autonomous execution. Secondly, during the transition support phase detailed in Section 9.3, the frequency and type of inquiries from various user groups are informally monitored, as a concentration of similar questions serves as an economical indicator that specific portions of the user documentation or video tutorials require updates or further detail prior to the conclusion of direct assistance from the HYDRA team.

## 9.2 Handover Plan

### 9.2.1 Formal handover deliverables

Upon the project's completion, in accordance with the Partnership Agreement established between the HYDRA team, Emeris, and PSG Electrical and Cables/Trite Solar, the following items are formally transferred to the client. Rather than a simple list, each component is provided with the level of detail specified in the checklist:

The complete source code for the three applications (the Customer, Employee, and Admin Apps as defined by the three journeys in Section 2.2) and the backend API, alongside the completed database schema (Section 4.4) and a written account of the design logic and decisions (Sections 3.1, 3.2, and 5.1–5.3). This ensures that subsequent developers receive both functional code and a documented rationale for its architecture, addressing a critical deficiency often found in inadequate software handovers (Sommerville, 2016; Fowler, 2018).

Deployment credentials for the active Azure environment (Section 4.2), supplemented by a demonstration of how to inspect application logs, track uptime, monitor costs relative to the budget-alert limits mentioned in Section 8.3.4, and execute a slot-swap rollback (Section 7.4.4) should future updates cause issues. This will be provided via both written documentation and a recorded screen-share session, ensuring that even non-technical client stakeholders have observed the procedure at least once.

The final versions of the user manual and video tutorials specified in Section 9.1, housed in the client's designated storage location.

The entire documentation suite, comprising the requirements and analysis materials (Sections 2–5), the risk register (Section 6.8), the operational cost projections and two-year financial forecast (Section 8), and the backup and recovery protocols (Section 6.6). This comprehensive, self-contained record of the system's development and purpose is intended to assist any future internal or external developers, thereby significantly mitigating the "key person dependency" risk previously identified in the Section 6.8 risk register (Risk 15) following the departure of the original HYDRA team.

### 9.2.2 The handover session itself

Rather than being transmitted to the client as a collection of files via email upon project completion, the aforementioned handover deliverables are presented through a structured, in-person session conducted at the client's location. This approach adheres to the "show, don't just tell" methodology utilized in the training modules described in Section 9.1, based on the premise that stakeholders acquire significantly more practical knowledge by observing a live rollback or navigating the Azure portal with a team member than by merely



reviewing written documentation (Cameron and Green, 2024). Furthermore, this session functions as the formal occasion for the explicit and mutual transfer of system responsibility from the HYDRA team to the client, thereby preventing the uncertainty that often occurs when a project concludes without a clearly defined and collectively recognized transition milestone.

### **9.2.3 Explicit boundaries of post-handover responsibility**

In accordance with the Partnership Agreement, the client is explicitly informed—both through this written document and during the aforementioned handover session—that neither the HYDRA team nor Emeris shall bear responsibility for continuous maintenance or support following the formal conclusion of the transition period specified in Section 9.3. This distinction is articulated clearly and repeatedly throughout the documentation and the handover process to prevent any ambiguity. Such clarity is necessary because vague expectations regarding maintenance frequently lead to friction at the conclusion of similar WIL engagements; specifically, the unintended perception that the team remains available for informal assistance can create an expectation of perpetual, unpaid support, which is unsustainable for the team and complicates the client's long-term planning. To ensure the client understands precisely when the team's direct involvement ceases and their own responsibility for system operation, maintenance, and future development commences, this section and Section 9.3 are designed to eliminate any uncertainty regarding these obligations.

### **9.2.4 Documentation as the primary post-handover risk mitigation**

Because the client will not retain a dedicated in-house developer once the HYDRA team's direct involvement ends (Section 8.3.4), the completeness and clarity of the documentation handed over is, in a very real sense, the single most important post-handover risk-mitigation measure available — more important, in practice, than any individual technical decision made earlier in the project — since a future developer working from thorough, well-organised documentation of the kind this report itself models can reconstruct context and make sound decisions even without direct access to the original team, whereas a future developer working from code alone, without the reasoning behind it, is far more likely to introduce regressions or misunderstand the system's intended behaviour (Fowler, 2018; Sommerville, 2016). This is the reasoning that connects the handover plan directly back to the DevOps documentation culture already discussed in Section 7.7.3, and is precisely why this document itself — comprehensive, reasoned and explanatory rather than a bare technical specification — forms part of the handover package.

## **9.3 Transition Support Period**

### **9.3.1 Rationale for a defined, time-boxed window**

Following the production slot swap detailed in Section 4.2, a fourteen-day period of transitional assistance is provided. During this time, the HYDRA team will be directly accessible to PSG Electrical and Cables/Trite Solar to provide clarifications on training, address inquiries, and implement critical bug fixes via the CI/CD pipeline specified in Section 7. This specific, time-limited duration was intentionally selected as a balanced approach to satisfy both stakeholders. It offers the client essential support during the most volatile phase of implementation—the initial period following deployment when personnel first utilize training within live operational environments (Cameron and Green, 2024). Simultaneously, it establishes a definitive conclusion to the HYDRA team's responsibilities, accommodating their subsequent academic and professional obligations in accordance with the constraints outlined in Section 9.2.3.

### **9.3.2 Parallel running as a low-risk operational safety net**

Throughout this period, administrative personnel are advised to maintain simplified manual logs alongside the HYDRA system. This measure is not prompted by anticipated system failure, but rather serves as a conventional, low-risk precaution during the transition from manual to digital workflows. Such redundancy ensures that no tasks, quotations, or invoices are lost should any unforeseen complications arise during the initial two weeks of live operation. This method of parallel operation aligns with established organizational change management principles, where running existing and new processes concurrently for a specific duration is a recognized strategy to safeguard business continuity without permanently postponing the transition to new methods (Cameron and Green, 2024). This dual-running phase is strictly limited to the same two-week support window to avoid an unnecessary and indefinite increase in staff workload once the system's reliability has been established.

### **9.3.3 Support channel and escalation**

Throughout the transition support period, the client will be provided with one centralized point of contact—consisting of a shared messaging platform and a telephone number managed via a rotation of HYDRA personnel. This ensures that all PSG Electrical and Cables/Trite Solar staff, including both office-based employees and field electricians, can direct inquiries or concerns to a specific location without having to identify the most appropriate individual for a particular matter. Reported issues will undergo informal triage based on their impact: critical problems that impede the processing of jobs, quotes, or payments are classified as urgent and addressed as swiftly as possible through the hotfix procedure detailed in Section 7.5.5. Conversely, less severe matters, such as minor cosmetic concerns or usability questions, will be recorded and resolved either through immediate verbal communication or integrated into the documentation refinement process outlined in Section 9.1.5.

### **9.3.4 Formal conclusion of the transition support period**

At the end of the two-week window, parallel running is formally discontinued and HYDRA becomes the business's sole system of record, with the transition support window itself also concluding at that point, consistent with the explicit boundary set out in Section 9.2.3. This formal conclusion is itself communicated clearly to the client in advance — not allowed to simply lapse silently — so that the client experiences the end of direct support as a planned, expected milestone rather than as the team unexpectedly becoming unavailable, which reinforces rather than undermines the trust the wider engagement has been built on.

## **9.4 Adoption Roadmap**

### **9.4.1 Sequencing rationale**

To mitigate the risk of overwhelming support resources and damaging corporate reputation by simultaneously introducing an unfamiliar system to the entire workforce, HYDRA will be implemented through a strategic, three-stage phased approach. This sequence prioritizes small, adaptable groups and delays external customer exposure until the system is internally validated. This strategy aligns with Kotter's (2014) methodology of securing early, visible successes to establish the organizational momentum and confidence required for a comprehensive rollout.

Phase 1 (Weeks 1–2 post-implementation) focuses exclusively on administrative staff using the Admin App for quoting and task assignment. During this period, electricians will maintain their existing manual workflows. This approach confines potential usability challenges to a small cohort of users who have already participated in user acceptance testing (Section 2.3), thereby preventing disruption for the broader, more time-sensitive workforce.

Phase 2 (Weeks 3–4) involves the onboarding of electricians via the Employee App for tasks such as QR check-ins, milestone updates, and materials logging. To prevent operational confusion, ongoing jobs at the time of transition will be completed using legacy manual processes. By this stage, the administrative team will have two weeks of experience with HYDRA, allowing them to provide essential internal support to electricians and reducing the burden on the primary HYDRA support channel (Section 9.3.3).

Phase 3 (Week 5 onwards) marks the full customer-facing rollout, where the Customer App and website become the primary tools for service requests, tracking, and payments. This stage is intentionally deferred until both internal user groups are proficient. Because customer-facing errors pose the greatest reputational threat to PSG Electrical and Cables/Trite Solar, this sequence ensures that any operational flaws are identified and rectified internally before external clients are exposed to the system.

### 9.4.2 Mapping the roadmap to Kotter's model

Each phase of the roadmap above corresponds to a recognisable stage of Kotter's model of organisational change: Phase 1 functions as the "guiding coalition" and "short-term wins" stages, in which a small, engaged group (office staff, already positively disposed toward the system through their involvement in UAT) builds early, visible proof of success; Phase 2 extends the change to a wider group under conditions where urgency and vision have already been established by Phase 1's success, addressing the "empower action" and "build on the change" stages; and Phase 3, extending the change to customers, corresponds to Kotter's final stage of anchoring the new approach as the organisation's normal way of operating, since once customers themselves are using the Customer App to request and track work, reverting to the previous manual process ceases to be a realistic option and the change becomes self-sustaining (Kotter, 2014).

### 9.4.3 Mapping the roadmap to ADKAR at the individual level

Although Kotter's framework outlines the organizational progression of the roadmap, the ADKAR model clarifies the individual psychological mechanisms behind this order. Specifically, the training modules detailed in Section 9.1 and the consultative meetings presented in Section 9.5.4 are designed to establish awareness and motivation prior to each group's deployment. Furthermore, the model ensures that necessary knowledge and capabilities are acquired just before a group's respective phase commences. The process also leverages the visible achievements of preceding groups to bolster the motivation of those awaiting onboarding. Consequently, the roadmap is intentionally designed as a series of staggered individual ADKAR transitions throughout the organization, rather than a single, collective occurrence (Prosci, 2026).

### 9.4.4 Contingency if a phase does not proceed smoothly

Should a given phase surface unresolved issues — for example, if Admin staff in Phase 1 report a workflow gap that meaningfully slows their work rather than merely representing an unfamiliar new process — the roadmap is designed to allow the following phase's start date to be pushed back rather than proceeding to plan on a fixed calendar regardless of readiness, since advancing to the next, larger and more time-pressured user group before the current phase's issues are resolved would compound rather than contain any problem. This contingency is a deliberate design choice consistent with Kotter's caution against declaring victory too early in a change process and easing off before new behaviours are genuinely anchored (Kotter, 2014), and is a decision the transition support channel described in Section 9.3.3 is explicitly empowered to make in real time, rather than requiring a formal change request to the wider project plan.

## 9.5 Managing Resistance to Change

### 9.5.1 Where resistance is most likely to arise, and why

Kotter's framework outlines the organizational progression of the roadmap, whereas the ADKAR model elucidates the underlying logic at the individual employee level. Specifically, the roadmap is intentionally structured as a series of staggered individual ADKAR journeys across the organization, rather than a singular, simultaneous occurrence (Prosci, 2026). This is achieved by utilizing the training detailed in Section 9.1 and the consultations mentioned in Section 9.5.4 to proactively cultivate Awareness and Desire prior to each group's implementation date. Furthermore, the model ensures Knowledge and Ability are established just before a group's phase commences, while leveraging the visible achievements of preceding phases to bolster Desire in subsequent groups.

The roadmap is designed to accommodate delays if a specific phase reveals significant problems—such as a workflow deficiency reported by Phase 1 Administrative staff that genuinely hinders productivity, as opposed to mere unfamiliarity with new procedures. Rather than adhering to a rigid schedule, the start of subsequent phases may be postponed to prevent the escalation of issues that would occur if a larger, more time-sensitive user group were onboarded before current obstacles are addressed. This flexibility is an intentional design feature aligned with Kotter's (2014) warning against premature declarations of success before new behaviors are firmly established. Consequently, the transition support channel described in Section 9.3.3 is granted the authority to make these real-time adjustments without the necessity of a formal change request to the overarching project plan.

### **9.5.2 Mitigation 1 — deliberately minimal scope**

To minimize the learning curve for electricians, whose primary focus is electrical tasks rather than software proficiency, the Employee App's scope is intentionally restricted to four essential functions (Section 9.1.3). This design directly addresses the previously noted apprehension regarding potential operational delays by providing a tool that is demonstrably easy to master and utilize rapidly. This strategy aligns with the Ability component of the ADKAR model: because the application can be mastered during a single brief training session and reviewed via a short video if necessary, it offers a significantly lower barrier to Ability than a complex, feature-heavy system, irrespective of the user's existing levels of Awareness, Desire, or Knowledge (Prosci, 2026).

### **9.5.3 Mitigation 2 — phased, staggered exposure**

Furthermore, the incremental implementation strategy outlined in Section 9.4 ensures that electricians are not required to utilize the system during their initial encounter with it under high-pressure, real-world circumstances. By the onset of Phase 2, administrative personnel will have achieved daily proficiency with HYDRA, allowing them to serve as internal resources for guidance and support, thereby preventing a simultaneous lack of familiarity across the entire organization. This staggered approach specifically mitigates the apprehension regarding perceived incompetence noted in Section 9.5.1; an electrician experiencing difficulty with a new functionality during Phase 2 can seek assistance from office staff who have already successfully employed the system for two weeks, rather than being the sole individual facing a novel issue without any internal precedent.

### **9.5.4 Mitigation 3 — genuine co-design rather than imposed change**

Furthermore, the QR check-in and milestone-update functionalities were confirmed with the client during a specific consultation on 19 June 2026 (meeting 12, Section 1.4) prior to development. Consequently, the feature most significantly impacting the daily operations of electricians was developed through client collaboration rather than being unilaterally implemented. This approach aligns with recognized change-management principles, which suggest that resistance is minimized when those impacted by a transition are permitted to influence its design, rather than merely receiving notification of a finalized decision. Engaging

stakeholders in the design phase fosters "Desire"—the "D" component of the ADKAR model—which cannot be artificially generated through communication once a design is finalized (Prosci, 2026; Cameron and Green, 2024). To mitigate the apprehension regarding increased surveillance noted in Section 9.5.1, the client's involvement ensured the QR check-in feature was positioned as a means of ensuring job-status precision and providing customer reassurance—specifically by confirming an electrician's arrival—instead of a tool for employee monitoring. This conceptual framing is maintained throughout the training documentation in Section 9.1.3, ensuring the feature is presented to electricians in accordance with its original design intent.

#### **9.5.5 Reactive support if resistance surfaces despite these mitigations**

If resistance emerges during the transition support phase (Section 9.3) notwithstanding the three proactive mitigation strategies previously mentioned, the two-week period of parallel operations offers a low-risk environment for impacted employees to communicate grievances and obtain specific, individualized assistance prior to the official conclusion of the manual process. This approach prevents unresolved friction from intensifying into total non-compliance. Any opposition voiced during this interval shall be addressed by the HYDRA team and, following the handover, by the client, as a legitimate indication of potential issues regarding usability or trust that warrant independent investigation in accordance with the framework in Section 9.5.1, rather than being dismissed or ignored as a mere impediment.

#### **9.5.6 Reinforcement — sustaining adoption beyond the transition period**

Ultimately, since the HYDRA team's official assistance terminates upon the conclusion of the transition support phase (Section 9.3.4), long-term adoption necessitates that the final stage of the ADKAR model, Reinforcement, is authentically integrated into the client's operational framework rather than depending on the continued involvement of the outgoing team (Prosci, 2026). Specifically, this requires that the shared documentation detailed in Section 9.1.4 stays both available and up to date; that the client's Manager/Owner-role personnel (previously designated in Section 8.3.4 for cost governance) assume informal responsibility for verifying process compliance following the decommissioning of the parallel manual record; and that future internal communications regarding the QR check-in and electrician-facing tools maintain the trust-building and reassuring tone established during the initial client consultation (Section 9.5.4), avoiding an informal shift toward a monitoring-focused narrative that could jeopardize the goodwill generated during the co-design phase. Such reinforcement—characterized by intentionality, continuity, and internal organizational ownership rather than external management—is the deciding factor between a change that becomes a permanent part of organizational culture and one that regresses to previous behaviors once the project team's focus shifts (Kotter, 2014; Prosci, 2026).

## 10. Appendices

This section provides supporting material referenced throughout the main body of this report: the diagram-to-section coverage map, full minutes for the two client meetings specifically called out in the module checklist, the signed client meeting attendance confirmation, the signed WIL Partnership Agreement letter from Emeris, and a glossary of terms used throughout this document.

### Appendix A — Diagram-to-Section Mapping and Coverage Summary

The table below maps every diagram artefact produced for HYDRA to the section of this document in which it is discussed and explained, confirming complete coverage against the checklist and giving a marker a single page from which to locate any diagram referenced elsewhere in the report.

**Table 13: Diagram-to-section coverage map**

| Artefact                                                | Diagram Type                                              | Discussed In |
|---------------------------------------------------------|-----------------------------------------------------------|--------------|
| Domain_Class_Diagram_1.png                              | Domain Class Diagram                                      | Section 3.1  |
| Design_Class_Diagram1.png                               | Design Class Diagram                                      | Section 3.2  |
| UX_JOURNEY_DIAGRAM.png                                  | UX Journey Map                                            | Section 2.2  |
| State_Diagrams.zip (11 diagrams; Figures 4–14)          | State Diagrams                                            | Section 4.1  |
| sequence_diagrams_new_.zip (11 diagrams; Figures 15–25) | Sequence Diagrams                                         | Section 4.1  |
| DEPLOYMENT-DEVELOPMENT_DIAGRAM_1.png                    | Deployment Diagram                                        | Section 4.2  |
| —                                                       | Data Schema: Choice of Technology (narrative, no diagram) | Section 4.3  |
| Client_ERD1.png                                         | Customer/Website ERD                                      | Section 4.4  |
| Employee_erd1.png                                       | Employee ERD                                              | Section 4.4  |
| Admin_erd1.png                                          | Admin ERD                                                 | Section 4.4  |
| ACCESS_ERD_WEBSITE-APP.png                              | Access Control ERD                                        | Section 4.4  |
| DESIGN_PATTERNS_FULL.png                                | Design Patterns (Repository)                              | Section 5.1  |
| Architecture_Pattern_Diagram.png                        | Architecture Pattern (Layered/N-tier)                     | Section 5.2  |
| CLOUD_ARCHITECTURE_1.png                                | Cloud Architecture                                        | Section 5.3  |
| —                                                       | Risk register (table, no diagram)                         | Section 6.8  |
| DEV-OPS_DIAGRAM.png                                     | DevOps Pipeline                                           | Section 7    |
| —                                                       | Running cost tables (no diagram)                          | Section 8    |
| —                                                       | Change management roadmap (no diagram)                    | Section 9    |
| HYDRA_WBS_GANTT_CHART.pdf                               | Work Breakdown Structure / Gantt Chart                    | Section 2.3  |

Coverage status: every diagram artefact produced for the project is written up under exactly one checklist-numbered section, with no diagram left undocumented and no diagram duplicated across two sections.

### Appendix B — Meeting Minutes

Table 2 (Section 1.4) summarises the topic and key outcome of all 20 meetings held between the project's inception and the close of the Design phase, and all 20 official Minutes of Meeting documents were produced and retained by the team. The module checklist specifically calls out two of these — the initial client meeting of 18 April 2026 and the client meeting of 16 May 2026 — as evidence of direct client engagement grounding this document's requirements. Both are reproduced in full below, followed by the client-signed Meeting Attendance Confirmation covering all 20 meetings.

#### B.1 Minutes of Meeting — 18 April 2026

##### Meeting details

| Field               | Detail                       |
|---------------------|------------------------------|
| Meeting Title       | First Meeting with Client    |
| Date                | 18 April 2026                |
| Time                | 11am to 6pm                  |
| Location / Platform | Umdloti (Salta Mall)         |
| Chairperson         | Kaedon Naidoo (Group Leader) |

|              |              |
|--------------|--------------|
| Minute Taker | Darsh Somayi |
|--------------|--------------|

Attendees present: Kaedon Naidoo, Darsh Somayi, Yashodha Govender, Katelyn Narain, Yajur Mohan and Lonwabo Gumedede

Apologies: None

### Discussion and decisions

| Agenda Topic                                | Key Points Raised                                                                                                                                                         | Decision Made                                                                               |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Introductions and Stakeholder Roles         | Identification of the main client representative; Introduction of developers, designers, and project manager; Preferred communication channels discussed                  | One main point of contact from both teams will handle communication and approvals.          |
| Overview of the Client's Business Goals     | Improve online visibility; Increase customer engagement; Provide easier access to services/products                                                                       | The platform will focus on improving customer experience and business growth.               |
| Purpose of the Website and Mobile App       | Customers should be able to browse information easily; Users should access services through both the app and website; Need for a professional and modern digital platform | Both platforms will share similar features and branding.                                    |
| Target Audience and User Requirements       | Mobile-friendly experience is important; Users should have simple and fast navigation; Accessibility for different age groups considered                                  | A responsive and user-friendly design will be prioritised.                                  |
| Features and Functionality Required         | User registration and login; Search functionality; Booking/order system; Notifications and admin dashboard                                                                | Essential features will be included in the first development phase.                         |
| Design Preferences and Branding Guidelines  | Company colours and logo usage; Preference for a modern and clean interface; Consistent branding across all platforms                                                     | The development team will prepare wireframes based on the branding requirements.            |
| Platform Requirements (Web, Android, iOS)   | Website should work on desktop and mobile browsers; Android app needed immediately; iOS support discussed as a future addition                                            | Development will begin with the website and Android app first.                              |
| Project Timeline and Development Milestones | Planning and design phase; Development and testing period; Final deployment schedule                                                                                      | A prototype will be delivered first for client feedback before final development continues. |
| Budget and Payment Discussion               | Development and hosting costs; Maintenance and future update expenses; Payment schedule linked to milestones                                                              | Payments will be made in stages as milestones are completed.                                |
| Summary of Next Steps and Action Items      | Client must provide branding assets and content; Developers to begin planning and wireframes; Future meetings required for progress updates                               | A follow-up meeting will be scheduled to review the first prototype.                        |

### Action items

| Task                                                       | Responsible                               | Deadline                           |
|------------------------------------------------------------|-------------------------------------------|------------------------------------|
| Share contact details and communication channels           | Project Manager and Client Representative | End of the meeting day             |
| Prepare a detailed business requirements document          | Client Representative                     | Within 3 business days             |
| Draft the system feature list and user flow                | Development Team                          | Within 1 week                      |
| Conduct user experience planning and audience analysis     | UI/UX Designer                            | Within 5 business days             |
| Create a technical specification document for features     | Lead Developer                            | Within 1 week                      |
| Provide branding materials and logo files                  | Client Marketing Team                     | Within 2 business days             |
| Set up development environments and project repositories   | Development Team                          | Within 3 business days             |
| Create and share the project timeline document             | Project Manager                           | Within 2 business days             |
| Prepare and send the final quotation and payment agreement | Finance Manager                           | Within 3 business days             |
| Schedule the next progress review meeting                  | Project Manager                           | Before the end of the current week |



## B.2 Minutes of Meeting — 16 May 2026

### Meeting details

| Field               | Detail                                         |
|---------------------|------------------------------------------------|
| Meeting Title       | Requirements Confirmation and Project Planning |
| Date                | 16 May 2026                                    |
| Time                | 1pm to 3pm                                     |
| Location / Platform | Microsoft teams                                |
| Chairperson         | Kaedon Naidoo (Group Leader)                   |
| Minute Taker        | Darsh Somayi                                   |

Attendees present: Kaedon Naidoo, Darsh Somayi, Yashodha Govender, Katelyn Narain, Yajur Mohan and Lonwabo Gumedede

Apologies: None

### Discussion and decisions

| Agenda Topic                   | Key Points Raised                                                                                                                                                                                                      | Decision Made                                                                                     |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Review of Initial Requirements | Customer, employee and administrator user roles confirmed; Website will attract customers while the app provides extended functionality; Electrical and solar services must remain clearly separated where appropriate | The requirements baseline was approved for use during analysis and design.                        |
| Business Scope Refinement      | PSG Electrical service information will be prominent; Trite Solar content will support the solar offering; No online sale of cables, lights or electrical stock will be included                                       | E-commerce product sales were excluded from the current project scope.                            |
| User Role Requirements         | Customers need service information, requests, rewards and job updates; Employees need customer/job information and calendar access; Administrators need oversight and management functions                             | Separate role-based access and dashboards will be designed.                                       |
| Functional Requirements        | Registration and secure login; Service request and job tracking; Rewards, discounts and notifications                                                                                                                  | Functional requirements will be prioritised before interface design is finalised.                 |
| Non-Functional Requirements    | Responsive performance across devices; Simple navigation and accessibility; Secure handling of user and business information                                                                                           | Usability, security, reliability and responsiveness will be treated as core quality requirements. |
| Branding Assets Review         | PSG Electrical and Trite Solar require distinct visual identities; Brand colours must remain consistent; Modern professional presentation is required                                                                  | Separate but related visual styles will be developed for the two brands.                          |
| Technical Platform Planning    | Website and mobile app require coordinated development; Shared data structures should reduce duplication; Version control will be used for team collaboration                                                          | A shared project structure and repository workflow will be adopted.                               |
| Project Schedule Review        | Work must be distributed across analysis, design, development and testing; Dependencies must be visible; The final project deadline must remain protected                                                              | The schedule will be updated with clearer milestones and dependencies.                            |
| Risk Identification            | Scope growth could affect delivery; Late client feedback could delay design approval; Integration and testing may require additional time                                                                              | A formal risk register will be maintained and reviewed at future meetings.                        |
| Next Steps                     | Requirements documents must be completed; Initial diagrams should begin; Branding concepts should be prepared                                                                                                          | The next meeting will focus on analysis models and early design outputs.                          |

### Action items

| Task                                                    | Responsible      | Deadline                |
|---------------------------------------------------------|------------------|-------------------------|
| Update and circulate the approved requirements baseline | Business Analyst | Within 2 business days  |
| Update the project scope statement and exclusions       | Project Manager  | Within 2 business days  |
| Create role-based user requirement lists                | Development Team | Within 4 business days  |
| Create a prioritised functional requirements register   | Business Analyst | Within 4 business days  |
| Document measurable non-functional requirements         | Lead Developer   | Within 5 business days  |
| Prepare initial colour and typography options           | UI/UX Designer   | Within 5 business days  |
| Set up repository branches and development conventions  | Lead Developer   | Within 3 business days  |
| Revise the WBS and project schedule                     | Project Manager  | Within 3 business days  |
| Create the first project risk register                  | Project Manager  | Within 3 business days  |
| Prepare analysis artefacts for review                   | Development Team | Before the next meeting |

### B.3 Client Meeting Attendance Confirmation

The document reproduced below was signed by the client to confirm that all 20 HYDRA project meetings listed in Table 2 (Section 1.4) were held on the dates and at the times recorded in the official minutes, and that the team members recorded as present in those minutes did attend. It was signed by Keoran Govender of PSG Electrical and Cables on 17 August 2026.

## HYDRA PROJECT

### Client Meeting Attendance Confirmation

#### *Confirmation of Meetings Held and Team Attendance*

This document is provided for the client to confirm that the HYDRA project meetings listed below were held on the dates and at the times recorded in the official meeting minutes, and that the project team members recorded as present attended each meeting.

**Team members recorded as present in the meeting minutes:** Kaedon Naidoo, Darsh Somayi, Yashodha Govender, Katelyn Narain, Yajur Mohan , Lonwabo Gumede.

| No. | Date          | Time        | Meeting                                        | Client Signature                                                                      |
|-----|---------------|-------------|------------------------------------------------|---------------------------------------------------------------------------------------|
| 1   | 18 April 2026 | 11am to 6pm | First Meeting with Client                      |    |
| 2   | 16 May 2026   | 1pm to 3pm  | Requirements Confirmation and Project Planning |   |
| 3   | 30 May 2026   | 1pm to 3pm  | Analysis Models and Use Cases                  |  |
| 4   | 6 June 2026   | 12pm to 3pm | UML Analysis Diagram Review                    |  |
| 5   | 13 June 2026  | 1pm to 3pm  | Wireframes and User Experience Review          |  |
| 6   | 20 June 2026  | 1pm to 4pm  | System Architecture and Database Design        |  |
| 7   | 27 June 2026  | 1pm to 3pm  | Website Prototype Review                       |  |
| 8   | 4 July 2026   | 1pm to 3pm  | Mobile App Prototype Review                    |  |
| 9   | 11 July 2026  | 1pm to 5pm  | Authentication and Role Access Review          |  |
| 10  | 17 July 2026  | 1pm to 3pm  | Customer Features Development Review           |  |
| 11  | 18 July 2026  | 12pm to 3pm | Employee Features Development Review           |  |
| 12  | 24 July 2026  | 1pm to 3pm  | Admin Features Development Review              |  |
| 13  | 25 July 2026  | 1pm to 3pm  | Rewards, Discounts and Notification Review     |  |
| 14  | 30 July 2026  | 1pm to 4pm  | Job Tracking and Calendar Integration Review   |  |
| 15  | 31 July 2026  | 1pm to 5pm  | Integration Testing Review                     |  |

HYDRA Project - Client Meeting Attendance Confirmation

|    |                |            |                                         |                                                                                     |
|----|----------------|------------|-----------------------------------------|-------------------------------------------------------------------------------------|
| 16 | 7 August 2026  | 1pm to 3pm | Usability and Client Feedback Review    |  |
| 17 | 8 August 2026  | 1pm to 3pm | Bug Fixing and Final Improvements       |  |
| 18 | 10 August 2026 | 1pm to 3pm | Final Testing and Documentation Review  |  |
| 19 | 14 August 2026 | 1pm to 5pm | Pre-Deployment and Submission Readiness |  |
| 20 | 15 August 2026 | 1pm to 5pm | Final Project Review and Handover       |  |

**Client Declaration**

By signing in the table above, I confirm that the corresponding meeting was held and that the HYDRA project team members recorded as present in the official minutes attended that meeting.

**Client Name:** KEORAN GOVENDER

**Company / Organisation:** PSG ELECTRICAL AND CABLES

**Final Client Signature:**  \_\_\_\_\_

**Date:** 17/08/2026

HYDRA Project - Client Meeting Attendance Confirmation

*Client Meeting Attendance Confirmation, signed by Keoran Govender (PSG Electrical and Cables), 17 August 2026.*

## **Appendix C — Partnership Agreement**

The Partnership Agreement letter reproduced below was issued by Emeris and formalises the roles and expectations of the HYDRA team, Emeris and the client (PSG Electrical and Cables/Trite Solar) for this Work-Integrated Learning engagement. It is the source document for the handover and post-handover support boundary described in Section 9.2 of this report — specifically its confirmation that Emeris and the student team are not responsible for ongoing maintenance of HYDRA once the transition support period (Section 9.3) has concluded — and is attached here in full as the authoritative reference for that boundary. It was signed by Denzyl Soobramoney Govender and Gareth Grant, WIL Coordinators, Emeris Durban North, School of Computer Science.



### **Partnership Agreement: Student Software Development Project**

To our valued Industry Partner,

Thank you for choosing to collaborate with our students at Emeris. This letter serves to formalize the roles and expectations regarding the development of bespoke software for your organization. This partnership provides our students with invaluable real-world experience while providing your company with tailored software solutions at no cost.

### **Intellectual Property and Confidentiality**

All software created during this project is provided free of charge. The Intellectual Property (IP) of the final product belongs entirely to your company.

Please note that if your organization requires a Non-Disclosure Agreement (NDA) to be signed, we ask that you reach out to the Work Integrated Learning (WIL) Coordinator to discuss the necessary protocols and legal frameworks.

### **Expectations of the Client**

By agreeing to participate as a project client, your organization commits to the following:

---

**Directors:** GD Whyte (UK), JW Boonzaaier, MD Aitken, SCD Lurie **Group Company Secretary:** CB Crouse

**KwaZulu-Natal** Durban North +27 31 573 2038 | Umhlanga +27 31 569 1415 | Westville-Durban +27 31 266 8400 | Pietermaritzburg +27 33 386 2376  
**Gauteng** Pretoria +27 12 348 2551 | Ruimsig +27 11 950 4000 | Sandton +27 11 784 6939 | Bordeaux +27 11 521 4600 | Waterfall-Midrand +27 10 224 4300  
Rosebank +27 11 234 1896 **Western Cape** Cape Town-City +27 21 461 8089 | Newlands-Cape Town +27 21 685 5021 **Eastern Cape** Nelson Mandela Bay  
+27 41 363 4223 **Online** +27 87 354 5884



Emeris is an educational brand of The Independent Institute of Education (Pty) Ltd which is registered with the Department of Higher Education and Training as a private higher education institution under the Higher Education Act, 1997 (reg. no. 2007/HE07/002).  
Company registration number: 1987/004754/07.



1. **Project Scope:** Providing the student group with a clear scope of work or initial idea.
2. **Engagement:** Offering regular feedback on the students' progress to ensure the project aligns with your vision.
3. **Implementation:** You are under no obligation to implement or use the final software if it does not meet your internal standards. However, regardless of implementation, the software remains your property.

### The Project Expo

As part of the academic process, students will present their work in an expo format. We warmly welcome you and your team to attend this event to see the culmination of their hard work and the final iterations of your project.

### Post-Handover and Support

It is important to note that Emeris is not responsible for the software, its maintenance, or any changes required after the students have completed their formal handover.

Once the academic project concludes:

- Any requests for further changes or support should be discussed directly with the students.
- Any arrangements for future work or updates beyond the project scope may involve remuneration, which must be negotiated directly between your company and the students.

### Contact and Queries

Should you experience any issues with your assigned student group, please direct your queries to the **WIL Coordinator**.

For any other questions, concerns, or clarifications regarding the project structure, please feel free to contact me directly using the details provided below. We look forward to a successful collaboration.

Kind regards,

A handwritten signature in black ink, appearing to be 'Denzyl Soobramoney Govender and Gareth Grant', written over a dashed line.

Denzyl Soobramoney Govender and Gareth Grant  
[dengovender@emeris.ac.za](mailto:dengovender@emeris.ac.za) and [gagrants@emeris.ac.za](mailto:gagrants@emeris.ac.za)

Wil Coordinators Emeris Durban North for School of Computer Science

**Directors:** GD Whyte (UK), JW Boonzaier, MD Aitken, SCD Lurie **Group Company Secretary:** CB Crouse

**KwaZulu-Natal** Durban North +27 31 573 2038 | Umhlanga +27 31 569 1415 | Westville-Durban +27 31 266 8400 | Pietermaritzburg +27 33 386 2376  
**Gauteng** Pretoria +27 12 348 2551 | Ruimsig +27 11 950 4000 | Sandton +27 11 784 6939 | Bordeaux +27 11 521 4600 | Waterfall-Midrand +27 10 224 4300  
**Rosebank** +27 11 234 1896 **Western Cape** Cape Town-City +27 21 461 8089 | Newlands-Cape Town +27 21 685 5021 **Eastern Cape** Nelson Mandela Bay +27 41 363 4223 **Online** +27 87 354 5884



Emeris is an educational brand of The Independent Institute of Education (Pty) Ltd which is registered with the Department of Higher Education and Training as a private higher education institution under the Higher Education Act, 1997 (reg. no. 2007/HE07/002).  
Company registration number: 1987/004754/07.

*WIL Partnership Agreement letter, Emeris, signed by Denzyl Soobramoney Govender and Gareth Grant.*

## Appendix D — Glossary of Terms

**Table 14: Glossary**

| Term                                    | Meaning                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Admin                                   | Office/management staff role with full or near-full system access; see Section 2.1                                                                                                                                                                                                                                               |
| CoC                                     | Certificate of Compliance — the legally required compliance document issued for South African electrical work (Republic of South Africa, 2009)                                                                                                                                                                                   |
| CI/CD                                   | Continuous Integration / Continuous Deployment — the automated build, test and release pipeline described in Section 7                                                                                                                                                                                                           |
| Electrician                             | Field technician role, also referred to as "Employee" throughout the applications and this document; see Section 2.1                                                                                                                                                                                                             |
| ERD                                     | Entity Relationship Diagram                                                                                                                                                                                                                                                                                                      |
| FK                                      | Foreign Key                                                                                                                                                                                                                                                                                                                      |
| HYDRA                                   | The name adopted by the six-member student team and, informally, the system they built                                                                                                                                                                                                                                           |
| JWT                                     | JSON Web Token — the signed authentication token format used across all three HYDRA apps (Section 6.1)                                                                                                                                                                                                                           |
| NFR                                     | Non-Functional Requirement                                                                                                                                                                                                                                                                                                       |
| PK                                      | Primary Key                                                                                                                                                                                                                                                                                                                      |
| POPIA                                   | Protection of Personal Information Act, No. 4 of 2013 (Republic of South Africa, 2013)                                                                                                                                                                                                                                           |
| PSG Electrical and Cables / Trite Solar | The client business; PSG Electrical and Cables is the electrical-services trading name, Trite Solar is its renewable-energy/solar installation division; both names refer to the same single client engagement and are used interchangeably in this document where the distinction between the two service lines is not material |
| RBAC                                    | Role-Based Access Control                                                                                                                                                                                                                                                                                                        |
| UML                                     | Unified Modelling Language                                                                                                                                                                                                                                                                                                       |
| WBS                                     | Work Breakdown Structure                                                                                                                                                                                                                                                                                                         |
| WIL                                     | Work-Integrated Learning                                                                                                                                                                                                                                                                                                         |



## References

The following sources are referenced throughout this document using the IIE Harvard (Angelica) referencing style.

- Agile Business Consortium, 2014. *DSDM Agile Project Framework: MoSCoW Prioritisation*. [Online] Available at: <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html> [Accessed 16 August 2026].
- AltexSoft, 2023. *User Stories: Documenting Requirements in Agile*. [Online] Available at: <https://www.altexsoft.com/blog/user-stories/> [Accessed 16 August 2026].
- Atlassian, 2023. *Comparing Workflows: Feature Branch Workflow*. [Online] Available at: <https://www.atlassian.com/git/tutorials/comparing-workflows> [Accessed 16 August 2026].
- Bass, L., Clements, P. and Kazman, R., 2021. *Software Architecture in Practice*. 4th ed. Boston: Addison-Wesley.
- Beyer, B., Jones, C., Petoff, J. and Murphy, N.R., 2016. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media.
- Cameron, E. and Green, M., 2024. *Making Sense of Change Management: A Complete Guide to the Models, Tools and Techniques of Organizational Change*. 6th ed. London: Kogan Page.
- Chacon, S. and Straub, B., 2014. *Pro Git*. 2nd ed. Berkeley: Apress.
- Cloud Security Alliance, 2017. *Security Guidance for Critical Areas of Focus in Cloud Computing, Version 4.0*. [Online] Available at: <https://cloudsecurityalliance.org/research/guidance> [Accessed 16 August 2026].
- Connolly, T. and Begg, C., 2015. *Database Systems: A Practical Approach to Design, Implementation and Management*. 6th ed. Harlow: Pearson Education.
- Elmasri, R. and Navathe, S.B., 2016. *Fundamentals of Database Systems*. 7th ed. Boston: Pearson Education.
- Erl, T., Puttini, R. and Mahmood, Z., 2013. *Cloud Computing: Concepts, Technology & Architecture*. Upper Saddle River: Prentice Hall.
- FinOps Foundation, 2026. *What is FinOps?* [Online] Available at: <https://www.finops.org/introduction/what-is-finops/> [Accessed 17 August 2026].
- Fitzgerald, B. and Stol, K-J., 2017. Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software*, 123, pp.176–189.
- Forsgren, N., Humble, J. and Kim, G., 2018. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland: IT Revolution Press.
- Fowler, M., 2018. *Refactoring: Improving the Design of Existing Code*. 2nd ed. Boston: Addison-Wesley.
- Garrett, J.J., 2011. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. 2nd ed. Berkeley: New Riders.
- Gibbons, S., 2018. *Journey Mapping 101*. Nielsen Norman Group. [Online] Available at: <https://www.nngroup.com/articles/journey-mapping-101/> [Accessed 16 August 2026].
- GitHub, 2024. *About Protected Branches*. [Online] Available at: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches> [Accessed 16 August 2026].
- Google, 2024. *Firebase Documentation*. [Online] Available at: <https://firebase.google.com/docs> [Accessed 17 August 2026].

- Humble, J. and Farley, D., 2010. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston: Addison-Wesley.
- Information Regulator (South Africa), 2021. *Guidance Note on Data Subject Participation and Security Safeguards under the Protection of Personal Information Act*. [Online] Available at: <https://inforegulator.org.za/> [Accessed 16 August 2026].
- International Organization for Standardization, 2011. *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva: ISO.
- International Organization for Standardization, 2013. *ISO/IEC 27001:2013 — Information Technology — Security Techniques — Information Security Management Systems — Requirements*. Geneva: ISO.
- International Organization for Standardization, 2018. *ISO 31000:2018 — Risk Management — Guidelines*. Geneva: ISO.
- ITWeb, 2024. *ZADNA Hikes Domain Name Registration Fees*. [Online] Available at: <https://www.itweb.co.za/article/zadna-hikes-domain-name-registration-fees/nWJad7bNxpX7bjO1> [Accessed 17 August 2026].
- Jones, M.B., Bradley, J. and Sakimura, N., 2015. *JSON Web Token (JWT), RFC 7519*. [Online] Internet Engineering Task Force. Available at: <https://datatracker.ietf.org/doc/html/rfc7519> [Accessed 16 August 2026].
- Kerzner, H., 2017. *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. 12th ed. Hoboken: Wiley.
- Kim, G., Humble, J., Debois, P. and Willis, J., 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability and Security in Technology Organizations*. Portland: IT Revolution Press.
- Kleppmann, M., 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol: O'Reilly Media.
- Kotter, J.P., 2014. *Accelerate: Building Strategic Agility for a Faster-Moving World*. Boston: Harvard Business Review Press.
- Larman, C., 2012. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd ed. Upper Saddle River: Prentice Hall.
- Lubbe, I. and Svensson, G., 2024. Work Integrated Learning (WIL) model – a win-win process between university, postgraduate business students and industry. *The Independent Journal of Teaching and Learning*, 19(1).
- Martin, R.C., 2017. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.
- Microsoft, 2023. *Set Up Staging Environments for Web Apps in Azure App Service*. [Online] Available at: <https://learn.microsoft.com/en-us/azure/app-service/deploy-staging-slots> [Accessed 16 August 2026].
- Microsoft, 2024. *Azure Well-Architected Framework*. [Online] Available at: <https://learn.microsoft.com/en-us/azure/well-architected/> [Accessed 16 August 2026].
- Microsoft, 2026a. *What is FinOps? Cloud Computing*. [Online] Available at: <https://learn.microsoft.com/en-us/cloud-computing/finops/overview> [Accessed 17 August 2026].
- MongoDB Inc., 2023. *MongoDB Documentation*. [Online] Available at: <https://www.mongodb.com/docs/> [Accessed 17 August 2026].
- National Institute of Standards and Technology, 2011. *The NIST Definition of Cloud Computing* (Special Publication 800-145). Gaithersburg: NIST.

- National Institute of Standards and Technology, 2017. *Digital Identity Guidelines: Authentication and Lifecycle Management* (Special Publication 800-63B). Gaithersburg: NIST.
- Newman, S., 2021. *Building Microservices: Designing Fine-Grained Systems*. 2nd ed. Sebastopol: O'Reilly Media.
- Object Management Group, 2017. *Unified Modeling Language (UML) Specification, Version 2.5.1*. [Online] Available at: <https://www.omg.org/spec/UML> [Accessed 16 August 2026].
- OWASP Foundation, 2021. *OWASP Top Ten*. [Online] Available at: <https://owasp.org/www-project-top-ten/> [Accessed 16 August 2026].
- OWASP Foundation, 2022. *Password Storage Cheat Sheet*. [Online] Available at: [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html) [Accessed 16 August 2026].
- Pathlock, 2026. *Role-Based Access Control (RBAC): A Comprehensive Guide*. [Online] Available at: <https://pathlock.com/blog/role-based-access-control-rbac/> [Accessed 16 August 2026].
- PCI Security Standards Council, 2022. *Payment Card Industry Data Security Standard, Version 4.0*. Wakefield, MA: PCI Security Standards Council.
- Pressman, R.S. and Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. New York: McGraw-Hill Education.
- Preston-Werner, T., 2013. *Semantic Versioning 2.0.0*. [Online] Available at: <https://semver.org/> [Accessed 16 August 2026].
- Project Management Institute, 2019. *Work Breakdown Structures Practice Standard*. 3rd ed. Newtown Square: Project Management Institute.
- Project Management Institute, 2021. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th ed. Newtown Square: Project Management Institute.
- Prosci, 2026. *The Prosci ADKAR® Model*. [Online] Available at: <https://www.prosci.com/methodology/adkar> [Accessed 17 August 2026].
- Republic of South Africa, 2009. *Electrical Installation Regulations, 2009* (made under the Occupational Health and Safety Act 85 of 1993, Government Notice R242). Pretoria: Department of Labour.
- Republic of South Africa, 2013. *Protection of Personal Information Act, No. 4 of 2013*. Pretoria: Government Printer.
- Richards, M., 2015. *Software Architecture Patterns*. Sebastopol: O'Reilly Media.
- Rozanski, N. and Woods, E., 2012. *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*. 2nd ed. Boston: Addison-Wesley.
- SA Shares, 2026. *PayFast Review South Africa Edition (Updated 2026)*. [Online] Available at: <https://sashares.co.za/payfast-review/> [Accessed 17 August 2026].
- Silberschatz, A., Korth, H.F. and Sudarshan, S., 2020. *Database System Concepts*. 7th ed. New York: McGraw-Hill Education.
- Snedaker, S., 2013. *Business Continuity and Disaster Recovery Planning for IT Professionals*. 2nd ed. Waltham: Syngress/Elsevier.
- Sommerville, I., 2016. *Software Engineering*. 10th ed. Harlow: Pearson Education.
- Sonatype, 2023. *State of the Software Supply Chain Report*. [Online] Available at: <https://www.sonatype.com/state-of-the-software-supply-chain> [Accessed 16 August 2026].

Vernon, V., 2013. *Implementing Domain-Driven Design*. Boston: Addison-Wesley.

## Appendix E — Prototype and Interactive Demonstrations

This appendix documents the clickable prototypes produced for HYDRA ahead of full development: a static HTML prototype of the public website, a deployed, fully interactive high-fidelity prototype of the Customer App, and flow diagrams with high-fidelity dashboard mockups for the Employee and Admin apps. Together these demonstrate, screen by screen, the same navigation flows described in the UX Journey Map (Section 2.2) and confirm that the visual design direction is consistent across all four surfaces of the platform before backend integration.

### E.1 System Overview

All four surfaces of the platform — the public website, the Customer App, the Employee App and the Admin App — are prototyped as a single connected system rather than four unrelated designs. Each surface has its own login and its own navigation, but all four read from and write to the same shared backend, so a quote request submitted on the website can be actioned by Admin, assigned to an Employee, and tracked live by the Customer, all from the same underlying data.

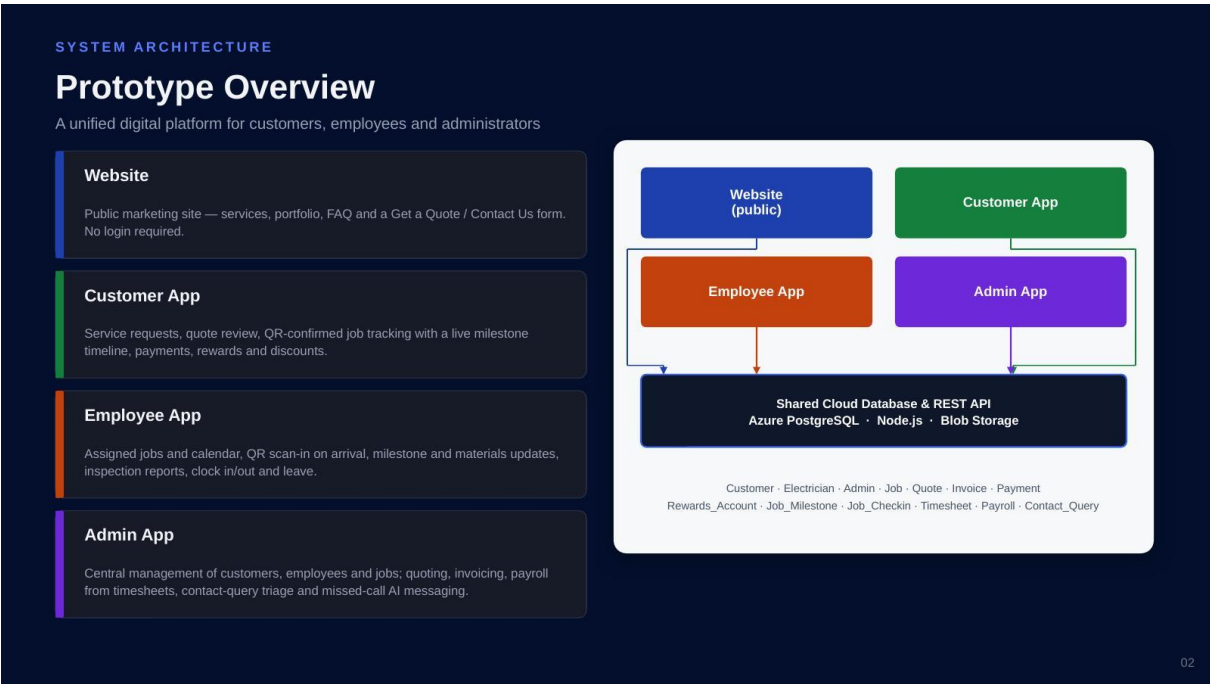


Figure 35: Prototype overview — the website and three apps sharing one backend.

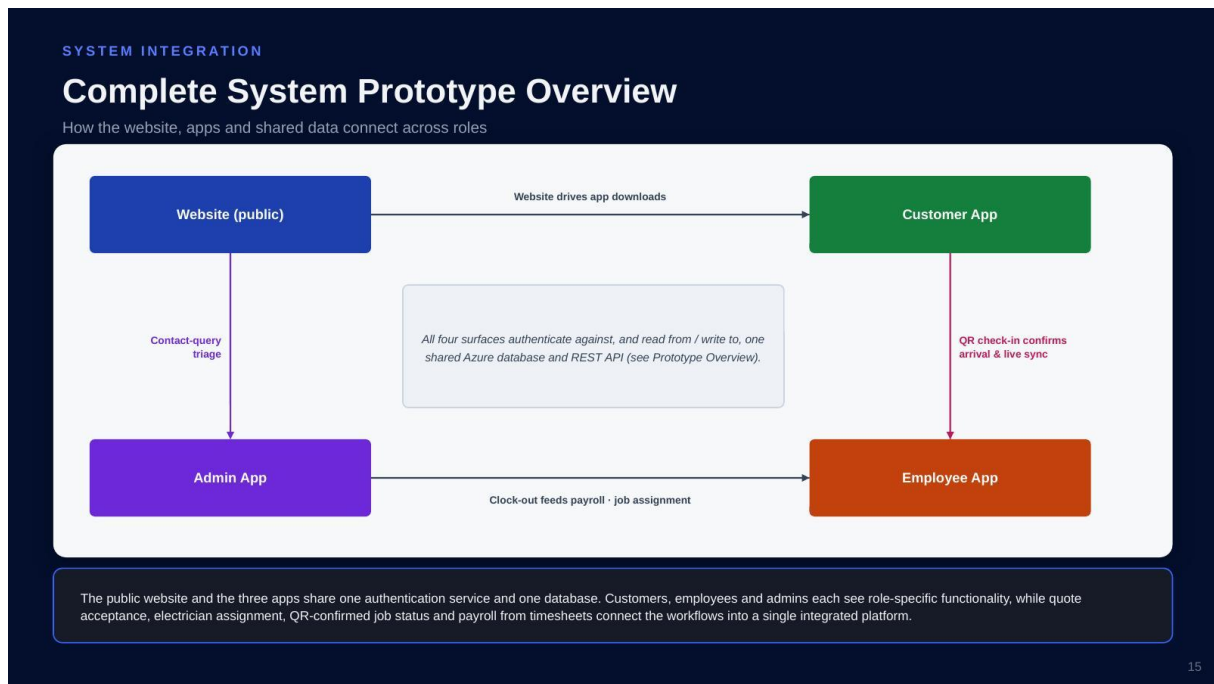


Figure 36: System integration — how actions on one surface propagate to the others.

## E.2 Website Prototype

The public website prototype was hand-coded as a standalone, static HTML file (PSG\_Electrical\_Website\_Prototype.html, included as a supplementary file alongside this document) rather than built from screenshots, so that its navigation, hover states and form interactions behave exactly as they would in the finished site. It requires no login, matching the "public, no login" access level confirmed for this surface in the Access Control ERD (Section 4.4) and Table 7 (RBAC permission matrix). The file can be opened directly in any browser.

**Website Prototype link:** [PSG Electrical and Cables — Website Prototype / file:///C:/Users/trite/AppData/Local/Temp/92259ca3-fcfd-4fbc-9e30-c8d6b3726877 Hydra To Send.zip/Hydra To Send/PSG Electrical Website Prototype%20\(1\).html](file:///C:/Users/trite/AppData/Local/Temp/92259ca3-fcfd-4fbc-9e30-c8d6b3726877%20Hydra%20To%20Send.zip/Hydra%20To%20Send/PSG%20Electrical%20Website%20Prototype%20(1).html)

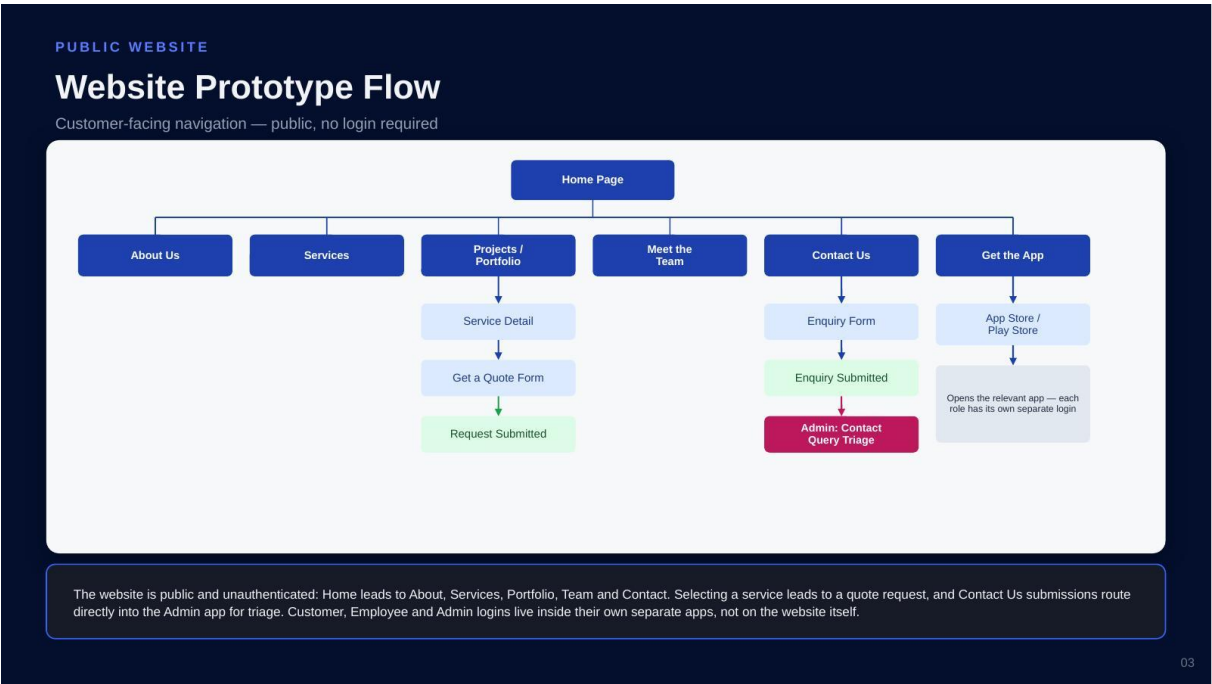


Figure 37: Website prototype navigation flow — Home to About, Services, Portfolio, Team and Contact.

Table 15: Website prototype — section coverage

| Section                | Covers                                                                                                                       |
|------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Home                   | Hero banner, trust badges, quick links to Services and Get a Quote                                                           |
| Our Services           | Service catalogue — Electrical Installations, Cabling Solutions, Maintenance & Repairs, Commercial & Industrial, Tribe Solar |
| Recent Projects        | Portfolio grid filterable by sector (Commercial / Industrial / Solar), with individual project case studies                  |
| Why Choose PSG         | Certifications, accreditations, values and track-record statistics                                                           |
| Meet the Team          | Staff directory with roles, experience and specialities                                                                      |
| Contact Us / Quotation | Contact form, regional offices, and a 4-step Quotation Request form                                                          |

The set of figures below shows each website section’s design evolution from a greyscale low-fidelity wireframe (left) through to the fully styled high-fidelity design (right), followed by a screen capture of the corresponding section as actually implemented in the working HTML prototype.

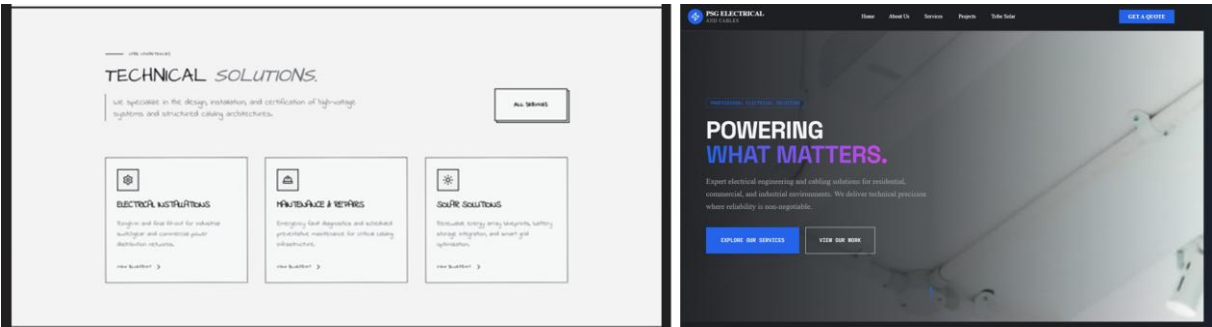


Figure 38: Website — Home — low-fidelity (left) and high-fidelity (right) wireframes.



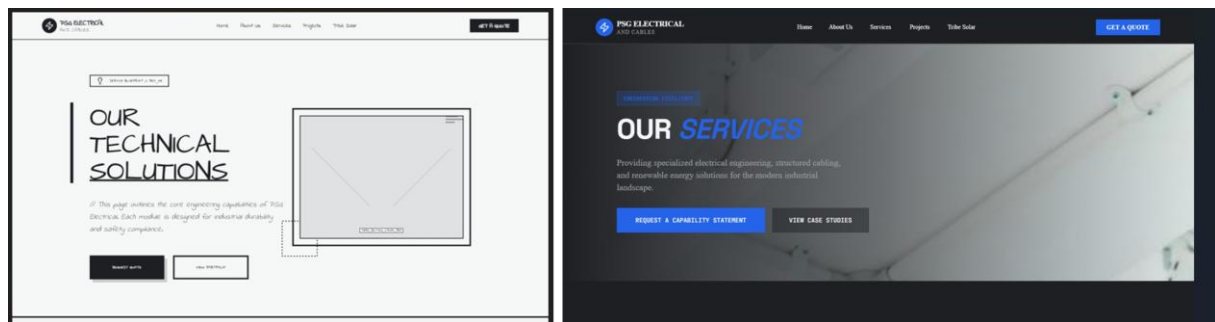


Figure 39: Website — Our Services — low-fidelity (left) and high-fidelity (right) wireframes.

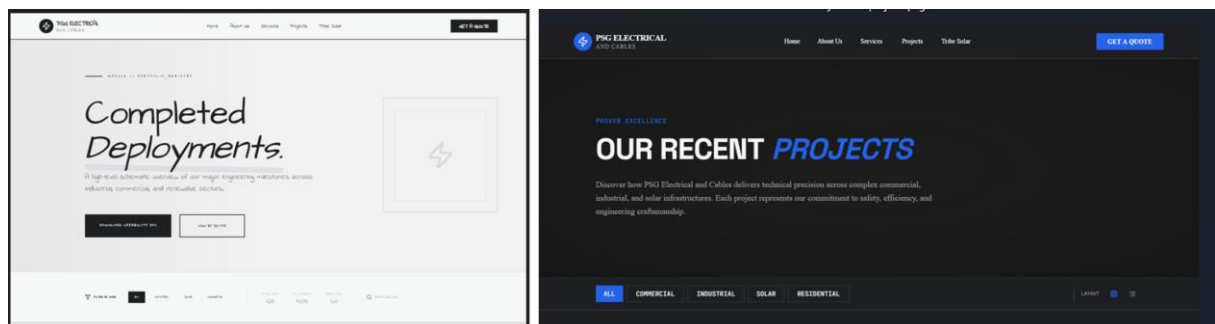


Figure 40: Website — Recent Projects — low-fidelity (left) and high-fidelity (right) wireframes.

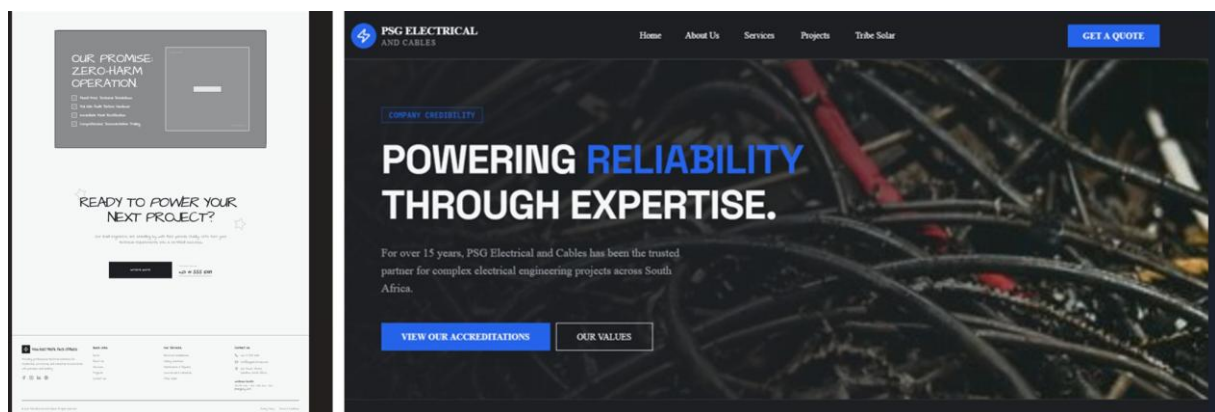


Figure 41: Website — Why Choose PSG — low-fidelity (left) and high-fidelity (right) wireframes.

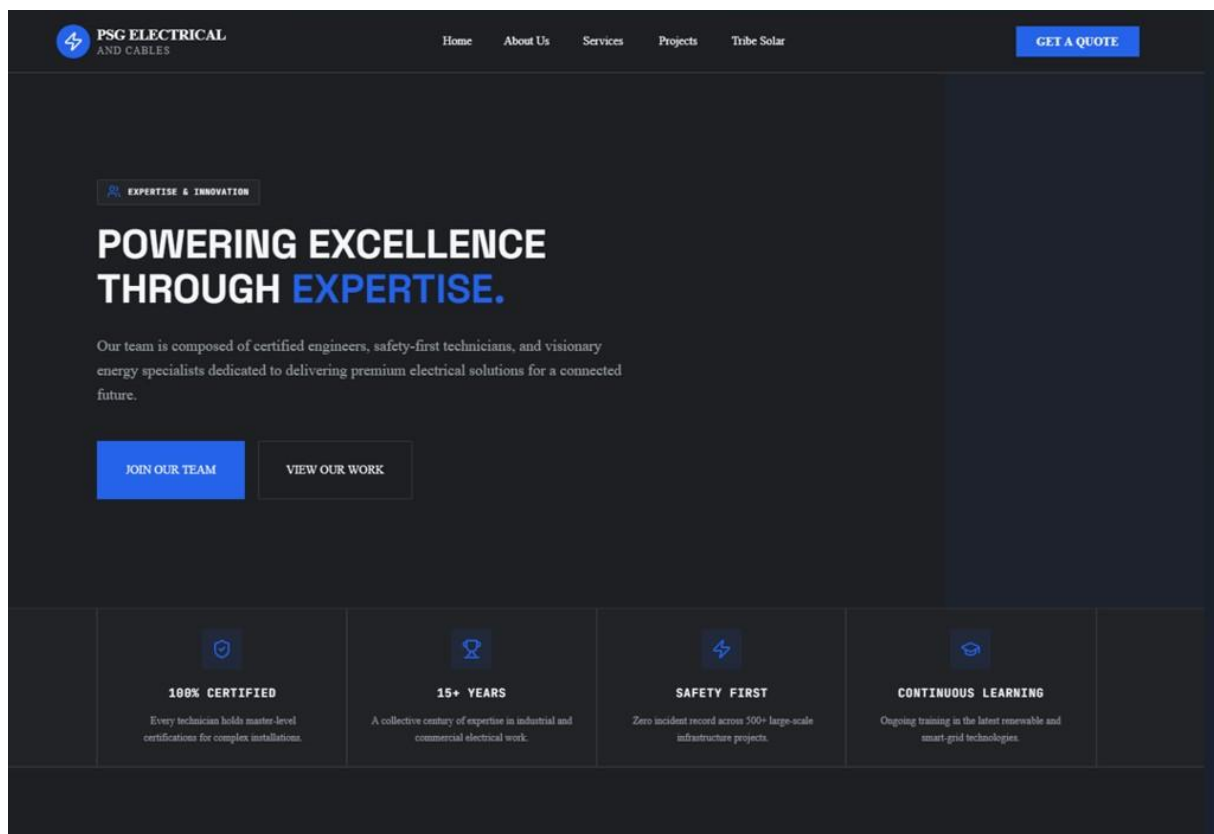


Figure 42: Website — Meet the Team — low-fidelity (left) and high-fidelity (right) wireframes.

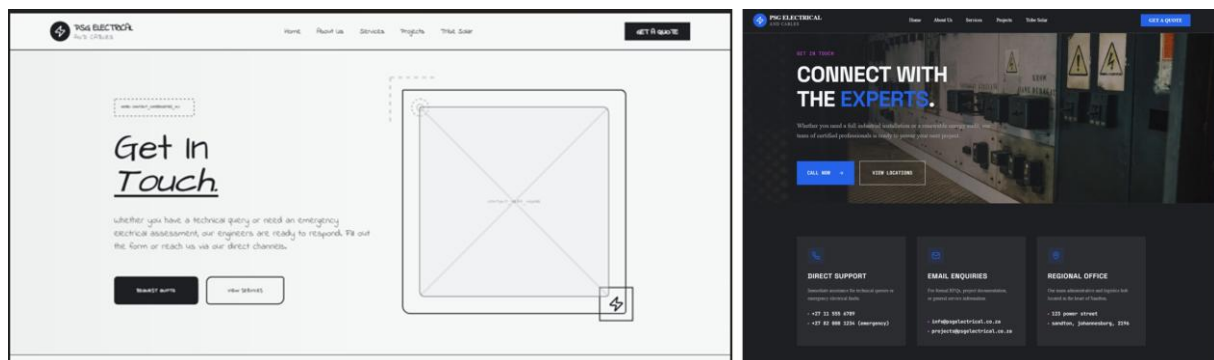


Figure 43: Website — Contact Us — low-fidelity (left) and high-fidelity (right) wireframes.

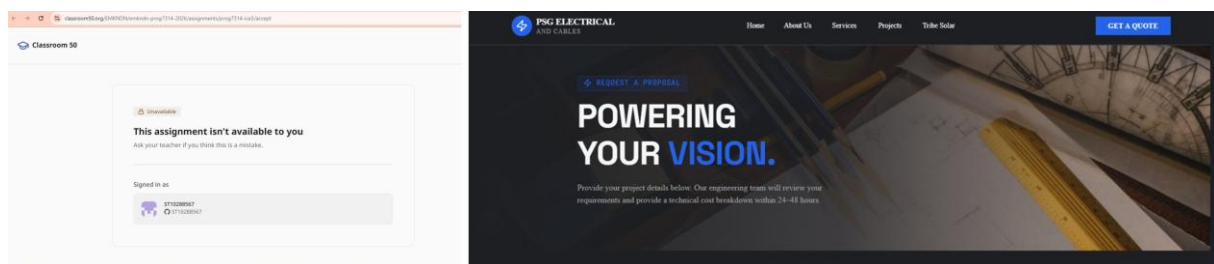


Figure 44: Website — Quotation — low-fidelity (left) and high-fidelity (right) wireframes.

### E.2.1 As-Built Website Prototype Screens

The screens below are direct captures of the working HTML prototype, confirming the high-fidelity designs above were carried through into functioning, clickable code.

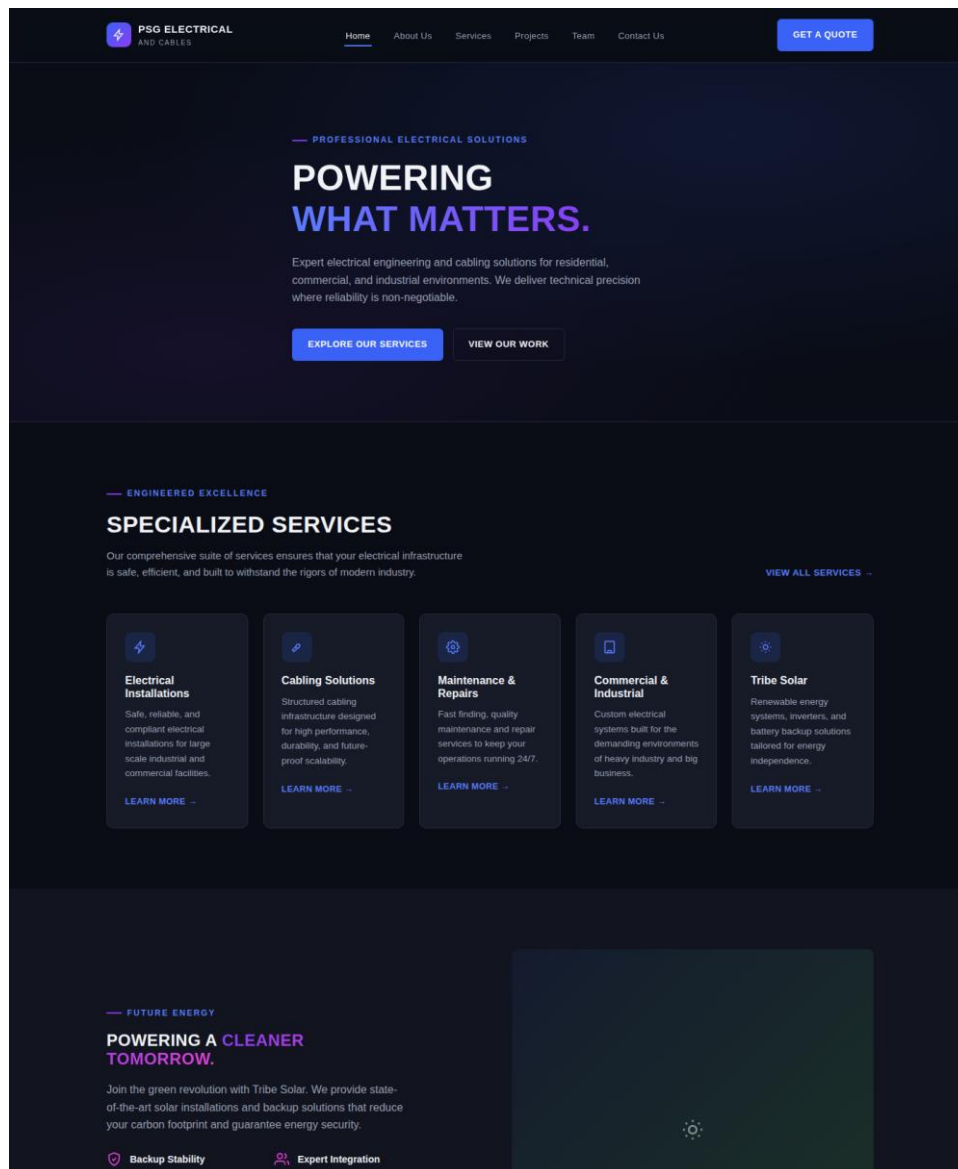


Figure 45: Website prototype (as-built) — Home.

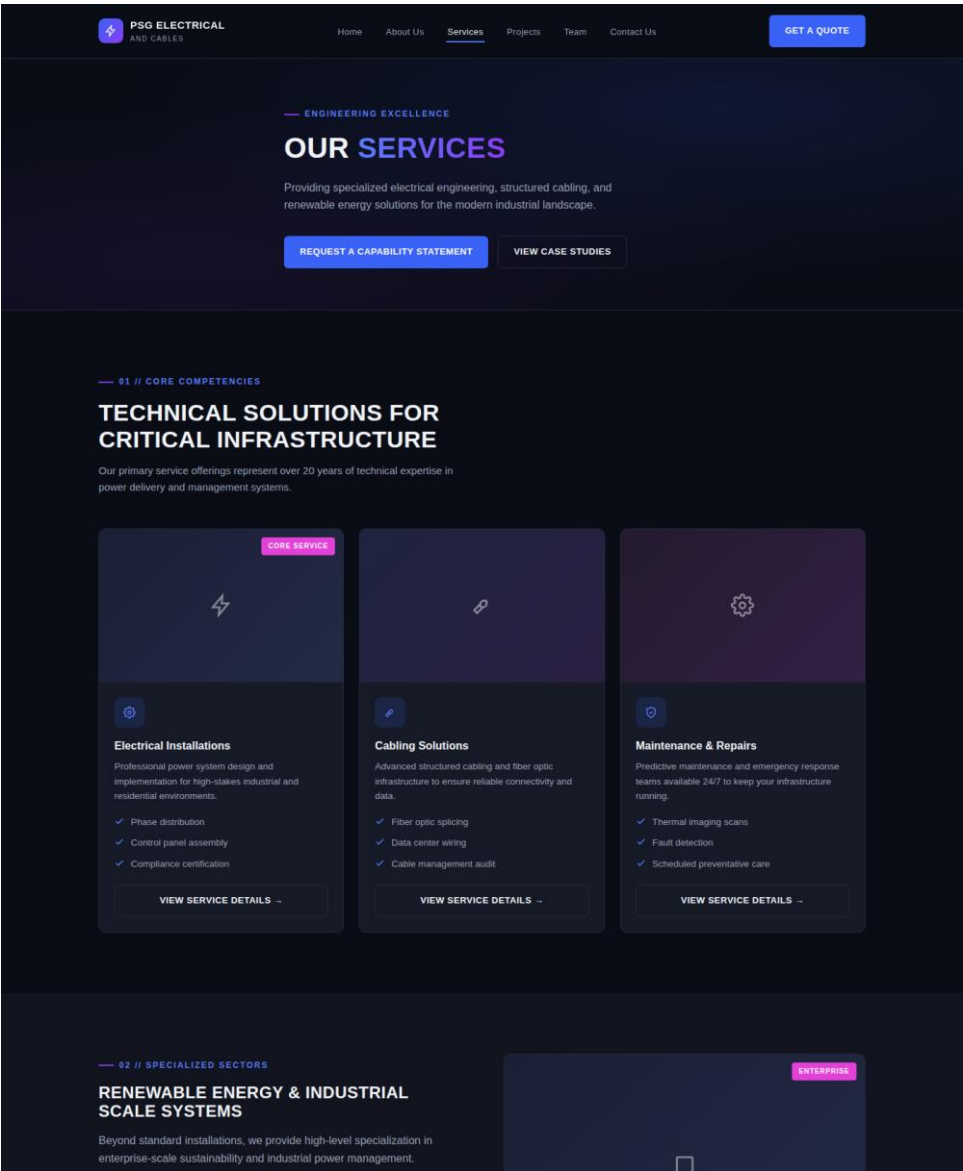


Figure 46: Website prototype (as-built) — Our Services.

**PSG ELECTRICAL**  
AND CABLES

[Home](#) [About Us](#) [Services](#) [Projects](#) [Team](#) [Contact Us](#)

GET A QUOTE

— GET IN TOUCH

CONNECT WITH  
THE EXPERTS.

Whether you need a full industrial installation or a renewable energy audit, our team of certified professionals is ready to power your next project.

CALL NOW — VIEW LOCATIONS

**DIRECT SUPPORT**  
Immediate assistance for technical queries or emergency electrical faults.  
+27 11 555 6789  
+27 82 000 1234 (emergency)

**EMAIL ENQUIRIES**  
For formal RFQs, project documentation, or general service information.  
[info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)  
[projects@psgelectrical.co.za](mailto:projects@psgelectrical.co.za)

**REGIONAL OFFICE**  
Our main administrative and logistics hub located in the heart of Sandton.  
123 Power Street  
Sandton, Johannesburg, 2196

**Send An Inquiry**  
Complete the form below and one of our senior consultants will respond within 24 business hours.

Full Name

John Doe

Email Address

[john@company.com](mailto:john@company.com)

Project Type / Subject

e.g., Commercial Solar Installation

Message / Brief Requirements

Tell us about your project requirements...

TRANSMIT MESSAGE ►

**STANDARD OPERATING HOURS**  
Monday — Friday: 07:30 - 17:00  
Saturday: 08:00 - 13:00  
Emergency Service: 24/7 Available

**LIVE CHAT SUPPORT**  
Technical consultants available during office hours for instant guidance.  
Launch Chat Console —

**HEALTH & SAFETY**  
All PSG installations comply with SANS 10142 regulations.  
Safety is our technical baseline.

  
Map data interface loading...

**PSG Headquarters**  
Sandton, Johannesburg

Figure 47: Website prototype (as-built) — Quotation Request.

### E.3 Customer App — Live Interactive Prototype

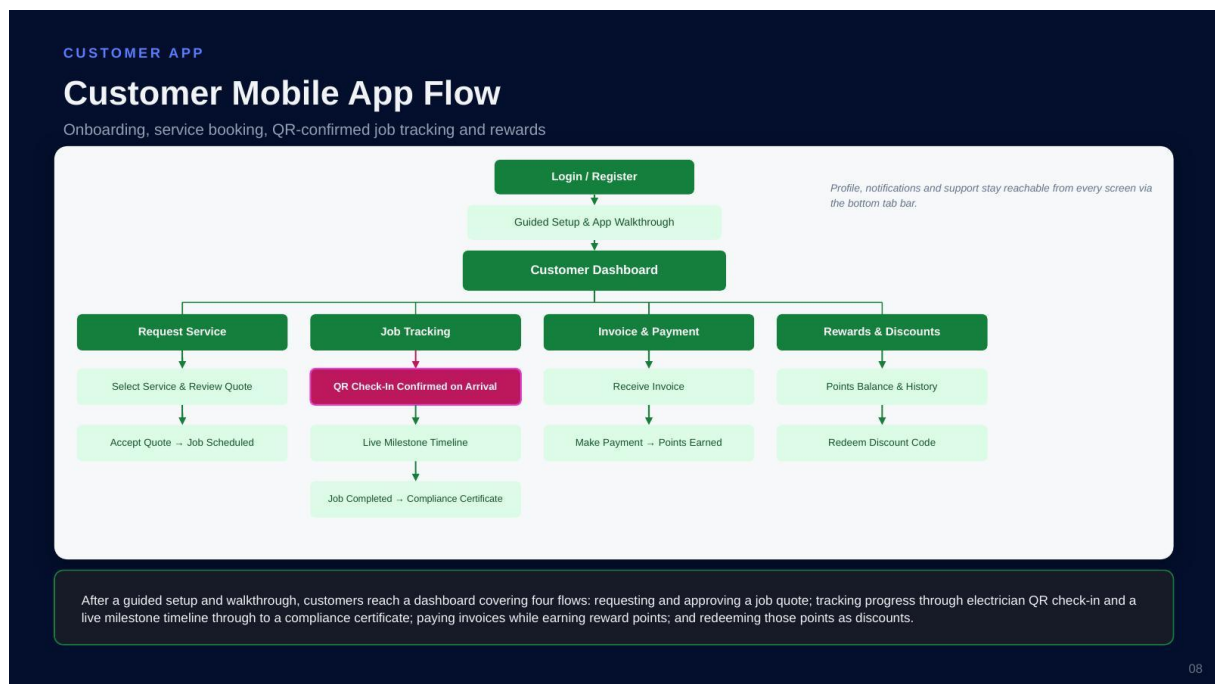
The Customer App was taken furthest of all four surfaces: 43 high-fidelity screens across 8 sections were built and deployed as a live, interactive, phone-frame click-through prototype, so that the full navigation experience — not just static screens — can be reviewed directly in a browser on any device.

**Live prototype:** <https://psg-electrical-app-prototype-yashgov13-3579s-projects.vercel.app/>

Open the link on any device to try it directly: use the left-hand tabs to jump between the 8 sections below, and the arrows or dot indicators beneath the phone frame to step through each screen within a section.

**Table 16: Customer App live prototype — 43 screens across 8 sections**

| Section       | Screens | Covers                                                                                                                                   |
|---------------|---------|------------------------------------------------------------------------------------------------------------------------------------------|
| Home          | 6       | Hero & quick actions, core services shortcuts, regulatory compliance, performance track record, featured services, certifications banner |
| Our Services  | 6       | Services overview, featured service detail, service cards, cross-sell CTA                                                                |
| Quotation     | 3       | Step 1 (sector & type), detailed quote CTA, process FAQ                                                                                  |
| Recent Work   | 7       | Portfolio overview, why we stand out, 4 project case studies, client testimonial                                                         |
| Why Choose Us | 5       | Hero standard, official accreditations, compliance FAQ/CTA, certifications list                                                          |
| Our Partners  | 6       | Tier-1 partnerships, partner directory, featured partners, spares dispatch, component quotation CTA                                      |
| Meet the Team | 6       | Team overview, 4 engineer profiles, request-a-specialist CTA                                                                             |
| Contact Us    | 4       | Head office & quick contact, regional offices, send an enquiry, safety & compliance                                                      |



**Figure 48: Customer App navigation flow — Dashboard to Request Service, Job Tracking, Invoice & Payment, and Rewards & Discounts.**

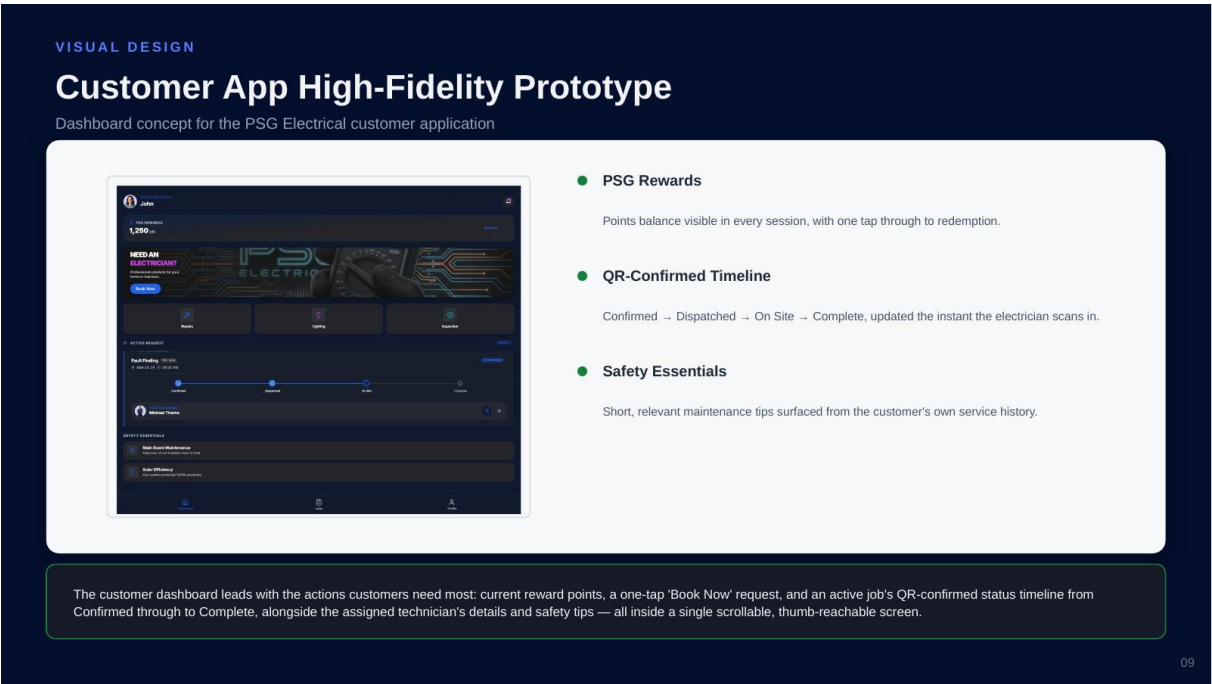


Figure 49: Customer App dashboard concept — rewards balance, QR-confirmed job timeline, and safety tips.

As with the website, the figures below show each section's low-fidelity wireframe (left) alongside its high-fidelity counterpart (right); the full 43-screen set is available to review directly in the live prototype linked above.

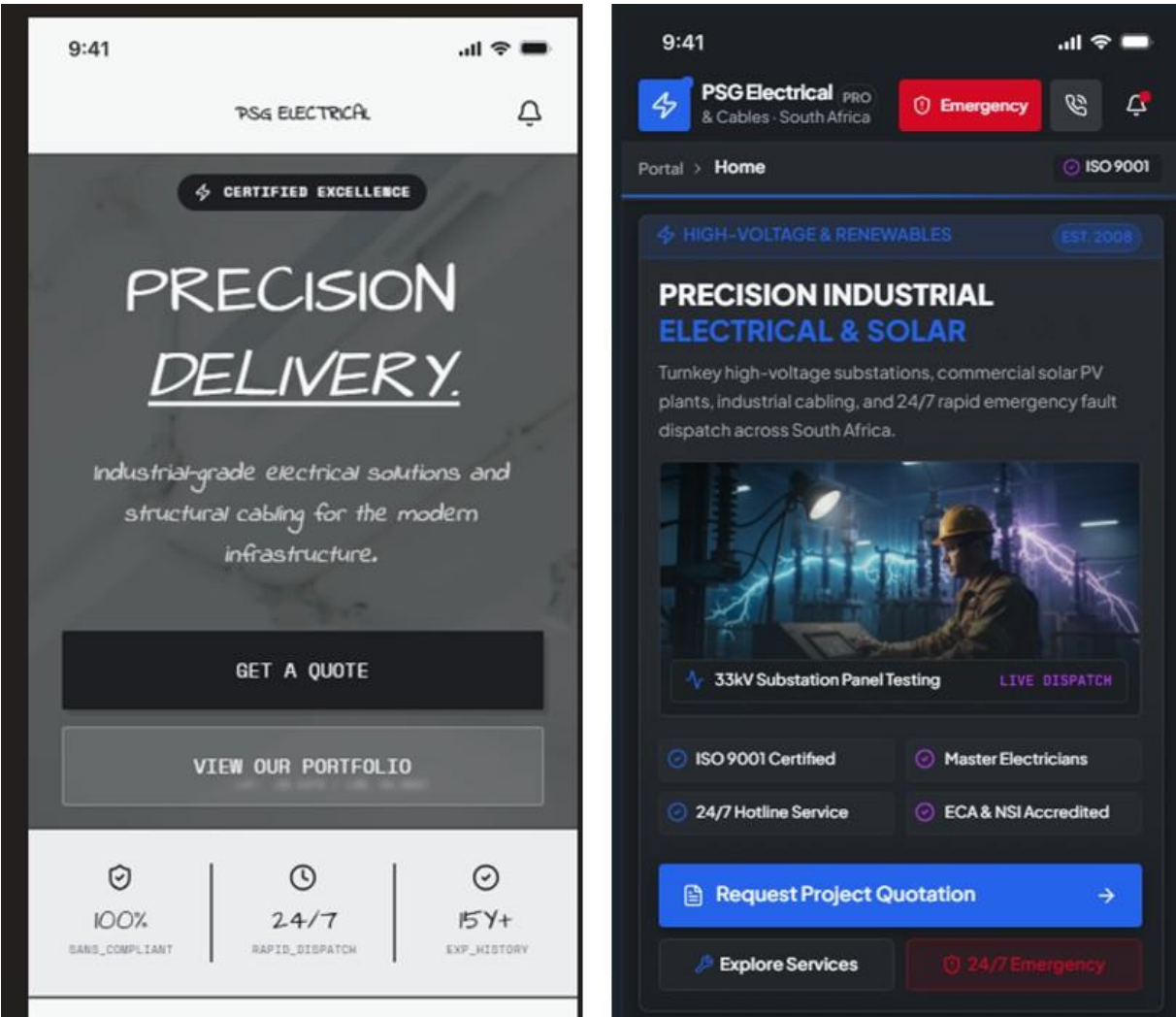




Figure 50: Customer App — Home — low-fidelity (left) and high-fidelity (right) wireframes.

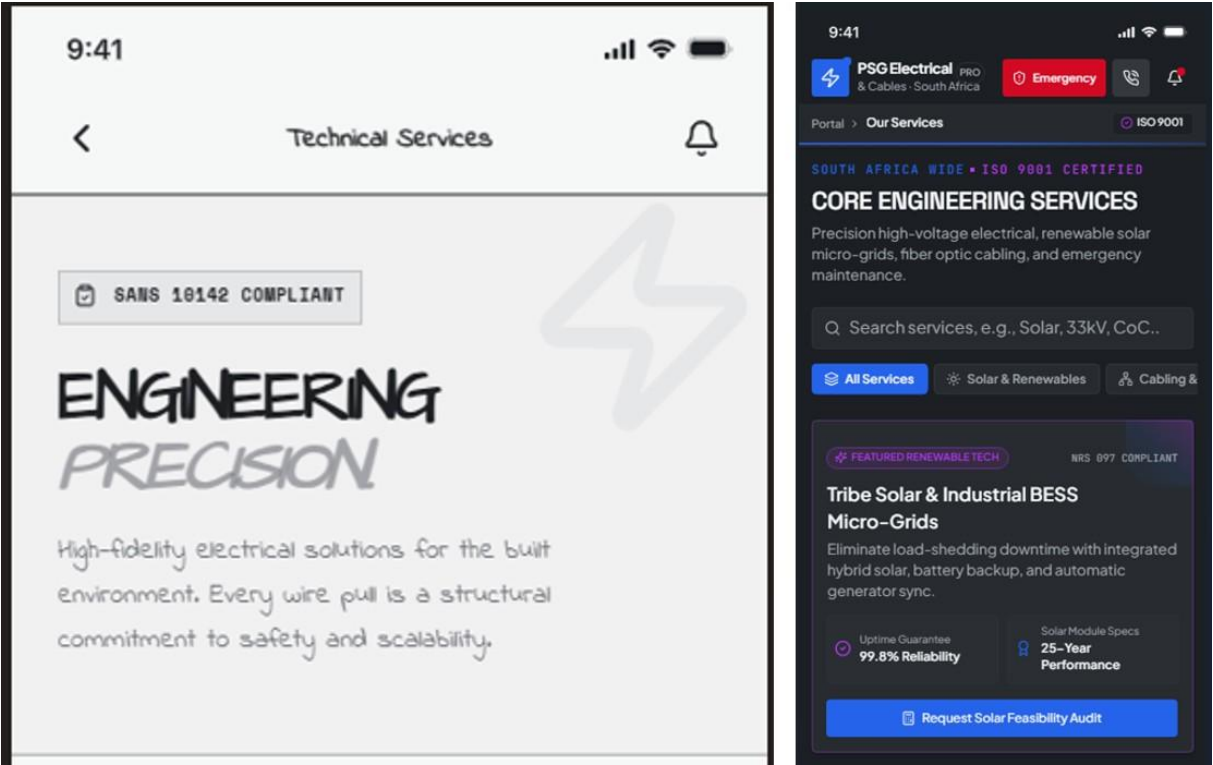


Figure 51: Customer App — Our Services — low-fidelity (left) and high-fidelity (right) wireframes.

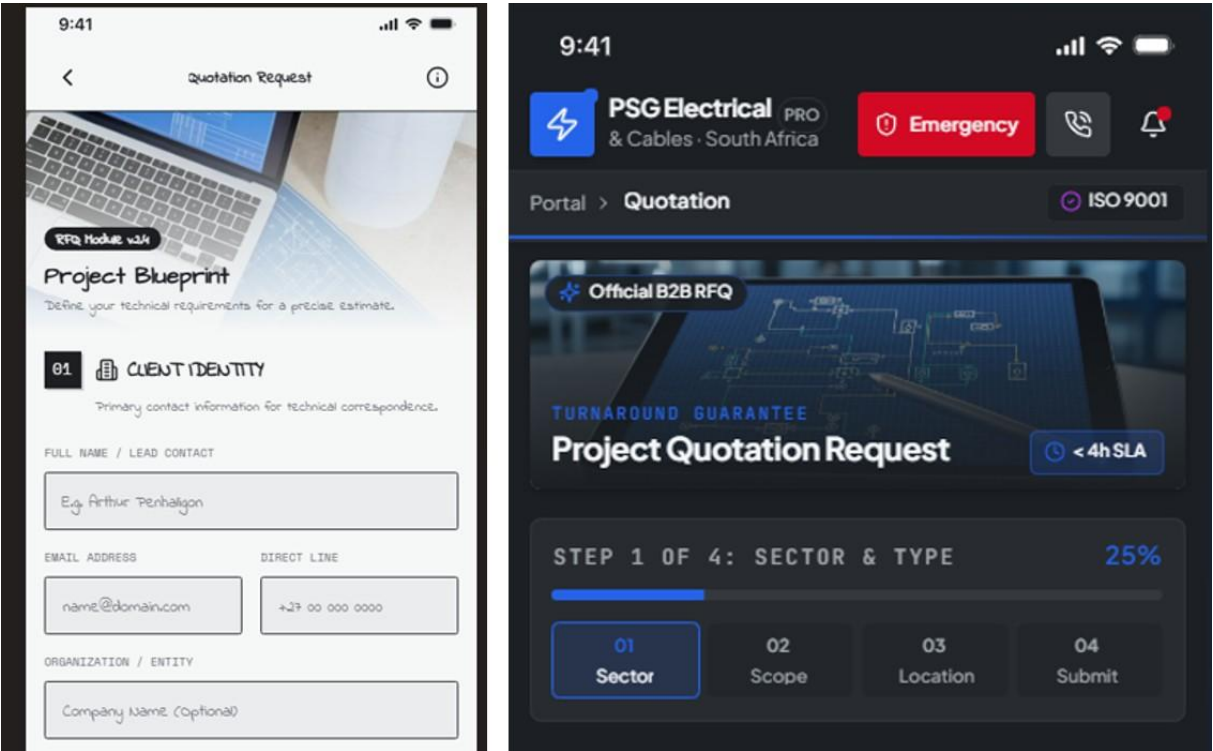


Figure 52: Customer App — Quotation — low-fidelity (left) and high-fidelity (right) wireframes.

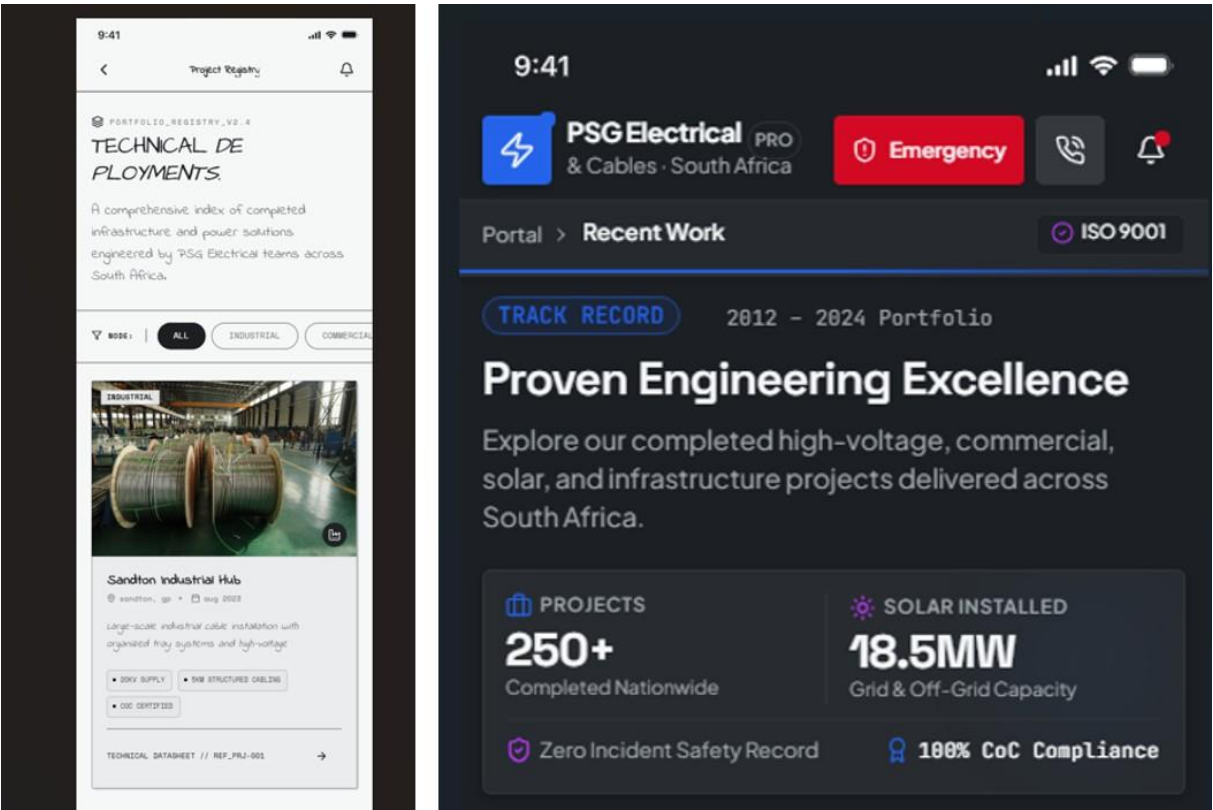


Figure 53: Customer App — Recent Work — low-fidelity (left) and high-fidelity (right) wireframes.

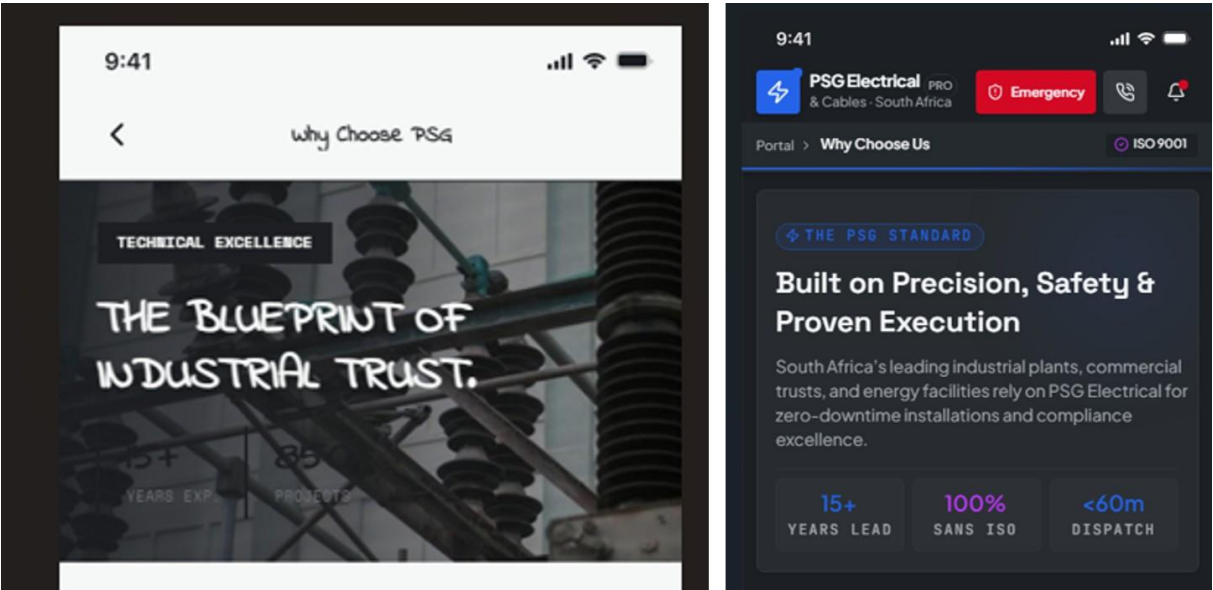


Figure 54: Customer App — Why Choose Us — low-fidelity (left) and high-fidelity (right) wireframes.

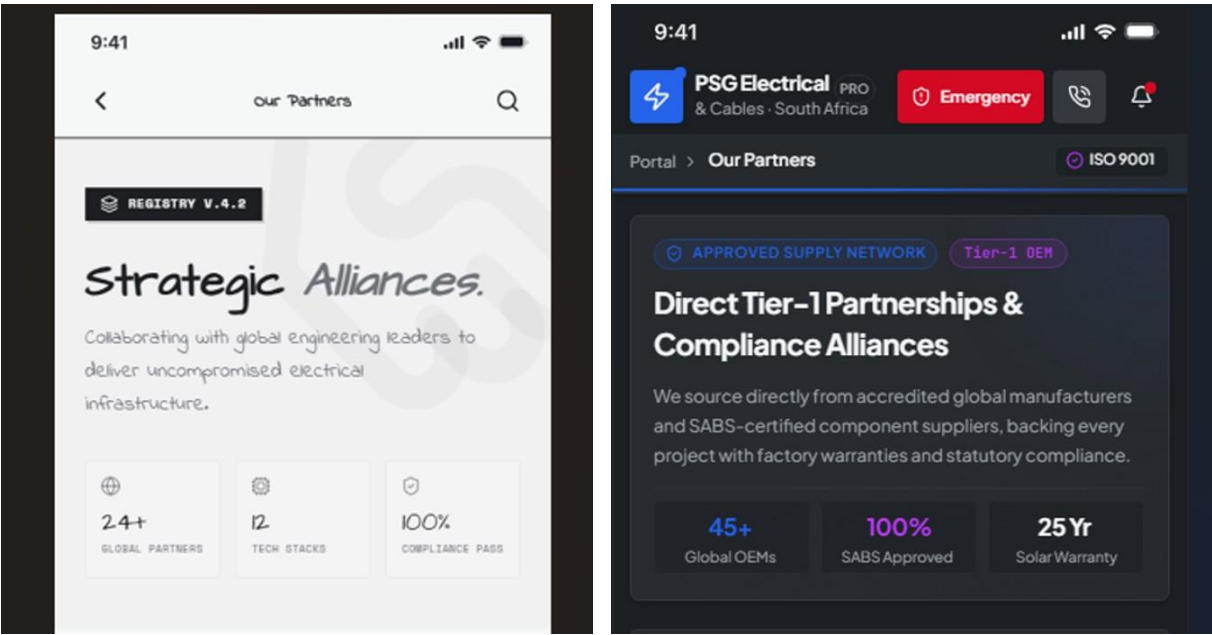


Figure 55: Customer App — Our Partners — low-fidelity (left) and high-fidelity (right) wireframes.

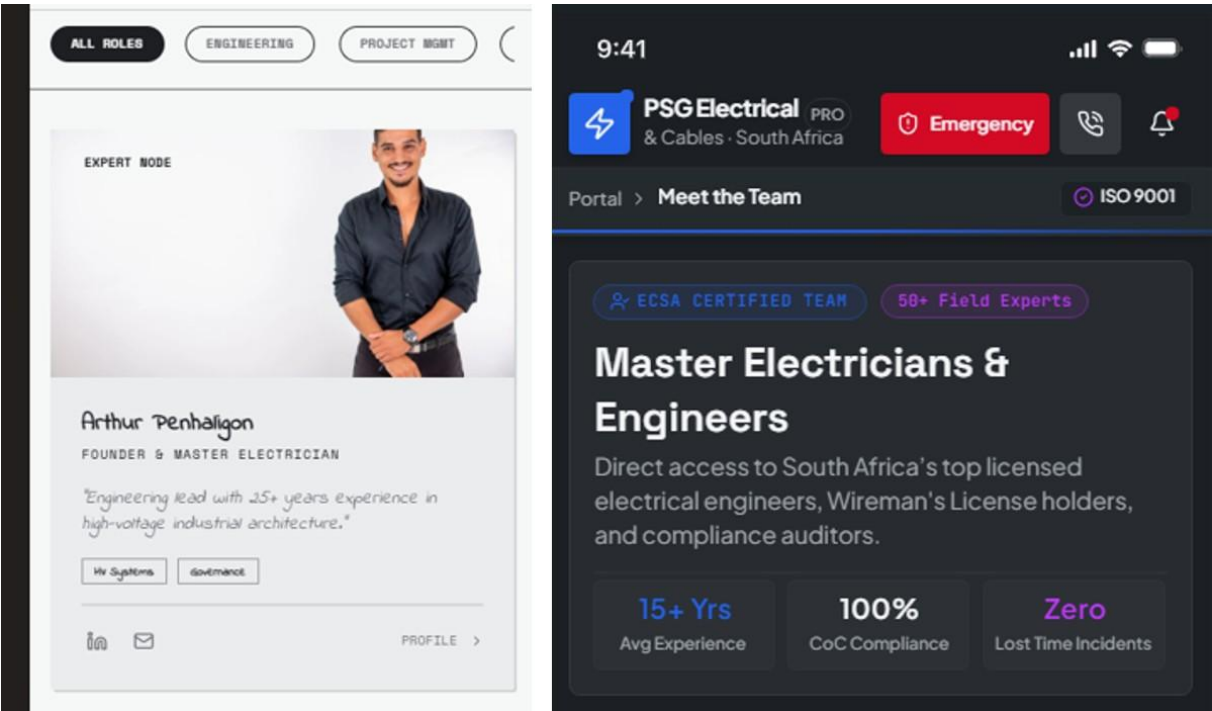


Figure 56: Customer App — Meet the Team — low-fidelity (left) and high-fidelity (right) wireframes.

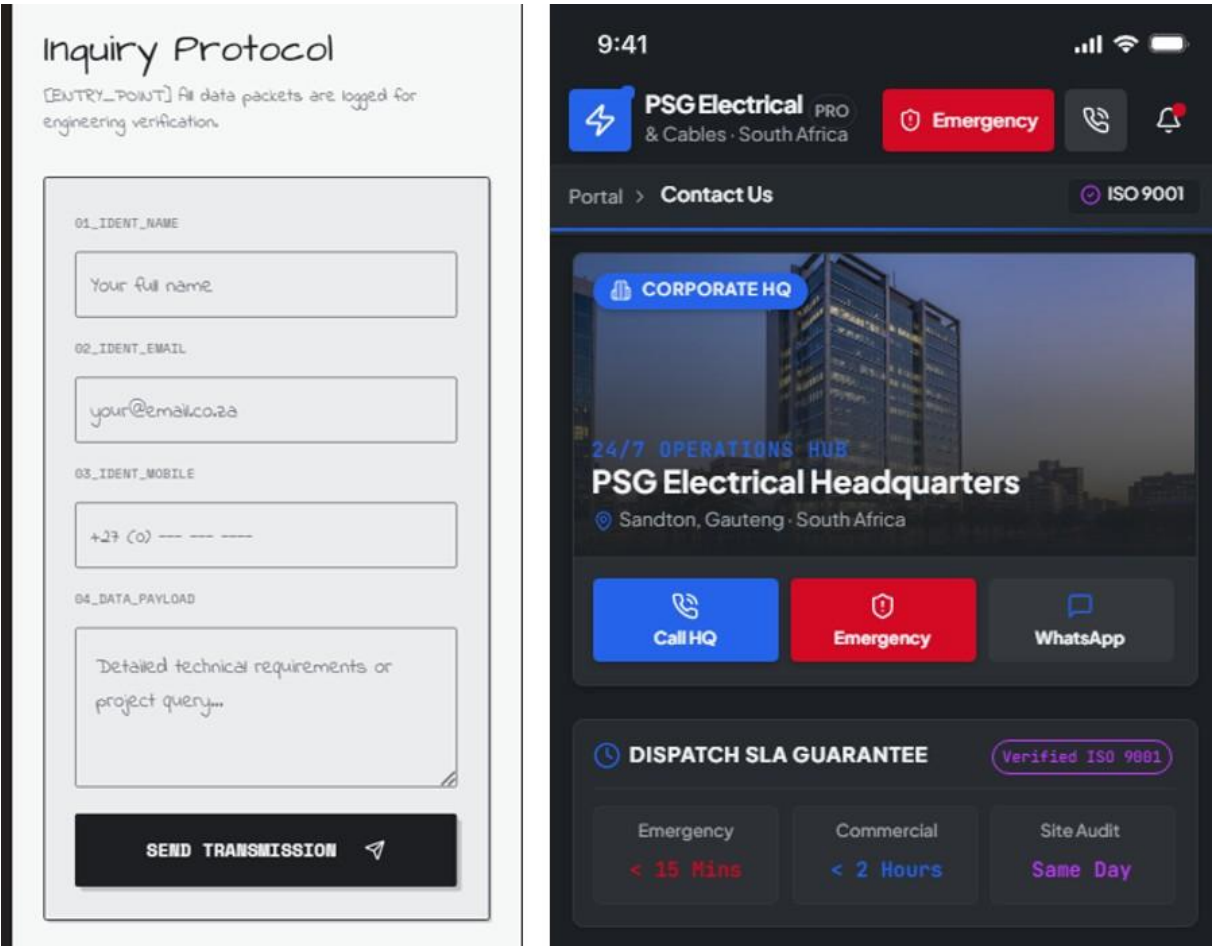


Figure 57: Customer App — Contact Us — low-fidelity (left) and high-fidelity (right) wireframes.

## E.4 Employee App — Flow and High-Fidelity Mockup

The Employee App does not yet have a deployed live prototype; it is documented as a full navigation flow together with a high-fidelity dashboard mockup, both produced to the same visual design language as the Customer App. After login, an electrician sees their assigned jobs and calendar, clocks in, and scans the customer's QR code on arrival to confirm the visit and start the live milestone timeline described in Section 2.2 and Figure 4 (Job Lifecycle State Diagram) — job notes, materials and an inspection report are captured before clocking out, while leave requests route to Admin for approval.

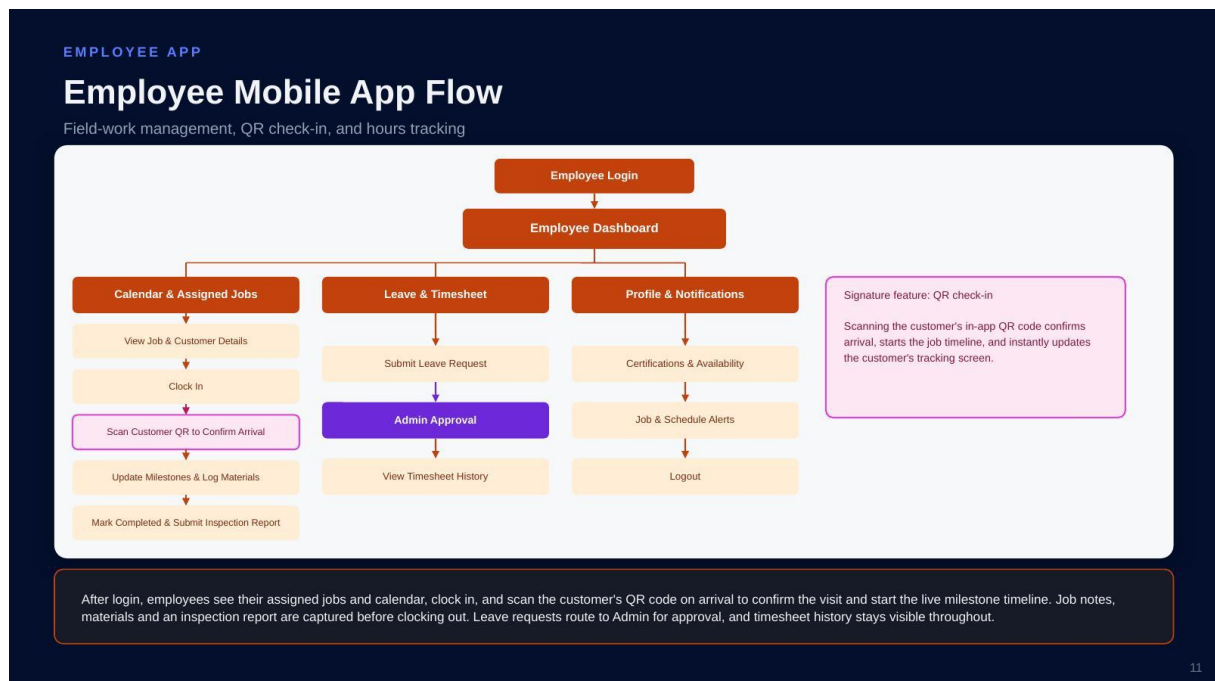


Figure 58: Employee App navigation flow — Calendar & Assigned Jobs, Leave & Timesheet, Profile & Notifications.

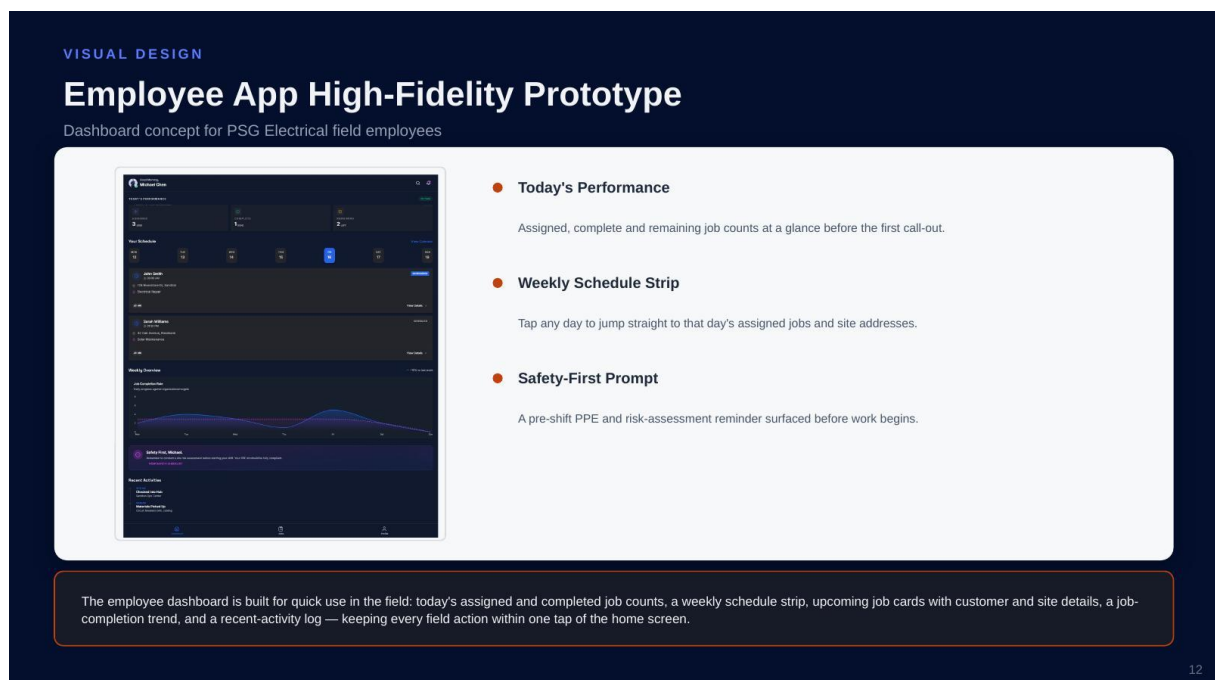


Figure 59: Employee App dashboard concept.

## E.5 Admin App — Flow and High-Fidelity Mockup

The Admin App is likewise documented as a full navigation flow with a high-fidelity dashboard mockup. Admin controls the whole system from one dashboard: triaging website and missed-call enquiries into quotes



and job assignments, monitoring job progress through to invoicing and stock, approving leave and processing payroll from submitted timesheets, and exporting data for reporting — the same operations described in Section 2.1.2 (User Stories, Admin Operations module) and Section 6 (RBAC, Table 7).

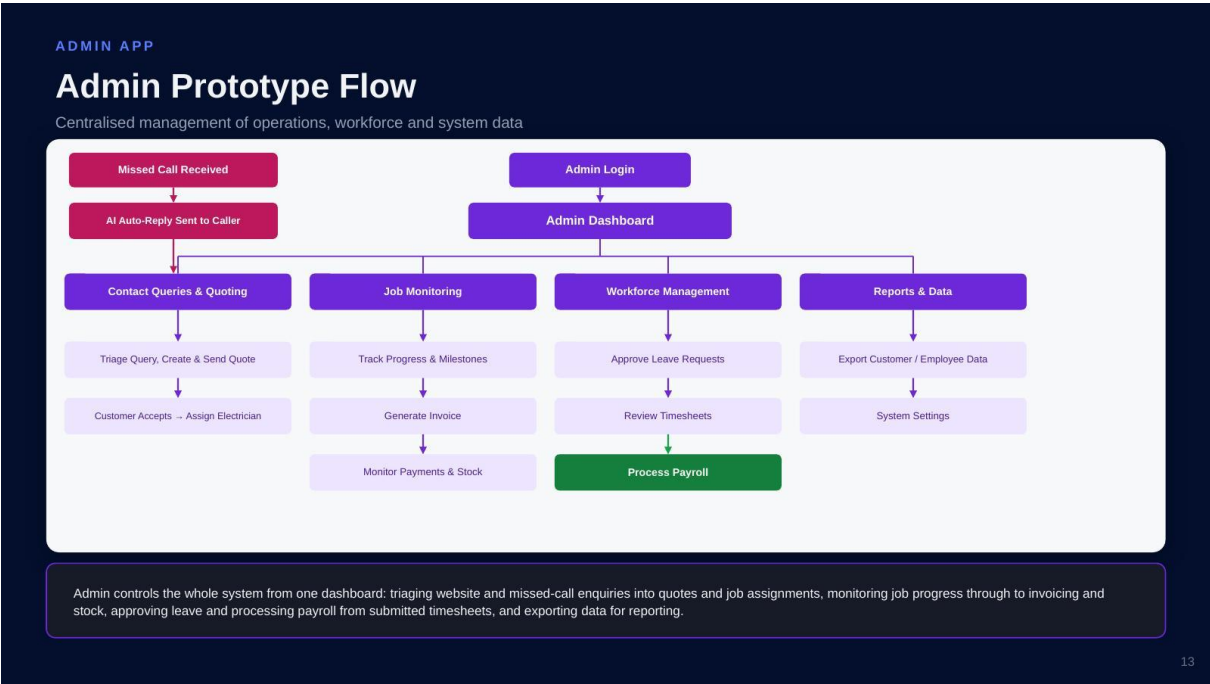


Figure 60: Admin App navigation flow — Contact Queries & Quoting, Job Monitoring, Workforce Management, Reports & Data.

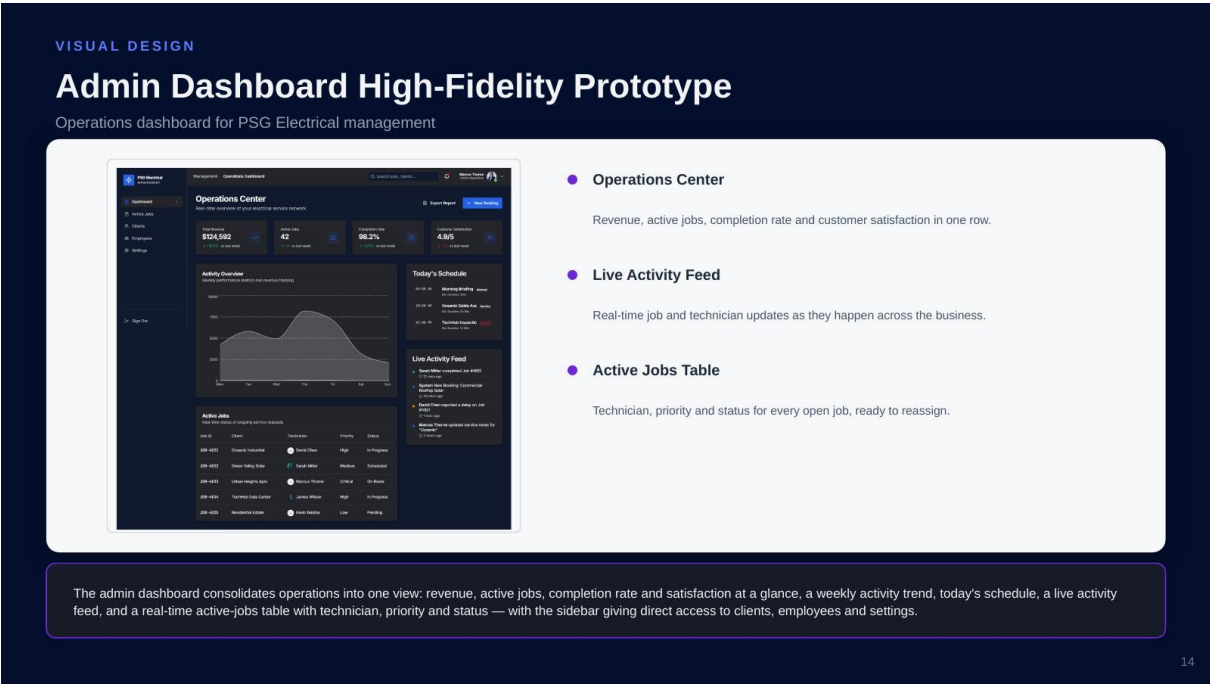
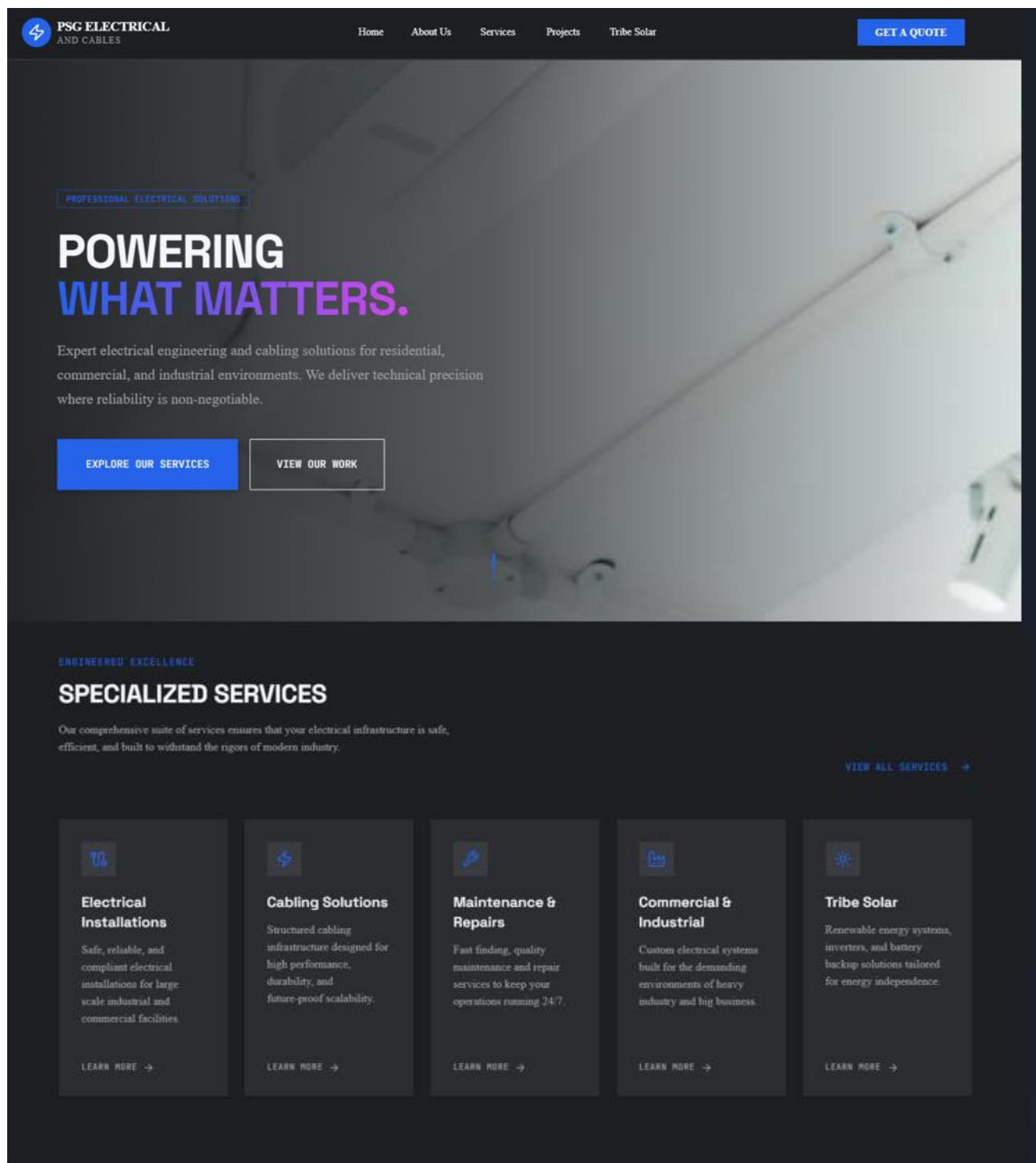


Figure 61: Admin App dashboard concept — Operations Center, live activity feed, active jobs table.

Open items: the Employee and Admin apps do not yet have deployed live click-through prototypes the way the Customer App does — only the flow diagrams and single dashboard mockups shown above. The same phone-frame prototype pattern used for the Customer App could be reused for both if a full wireframe set is produced for them in a later iteration.

High Fidelity Website pages:

Homepages:





FUTURE ENERGY

## POWERING A CLEANER TOMORROW.

Join the green revolution with Tribe Solar. We provide state-of-the-art solar installations and backup solutions that reduce your carbon footprint and guarantee energy security.

**Backup Stability**  
Load-shedding proof systems for home and work.

**Expert Integration**  
Seamless hybrid systems for existing infrastructure.

EXPLORE TRIBE SOLAR



THE PSG ADVANTAGE

## WHY INDUSTRY LEADERS TRUST US

**Qualified Electricians**  
Expertly trained & fully certified teams.

**Quality Workmanship**  
Visible, durable, and testing-verified results.

**On Time, Every Time**  
Strict adherence to project deadlines.

**Safety First**  
Rigorous OSHA compliant practices.

**Customer Focused**  
Your requirements are our priority.

OUR PORTFOLIO

RECENT PROJECTS

VIEW ALL PROJECTS



READY TO POWER UP YOUR NEXT PROJECT?

GET A FREE QUOTE

CONTACT SUPPORT



PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



Quick Links

- Home
- About Us
- Services
- Projects
- Contact Us

Our Services

- Electrical Installations
- Cabling Solutions
- Maintenance & Repairs
- Commercial & Industrial
- Tribe Solar

Contact Us

- +27 11 555 6789
- info@psgelectrical.co.za
- 123 Power Street, Sandton, South Africa

WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00  
Emergency: 24/7



PSG ELECTRICAL  
AND CABLES

[Home](#)[About Us](#)[Services](#)[Projects](#)[Trite Solar](#)

GET A QUOTE

EXPERTISE & INNOVATION

POWERING EXCELLENCE  
THROUGH EXPERTISE.

Our team is composed of certified engineers, safety-first technicians, and visionary energy specialists dedicated to delivering premium electrical solutions for a connected future.

JOIN OUR TEAMVIEW OUR WORK



100% CERTIFIED

Every technician holds master-level certifications for complex installations.



15+ YEARS

A collective century of expertise in industrial and commercial electrical work.



SAFETY FIRST

Zero incident record across 500+ large-scale infrastructure projects.



CONTINUOUS LEARNING

Ongoing training in the latest renewable and smart-grid technologies.

Page 205 of 235

# MEET THE EXPERTS

The faces behind PSG's reputation for precision. Our leadership and technical teams combine decades of field experience with innovative engineering approaches.

FILTER BY:

All Departments



**Marcus Chen**

CHIEF TECHNICAL OFFICER

With over 15 years in industrial electrical engineering, Marcus leads our technical strategy and complex infrastructure projects.

INDUSTRIAL SYSTEMSINFRASTRUCTURE





**Sarah Williams**

SENIOR ELECTRICAL ENGINEER

Sarah specializes in renewable energy integration and smart grid solutions for high-end residential development.

SOLAR PVSMART HOMES





**David Nkosi**

OPERATIONS MANAGER

David ensures every project meets our rigorous safety standards and delivery timelines across all commercial sites.

SITE MANAGEMENTSAFETY COMPLIANCE





**James Peterson**

LEAD PROJECT COORDINATOR

James bridges the gap between client requirements and technical execution, managing large-scale commercial deployments.

CLIENT RELATIONSRISK MANAGEMENT





**Elena Rodriguez**

COMPLIANCE & SAFETY OFFICER

Elena maintains our 100% safety record through meticulous inspections and staff certification programs.

ISO COMPLIANCEOSHA STANDARDS





**Liam Smith**

TECHNICAL APPRENTICE

A rising star in our technical team, Liam focuses on precision cabling and modern automation systems.

DATA CABLINGAUTOMATION



Page 206 of 235

# WANT TO BE PART OF THE POWER?

We are always looking for certified master electricians, engineers, and apprentices who share our commitment to precision and technical excellence.

EXPLORE CAREERS



PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



Quick Links

Home

About Us

Services

Projects

Contact Us

Our Services

Electrical Installations

Cabling Solutions

Maintenance & Repairs

Commercial & Industrial

Tribe Solar

Contact Us

 +27 11 555 6789

 [info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)

 123 Power Street,  
Sandton, South Africa

WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00

[Emergency: 24/7](#)

© 2024 PSG Electrical and Cables. All rights reserved.

Privacy Policy

Terms & Conditions

Page 207 of 235

**PSG ELECTRICAL  
AND CABLES**

HomeAbout UsServicesProjectsTrite Solar

GET A QUOTE

ENGINEERING EXCELLENCE

# OUR SERVICES

Providing specialized electrical engineering, structured cabling, and renewable energy solutions for the modern industrial landscape.

REQUEST A CAPABILITY STATEMENTVIEW CASE STUDIES

01 // CORE COMPETENCIES

## TECHNICAL SOLUTIONS FOR CRITICAL INFRASTRUCTURE

Our primary service offerings represent over 20 years of technical expertise in power delivery and management systems.



CORE SERVICE



### Electrical Installations

Professional power system design and implementation for high-stakes industrial and residential environments

- Phase distribution
- Control panel assembly
- Compliance certification

VIEW SERVICE DETAILS →





### Cabling Solutions

Advanced structured cabling and fiber optic infrastructure to ensure reliable connectivity and data

- Fiber optic splicing
- Data center wiring
- Cable management audit

VIEW SERVICE DETAILS →





### Maintenance & Repairs

Predictive maintenance and emergency response teams available 24/7 to keep your infrastructure

- Thermal imaging scans
- Fault detection
- Scheduled preventative care

VIEW SERVICE DETAILS →

02 // SPECIALIZED SECTIONS

# RENEWABLE ENERGY & INDUSTRIAL SCALE SYSTEMS

Beyond standard installations, we provide high-level specialization in enterprise-scale sustainability and industrial power management.



### High-Voltage Substation Design

Design, installation, and ongoing technical maintenance for critical sub-distribution centers.



### Solar PV Integration

Industrial-scale solar arrays tailored for significant energy cost reduction and grid independence.

DOWNLOAD SERVICES BROCHURE



ENTERPRISE



### Commercial & Industrial

Heavy-duty electrical solutions for factories, warehouses, and corporate headquarters with complex power needs.

- Substation maintenance
- UPS systems
- Lighting automation

VIEW SERVICE DETAILS →



### Tribe Solar Solutions

Leading-edge renewable energy systems designed to reduce dependency on the grid and lower carbon footprints.

- Photovoltaic arrays
- Battery storage systems
- Energy efficiency audits

VIEW SERVICE DETAILS →



ENTERPRISE

Page 209 of 235



03 // OUR STANDARDS

PRECISION. SAFETY. RELIABILITY.



Certified Safety

Full compliance with ISO standards and local regulatory electrical safety protocols.



24/7 Response

Rapid deployment teams for emergency repairs and critical system failure recovery.



Energy Efficiency

Optimizing existing systems for maximum power performance and reduced operational cost.



Qualified Experts

Every technician is fully vetted, licensed, and undergoes continuous technical training.

READY TO POWER YOUR NEXT PROJECT?

Connect with our lead engineers today to discuss your project requirements or request a custom quotation for specialized installations.

GET A QUOTE NOW

CONTACT SUPPORT



PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



Quick Links

[Home](#)[About Us](#)[Services](#)[Projects](#)[Contact Us](#)

Our Services

[Electrical Installations](#)[Cabling Solutions](#)[Maintenance & Repairs](#)[Commercial & Industrial](#)[Trite Solar](#)

Contact Us

 +27 11 555 6789 [info@psgelectrical.co.za](mailto:info@psgelectrical.co.za) 123 Power Street, Sandton, South Africa

WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00  
Emergency: 24/7

© 2024 PSG Electrical and Cables. All rights reserved.

[Privacy Policy](#) [Terms & Conditions](#)

Page 210 of 235

 **PSG ELECTRICAL**  
AND CABLES

HomeAbout UsServicesProjectsTrite Solar

GET A QUOTE

REQUEST A PROPOSAL

# POWERING YOUR VISION.

Provide your project details below. Our engineering team will review your requirements and provide a technical cost breakdown within 24–48 hours.

 **Quotation Request Form**

Fields marked with an asterisk (\*) are required for a valid technical assessment.

1 CONTACT INFORMATION

Full Name \*

Email Address \*

Phone Number \*

Company Name (Optional)

2 PROJECT DETAILS

Service Required \*

Select a service

Project Sector \*

Select sector

Project Description \*

Please describe the scope of work, project goals, and any specific technical requirements...

3 ADDITIONAL OPTIONS

☐ In-person site assessment required

☐ Urgent turnaround requested (< 24h)

☐ Provide all materials and consumables

☐ Include backup power options

SUBMIT QUOTATION REQUEST

By submitting this form, you agree to our [Service Terms](#) and [Privacy Policy](#).

OUR CAPABILITY SPECTRUM

 RESIDENTIAL

 COMMERCIAL

 INDUSTRIAL

 TRITE SOLAR

4 THE ESTIMATING PROCESS

01 Technical Intake

Our engineers analyze your specific electrical load and site requirements.

02 Site Assessment

Where necessary, we conduct a physical survey to identify structural constraints.

03 Cost Engineering

A detailed breakdown of materials, labor, and compliance certification.

04 Proposal Delivery

Final documentation delivered via secure portal with digital sign-off options.

PROFESSIONAL ACCREDITATIONS

SABS APPROVED

ECASA MEMBER

PV GREEN CARD

PSG is fully insured and all technicians are licensed by the Department of Labour.

NEED IMMEDIATE ASSISTANCE?

Speak with an Engineer

+27 11 555 4780

Page 211 of 235

## Explore Our Recent Portfolio

View how we have powered large-scale industrial and premium residential projects across South Africa.

[VIEW ALL PROJECTS](#) →





### PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



#### Quick Links

- [Home](#)
- [About Us](#)
- [Services](#)
- [Projects](#)
- [Contact Us](#)

#### Our Services

- [Electrical Installations](#)
- [Cabling Solutions](#)
- [Maintenance & Repairs](#)
- [Commercial & Industrial](#)
- [Trite Solar](#)

#### Contact Us

 +27 11 555 6789

 [info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)

 123 Power Street,  
Sandton, South Africa

#### WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00  
[Emergency: 24/7](#)

© 2024 PSG Electrical and Cables. All rights reserved.

[Privacy Policy](#) [Terms & Conditions](#)

Page 212 of 235



PSG ELECTRICAL  
AND CABLES

[Home](#)[About Us](#)[Services](#)[Projects](#)[Trite Solar](#)

GET A QUOTE

PROVEN EXCELLENCE

OUR RECENT *PROJECTS*

Discover how PSG Electrical and Cables delivers technical precision across complex commercial, industrial, and solar infrastructures. Each project represents our commitment to safety, efficiency, and engineering craftsmanship.

ALLCOMMERCIALINDUSTRIALSOLARRESIDENTIAL

LAYOUT



COMMERCIAL

SANDTON, GP

OCT 2023

Global Tech Headquarters

Complete electrical infrastructure overhaul including smart lighting controls and backup power systems for a

VIEW CASE STUDY →



INDUSTRIAL

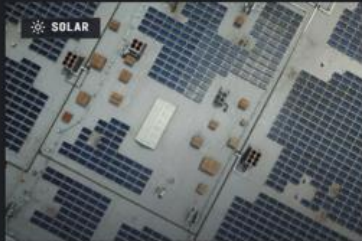
MIDRAND, GP

AUG 2023

Precision Manufacturing Plant

Installation of high-voltage distribution boards and automated machinery power lines for a heavy industrial

VIEW CASE STUDY →



SOLAR

PRETORIA, GP

DEC 2023

Eco-Park Solar Array

Design and commissioning of a 250kW rooftop solar farm with integrated battery storage solutions for

VIEW CASE STUDY →



COMMERCIAL

CAPE TOWN, WC

SEP 2023

Metro Data Center

Critical power distribution and redundant UPS cabling for a Tier III data center, ensuring 99.99% uptime

VIEW CASE STUDY →



INDUSTRIAL

DURBAN, KZN

NOV 2023

Advanced Logistics Hub

Complex conduit runs and industrial lighting for a 50,000sqm distribution center with specialized sorting

VIEW CASE STUDY →



RESIDENTIAL

STELLENBOSCH, WC

JUL 2023

Signature Smart Residence

Premium smart home automation system featuring home cinema wiring, multi-room audio, and luxury lighting

VIEW CASE STUDY →

END OF CURRENT LIST

LOAD MORE PROJECTS

Page 213 of 235

# HAVE A COMPLEX *PROJECT* IN MIND?

Our team of certified engineers and electricians are ready to bring technical excellence to your infrastructure. Let's discuss your requirements today.

REQUEST A QUOTATION

OUR SERVICES



PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



Quick Links

Home

About Us

Services

Projects

Contact Us

Our Services

Electrical Installations

Cabling Solutions

Maintenance & Repairs

Commercial & Industrial

Tribe Solar

Contact Us

 +27 11 555 6789

 [info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)

 123 Power Street,  
Sandton, South Africa

WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00

[Emergency: 24/7](#)

© 2024 PSG Electrical and Cables. All rights reserved.

Privacy Policy

Terms & Conditions

Page 214 of 235

**PSG ELECTRICAL**  
AND CABLES

HomeAbout UsServicesProjectsTrite Solar

GET A QUOTE

COMPANY CREDIBILITY

# POWERING **RELIABILITY** THROUGH EXPERTISE.

For over 15 years, PSG Electrical and Cables has been the trusted partner for complex electrical engineering projects across South Africa.

[VIEW OUR ACCREDITATIONS](#)[OUR VALUES](#)

THE PSG ADVANTAGE

## ENGINEERED FOR EXCELLENCE.

We don't just provide wiring; we deliver robust energy infrastructures that power growth and stability.



### Certified Safety

We adhere to the highest SHEQ standards, ensuring every installation is safe, compliant, and durable.



### Industry Experts

Our team consists of master electricians and engineers with decades of collective experience in industrial power.



### 24/7 Response

Electrical emergencies don't wait. Our rapid response teams are available around the clock for critical support.



### Premium Materials

We only source components from world-class manufacturers, guaranteeing longevity and peak performance.



### Client-Centric

From initial audit to final sign-off, our transparent communication keeps your project on track and in budget.



### Optimized Efficiency

Our solutions are designed to reduce energy waste and lower operational costs for your facility.

Page 215 of 235



## Industry Compliance & Standards

At PSG, quality isn't just a promise—it's a measurable standard. We are fully registered and compliant with all South African electrical regulatory bodies, ensuring that our work meets international safety benchmarks.



QUALITY  
MANAGEMENT

**ISO  
9001:2015**



ELECTRICAL  
COMPLIANCE

**ECB  
Registered**

 **FULL DOCUMENTATION**

*"Every project we undertake is accompanied by a comprehensive Certificate of Compliance (CoC), guaranteeing the integrity and legal standing of your electrical system."*



**100% Safety Track Record**

ZERO INCIDENTS OVER 300+ INDUSTRIAL PROJECTS

## DRIVEN BY VALUES

Our culture is built on the pillars of integrity, technical mastery, and a relentless commitment to our clients' success.



**Our Vision**

**Powering a Sustainable Future**

-  **Innovation**  
Embracing the latest in solar and smart-grid technology to future-proof your energy needs.
-  **Integrity**  
Transparent pricing, honest assessments, and no hidden costs in any of our quotations.
-  **Sustainability**  
Prioritizing renewable solutions like Trite Solar to reduce carbon footprints.



**Our Mission**

**Uncompromising Quality Control**

-  **Precision**  
Every cable run and panel installation is executed with millimetre accuracy.
-  **Reliability**  
Ensuring your operations stay online with robust failover and maintenance protocols.
-  **Community**  
Investing in local skills and development through our ongoing technician training programs.



# READY TO POWER YOUR NEXT PROJECT?

Consult with our lead engineers today and discover why enterprise clients choose PSG for their critical infrastructure needs.

REQUEST A FREE AUDIT >

MEET THE TEAM



## PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



### Quick Links

- Home
- About Us
- Services
- Projects
- Contact Us

### Our Services

- Electrical Installations
- Cabling Solutions
- Maintenance & Repairs
- Commercial & Industrial
- Tribe Solar

### Contact Us

- +27 11 555 6789
- info@psgelectrical.co.za
- 123 Power Street, Sandton, South Africa

### WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00  
Emergency: 24/7

**PSG ELECTRICAL**  
AND CABLES

HomeAbout UsServicesProjectsTrite Solar

GET A QUOTE

GET IN TOUCH

# CONNECT WITH THE EXPERTS.

Whether you need a full industrial installation or a renewable energy audit, our team of certified professionals is ready to power your next project.

CALL NOW →VIEW LOCATIONS



### DIRECT SUPPORT

Immediate assistance for technical queries or emergency electrical faults.

- +27 11 555 6789
- +27 82 808 1234 (emergency)



### EMAIL ENQUIRIES

For formal RFQs, project documentation, or general service information.

- info@psgelectrical.co.za
- projects@psgelectrical.co.za



### REGIONAL OFFICE

Our main administrative and logistics hub located in the heart of Sandton.

- 123 power street
- sandton, johannesburg, 2196

## SEND AN INQUIRY

Complete the form below and one of our senior consultants will respond within 24 business hours.

FULL NAME

John Doe

EMAIL ADDRESS

john@company.com

PROJECT TYPE / SUBJECT

e.g., Commercial Solar Installation

MESSAGE / BRIEF REQUIREMENTS

Tell us about your project requirements...

TRANSMIT MESSAGE



### STANDARD OPERATING HOURS

Monday — Friday: 07:30 - 17:00  
Saturday: 08:00 - 13:00

Emergency Service: 24/7 Available



### LIVE CHAT SUPPORT

Technical consultants available during office hours for instant guidance.

[LAUNCH CHAT CONSOLE](#)



### HEALTH & SAFETY

All PSG installations comply with SANS 10142 regulations. Safety is our technical baseline.



MAP DATA INTERFACE LOADING...

### PSG HEADQUARTERS

Sandton, Johannesburg

## READY TO POWER YOUR NEXT BIG PROJECT?

Our engineering team is ready to provide detailed quotations and technical consultations for industrial, commercial, and residential requirements.

[REQUEST A CUSTOM QUOTE](#)



### PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



### Quick Links

[Home](#)  
[About Us](#)  
[Services](#)  
[Projects](#)  
[Contact Us](#)

### Our Services

[Electrical Installations](#)  
[Cabling Solutions](#)  
[Maintenance & Repairs](#)  
[Commercial & Industrial](#)  
[Trite Solar](#)

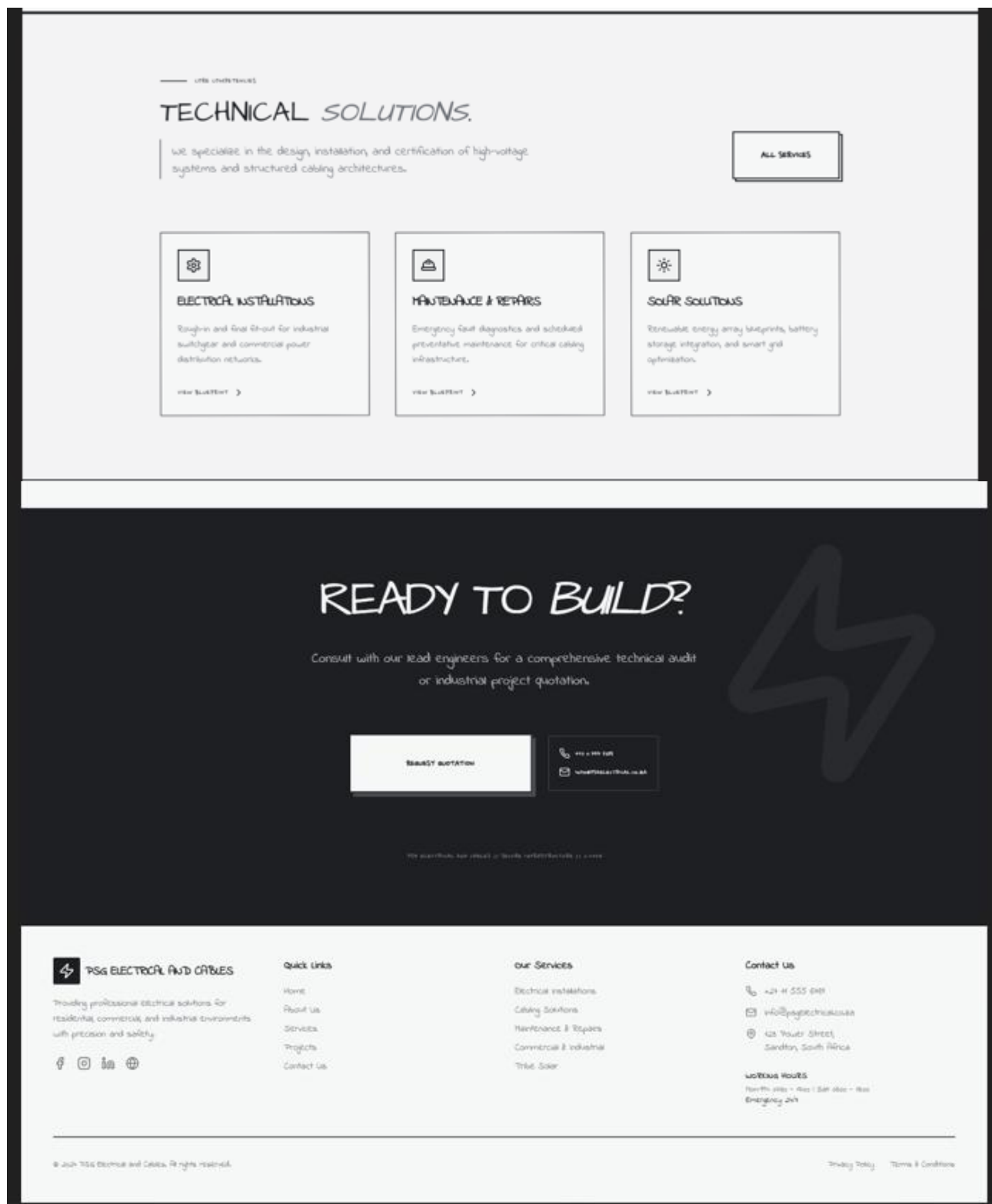
### Contact Us

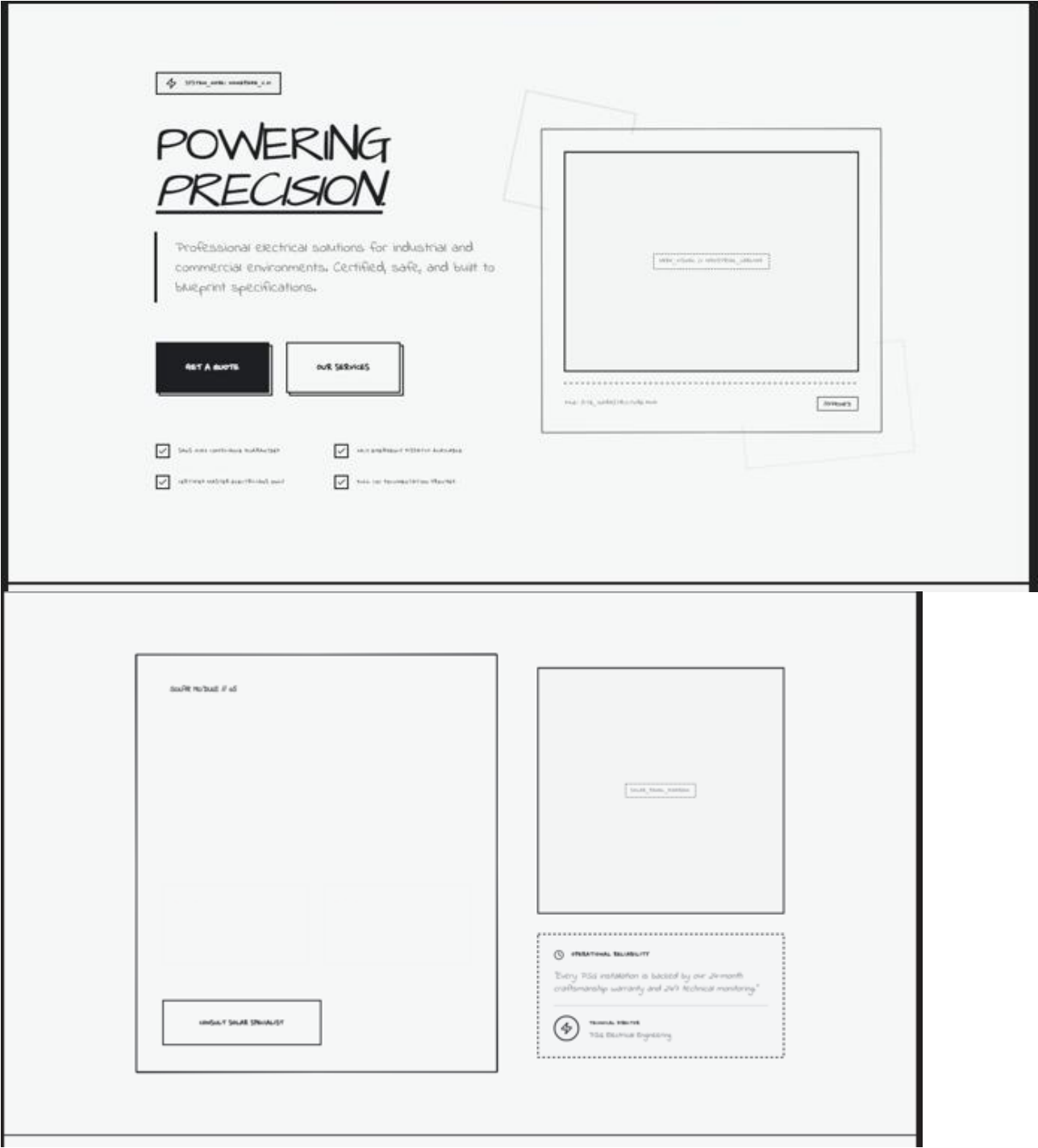
[+27 11 555 6789](#)  
[info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)  
123 Power Street,  
Sandton, South Africa

### WORKING HOURS

Mon-Fri: 07:30 - 17:00 | Sat: 08:00 - 13:00  
Emergency: 24/7

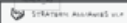
## Low Fidelity Website pages:








[Home](#)
[About Us](#)
[Services](#)
[Projects](#)
[Trite Solar](#)
[GET A QUOTE](#)



# Our Partners.

Collaborating with global engineering leaders to deliver uncompromised electrical infrastructure. Our network is the foundation of our technical reliability.

[BECOME A PARTNER](#)
[NETWORK STATS](#)



[ALL PARTNERS](#)
[TECHNOLOGY](#)
[SUPPLY CHAIN](#)
[MANUFACTURING](#)
[SAFETY & COMPLIANCE](#)

TECHNOLOGY

Partners (1) 7%



VOLTMETER SYSTEMS

"Advanced industrial multimeter and circuit protection modules."

Partnership with: 2020-2025

VIEW

MANUFACTURING

Partners (1) 3%



SOLARBOARD SOLUTIONS

"Automated panel manufacturing and battery storage innovation."

Partnership with: 2020-2025

VIEW

SUPPLY CHAIN

Partners (multiple) 12%



ATPEX CABLES CORP

"Heavy-duty copper and fiber optic cabling for industrial infrastructure."

Partnership with: 2020-2025

VIEW

SAFETY & COMPLIANCE

Partners (1) 8%



SAFEGUARD INDUSTRIAL

"Certified PPE and electrical safety audit equipment."

Partnership with: 2020-2025

VIEW

TECHNOLOGY

Partners (1) 5%



CIRCUITLOGIC LTD

"Smart grid automation and load management software systems."

Partnership with: 2020-2025

VIEW

SUPPLY CHAIN

Partners (1) 2%



POWERLINK LOGISTICS

"Rapid deployment and regional distribution of electrical components."

Partnership with: 2020-2025

VIEW

VIEW\_MORE\_PARTNERS

[VIEW ADDITIONAL PARTNERS](#)




## Technical Integration Protocol

We don't just use partner products; we integrate their engineering logic into our custom-built electrical architectures.

**#1**

### TECHNICAL AUDIT

Verifying partner component compatibility with PSG's safety standards.

**#2**

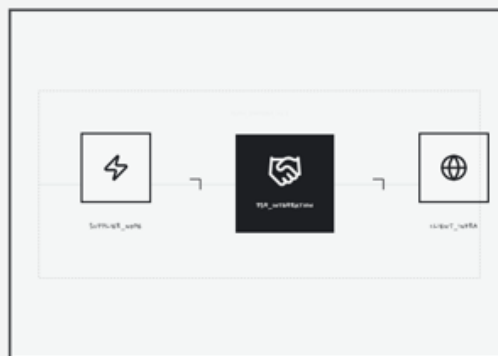
### SYSTEM DESIGN

Merging partner hardware into PSG's electrical blueprint schematics.

**#3**

### FIELD TESTING

Real-world operational stress testing before final commissioning.

[Download Partner Guide](#)

REGULATORY ALIGNMENT // ACCREDITATIONS



0



0



0



0



0

## Scale Your Potential

We are always looking to integrate innovative technology into our electrical solutions. If your organization leads in engineering or renewable energy, let's connect.

[Initiate Proposal Flow](#)[View Case Studies](#)

### PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



### Quick Links

[Home](#)  
[About Us](#)  
[Services](#)  
[Projects](#)  
[Contact Us](#)

### Our Services

[Electrical Installations](#)  
[Cabling Solutions](#)  
[Maintenance & Repairs](#)  
[Commercial & Industrial](#)  
[Trite Solar](#)

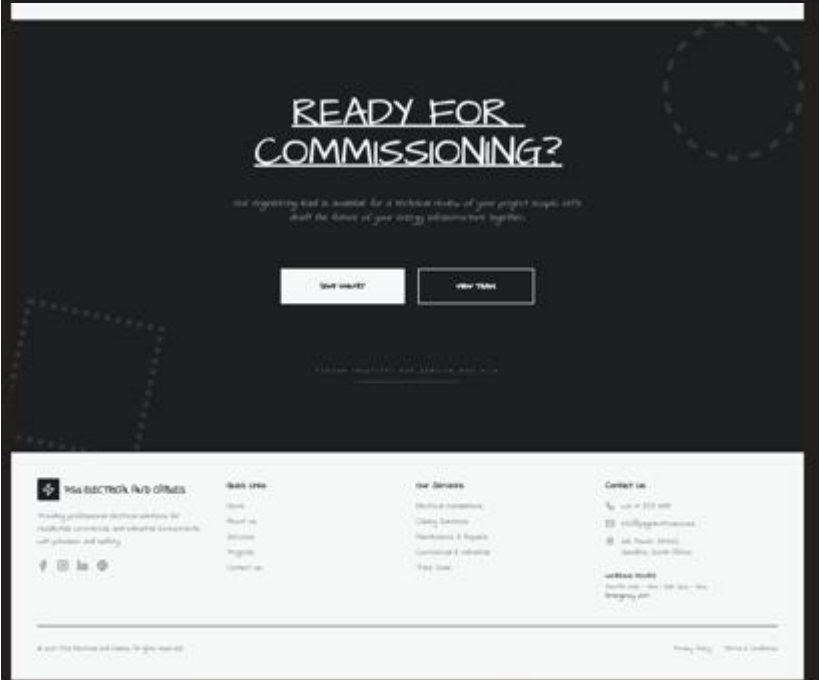
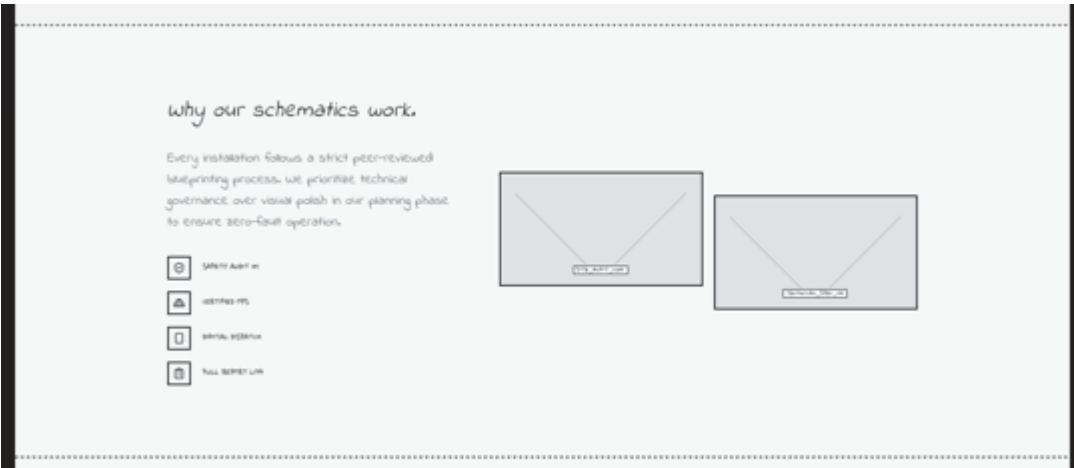
### Contact Us

[+27 11 555 6981](#)  
[info@psgelectrical.co.za](mailto:info@psgelectrical.co.za)  
425 Power Street,  
Sandton, South Africa

### WORKING HOURS

Mon-Fri 08:00 - 18:00 | Sat 08:00 - 16:00  
Emergency 24/7





classroom50.org/EMKNDN/emkndn-prog7314-2026/assignments/prog7314-ice3/accept

 Classroom 50

Unavailable

### This assignment isn't available to you

Ask your teacher if you think this is a mistake.

Signed in as

 ST10288567  ST10288567

Dash BETA

Queue Chan... Previ... Termi... Notes

Select a thread to show tools.



### Start building with Freebuff

Pick a project and start a thread, or pick up where you left off.

+ Open project

Press Ctrl+T anytime for a new thread

Page 227 of 235



HomeAbout UsServicesProjectsTrite Solar

GET A QUOTE

THE EXPERTS  
BEHIND THE  
POWER.

our team consists of industry veterans and certified engineers dedicated to technical precision and uncompromising safety. Each member brings a unique blueprint of excellence to PSG.

JOIN OUR TEAM

VIEW SERVICES



MEET THE ENGINEERS

Qualified Expertise.

our rigorous vetting process ensures that every PSG employee is not just certified, but aligned with our core philosophy of technical craftsmanship.

WORKER SFT | **ALL** | SUBMITTALS | SPECT | PROJECT MANAGEMENT



Arthur Penhalgon

MASTER ELECTRICIAN / FOUNDER





Sarah Jenkins

LEAD SYSTEMS ENGINEER





Marcus Thome

SENIOR PROJECT MANAGER



Page 228 of 235



SHOTIN\_REP: ELENA

**Elena Rodriguez**

CHIEF SAFETY OFFICER





SHOTIN\_REP: DAVID

**David Ukosi**

HEAD OF LABELING DIVISION





SHOTIN\_REP: CLARA

**Clara Venter**

TECHNICAL CONSULTANT



**LEADERSHIP PHILOSOPHY**

At PSG, leadership is defined by technical mentorship. Our seniors aren't just managers—they are master practitioners who remain involved in the critical circuit design and safety audit of every project.

"Precision is our only currency. If a connection isn't perfect, the entire infrastructure is a liability."

— ARTHUR FRANKLIN, FOUNDER



**Safety First**

Adherence to OSHA and SANS wiring codes is non-negotiable.



**Structural Integrity**

We design for longevity, ensuring future-proof scalability.



**Rapid Response**

24/7 technical support for critical industrial failure management.



**Master Level**

Top-tier certifications maintained through continuous R&D.



**15+**

YEARS EXPERIENCE



**45**

CERTIFIED STAFF



**500+**

PROJECTS MANAGED



**100%**

SAFETY RECORD

Page 229 of 235

# READY TO COLLABORATE WITH OUR ENGINEERS?

Connect with our senior consultants today to initiate a technical audit or request a detailed proposal for your next infrastructure upgrade.

REQUEST A QUOTATION

CONTACT SUPPORT



CONTACT: 021 555 5601 | 021 555 5602



## PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.



### Quick Links

[Home](#)  
[About Us](#)  
[Services](#)  
[Projects](#)  
[Contact Us](#)

### Our Services

[Electrical Installations](#)  
[Cabling Solutions](#)  
[Maintenance & Repairs](#)  
[Commercial & Industrial](#)  
[Trite Solar](#)

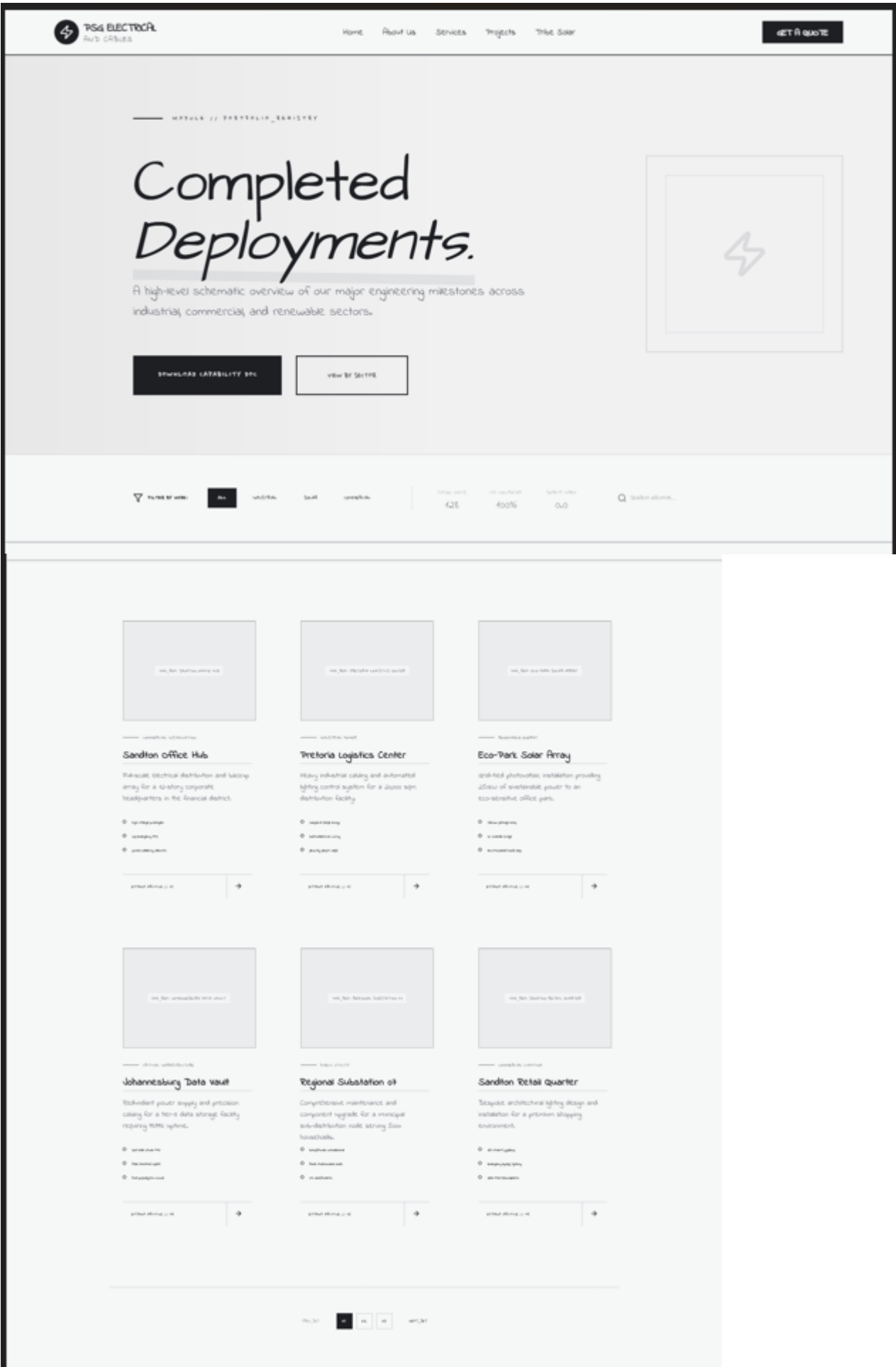
### Contact Us

[+27 11 555 5601](#)  
[info@psgelectricalcables.co.za](mailto:info@psgelectricalcables.co.za)  
425 Power Street,  
Sandton, South Africa

### WORKING HOURS

Mon-Fri 08:00 - 18:00 | Sat 08:00 - 18:00  
Emergency 24/7







OUR PROMISE:  
ZERO-HARM  
OPERATION.

☐ Fixed-Price Technical Solution

☐ Full Site Audit Before Handover

☐ Immediate Final Verification

☐ Comprehensive Documentation Trail

01753 555 6781

01753 555 6781

READY TO POWER YOUR  
NEXT PROJECT?

Our lead engineers are standing by with their pencils ready. Let's turn your technical requirements into a certified success.

INITIATE AUDIT

01753 555 6781

PSG ELECTRICAL AND CABLES

Providing professional electrical solutions for residential, commercial, and industrial environments with precision and safety.

TEAM LEADS

David

David Lee

Director

HydroLife

Contact us

OUR SERVICES

Electrical Installation

Cabling Solutions

Maintenance & Repairs

Commercial & Industrial

More...

CONTACT US

Call us on 01753 555 6781

info@psgelectrical.co.uk

145 River Street

Scarborough, North Yorkshire

WORKING HOURS

Monday - Friday: 9am - 5pm

Saturday: 9am - 12pm

Emergency 24/7

© 2024 PSG Electrical and Cables. All rights reserved.

Privacy Policy

Terms & Conditions

Page 233 of 235



